

INFORMATIQUE

2024 - 2025

Master

Méthode et outils pour l'Intelligence Artificielle (MIA)

Anne BOUQUET

Fabrice BOUQUET

SUP.

Table des matières

Liste des illustrations	vi
0. Initiation au langage Prolog	2
0.1 Présentation	2
0.1.1 Historique	2
0.1.2 Motivation	2
0.1.3 Mise en œuvre	3
0.1.4 Conclusion	3
0.2 Premiers pas	3
0.2.1 Création des fichiers PROLOG	3
0.2.2 Ecriture des fichiers PROLOG	4
0.2.3 Utiliser le logiciel SWI-Prolog	4
0.2.4 Utilisation du logiciel SWI-Prolog	4
0.3 Didacticiel PROLOG	5
0.3.1 Les types de base	5
0.3.2 Les termes de base	6
0.3.3 Déclaration des faits	6
0.3.4 Poser des questions ou buts	6
0.3.5 Le principe d'unification	7
0.3.6 Les variables partagées	7
0.3.7 Les règles	8
0.3.8 Test unitaire	9
0.3.9 Primitive de trace	11
0.3.10 La structure de liste	11
0.3.11 Etude d'une exécution d'un programme	13
0.3.12 Le prédicat cut : "!"	15
0.3.13 Le prédicat de la négation : "\+"	18
0.3.14 L'opérateur numérique "is"	19
0.3.15 Comparaison de valeurs numériques	19
0.4 Programmation avec contraintes	20
0.4.1 Objectifs de CLP(R) et CLP(Q)	20
Chargement de la bibliothèque CLP(R)	20
Poser de contraintes CLP(R) ou CLP(Q)	21
Exemple de solutions multiples	22
0.4.2 La bibliothèque CLP(FD)	22
Chargement de la bibliothèque CLP(FD)	23
Mode d'utilisation de CLP(FD)	23
Déclaration des domaines	23
Pose des contraintes	24
Génération de valeurs	25

0.5	Exercices chapitre 0	27
1	Introduction	32
1.1	Pourquoi MOIA ?	32
1.1.1	Plan du cours	32
1.1.2	Que signifie l'Intelligence Artificielle ?	33
1.2	Historique	33
1.3	Qu'est-ce que les systèmes à base d'IA savent faire ?	35
2	Représentation des connaissances	37
2.1	Les logiques classiques	37
2.1.1	La logique propositionnelle	37
2.1.2	La logique des prédicats	40
2.2	Les logiques non-classiques	42
2.2.1	La logique des défauts	42
2.2.2	La logique floue	43
2.3	Les règles de Production	43
2.3.1	Qu'est-ce qu'une règle de production ?	44
2.4	Les objets structurés	45
2.4.1	Comment raisonner ?	45
2.4.2	Les réseaux bayésiens	45
2.4.3	Les chaînes de Markov	46
2.4.4	Les réseaux de neurones	46
2.5	Exercices chapitre 2	48
3	Algorithmes classiques	51
3.1	Définitions	51
3.1.1	Définir un problème ?	51
3.1.2	Méthode de résolution	52
3.2	Les stratégies	53
3.2.1	Parcours en profondeur	53
3.2.2	Profondeur bornée	54
3.2.3	Parcours en largeur	55
3.2.4	Comparaison des approches	57
3.3	Parcours avec heuristiques	58
3.3.1	Définitions	58
3.3.2	Les méthodes à base de gradient	59
3.4	Best-First	60
3.4.1	Algorithme Gourmand	60
3.4.2	Algorithme A*	60
3.4.3	Variant A*	63
3.5	Exercices chapitre 3	68
4	Programmation par contraintes	72
4.1	Définitions des contraintes	72
4.1.1	Constraints Satisfaction Problem	73
4.2	Méthodes de résolution	74
4.2.1	Générer et Tester	74
4.2.2	Backtracking	75

4.3	Consistances - Enumérations	76
4.3.1	Techiques de consistance	76
4.3.2	Propagation de contraintes	86
4.3.3	Branch and Bound	87
4.4	Exercices chapitre 4	89
5	Systèmes experts	94
5.1	Présentation	94
5.1.1	But	94
5.1.2	Les éléments clefs	94
5.1.3	Cycle du moteur d'inférence	95
5.2	Mode de raisonnement	96
5.2.1	Chaînage avant	96
5.2.2	Chaînage arrière	97
5.3	Exercices chapitre 5	98
6	Algorithmes des jeux	101
6.1	Algorithmes	101
6.1.1	Algorithme Minimax	101
6.1.2	Algorithme $\alpha - \beta$	102
6.2	Exemples	103
6.2.1	Othello	103
6.2.2	Les échecs	104
6.2.3	La contrée	106
6.3	Exercices chapitre 6	107
7	Planification	108
7.1	Contextes	108
7.1.1	Problématique	108
7.1.2	Job-Shop	109
7.1.3	Les autres types de machines	110
7.2	Graphe de potentiels	113
7.3	Exercices chapitre 7	114
8	Apprentissage	116
8.1	Présentation	116
8.1.1	Apprentissage et inférence	117
8.1.2	Les phases pour faire de l'apprentissage	118
8.1.3	Traitement(s) des données	119
8.1.4	L'évaluation	121
8.1.5	Les dilemmes	124
8.2	Les approches	126
8.2.1	Régression	127
8.2.2	Machine à vecteur de support (SVM)	129
8.2.3	Version space	130
8.2.4	Arbre de décisions	132
8.2.5	K-Means	137
8.2.6	MeanShift	139
8.2.7	Classification ascendante hiérarchique	141

8.3	Réseau de neurones	144
8.3.1	Apprentissage	145
8.3.2	Le perceptron	145
8.3.3	Les réseaux multi-couches	146
8.4	Exercices chapitre 8	149
9	Méthode incomplète	154
9.1	Intérêt	154
9.2	Fonctionnement	154
9.3	Exemples	154
9.3.1	Les reines	154
9.3.2	3-Sat	155
9.4	Exercices chapitre 9	155
10	Linguistique	157
10.1	Sensibilisation	157
10.1.1	Problème : Homonyme et polysémie	157
10.1.2	Problème : Synonyme	157
10.1.3	Problème : Contexte et Figure de style	158
10.1.4	Solution	158
10.2	Exercices chapitre 10	159
11	Correction des exercices	160
11.1	Exercices chapitre 0	160
11.2	Exercices chapitre 2	171
11.3	Exercices chapitre 3	178
11.4	Exercices chapitre 4	188
11.5	Exercices chapitre 5	201
11.6	Exercices chapitre 6	204
11.7	Exercices chapitre 7	208
11.8	Exercices chapitre 8	213
11.9	Exercices chapitre 9	224
11.10	Exercices chapitre 10	226
	Annexes	228
A	Principaux prédicats prolog	229

Liste des illustrations

1	Graphe d'exécution	15
2	Le cycle de création de test	30
2.1	Formalisation français et logique	41
2.2	Exemple de règles de production	44
2.3	Neurone	47
2.4	Synapse	47
2.5	Neurone artificiel	48
3.1	Description d'un problème	52
3.2	Espace d'états	52
3.3	Parcours en profondeur	53
3.4	Problème du parcours en profondeur	54
3.5	Parcours en profondeur bornée	55
3.6	Parcours en largeur	56
3.7	Exemple d'algorithme gourmand	60
3.8	Exemple de l'algorithmique A^*	62
3.9	Exemple Graphe ET-OU	64
3.10	Exemple de graphe AO^*	66
3.11	Le labyrinthe	68
3.12	Les configurations	70
4.1	Exemple backtracking	75
4.2	Exemple de CSP	76
4.3	Exemple de consistance d'arcs	77
4.4	Exemple d'insuffisance pour consistance d'arcs	78
4.5	Exemple zèbre - contraintes différences	80
4.6	Exemple zèbre - après 6 contraintes	81
4.7	Exemple zèbre - après 13 contraintes	82
4.8	Exemple zèbre - après 14 contraintes	83
4.9	Les deux approches pour la propagation	86
4.10	Exemple de Branch and Bound	88
4.11	Graphe de nombres	92
5.1	Composant d'un Système expert	95
5.2	Exemple chainage avant	97
5.3	Exemple chainage avant suite	97
5.4	Exemple de chainage Arrière	98
6.1	Exemple de graphe Minmax	102
6.2	Exemple de graphe $\alpha - \beta$	102

6.3	Exemple du jeu Othello	104
6.4	Exemple du jeu d'échec	105
6.5	Exemple effet Bloc	105
6.6	Exemple effet d'horizon	106
6.7	Exemple jeu de la contrée	106
7.1	Exemple Job shop sans ordre avec heuristique	109
7.2	Exemple Job shop sans ordre	110
7.3	Exemple Job shop avec ordre et heuristique	110
7.4	Exemple Job shop avec ordre	110
7.5	Exemple de graphe de potentiel	113
7.6	Exemple de graphe au plus tôt	114
7.7	Exemple de graphe au plus tard	114
8.1	Schéma du scientifique des données	120
8.2	Exemple d'erreur irréductible	124
8.3	Exemple d'apprentissage pour les lapins	125
8.4	Exemple de variance pour les lapins	125
8.5	Courbe avec explication du minimum global et local	128
8.6	Régression polynomiale	129
8.7	SVM linéaire à marge rigide	130
8.8	Exemple de méthode de généralisation	132
8.9	Exemple de la méthode ID3	136
8.10	Exemple graphe de la méthode elbow	139
8.11	Dendrogramme de l'exemple	144
8.12	Espace représentant le fonction <i>xor</i>	146
8.13	Exemple de réseau une couche	147
8.14	Exemple de réseau multi-couches	148
8.15	Exemple de réseau Hopfield	148
8.16	Exemple de réseau en treillis	149
8.17	Arbre de hiérarchie de couleur	151
9.1	Seuil pour les problèmes 3-Sat	155

0. Initiation au langage Prolog

Ce chapitre transversal est consacré au langage PROLOG. Ce langage est en effet utilisé pour mettre en pratique les techniques et algorithmes étudiés dans ce module. L'assimilation des leçons [Leçon 1](#) et [Leçon 2](#) est nécessaire pour effectuer les exercices. La compréhension de la [leçon 3](#) est nécessaire pour effectuer les exercices du sujet 5, mais cette leçon ne nécessite pas d'être assimilée avant l'étude du [Chapitre 4](#).

- Nous introduisons tout d'abord ce chapitre par une brève présentation du langage PROLOG dans la partie intitulée : [Leçon 1](#)
- Nous précisons ensuite les commodités d'utilisation de ce langage au moyen du logiciel SWI-Prolog : [Leçon 2](#)
- Puis, nous présentons de manière informelle les techniques de Programmation Logique avec SWI-Prolog : [Leçon 3](#)
- Enfin, nous terminons en présentant trois bibliothèques de SWI-Prolog permettant d'utiliser la Programmation Logique avec Contraintes : [Leçon 4](#)

0.1 Présentation

Que signifie PROLOG ?

PROLOG : **PRO**grammation en **LOG**ique

0.1.1 Historique

Le langage PROLOG est né en 1972 au Groupe d'Intelligence Artificielle de Luminy (Université de Marseille) dirigé par Alain Colmerauer. Ce n'est cependant qu'au début des années 1980 que PROLOG est projeté sur le devant de la scène par l'intermédiaire de projets internationaux de développement de l'Intelligence Artificielle. Ce langage n'était auparavant connu que par un cercle étroit de quelques chercheurs. Aujourd'hui, PROLOG est largement répandu, non seulement dans les Universités et laboratoires de recherche, mais aussi au sein du monde industriel.

0.1.2 Motivation

Le langage PROLOG est né d'un pari : créer un langage de très haut niveau sans se préoccuper de son efficacité au sens des informaticiens de l'époque. L'efficacité consistait alors à faire exécuter très rapidement par une machine des programmes écrits de manière laborieuse. Le pari était donc de pouvoir écrire rapidement des programmes, quitte à ce que la machine les exécute de manière laborieuse.

0.1.3 Mise en œuvre

Débarassé de cette obsession d'efficacité, les chercheurs du Groupe d'Intelligence Artificielle de Luminy se sont appuyés sur les formalismes et les mécanismes d'inférence de la logique mathématique pour inventer le langage PROLOG (d'où il tire son nom). Ces fondements poussent la machine, sur laquelle les programmes PROLOG sont exécutés, à être plus intelligente que d'habitude. Elle doit en effet être non-déterministe, c'est-à-dire capable d'explorer elle-même plusieurs possibilités.

0.1.4 Conclusion

Le langage PROLOG est à la fois facile et difficile. En effet, sa simplicité permet très vite aux débutants d'écrire des premiers programmes souvent spectaculaires. Mais la réalisation de programmes plus conséquents demande une rigueur de conception et une maîtrise du langage acquises bien souvent par une longue pratique.

0.2 Premiers pas

Les prolog disponible

On distingue en fait deux filières dont émanent les logiciels PROLOG. D'une part, la filière française (historique) issue du Groupe d'Intelligence Artificielle de Luminy (logiciels PROLOG I, PROLOG II, PROLOG III,...). D'autre part, la filière écossaise issue de l'Université d'Edimbourg. Il existe des versions payantes et des versions gratuites pour des usages éducatifs. Dans ce cours, nous vous proposons d'utiliser l'environnement SWI-Prolog disponible sur le site <http://www.swi-prolog.org>. Celui-ci offre l'avantage d'avoir la possibilité de travail dans un navigateur <https://swish.swi-prolog.org> ou en téléchargeant une exécutable pour les plateformes Microsoft Windows (32 ou 64 bits), Mac OS Mojave ou supérieur et différentes versions de Linux <http://www.swi-prolog.org/download/stable>.

0.2.1 Création des fichiers PROLOG

Il y a trois méthodes. La première dite classique. Après avoir lancé l'application SWI-Prolog, vous allez dans le menu File $\hookrightarrow$ New... En suite, vous indiquez le nom du fichier avec l'extension `.pl` (ce renommage n'est en aucun cas une nécessité de bon fonctionnement mais il correspond à une bonne pratique). Ensuite, l'éditeur s'ouvre et vous pouvez écrire votre programme prolog.

La deuxième est d'utiliser un éditeur tiers. Les programmes PROLOG doivent être écrits puis enregistrés dans un fichier texte.

ATTENTION : n'utilisez pas MICROSOFT WORD ou tout logiciel apparenté pour rédiger vos programmes. En effet, des balises viendraient alors surcharger le code source de votre fichier sans que vous n'ayez l'opportunité de les ôter par la suite (ce qui explique pourquoi un document WORD vide occupe près de 20Ko). Or, ces balises ne sont pas supportées par SWI-Prolog qui vous renvoie alors une liste d'erreurs souvent incompréhensibles. Préférez donc des éditeurs de texte plus classiques, de type "notepad" ou "emacs" qui permettent sans risque de construire de réels fichiers en texte seulement (extension `*.txt`).

Notez, pour les puristes, que l'extension par défaut des fichiers texte (`*.txt`) peut être modifiée en `*.pl`, qui est l'extension traditionnelle des fichiers PROLOG (ce renommage n'est en aucun cas une nécessité de bon fonctionnement mais il correspond à une bonne pratique).

et vous permettra de bénéficier du filtre pour retrouver vos fichiers dans l'application SWI-Prolog).

La troisième approche est d'utiliser l'éditeur en ligne SWISH. Vous pouvez regarder la vidéo qui vous explique les premiers pas. En résumé, il faut créer un nouveau programme basé sur un projet vide (*empty*). Hélas cette troisième option atteint c'est limite dès que vous utiliserez des bibliothèques comme PLUnit pour les tests.

0.2.2 Ecriture des fichiers PROLOG

Les programmes PROLOG s'écrivent au moyen des caractères traditionnels présents sur les claviers. Il est possible de renseigner son code avec des commentaires. Il existe pour cela deux types différents de balisage qui sont les suivants :

- Tous les caractères qui suivent le signe "%" sont considérés comme des commentaires. Dès qu'un retour chariot est effectué (appui de la touche entrée), la zone de commentaires prend fin. On utilise ainsi le signe "%" pour placer des commentaires sur une ligne complète ou à la fin d'une ligne.
- Tous les caractères se trouvant entre les deux signes "/*" et "*/" sont considérés comme des commentaires. On utilise ce type de balisage pour définir un bloc de commentaires d'une ou de plusieurs lignes.

0.2.3 Utiliser le logiciel SWI-Prolog

Vous pouvez consulter la vidéo du cours pour l'utilisation de SWI-Prolog en version en ligne. Pour utiliser le logiciel SWI-Prolog, il suffit de lancer le programme "swiprolog" (ou "swiprolo.exe" pour les utilisateurs de WINDOWS) situé dans le dossier "bin" de l'application.

Au lancement de l'application, une fenêtre similaire à la suivante doit apparaître :

```
Welcome to SWI-Prolog (threaded, 64 bits, version 8.4.3)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit http://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?-
```

Le curseur clignotant n'attend plus que vos instructions !

0.2.4 Utilisation du logiciel SWI-Prolog

Dans la console de SWI-Prolog, les symboles "|?-" indique en fait que le logiciel est en attente d'une question. Avant de poser toute question, il est nécessaire de mettre votre programme dans la mémoire.

Cette opération est effectuée au moyen de la commande "consult" (si vous chargez votre programme pour la première fois) ou "reconsult" (si vous rechargez votre programme pour faire des mises à jour). Vous pouvez le faire à travers le menu File ↔ Consult... ou Reload modified files. Vous pouvez aussi ouvrir directement l'éditeur. de la manière suivante :

```
?- consult('C:\mon_repertoire\mon_fichier.pl').
```

ou

```
?- reconsult('C:\mon_repertoire\mon_fichier.pl').
```

Il est à noter que sous windows, il faut doubler les \ en \\ :

```
?- consult('C:\\mon_repertoire\\mon_fichier.pl').
```

Suite à cette (re)consultation, différents messages peuvent apparaître à l'écran. Le symbole qui débute la ligne permet de connaître leur nature :

- "%" : les messages commençant par le symbole pourcentage décrivent une action réalisée par PROLOG. Ces messages sont simplement informatifs.
Exemples : information de la consultation d'un fichier ou chargement d'une bibliothèque.
- "*" : les messages commençant par une étoile sont des avertissements. Ils indiquent que certaines anomalies ont été trouvées dans le programme consulté. Néanmoins ces anomalies ne sont pas bloquantes à l'exécution de celui-ci. **Exemples** : découverte de variables muettes ou définitions de prédicats (fonctions) disséminées le long du code.
- "!" : les messages commençant par un point d'exclamation notifient des erreurs bloquantes dans le code. Ces erreurs doivent absolument être corrigées avant de procéder à la pose de questions.

Nous reviendrons plus en détails sur les deux derniers types de messages (anomalies et erreurs) dans le didacticiel de la [section 0.3](#), au cours duquel nous allons mettre en place et questionner nos premiers programmes PROLOG.

0.3 Didacticiel PROLOG

- L'objectif de ce didacticiel consiste à vous présenter, à travers de petits exemples, la philosophie et les mécanismes de base du langage PROLOG. Une documentation en anglais complète est disponible en ligne dont les informations pourront vous être utiles pour répondre aux questions techniques plus précises.
- Il est impératif que ce didacticiel soit compris pour effectuer les exercices des sujet0, sujet2, sujet3 et sujet4.

0.3.1 Les types de base

PROLOG s'appuie sur deux types de base :

- Les identificateurs : ils sont représentés par une suite de caractères alphanumériques commençant par une lettre minuscule (les lettres accentuées sont à éviter).
Exemples : f, aLPHA, jean-paul, h1
- Les variables : elles sont représentées par une suite de caractères alphanumériques commençant par une lettre majuscule (les lettres accentuées sont également à éviter) ou par le caractère "_".
Exemples : X, X1, MACHIN, Truc, _truc, _

0.3.2 Les termes de base

Les identificateurs et variables sont utilisés pour définir les termes de base du langage PROLOG :

- Les prédicats (un prédicat peut être vu comme une fonction dont le paramètre de retour est une valeur booléenne) sont représentés à l'aide d'un identificateur. Les éventuels arguments sont écrits après, entre parenthèses, séparés par des virgules (notation préfixée et parenthésée).

Ainsi, la forme générale d'un prédicat est la suivante : *prédicat(argument1, argument2, ...)*. Un prédicat est un symbole qui traduit une relation. Le poids est le nombre de ses arguments (on utilise le terme poids pour les prédicats car l'arité est utilisée pour les fonctions). On identifie un prédicat par son nom et son poids : *prédicat/poids*. Un prédicat se termine toujours par un point.

- Les constantes sont représentées par un nombre ou un identificateur.
- Les variables.

0.3.3 Déclaration des faits

Les faits sont des affirmations qui décrivent des relations ou des propriétés. Ils sont introduits par l'intermédiaire de prédicats.

Exemple : le prédicat *habite/2* ("*/2*" définit que le prédicat "habite" possède deux arguments)

```
% liste de faits d'un programme PROLOG
habite(christophe, vesoul) .
habite(fabrice, marseille) .
habite(fabien, belfort) .
habite(laurent, vesoul) .
```

Dans cet exemple, le prédicat *habite/2* exprime qu'un individu (premier argument) réside dans une ville (second argument). On peut noter que les noms propres commencent par une minuscule et non par une majuscule (ce sont des constantes et non pas des variables).

Ces déclarations sont à sauvegarder dans un fichier texte que l'on consulte ensuite depuis la console PROLOG à l'aide de l'instruction "consult".

0.3.4 Poser des questions ou buts

Une fois ce petit programme consulté, on peut poser des questions (appelées également buts) dans la console PROLOG. Ces questions prennent elles-aussi la forme de prédicats :

```
?- habite(fabrice, marseille) .
true.

?- habite(fabien, X) .
X = belfort.

?- habite(X, vesoul) .
X = christophe.

?- habite(X, vesoul) .
X = christophe ;
X = laurent.
```

Les quatre questions posées permettent de présenter l'ensemble des possibilités de réponses possibles.

Si la question ne contient aucune variable, PROLOG répond `true.` ou `false.` suivant que le fait est dans la base de données ou non.

On peut aussi utiliser des variables. Elles peuvent s'identifier à toutes les autres valeurs : aux constantes et aux variables elles-mêmes. On dit alors que les variables **s'unifient** aux autres termes.

Si la question contient une variable ou plus, PROLOG fournit le premier résultat trouvé (les faits sont explorés à la volée dans l'ordre d'apparition dans le code PROLOG). En frappant un point virgule, on obtiendra le second résultat, et ainsi de suite. En frappant la touche entrée (retour chariot), on arrête le parcours des solutions. Le point à la fin d'une réponse précise qu'il n'y a pas ou plus de solution, tandis que si le prompt n'est pas affiché, cela signifie que l'ensemble des faits n'ont pas été parcouru, et qu'il reste des solutions potentielles.

0.3.5 Le principe d'unification

Lorsqu'une variable s'identifie à un autre terme, on dit qu'il y a unification de cette variable. Pour ceci, PROLOG réalise la substitution de la variable par le terme correspondant. Dans le cas de la question `habite(fabien, X)`, on note cette substitution : `X = belfort.`

Les variables peuvent s'unifier à tous les autres termes, contrairement aux identificateurs qui ne s'unifient qu'avec les identificateurs du même nom (en plus des variables bien entendu). Ainsi l'unification `fabien = christophe` échoue, ce qui explique que la réponse `X = vesoul` à la question `habite(fabien, X)` ne soit pas générée. Lors d'un tel échec, PROLOG tente l'unification avec les termes restant de sa base de donnée. Si tout échoue, PROLOG répond `false.`

0.3.6 Les variables partagées

On utilise une même variable pour contraindre deux arguments à avoir la même valeur. Par exemple, pour chercher un individu qui porterait le nom de sa ville de résidence (l'exemple est un peu idiot), on note :

```
?- habite(X, X) .  
false.
```

Les questions peuvent être des conjonctions et on peut partager des variables entre les buts. Pour chercher les individus habitant au même endroit, on tapera :

```
?- habite(X,V),habite(Y,V).
X = Y, Y = laurent,
V = vesoul ;

X = laurent,
V = vesoul,
Y = christophe ;

X = Y, Y = fabrice,
V = marseille ;

X = Y, Y = fabien,
V = belfort ;

X = christophe,
V = vesoul,
Y = laurent ;

X = Y, Y = christophe,
V = vesoul.
```

0.3.7 Les règles

Les règles permettent d'exprimer des conjonctions de buts. Leur forme générale est :

$$\text{T\^ETE} \text{ :- } C1, C2, \dots, Cn.$$

La tête de la règle est formée par un seul prédicat, tandis que le corps est constitué de un ou plusieurs prédicats qui sont séparés par des virgules. La tête est séparée du corps par le signe ":-". Une règle PROLOG se termine toujours par un point.

La tête de la règle est vraie si chacun des éléments du corps de la règle $C1, \dots, Cn$ est vrai. On appelle ce type de règles des clauses de Horn.

Grâce à ce type de déclaration, il est possible de définir des règles directement dans le fichier PROLOG. Par exemple, introduisons la règle du dernier exemple dans notre fichier PROLOG initial au moyen de la clause `meme_endroit/2` :

```
% liste de faits d'un programme PROLOG
habite(christophe,vesoul).
habite(fabrice,marseille).
habite(fabien,belfort).
habite(laurent,vesoul).

% règle PROLOG
meme_endroit(X,Y) :- habite(X,V),habite(Y,V).
```

Le programme reconsulté, on peut poser directement la question suivante :

```
?- meme_endroit(X,Y).
X = Y, Y = laurent ? ;

...
```

Lors de l'exécution d'une telle requête, PROLOG examine les règles ou les faits correspondants dans l'ordre de leur écriture dans le programme : de haut en bas. Il utilise la première règle (ou le premier fait) du prédicat pour répondre. Si elle échoue, alors il passe à la règle suivante et ainsi de suite jusqu'à épuiser toutes les règles (ou tous les faits) définies pour ce prédicat. Lorsqu'une règle est récursive, l'interprète PROLOG rappelle le prédicat du même nom en examinant les règles (ou les faits) de ce prédicat dans le même ordre.

Dans le corps d'une règle, la virgule "," est le symbole représentant un ET logique : la conjonction de buts. Le symbole ";" représente le OU logique, la disjonction de buts.

0.3.8 Test unitaire

Prolog permet d'écrire des tests unitaires pour valider vos prédicats au fur et à mesure. Pour cela, il met à disposition une bibliothèque particulière `plunit`. Pour l'utiliser il suffit d'ajouter au début de votre programme `prolog :-use_module(library(plunit))`. Il est possible de mettre les tests dans un fichier différent de celui du programme. C'est ce fichier qui inclura la bibliothèque. Il faut juste bien avoir consulté les deux fichiers (celui du programme et celui des tests). Vous pouvez aussi mettre `:-consult('monFichierProgramme.pl')` dans votre fichier de tests pour qu'il inclue directement le fichier du programme.

En suite, vous identifiez vos tests en l'insérant entre deux balises `:-begin_tests(un_nom)` et `:-end_tests(un_nom)`.

Pour terminer l'écriture du test s'effectue de l'une des façons suivantes :

- `test('Le nom du test'):-predicat_a_tester`
- `test('Le nom du test',Liste_valeurs_attendues):-predicat_a_tester,`
où `Liste_valeurs_attendues` est une liste pouvant avoir cinq formes d'éléments :
 1. `true(Variable op valeur)`, où `op` est un opérateur de comparaison comme `==`, `\=`, `<` ...
 2. `all(Variable op liste_de_valeurs)`, les valeurs de la liste étant dans l'ordre d'apparition.
 3. `set(Variable op liste_de_valeurs)`, les valeurs de la liste étant dans l'ordre croissant.
 4. `fail` pour indiquer que le prédicat échoue
 5. `error` ou `exception` pour indiquer que le prédicat termine sur une erreur ou une levée d'exception.

Si, nous reprenons l'exemple précédent cela donnerait :

```
% La bibliothèque de test unitaire
:-use_module(library(plunit)).

% liste de faits d'un programme PROLOG
habite(laurent,vesoul).
habite(fabrice,marseille).
habite(fabien,belfort).
habite(christophe,vesoul).

% règle PROLOG
meme_endroit(X,Y):- habite(X,V),habite(Y,V).

% Les tests unitaires :
:-begin_tests(chp0).
test('habite(X,vesoul)',[true(X==laurent)]) :-
    habite(X,vesoul).
test('habiteA(X,vesoul)',[true(X\=christophe)]) :-
    habite(X,vesoul).
test('habiteAll(X,vesoul)',[all(X==[laurent,christophe])]) :-
    habite(X,vesoul).
test('habiteAllF(X,vesoul)',[all(X==[christophe,laurent])]) :-
    habite(X,vesoul).
test('habiteSets(X,vesoul)',[set(X==[christophe,laurent])]) :-
    habite(X,vesoul).
test('habite(X,besancon)',[fail]) :-
    habite(X,besancon).
:-end_tests(chp0).
```

Le programme reconsulté, on peut exécuter tous les tests :

```
?- run_tests.
% PL-Unit: chp0
```

On retrouve au niveau de PL-Unit le nom de notre groupe de tests chp0.

Pour les 6 tests les résultats : cinq sont passés et un a échoué (comme indiqué dans la synthèse à la fin de l'exécution après le done) :

1. Le premier est correct puisque la valeur attendue correspond à la première personne 'laurent', mais il y a un warning pour indiquer qu'il y a des points de choix qui n'ont pas été explorés.

```
Warning: exempleChp0.pl:11:
PL-Unit: Test habite(X,vesoul): Test succeeded with choicepoint
```

2. Le second est correct puisque la valeur attendue doit être différente de 'christophe', nous avons 'laurent'.

```
Warning: exempleChp0.pl:13:
PL-Unit: Test habiteA(X,vesoul): Test succeeded with choicepoint
```

3. Le troisième est correct puisque la liste prend en compte l'ordre d'apparition des personnes dans le prédicat habite. Il est à noter que le test est passé est un "." est affiché.
4. Le quatrième échoue car il teste par ordre alphabétique qui n'est pas l'ordre d'apparition des personnes dans le prédicat habite.

```
ERROR: exempleChp0.pl:17:
test habiteAllF(X,vesoul): wrong "all" answer:
ERROR:      Expected: [christophe,laurent]
ERROR:      Found: [laurent,christophe]
```


5. Pour permettre de tester sans tenir compte de l'apparition mais bien de l'ordre alphabétique il faut utiliser le `set` comme dans le cinquième test.
6. Le sixième est correct puisque la ville n'existe pas dans le prédicat `habite` et donc l'appel doit échouer. Il y a donc 2 fois "." pour les 5e et 6e tests est donc pour indiquer la fin des tests. Puis le résumé des différentes exécutions.

```
.. done
% 1 test failed
% 5 tests passed
```

Si vous voulez cibler un seul test, il suffit de le préciser dans l'appel avec `run_tests` en ajoutant derrière le nom du groupe le nom du test, par exemple :

```
?- run_tests(chp0:'habite(X,vesoul)').
...
```

0.3.9 Primitive de trace

Il est possible de suivre pas à pas l'exécution d'un programme PROLOG. Pour cela, il suffit de taper `"trace."` dans la console PROLOG avant de poser des questions. Le prompt se transforme et le mot `[trace]` apparaît avant le `?-`. Cette routine de trace se désactive par la saisie de `"notrace."` dans la console.

L'exécution d'une trace peut être abandonnée par la saisie de la lettre "a" (signifiant *abort*) en fin de ligne. Les touches les plus usuelles sont :

- "barre d'espace" : pour passer à l'instruction suivante
- "s" : (skip) pour ne pas rentrer dans l'appel du prédicat
- "f" : (finish) pour finir le traitement du prédicat
- "r" : (redo) pour rappeler le prédicat lorsqu'il échoue

SWI-Prolog propose une interface graphique pour le mode trace qui peut être activée en faisant `"gtrace."` au niveau du prompt.

0.3.10 La structure de liste

En PROLOG, les listes sont décrites à l'intérieur de crochets. Pour former des listes, on utilise en plus des crochets l'un ou l'autre symbole : `"` ou `|`. Mais attention, leur signification n'est pas du tout la même.

La virgule est un séparateur permettant d'énumérer les éléments d'une liste. Ainsi, `[a, b, c]` est la liste formée par `a`, `b` et `c`.

`[A|L]` définit une liste dont `A` est le premier élément et `L` le reste de la liste. Ce symbole permet ainsi soit de récupérer le premier élément d'une liste connue, soit d'ajouter un élément en début d'une liste connue.

Il est possible de construire plusieurs profondeurs de listes. Pour cela, il suffit d'imbriquer la structure de crochets. `[[a, b], [c, d]]` définit ainsi une liste de listes.

`[]` étant la liste vide, les notations suivantes désignent le même objet : `[a, b]`, `[a|[b]]`, `[a|[b|[]]]`.

Voici quelques exemples simples utilisant la structure de liste :

```
?- [a, b] = [X | Y].
X = a, Y = [b]

?- [a] = [X | Y].
X = a, Y = []

?- [a, [b]] = [X | Y].
X = a, Y = [[b]]

?- [a, b, c, d] = [X, Y | Z].
X = a, Y = b, Z = [c, d]

?- [[a, b, c], d, e] = [X | Y].
X = [a, b, c], Y = [d, e]
```

Premier exemple de programme utilisant les listes :

```
% Exemple : appartient(X,L) : X appartient à L
appartient(X, [X|Y]).
appartient(X, [_Z|Y]) :- appartient(X,Y).
```

Ces prédicats permettent de définir la relation d'appartenance d'un élément à une liste. Lors de la consultation de ces prédicats, PROLOG détecte deux anomalies non bloquantes dans notre code (identifiées par une étoile en début de ligne) :

```
?- consult('mon_fichier.pl').
% consulting mon_fichier.pl...
Warning: mon_fichier.pl:2:
Singleton variables: [Y]
Warning: mon_fichier.pl:3:
Singleton variables: [Z]
% mon_fichier.pl compiled, 0.00 sec, 1 clause
```

Ces deux warning ont la même source. Il s'agit de variables non partagées dans notre code : Y dans le premier prédicat et Z dans le second. Ces variables n'étant pas utilisées (quelque soit la valeur qu'elles prennent, l'exécution reste la même) sont appelées variables muettes.

Avant de poursuivre, il est préférable d'éliminer les anomalies provoquées par ces variables muettes. Il arrive en effet souvent qu'une faute de frappe ait introduit ce genre d'erreur : une variable partagée (XX et XX) donne naissance à deux variables muettes (XX et Xx par exemple).

Si les variables muettes détectées le sont effectivement, il suffit de placer le caractère "_" devant celles-ci pour avertir PROLOG de leur nature et ainsi faire disparaître les warning correspondant :

```
% Exemple : appartient(X,L) : X appartient à L
appartient(X, [X|_Y]).
appartient(X, [_Z|Y]) :- appartient(X,Y).
```

Une fois le programme reconsulté, aucune anomalie n'est plus détectée. Vous pouvez alors suivre pas à pas (utilisation de "trace.") et comprendre la construction du résultat de ce prédicat, avec par exemple le traitement de la question `appartient(X, [a,b,c])`.

```
?- trace.
true.

[trace]  ?- appartient(X,[a,b,c]).
  Call: (8) appartient(_19322, [a, b, c]) ? creep
  Exit: (8) appartient(a, [a, b, c]) ? creep
X = a
  Redo: (8) appartient(_19322, [a, b, c]) ?
...
```

Au travers de cette trace, notez que les chaînes de caractères débutant par "_" et terminant par un nombre sont les variables (pas encore unifiées) de votre programme. Il s'agit de la représentation interne des variables par PROLOG.

Il est à noter que lorsqu'une liste est trop longue le résultat renvoyé peut être tronqué. Pour forcer l'affichage complet, vous pouvez indiquer les instructions suivantes lors de l'invite de commande. La valeur dans `max_depth` indique le nombre de caractères. 0 indique que l'on souhaite tous les afficher.

```
?- set_prolog_flag(answer_write_options, [max_depth(0)]).
```

0.3.11 Etude d'une exécution d'un programme

Pour étudier plus précisément l'exécution d'un programme PROLOG, nous allons utiliser un nouvel exemple utilisant les listes : le prédicat `intersection(L1,L2,R)` dans lequel R est l'intersection des listes L1 et L2.

```
% Exemple : intersection(L1,L2,R) :
% R est l'intersection de L1 et L2

intersection([], _, []).                                % CLAUSE 1

intersection([X|L1], L2, R) :-                          % CLAUSE 2
    appartient(X, L2),
    intersection(L1, L2, R1),
    R=[X|R1].

intersection(_|L1, L2, R) :-                            % CLAUSE 3
    intersection(L1, L2, R1),
    R=R1.
```

Ce prédicat consiste à parcourir récursivement la liste L1 en dépilant à chaque appel récursif le premier élément de L1 et en continuant le traitement avec le reste de cette liste. Si le premier élément de L1 appartient à la liste L2, il sera ajouté au résultat R (CLAUSE 2), sinon on poursuit le traitement sans utiliser cet élément (CLAUSE 3). Lorsque la liste L1 a été totalement parcourue (L1 est *vide*), la CLAUSE 1 stoppe la récursivité et initialise la réponse R (sous sa quatrième instance R⁴) à *vide* : ce prédicat joue le rôle de prédicat d'arrêt de la récursivité. La réponse R est construite lors du retour d'appel de chaque prédicat. Ce retour est appelé **dépilement récursif**. Ainsi, dans la CLAUSE 2, le premier élément de L1 traité lors de l'exécution est ajouté à la réponse (R=[X|R1]), tandis la CLAUSE 3 retourne le même résultat R.

Le tableau suivant présente l'exécution d'un appel au prédicat `intersection` avec L1=[a,b,c], L2=[d,b,e,a] et R laissé en variable : `intersection([a,b,c],[d,b,e,a],R)`.

N°	Clauses appelées	Variables lors de la descente	Dépilement récursif
1)	CLAUSE 2 _{intersection(X L1,L2,R')} - appartient(X,L2) - intersection(L1,L2,R1)	X = a, L1 = [b, c] L2 = [d, b, e, a] le prédicat est vrai.	R' = R1
2)	CLAUSE 2 _{intersection(X' L1',L2',R'')} - appartient(X',L2') - intersection(L1',L2',R1')	X' = b, L1' = [c] L2' = [d, b, e, a] le prédicat est vrai.	R1 = [a, b] R'' = R1'
3)	CLAUSE 2 _{intersection(X'' L1'',L2'',R3)} - appartient(X'',L2'')	X'' = c, L1'' = [] L2'' = [d, b, e, a] le prédicat échoue.	R1' = [b] fail
4)	CLAUSE 3 _{intersection([_ L13],L23,R4)} - intersection(L13,L23,R4)	L13 = [] L23 = [d, b, e, a]	R4 = []
5)	CLAUSE 1 _{intersection([],_,[])}	le prédicat est vrai.	

Pour les autres exécution, comme nous le verrons plus loin, on peut se reporter à la figure 1 qui donne l'ensemble des chemins d'exécution pour lesquels les prédicats sont satisfaits. Par exemple le premier chemin (le plus à gauche) ne fait pas apparaître le troisième appel à la clause 2 (comme dans le tableau ci-dessus) car elle échoue.

Traditionnellement en PROLOG, on évite au maximum d'utiliser le signe d'unification "=" dans le code. En effet, en posant A=B, on souhaite unifier les deux variables A et B. Le résultat attendu est donc de faire en sorte que les variables A et B soient les mêmes. Ainsi, le code a la même signification si on remplace toutes les occurrences de A par B (ou inversement). En adoptant cette règle, le programme devient le suivant :

```
% Exemple : intersection(L1,L2,R) :
% R est l'intersection de L1 et L2

intersection([], _, []).                                % CLAUSE 1

intersection([X|L1], L2, [X|R1]) :-                    % CLAUSE 2
    appartient(X, L2),
    intersection(L1, L2, R1).

intersection([_|L1], L2, R) :-                          % CLAUSE 3
    intersection(L1, L2, R).
```

Lors de l'appel à la question `intersection([a,b,c],[d,b,e,a],R)` dans la console PROLOG, le premier résultat retourné est celui attendu, à savoir `R=[a,b]`. On peut néanmoins s'apercevoir qu'en demandant les autres réponses possibles, PROLOG nous renvoie des solutions erronées :

```
?- intersection([a,b,c],[d,b,e,a],R).
R = [a,b] ;
R = [a] ;
R = [b] ;
R = [] .
```

Étudions maintenant l'appel successif des clauses `intersection` lors de l'exécution pour comprendre la cause de ces erreurs. En fait, on peut représenter ces appels sous la forme d'un arbre dont les alternatives correspondent aux points de choix rencontrés par PROLOG lors de l'appel récursif au prédicat `intersection`.

En tapant ";", on demande à PROLOG d'explorer le dernier point de choix rencontré. Lors de l'appel N°4, PROLOG va tenter maintenant d'appliquer les clauses 2 puis 3 de notre programme. Ces tentatives échouent lors de l'unification de [] avec [X|L1] (la liste vide ne peut

pas être découpée). L'appel N°3 ne présente plus aucun point de choix (nous étions rentrés dans la dernière clause possible du prédicat intersection). PROLOG revient donc à l'appel N°2 pour lequel, après être passé dans la CLAUSE 2 lors de la première exécution, il peut maintenant passer dans la CLAUSE 3 et poursuivre une nouvelle exécution...

Ce mécanisme, appelé **retour en arrière** ou **backtrack**, correspond à un parcours de l'arbre d'exécution en profondeur gauche. Lors d'un échec de l'unification ou d'une demande de nouvelle réponse, PROLOG prendra le premier chemin encore explorable sur la droite (qui correspond au dernier point de choix de l'exécution).

La figure 1 présente le résultat avec l'appel `intersection([a,b,c],[d,b,e,a],R)` pour le prédicat `intersection/3`.

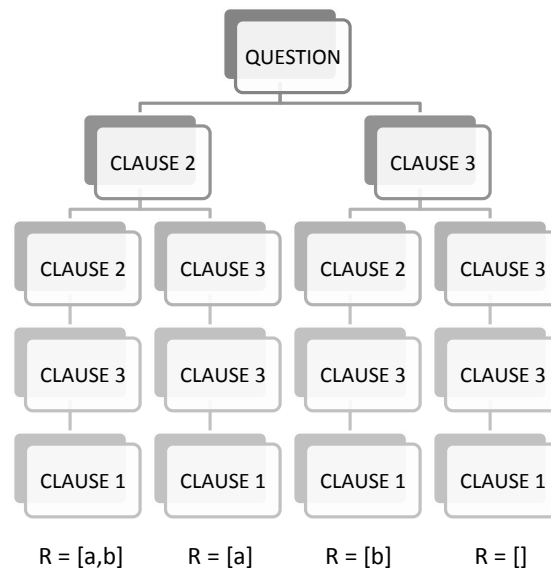


FIGURE 1 – Graphe d'exécution

Pour éviter que l'effet de backtrack ne nous propose des solutions erronées, deux opérateurs prédéfinis de PROLOG peuvent être utilisés : le prédicat `cut "!"` ou le prédicat de négation `"\+"`.

0.3.12 Le prédicat cut : "!"

Le prédicat prédéfini `cut "!"` le coupe-choix ou cut permet d'empêcher le retour en arrière. Lorsqu'il est traversé lors de l'exécution, il a pour effet d'effacer les points de choix potentiellement encore explorables sur les prédicats situés avant lui dans l'appel courant (les prédicats placés à sa gauche). En revanche, il laisse la possibilité de retour arrière sur les appels des prédicats situés à sa droite.

Dans l'exemple de l'intersection, pour chaque appel du prédicat `intersection`, nous voulons en fait empêcher PROLOG de rentrer dans la CLAUSE 3 si, dans la CLAUSE 2, l'élément `x` appartient à `L2`. Pour ce faire, il suffit de placer le cut juste après l'appel à appartient dans la CLAUSE 2 :

```

% Exemple : intersection(L1,L2,R) :
% R est l'intersection de L1 et L2

intersection([], _, []).                % CLAUSE 1

intersection([X|L1], L2, [X|R1]) :-    % CLAUSE 2
    appartient(X, L2),
    !,
    intersection(L1, L2, R1).

intersection([_|L1], L2, R) :-          % CLAUSE 3
    intersection(L1, L2, R).

```

Le *cut* placé à cet endroit aura pour effet de couper les points de choix existant sur l'appel courant de *intersection*, mais également de *appartient*. Il ne coupe pas cependant les points de choix de l'appel récursif de *intersection(L1, L2, R1)* puisque cet appel est situé à sa droite.

En résumé, le principe est de supprimer les points de choix des prédicats avant le *cut* et dont les appels sont effectués au même niveau que le *cut*. Soient deux cas :

1. les autres occurrences du prédicat de tête se trouvant après dans l'ordre du source,
2. les point de choix laissés par les appels des prédicats avant le *cut* dans le corps du prédicat.

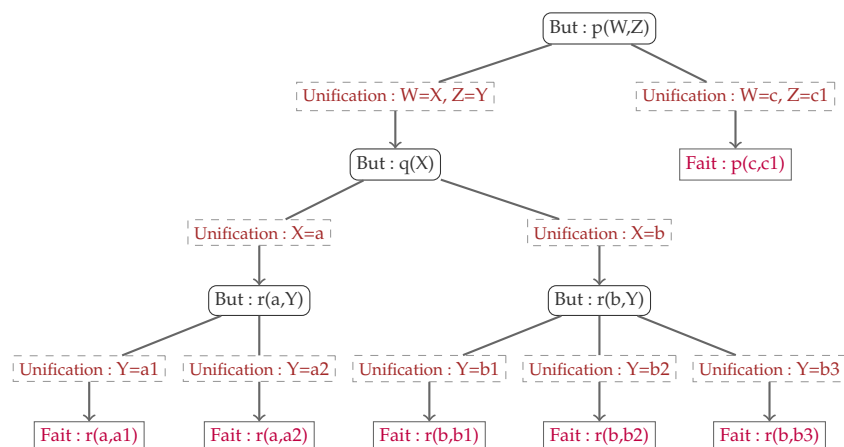
Nous allons voir quatre cas de positionnement du *cut* pour voir son impact sur le traitement fait par prolog et les solutions produites.

Ce premier cas, n'utilise pas le *cut* et donc l'ensemble des cas est exploré. Nous avons à gauche le programme et à droite le graphe d'exploration effectué :

p(X,Y) :- q(X), r(X,Y).
p(c,c1).

q(a).
q(b).

r(a,a1).
r(a,a2).
r(b,b1).
r(b,b2).
r(b,b3).

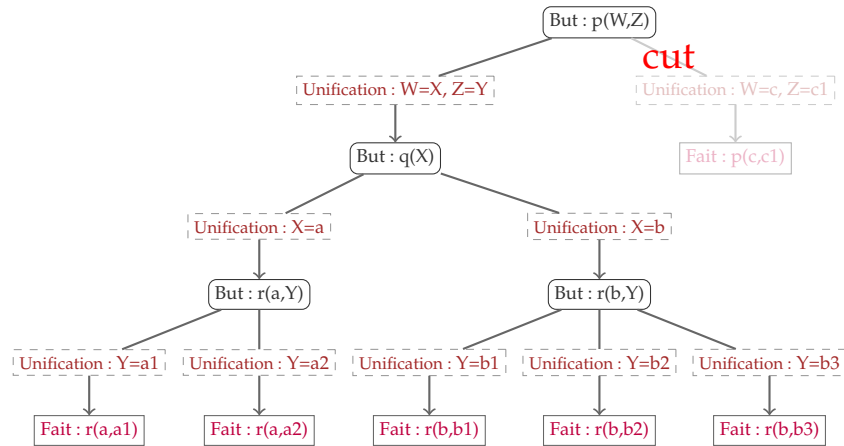


Dans le second cas, nous insérons le *cut* au début du corps du prédicat. Ainsi, seules les autres occurrences du prédicat, placés après dans le code, seront ignorés. Nous avons à gauche le programme (avec les prédicats n'ont explorés par l'usage du *cut* sont rayés) et à droite le graphe d'exploration effectué :

$p(X,Y) :- !, q(X), r(X,Y).$
 ~~$p(c,cl).$~~

$q(a).$
 ~~$q(b).$~~

$r(a,a1).$
 $r(a,a2).$
 $r(b,b1).$
 $r(b,b2).$
 $r(b,b3).$

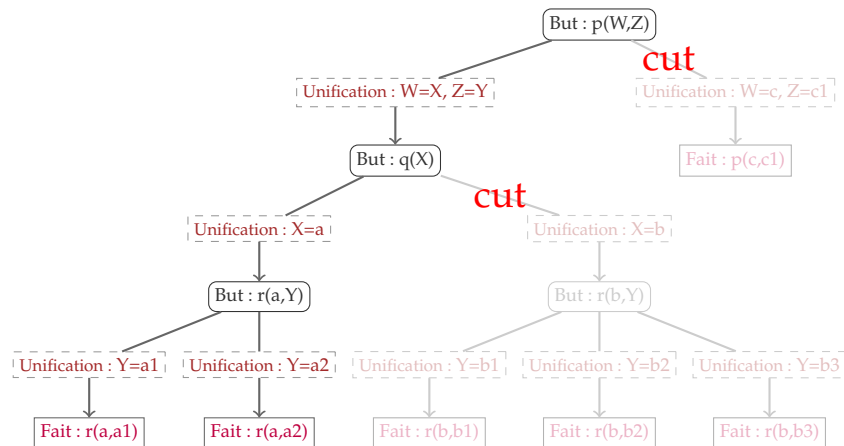


Dans le troisième cas, nous insérons le `cut` dans le corps du prédicat après un premier appel de prédicat. Ainsi, les autres occurrences du prédicat, placés après dans le code, seront ignorés mais aussi les points de choix associés à l'appel du prédicat avant le `cut`. Puisque le `cut` ne permettra pas l'unification de la variable X à la valeur b , les prédicats $r(b, _)$ ne donneront pas lieu à des solutions. Nous avons à gauche le programme (avec les prédicats n'ont explorés par l'usage du `cut` sont rayés) et à droite le graphe d'exploration effectué :

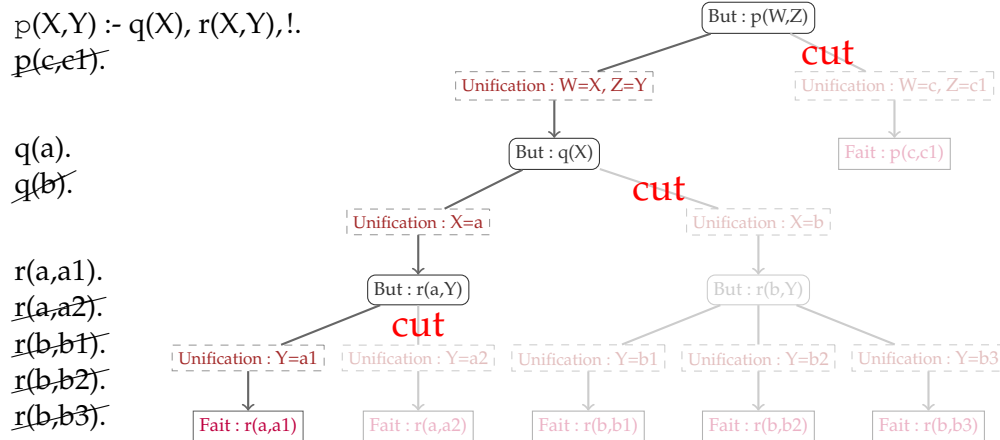
$p(X,Y) :- q(X), !, r(X,Y).$
 ~~$p(c,cl).$~~

$q(a).$
 ~~$q(b).$~~

$r(a,a1).$
 $r(a,a2).$
 $r(b,b1).$
 $r(b,b2).$
 $r(b,b3).$



Dans le quatrième cas, nous insérons le `cut` à la fin du corps du prédicat. Ainsi, les autres occurrences du prédicat, placés après dans le code, seront ignorés mais aussi tous les points de choix associés aux appels de prédicats avant le `cut`. Nous avons à gauche le programme (avec les prédicats n'ont explorés par l'usage du `cut` rayés) et à droite le graphe d'exploration effectué :



0.3.13 Le prédicat de la négation : "\+"

Le prédicat prédéfini "\+" la **négation** permet de considérer la négation d'un prédicat. Ainsi, soit P est un prédicat, si l'appel à P réussit, l'appel à \+P échoue, si l'appel à P échoue, l'appel à \+P réussit.

Sa définition est la suivante (le prédicat prédéfini "fail" force l'échec) :

```
not (P) :-
    P,
    !,
    fail.
not (_).
```

Dans l'exemple de l'intersection, pour chaque appel au prédicat intersection, nous voulons en fait empêcher PROLOG d'exécuter la CLAUSE 3 si le premier élément X de L1 appartient à L2. Pour ce faire, il suffit de considérer la négation de l'appartenance dans la CLAUSE 3 :

```
% Exemple : intersection(L1,L2,R) :
% R est l'intersection de L1 et L2

intersection([], _, []).                                % CLAUSE 1

intersection([X|L1], L2, [X|R1]) :-                    % CLAUSE 2
    appartient(X, L2),
    intersection(L1, L2, R1).

intersection([X|L1], L2, R) :-                          % CLAUSE 3
    \+ appartient(X, L2),
    intersection(L1, L2, R).
```

ATTENTION : ce prédicat de négation est à utiliser avec précaution. Dans un cadre général, ne l'utilisez que lorsque les variables du prédicat nié sont instanciées. A titre d'exemple, l'appel du prédicat `intersection([a,b,c],B,[a,b])` boucle indéfiniment avec la version utilisant la négation (visible en traçant l'exécution).

0.3.14 L'opérateur numérique "is"

Les nombres sont des constantes. Ils peuvent faire l'objet d'opérations notées de façon infixée avec `+`, `-`, `*`, `/`, `//` (division entière), `mod` (modulo). Ces opérations sont utilisées avec l'opérateur numérique `is`. Cet opérateur permet de poser l'égalité entre la variable à affecter et le résultat du calcul numérique.

ATTENTION : n'utilisez pas le signe `=` qui est réservé à l'unification (si vous l'utilisez, PROLOG tentera d'unifier les termes (équivalent aux chaînes de caractères) placées de part et d'autre du signe `=`).

```
?- X=1+2.  
X = 1+2.  
  
?- X is 1+2.  
X = 3.
```

Les conditions d'utilisation de l'opérateur `is` sont assez strictes. Ainsi, on doit écrire à gauche de `is` un nombre ou une variable. À sa droite, on doit trouver une expression numérique. Cette expression peut contenir des variables à condition que ces dernières aient fait l'objet d'une unification avec un nombre avant le traitement de la clause contenant `is` (une sorte d'initialisation des variables du membre droit). Ainsi, la question `X is Y+2` n'est pas acceptée mais la question `Y is 1, X is Y+3` est valable. Lorsqu'une variable doit être affectée à un nombre (terme unique), l'utilisation du signe d'unification `=` est tolérée. Ainsi, `X is 1` est équivalent à `X=1`.

```
?- X is Y+2.  
ERROR: Arguments are not sufficiently instantiated  
ERROR: In:  
ERROR:      [8] _19896 is _19902+2  
ERROR:      [7] <user>  
  
?- Y=1, X is Y+2.  
Y = 1,  
X = 3.
```

Le traitement de `is` consiste en une unification. PROLOG va effectivement tenter d'unifier le membre de gauche avec le résultat du membre droit. Si l'unification échoue, la valeur `no` est renvoyée. Si elle aboutit, l'unification est effectuée (le membre gauche prend la valeur du résultat du membre droit) et la valeur est renvoyée.

0.3.15 Comparaison de valeurs numériques

Il est possible de comparer des nombres au moyen des opérateurs `<`, `=<`, `>`, `=>`, `<=`, `>=`. La notation est infixée et, les expressions à droite et à gauche du prédicat de comparaison peuvent également contenir des variables si celles-ci sont instanciées par unification à un nombre avant le traitement de la clause de comparaison (mêmes contraintes que pour le membre droit de `is`). Un moyen mnémotechnique pour savoir où mettre le `=` pour les opérateurs "supérieur ou égal" et "inférieur ou égal", cela ne doit pas faire une flèche. On utilisera donc `>=` et `<=`.

```

?- X>=Y.
ERROR: Arguments are not sufficiently instantiated
ERROR: In:
ERROR:      [8] _21096>=_21098
ERROR:      [7] <user>

?- X=2, X>=Y.
ERROR: Arguments are not sufficiently instantiated
ERROR: In:
ERROR:      [9] 2>=_22486
ERROR:      [8] '<meta-call>'(user:(...,...)) <foreign>
ERROR:      [7] <user>

?- Y=1, X>=Y.
ERROR: Arguments are not sufficiently instantiated
ERROR: In:
ERROR:      [9] _23558>=1
ERROR:      [7] <user>

?- X=2, Y=1, X>=Y.
X = 2,
Y = 1.

```

0.4 Programmation avec contraintes

L'objectif de cette leçon consiste à vous présenter trois bibliothèques de SWI-Prolog permettant d'utiliser des techniques de **Programmation Logique avec Contraintes** (PLC). La première section de la leçon présente les bibliothèques CLP(R) et CLP(Q) tandis que la seconde section introduit les concepts de la bibliothèque CLP(FD). Noter que CLP est l'acronyme anglophone pour désigner la Programmation Logique avec Contraintes (Constraint Logic Programming).

Les principes de programmation présentés dans cette leçon sont à mettre en œuvre dans le cadre du chapitre 4 et de ses exercices.

0.4.1 Objectifs de CLP(R) et CLP(Q)

CLP(R) et CLP(Q) représentent deux modes de Programmation Logique avec Contraintes.

CLP(R) permet de résoudre des systèmes d'équations (ou d'inéquations) dont les variables-résultats peuvent être des valeurs numériques **réelles**. CLP(Q) permet de résoudre des systèmes d'équations (ou d'inéquations) dont les variables-résultats peuvent être sur des valeurs numériques **rationnelles**.

Dans les deux cas, les systèmes d'équations sont construits à partir des opérateurs mathématiques classiques (+,-,*,/,=,>,...) et de nombres entiers.

Ces deux bibliothèques utilisent des algorithmes de résolution équivalents (ces algorithmes sont implantés dans des solveurs utilisant des routines communes) et, de ce fait, elles ne peuvent pas être utilisées en même temps : seule une des deux bibliothèques peut être utilisée à la fois.

Chargement de la bibliothèque CLP(R)

La bibliothèque CLP(R) permettant l'utilisation de contraintes, dont les résultats sont des nombres réels, se charge en tapant dans la console de SWI-Prolog l'appel suivant :

```

?- use_module(library(clpr)).

```

Pour éviter d'avoir à charger la bibliothèque depuis la console à chaque utilisation d'un programme utilisant cette bibliothèque, il est possible d'inclure un appel automatique à la bibliothèque dans vos programmes à l'aide de la commande suivante :

```
% Appel automatique à CLP(R)
:- use_module(library(clpr)).
```

Prenez garde à ne jamais appeler successivement la consultation des deux bibliothèques de gestion de contraintes dans le même fichier : SWI-Prolog vous indiquera une erreur car il refusera de remplacer les prédicats déjà présent dans la première bibliothèque. Pour passer de l'utilisation de CLP(R) à CLP(Q) (ou inversement), il vous faudra quitter SWI-Prolog.

Poser de contraintes CLP(R) ou CLP(Q)

L'utilisation d'une écriture spécifique des contraintes numériques permet à PROLOG d'invoquer le solveur de la forme de PLC déclarée. Ainsi, pour spécifier à PROLOG que des contraintes doivent être soumises au solveur CLP(R) ou CLP(Q), il faut que ces contraintes soient écrites entre accolades. D'autre part, le signe d'égalité numérique de la Programmation Logique ("is") est ici remplacé par le signe traditionnel mathématique "=".

Prenons comme exemple un système de deux équations à deux inconnues x et y tel que : $2X = 3Y + 5$ et $X = Y - 2$. Ce problème se modélise en SWI-Prolog de la manière suivante :

```
% exemple de système d'équations
equation(X,Y):-
    {2*X = 3*Y + 5, X = Y -2}.
```

L'écriture suivante est équivalente :

```
% exemple de système d'équations
equation1(X,Y):-
    {2*X = 3*Y + 5},
    {X = Y -2}.
```

Après avoir consulté le fichier dans lequel le prédicat `equation(X,Y)` est déclaré, les réponses à la question avec CLP(R) et CLP(Q) seront les suivantes :

```
?- use_module(library(clpr)).
?- equation(X,Y).
X = -10.999999999999998,
Y = -8.999999999999998.
```

```
?- use_module(library(clpq)).
?- equation(X,Y).
X = -11,
Y = -9.
```

A travers ce petit exemple, on pourra noter que l'utilisation de CLP(R), au contraire de CLP(Q), peut entraîner un défaut de précision lorsque les résultats attendus sont entiers.

Exemple de solutions multiples

On a vu, à travers de l'exemple précédent, que les solveurs CLP(R) et CLP(Q) sont capables de raisonner sur un système d'équations, et de donner la solution d'un tel système lorsqu'elle existe. Cependant, lorsque les informations données ne permettent pas d'identifier une réponse unique, ces solveurs ne vous proposent pas une réponse évaluée possible, mais vous présentent le système d'équations soumis (ce système a pu éventuellement être réécrit par PROLOG).

Rajoutons par exemple une inconnue z au système de l'exercice précédent telle que le système à résoudre est maintenant $2X = 3Y - Z + 5$ et $X = Y - 2$:

```
% exemple de système d'équations
equation(X,Y,Z):-
    {2*X = 3*Y - Z + 5, X = Y -2}.
```

Après avoir consulté le fichier dans lequel le prédicat `equation(X,Y,Z)` est déclaré, les réponses à la question avec CLP(R) et CLP(Q) seront les suivantes :

```
?- use_module(library(clpr)).
?- equation(X,Y,Z).
{Y=2.0+X,Z=11.0+X}.
```

```
?- use_module(library(clpq)).
?- equation(X,Y,Z).
{Y=2+X, Z=11+X}.
```

Dans cet exemple, le système d'équations ne donne pas lieu à une réponse évaluée car ce système est incomplet. Néanmoins, PROLOG renvoie un nouveau système (simplifié) dans lequel les valeurs des inconnues y et z sont paramétrées par x .

Dans le cadre de l'utilisation de CLP(R) et CLP(Q), il faut ainsi être particulièrement vigilant au système de contraintes proposé si vous désirez avoir une réponse évaluée : en effet, si le nombre de réponses est infini, seule une réécriture du système initial est proposée.

0.4.2 La bibliothèque CLP(FD)

CLP(FD) est une bibliothèque permettant d'utiliser la Programmation Logique avec Contraintes pour résoudre des équations (ou inéquations) dont les variables-résultats sont des valeurs numériques **entières** pour lesquelles on a préalablement défini un domaine discret.

CLP(FD) définit la programmation Logique avec Contraintes sur Domaine Fini (Constraint Logic Programming on Finite Domain en anglais).

Cette bibliothèque est de loin plus efficace en terme de résultat que les deux bibliothèques introduites dans la section précédente (CLP(R) et CLP(Q)). Elle utilise des algorithmes spécifiques qui tirent leur efficacité du fait que toutes les inconnues des systèmes à résoudre possèdent un domaine fini. Pour cette raison, lorsque les domaines des variables d'un système, devant être résolu en Programmation Logique avec Contraintes, sont connus et bornés, on utilise de manière préférentielle CLP(FD) plutôt que CLP(R) ou CLP(Q).

Utilisant des algorithmes spécifiques, il n'y a aucun conflit de chargement lors de l'utilisation de CLP(FD) avec CLP(R) - ou CLP(Q) -.

Chargement de la bibliothèque CLP(FD)

La bibliothèque CLP(FD) se charge en tapant dans la console de SWI-Prolog l'appel suivant :

```
?- use_module(library(clpfd)).
```

Pour éviter d'avoir à charger la bibliothèque depuis la console à chaque utilisation d'un programme utilisant cette bibliothèque, il est possible d'inclure un appel automatique à la bibliothèque dans vos programme à l'aide de la commande suivante :

```
% Appel automatique à CLP(FD)
:- use_module(library(clpfd)).
```

Mode d'utilisation de CLP(FD)

L'utilisation de cette forme de PLC requiert, dans l'ordre, les déclarations suivantes :

1. La **déclaration des domaines** de chacune des variables utilisées,
2. La **pose des contraintes** sur ces variables (en utilisant une écriture spécifique),
3. Eventuellement une **génération de valeurs** vérifiant les domaines et les contraintes.

Ces trois phases sont à mettre en œuvre dans l'ordre précisé pour chacune des variables du problème à résoudre. La pratique habituelle consiste à n'utiliser qu'une seule fois ce processus dans un programme donné : déclaration des domaines de toutes les variables, pose de toutes les contraintes puis éventuellement une génération de valeurs pour toutes les variables.

Il peut cependant être intéressant, sur certains types de problèmes, d'effectuer plusieurs fois ces trois phases sur des sous-ensembles de variables. Dans ce cas, pour un même sous-ensemble de variables, les trois phases peuvent ne pas être consécutives, mais leur ordre doit obligatoirement être respecté.

Déclaration des domaines

Par défaut, toute variable a un domaine dont les bornes minimum et maximum sont *inf* (pour moins l'infini) et *sup* (pour plus l'infini), mais les algorithmes de résolution de CLP(FD) sont inefficaces avec un tel domaine.

La déclaration des domaines de chaque variable est de ce fait la première étape pour utiliser CLP(FD). Cette déclaration peut prendre plusieurs formes.

La façon la plus commune est d'utiliser le prédicat `domain/3`. Le premier argument est la liste des variables dont le domaine est déclaré, tandis que les deuxième et troisième arguments constituent respectivement la borne minimum et la borne maximum (incluses) du domaine.

ATTENTION : il faut absolument que les bornes minimum et maximum soient évaluées lors de la pose de ce prédicat.

Ainsi, soient *X* et *Y* deux variables du système à résoudre dont le domaine est $0..8$. La définition de leur domaine en PROLOG, pour le solveur CLP(FD), à l'aide du fonction `in/2` pour déclarer le domaine d'une seule variable ou `ins/2` pour déclarer le même domaine à une liste de variables. Au contraire de la majorité des prédicats PROLOG, ce prédicat n'est pas préfixé. Soit l'exemple suivant :

```
% Declaration par variable
X in 0..8, Y in 0..8.

% Declaration liste de variables
[X,Y] ins 0..8.
```

Dans ce cas où le domaine est discontinu, on peut définir dans le second argument des prédicats `in` et `ins` un ensemble en extension défini entre accolades (s'il n'y a qu'une seule valeur on peut se passer de ces dernières). Son utilisation peut être encore être plus commode avec le prédicat `union /2` (non infixé), et avec le prédicat `intervalle . .` (non infixé) vu précédemment.

ATTENTION : il faut absolument que tous les éléments du domaine soient valués lors de la pose de ce(s) prédicat(s).

Ainsi, soit Z une variable du système à résoudre dont le domaine est $0, 1, 2, 3, 6, 8$. La définition de son domaine en PROLOG, pour le solveur CLP(FD), peut prendre une des écritures suivantes :

```
Z in {0,1,2,3,6,8}.
Z in {0,1,2,3}\/{6,8}.
Z in {0,1,2,3}\/{6}\/{8}.
Z in (0..3) \ / {6,8}.
Z in 0..3 \ / 6 \ / 8.
```

Lorsque le domaine d'une variable est continu, on peut indifféremment choisir de déclarer son domaine avec l'une ou l'autre forme de déclaration. On pourra utiliser `in/2` ou pour se simplifier la tâche lorsqu'on a une liste de variables avec le même domaine utiliser `ins/2`. Souvent nous utiliserons le prédicat `domain/3`. Ce prédicat est à ajouter, il permet de simplifier la définition de domaine d'un ensemble de variables, il se définit de la façon suivante : `domain(L,Bmin,BMax):- L ins Bmin..Bmax..`

Cependant, il est impossible de déclarer, avec ces deux formes, un domaine dans lequel subsiste des variables PROLOG non valuées au moment de la pose du domaine. Pour ce faire, on utilisera des contraintes faisant fonction, dans ce cas extrême, de délimiteurs de domaine (exemple pour une variable W : $X < W$ et $W < Y$). Ce type de déclaration suit la méthode utilisée plus généralement lors de la pose des contraintes (seconde phase) présentée par la suite.

Pose des contraintes

Comme pour CLP(R) ou CLP(Q), les contraintes CLP(FD) sont écrites de manière spécifique permettant ainsi à PROLOG de repérer que ces contraintes sont à traiter avec le solveur CLP(FD). Ainsi, les contraintes soumises à CLP(FD) se distinguent par la présence d'un dièse devant chaque opérateur d'égalité, de différence, de supériorité, d'infériorité...

La réponse de PROLOG à une question consiste alors à répondre "no" si le système de contraintes est inconsistant, sinon, à répondre en donnant un domaine (intervalle de valeurs) pour chacune des variables du système initial, ou par une valeur dans le cas d'un intervalle réduit à un élément.

Par exemple, soient les variables X, Y et Z de domaine $0..5$, et le système d'équations $X = Y + 5$, $X \neq 2Y$ et $X > Y + Z$. Ce système se réécrit en PROLOG de la manière suivante :

```
equation(X,Y,Z):-  
    [X,Y,Z] ins 0..5, % pose des domaines  
    X #= Y+5,         % pose  
    X #\= 2*Y,        % des  
    X #> Y+Z.         % contraintes
```

Après consultation, à la question dans la console SWI-Prolog, la réponse est :

```
?- equation(X,Y,Z).  
X = 5,  
Y = 0,  
Z in 0..4.
```

Ainsi, les variables X et Y de notre système valent respectivement 5 et 0, tandis que la variable Z peut prendre une valeur entre 0 et 4.

ATTENTION : lorsque PROLOG renvoie un intervalle de valeurs comme domaine d'une variable, cela ne signifie pas que cette variable peut forcément prendre toutes les valeurs de ce domaine ! Pour obtenir les différentes réponses évaluées correctes, il est alors nécessaire d'effectuer une phase de génération de valeurs.

D'autre part, les algorithmes du solveur CLP(FD) ne sont pas capables de détecter toutes les inconsistances du système posé. Pour preuve, en posant l'égalité puis la différence de deux variables, CLP(FD) ne détecte pas que le système posé est inconsistant :

```
?- [A,B] in 0..5, A#=B, A#\=B.  
A in 0..5,  
B in 0..5 .
```

Seule une génération de valeurs pour les variable A et B permet à CLP(FD) de s'apercevoir qu'aucune réponse n'est possible pour ce type de système. En conclusion, il est fortement conseillé d'effectuer une génération de valeurs pour être certain de la satisfiabilité d'un système et obtenir un résultat juste.

Génération de valeurs

La génération de valeurs (ou instanciation des variables) est appelée au moyen du prédicat `labeling/2` dont les deux arguments représentent respectivement la liste d'options de recherche et la liste des variables à instancier.

Ce prédicat retourne vrai si la recherche permet de trouver une instanciation possible pour toutes les variables de la liste donnée en second argument satisfaisant les contraintes posées auparavant.

Les arguments de la liste d'options contrôlent l'ordre et la manière dont les variables sont affectées. En renseignant cet argument par la liste vide, on précise à PROLOG que les options par défaut doivent être utilisées.

Les options de génération influent en fait sur l'efficacité de la recherche et le sens du résultat. Ces options sont divisées en quatre groupes, et une seule option par groupe peut être sélectionnée à la fois. Les différentes options sont présentées dans le manuel SWI-Prolog, et libre à vous d'utiliser celles de votre choix, mais celles définies par défaut seront satisfaisantes pour l'ensemble des exercices proposées.

Néanmoins, on pourra noter les options les plus intéressantes :

- *up* : le domaine des variables est exploré dans l'ordre croissant (cette stratégie est celle utilisée par défaut),
- *down* : le domaine est exploré dans l'ordre décroissant (incompatible avec *up*),
- *min(X)* : cette option force l'utilisation d'un algorithme permettant de trouver des solutions qui minimisent la valeur de la variable *X*,
- *max(X)* : cette option force l'utilisation d'un algorithme permettant de trouver des solutions qui maximisent la valeur de la variable *X* (incompatible avec *min(X)*).

Ainsi, pour énumérer les solutions valuées du système de variables *X, Y, Z* présenté dans le courant de cette page, on transformera le programme PROLOG de la manière suivante :

```
equation(X,Y,Z):-
    [X,Y,Z] in 0..5,      % pose des domaines
    X #= Y+5,             % pose
    X #\= 2*Y,            % des
    X #> Y+Z,             % contraintes
    labeling([], [X,Y,Z]).% génération de valeurs avec
                        % options par défaut
```

Après consultation, à la question dans la console SWI-Prolog, les réponses sont désormais les suivantes :

```
?- equation(X,Y,Z).
X = 5,
Y = Z, Z = 0 ;

X = 5,
Y = 0,
Z = 1 ;

X = 5,
Y = 0,
Z = 2 ;

X = 5,
Y = 0,
Z = 3 ;

X = 5,
Y = 0,
Z = 4.
```

Avec une génération de valeurs, le petit exemple, dans lequel deux variables sont à la fois égales et différentes, donne lieu à une réponse négative de la part de PROLOG :

```
?- [A,B] in 0..5, A#=B, A#\=B, labeling([], [A,B]).
false.
```


0.5 Exercices chapitre 0

- Les exercices 1-9 consistent à écrire de petits prédicats PROLOG. Chaque prédicat demandé est accompagné d'un exemple. Il vous appartient toutefois de vérifier que les prédicats que vous aurez écrits rendent les bonnes solutions dans tous les cas d'utilisation (emploi avec des arguments constants et/ou variables).
- Les sujets 10-12 proposent des exercices plus complets permettant de s'assurer que les notions abordées en PROLOG sont définitivement assimilées. Ces exercices sont destinés à être réalisés au cours de la semaine 2.
- Le dernier exercice va vous permettre de connaître les primitives mises à disposition par SWI-Prolog que vous pourrez utiliser par la suite.

Exercice 1 - Prédicat `Premier/2`

Définir le prédicat `premier(X, L)` ayant la signification : 'X est le premier terme de la liste L'. La réponse à la question `premier(X, [2, 4, 6])` sera `X=2` (une clause suffit).

[Correction.](#)

Exercice 2 - Prédicat `Dernier/2`

Définir `dernier(X, L)` ayant la signification : 'X est le dernier terme de la liste L'. La réponse à la question `dernier(X, [2, 4, 6])` sera `X=6` (deux clauses sont nécessaires).

[Correction.](#)

Exercice 3 - Prédicat `ajout_debut/3`

Définir `ajout_debut(X, L, R)` ayant la signification : 'R est obtenue en ajoutant X au début de la liste L'. La réponse à la question `ajout_debut(9, [2, 4, 6], R)` sera `R=[9, 2, 4, 6]` (une clause suffit).

[Correction.](#)

Exercice 4 - Prédicat `ajout_fin/3`

Définir `ajout_fin(X, L, R)` ayant la signification : 'R est obtenue en ajoutant X à la fin de la liste L'. La réponse à la question `ajout_fin(9, [2, 4, 6], R)` sera `R=[2, 4, 6, 9]` (deux clauses sont nécessaires).

[Correction.](#)

Exercice 5 - Prédicat `inclus/2`

Définir `inclus(L, M)` ayant la signification : 'Tous les éléments de la liste L se trouvent dans la liste M', L est une sous-liste de M sans tenir compte de l'ordre des éléments mais seulement de leurs occurrences. La réponse à la question `inclus([3, 1], [1, 2, 3, 4])` doit être `true`. La réponse à la question `inclus([1, 1], [1, 2, 3, 4])` doit être `false`.

[Correction.](#)

Exercice 6 - Prédicat `renverse_liste/2, 3`

Définir `renverse_liste(L, Acc, M)`, dont le poids peut être de 2 ou 3, qui permet de renverser une liste `L` donnée et retourne le résultat dans `M` avec ou sans accumulateur `Acc`. Ainsi, la liste renversée de `[a, b, c]` est la liste `[c, b, a]`.

[Correction.](#)

Exercice 7 - Prédicat `sous_liste_droite/2`

Définir `sous_liste_droite(L, K)` ayant la signification : '`K` est une sous-liste droite de la liste `L`'. Les sous-listes droites de `[a, b, c]` sont `[]`, `[c]`, `[b, c]` et `[a, b, c]`.

[Correction.](#)

Exercice 8 - Prédicat `sous_liste_gauche/2`

Définir `sous_liste_gauche(L, K)` ayant la signification : '`K` est une sous-liste gauche de la liste `L`'. Les sous-listes gauches de `[a, b, c]` sont `[]`, `[a]`, `[a, b]` et `[a, b, c]`.

[Correction.](#)

Exercice 9 - Prédicat `longueur/2`

Définir `longueur(L, A)` ayant la signification : '`L` est de longueur `A`'. La réponse à la question `longueur([2, 4, 6], A)` sera `A=3`.

Notez qu'un prédicat prédéfini effectue ce traitement : il s'agit de `length(L, A)`.

[Correction.](#)

Exercice 10 - Prédicat `somme/2`

Définir `somme(L, S)` ayant la signification : '`S` est la somme des nombres appartenant à `L`'. La réponse à la question `somme([2, 4, 6], S)` sera `S=12`. [Correction.](#)

Exercice 11 - L'agence matrimoniale

Nous sommes dans une agence matrimoniale qui possède un fichier de candidats au mariage organisé selon les trois catégories suivantes :

1. `homme(N, T, C, A)`.
`femme(N, T, C, A)`.

Où `N` est le nom d'une personne (d'un homme ou d'une femme), `T` sa taille (grande, moyenne, petite), `C` la couleur de ses cheveux (blonds, bruns, roux, châains), `A` son âge (jeune, mûr, vieux).

2. `gout(N, M, L, S)`.

Indique que la personne `N` aime le genre de musique `M` (classique, pop, jazz), le genre de littérature `L` (aventure, science-fiction, policier) et pratique le sport `S` (tennis, natation, jogging).

3. `recherche(N, T, C, A)`.

Exprime que la personne `N` recherche un (une) partenaire de taille `T`, ayant des cheveux de couleur `C`, et dont l'âge est `A`.

On considère que deux personnes X et Y de sexe différent sont assorties si X convient à Y et si Y convient à X. On dira que X convient à Y si d'une part, X convient physiquement à Y (taille, age, couleur de cheveux) et si d'autre part, les goûts de X et Y en matière de musique, littérature et sport sont identiques.

Par soucis de simplification, nous définirons chaque personne uniquement par son prénom. Les données sont les suivantes :

Nom	T	C	A	Musique	Littérature	Sport	T	C	A
Luc	g	bl	m	pop	policier	tennis	m	r	j, m
Gaston	m	bl	m	classique	aventure	natation	p	bl	j, m
Kevin	g	br	j	jazz	sf	jogging	m	r	j
Remi	m	bl	v	classique	aventure	jogging	p	bl	m
Martine	p	bl	m	classique	aventure	natation	m	bl	m, v
Odile	m	bl	j	classique	sf	natation	g	br	j
Lea	p	br	m	classique	policier	tennis	m	br	m, v
Elsa	g	br	j	jazz	sf	jogging	g	br, r	m
Sylvie	m	bl	j	classique	sf	natation	g	indif	m
Julie	p	bl	m	pop	policier	tennis	p, m	r	j
Mathilde	g	bl	m	classique	policier	tennis	g	br	m, v
Anne	g	bl	v	classique	aventure	natation	g	br, bl	v

Questions :

1. Donner un ensemble d'assertions présentant le fichier des candidats.
2. Ecrire les règles définissant `convient_physiquement(X, Y)`, ensuite les règles définissant `ont_memes_gouts(X, Y)`. (Application avec `X = martine`)
3. En déduire le programme qui détermine les couples assortis.
4. Etudier les simplifications que l'on pourrait avoir si `convient_physiquement(X, Y)` était symétrique.

[Correction.](#)

Exercice 12 - Coloriage d'une carte plane

En tant que graphiste, il nous faut colorier une carte de n régions de telle sorte que deux régions adjacentes ne soient pas de la même couleur, afin de bien mettre en évidence les différentes frontières. Un théorème montre que toute carte plane est coloriable avec au plus 4 couleurs de base.

Considérons à titre d'exemple la carte des régions de la France métropole suivante :



FIGURE 2 – Le cycle de création de test

Après avoir modélisé la carte en Prolog par un système d'adjacence, il suffit de tenter un coloriage avec une couleur, puis deux,... Pour ce faire, on colorie une à une les régions en prenant garde à chaque fois que la contrainte évoquée plus haut est respectée. Lorsque toutes les régions ont pu être coloriées, la couleur de chacune d'elles est affichée à l'écran sous forme textuelle.

[Correction.](#)

Exercice 13 - Le loup, la chèvre et le chou

Un loup, une chèvre et un chou doivent traverser une rivière et s'adressent à un batelier qui ne peut transporter que l'un d'entre eux à la fois. Hors de la présence du batelier, la chèvre croquera le chou et le loup dégustera la chèvre.

On se propose d'écrire un programme PROLOG qui calcule et affiche à l'écran la suite des traversées permettant le passage sur l'autre rive.

Technique conseillée :

Les rives sont numérotées 1 et 2. Un état est représenté par un triplet $[R1, R2, B]$ où $R1$ est la liste des individus situés sur la rive 1, $R2$ la liste des individus situés sur la rive 2, et le bateau se trouve sur la rive B . L'état de départ est ainsi $[[chèvre, chou, loup], [], 1]$.

Lorsque l'état $[], [chèvre, chou, loup], 2]$ est atteint, la mission est remplie. La liste des états parcourus constitue alors la solution au problème.

Notez qu'à chaque passage de la rive i à la rive j , on vérifiera qu'on n'a pas laissé sur la rive i un des couples interdits, et qu'on ne se retrouve pas dans un des états dans lesquels on est déjà passé.

[Correction.](#)

Exercice 14 - Primitive prolog

Prendre chacune des primitives prolog proposées en [Annexe](#) du cours et les compléter à l'aide de la documentation SWI-Prolog.

1. Introduction

Nous présentons dans ce chapitre d'introduction un certain nombre de généralités permettant de comprendre le pourquoi de ce module :

- Nous passerons en revue les différentes visions et définitions de l'Intelligence Artificielle (IA) : [leçon 1](#).
- Nous reviendrons sur les grandes étapes de l'IA : [leçon 2](#).
- Nous finirons par une synthèse sur l'IA aujourd'hui : [leçon 3](#).

1.1 Pourquoi MOIA ?

Que signifie MOIA ?

Méthodes et Outils pour l' Intelligence Artificielle

1.1.1 Plan du cours

Nous allons explorer les différentes notions qui se cachent derrière le terme d'Intelligence Artificielle. L'image que peut en avoir notre société est le reflet de la littérature ou du cinéma. En réalité, celle-ci est incomplète. Elle ne correspond qu'à une partie des travaux dans ce domaine, même s'il en demeure la finalité, celle de rendre le traitement de l'information le plus efficace possible. Ce cours se décompose en cinq grandes parties qui forment 11 chapitres :

- **Présentation :**
Cette première partie présente le contexte du module. Nous donnerons un bref aperçu de l'évolution de l'Intelligence Artificielle.
Cette partie est composée du premier chapitre *Introduction*.
- **Structuration de la connaissance :**
Pour pouvoir traiter l'information ou raisonner sur celle-ci, nous avons besoin de lui donner une structuration voire une sémantique. Nous allons, dans cette partie, nous intéresser à la formalisation de celle-ci.
Cette partie est composée du deuxième chapitre *Représentation des connaissances*.
- **Algorithmes généraux :** Cette partie présente les méthodes et les outils utilisés de façon générale pour résoudre des problèmes du domaine de l'Intelligence Artificielle. Ils se décomposent en deux grandes familles. La première regroupe les approches sur les états/transitions. La seconde regroupe les approches sur la résolution de contraintes. Cette partie est composée du troisième chapitre *Algorithmes classiques* et du quatrième chapitre *Programmation par contraintes*.
- **Domaine d'application :** Cette dernière partie regroupe un certain nombre de domaines d'application où l'Intelligence Artificielle a été très productive. Nous verrons les solutions spécifiques qui ont été développées dans ceux-ci.
Cette partie regroupe les chapitres cinq à dix.

- **Expérimentation et implémentations** : Afin de mieux appréhender l'ensemble des notions abordées dans ce cours, nous vous proposerons d'implémenter un certain nombre d'algorithmes pour résoudre des problèmes typiques du domaine. Pour cela, nous utiliserons un des langages de référence en Intelligence Artificielle, le langage Prolog. Cette partie est composée du chapitre Prolog, qui est en réalité un chapitre omniprésent pendant l'ensemble du cours.

Maintenant que le canevas du cours est posé, nous pouvons nous intéresser à son contenu.

1.1.2 Que signifie l'Intelligence Artificielle ?

Nous allons nous intéresser à déterminer ce qui se cache derrière la notion d'Intelligence Artificielle. Pour cela, regardons comment est perçu ce terme dans différents domaines.

- En **Philosophie** :
C'est la logique tout ce qui a trait à une méthode de raisonnement.
C'est l'esprit comme un système physique.
C'est le fait de pouvoir apprendre (l'apprentissage).
C'est le fait de pouvoir communiquer (le langage).
- En **Mathématiques** :
C'est la représentation formelle et la preuve (formaliser pour raisonner).
Ce sont les algorithmiques (in)décidables (analyser et décomposer pour résoudre).
Ce sont les probabilités (décrire des lois pour prédire des comportements).
- En **Psychologie** :
C'est le fait de pouvoir s'adapter à son environnement.
C'est le phénomène de perception et de contrôle moteur.
C'est tout ce qui a un rapport aux techniques expérimentales liées à l'Individu.
- En **Linguistique** :
C'est la représentation des connaissances.
C'est la formalisation de dialogue avec les grammaires.
- En **Neuroscience** :
C'est le substrat physique de l'activité mentale.
- En **Théorie du contrôle** :
C'est la base des systèmes asservis, il permet d'en assurer la stabilité.
C'est le concept d'agent qui permet de caractériser un comportement.

En résumé : Le concept d'Intelligence Artificielle est utilisé dans de nombreux domaines des sciences mais ne définit pas toujours la même notion. Dans le cadre de l'informatique, l'Intelligence Artificielle a beaucoup évolué. Nous allons voir cela dans la leçon suivante.

1.2 Historique

Nous allons faire un peu d'histoire afin de voir comment a évolué l'Intelligence Artificielle au cours des siècles.

Il y a très très longtemps...

- -3000 : on a retrouvé un papyrus sur lequel étaient écrites des procédures médicales sous une forme que l'on peut rapprocher de celle des règles de production.

- -350 : Aristote formalise le premier type de raisonnement : le syllogisme. Il utilise les démarches initiées par Platon et Socrate basées sur le principe de non-contradiction et celui du tiers-exclus.
- ... Plus rien jusqu'au siècle des lumières
- 1679 : Leibnitz invente l'arithmétique binaire.
- 1854 : Boole propose l'algèbre de Boole.
- 1938 : Shannon fait le lien entre l'algèbre de Boole et le traitement du signal dans l'ordinateur. BInary digiT est né.

Un des précurseurs en Intelligence Artificielle et qui reste sans nul doute la référence en matière de littérature de sciences-fictions dans ce domaine est Isaac Asimov. La cohérence de ses écrits vient du fait, qu'il a été un grand scientifique et s'est posé des questions de fond sur le rôle d'Intelligence Artificielle dans la société. Avec l'apparition de Robbie en 1941, il ouvre ainsi l'aire de la robotique. On lui doit les trois lois de la robotique (1950) qui sont la référence aussi bien dans la littérature mais surtout dans les milieux scientifiques :

1. Un robot ne peut pas nuire à la vie d'un être humain.
2. Un robot doit obéir aux ordres des humains sauf si c'est contraire à la première règle.
3. Un robot ne doit pas se mettre en danger sauf si c'est contraire à la première règle.

Dans le même esprit, Alan Turing (inventeur de la Machine de Turing 1937) propose le test de Turing (1950) pour déterminer si une machine est devenue suffisamment intelligente. Ce test consiste à mettre un individu devant un terminal d'ordinateur et de le faire dialoguer avec un interlocuteur invisible. Il doit décider si l'interlocuteur est un être humain ou un système d'IA imitant un être humain. Il avait prédit qu'avant l'an 2000, une machine aurait 30% de chance de tromper une personne non avertie pendant 5 minutes.

Ceci nous amène aux débuts de l'IA et de l'informatique :

- 1943 : J. McCulloch et W. Pitts modélisent les neurones et le cerveau pour pouvoir faire une machine de Turing.
- 1944 : Electronic Numerical Integrator And Calculator, le premier ordinateur fait pour résoudre toutes sortes de problèmes (non spécialisés).
- 1945 : J. Von Neumann et O. Morgenstein proposent une solution théorique pour résoudre les jeux de stratégie et les problèmes de démonstration automatique.
- 1950 : Alan Turing édite son livre "Computing Machinery and Intelligence". A cette époque, on cherche l'algorithme universel qui va résoudre tous les problèmes c'est le "General Problem Solver".
- 1955 : John McCarthy propose le terme "Intelligence Artificielle" pour l'opposer à la "Bêtise naturelle".

Puis, vient l'IA moderne :

- 1957 : N. Chomsky chercheur au Massachusetts Institute of Technology (MIT) en linguistique va définir la notion de sémantique comme différente de la syntaxe.
- 1958 : J. McCarthy crée le premier laboratoire de recherche en Intelligence Artificielle au MIT. Il est l'inventeur du langage Lisp (1958) et de l'algorithme Alpha-Beta (1961).
- 1965 : Premier système expert DENDRAL. C'est un système d'aide aux chimistes pour identifier une structure chimique inconnue à partir de sa formule de base et de résultats expérimentaux. J. A. Robinson invente la procédure de résolution. Au MIT une équipe met au point le premier logiciel qui permet de simuler le dialogue avec un psychologue : ELIZA.

- 1969 : Naissance du premier robot qui se déplace, qui perçoit et résout des problèmes : Shakey 1er.
 - 1972 : A. Colmerauer et P. Roussel inventent le langage PROgrammation en LOGique. Prolog est un langage basé sur la logique des prédicats et le principe de résolution. Il va devenir le langage de référence en Intelligence Artificielle sauf peut être aux Etats-unis où Lisp reste très présent.
 - 1980 : Création de l'American Association of Artificial Intelligence qui va permettre d'avoir des moyens pour développer l'IA dans le monde. Elle est aujourd'hui une référence dans le monde scientifique et un label de qualité pour les travaux scientifiques.
 - 1985 et + : Les travaux en linguistique font un bond en avant avec des moyens importants. On arrive à avoir une gestion efficace des dictionnaires (ceci permet d'avoir un vocabulaire riche) permettant de faire de la reconnaissance de texte et du discours avec l'utilisation de phonème pouvant ainsi travailler sur des systèmes multi-orateurs.
- Etude des jeux de stratégie pour la validation d'algorithmes de recherche.
- 1997 : Victoire du Deep Blue, ordinateur mis au point par IBM, face au champion du monde d'échecs G. Kasparov. Premier match de football avec des robots.
 - 2000... : Animaux de compagnie robot, aspirateur robotisé faisant le ménage seul. L'université de Caneggie Mellon lance des robots nomades pour visiter l'antarctique ou Mars.

1.3 Qu'est-ce que les systèmes à base d'IA savent faire ?

Jouer correctement au ping-pong : seulement un bras et une caméra sont nécessaires pour cela. Un logiciel de suivi de cible permet de connaître la position de la balle, une analyse du mouvement de la raquette de l'adversaire permet de prédire la trajectoire et le mouvement de la balle.

Conduire un véhicule sur une route de montagne / dans la circulation : de nombreux essais ont déjà été réalisés même avec des convois de camions autoconduits sur différents types de terrains. Mais une des grandes difficultés est le balisage de la route et la gestion des événements imprévisibles : traversée de la voie par des objets, comportement erratique des autres usagers et panne mécanique.

Jouer aux cartes : il existe de nombreux logiciels de jeux de cartes. La difficulté est de pouvoir graduer le niveau du joueur virtuel et de pouvoir modéliser ce qu'est un coup pertinent. Nous verrons cela dans le détail dans le chapitre 6 *Algorithmes des jeux*.

Découvrir et prouver un nouveau théorème mathématique : la preuve automatique commence à donner des résultats. Mais on est encore loin de pouvoir prouver complètement un théorème sans un appui important de bibliothèques et d'un expert qui indique le cheminement de la preuve par des lemmes intermédiaires.

Inventer une histoire drôle (de façon intentionnelle) : le rire est le propre de l'homme même si les hyènes le font aussi. En partant de ce postulat on peut considérer que faire de l'humour ne peut être que le propre d'être intelligent. Mais réaliser un bon mot par rapport à un contexte reste facile, la difficulté reste l'à propos (éviter le systématique ou le cynisme).

Donner des conseils avisés dans des domaines pointus : c'est le rôle des systèmes experts. Ils sont généralement le fruit de la synthèse de connaissances issues des experts du domaine, auquel on peut coupler un processus d'apprentissage pour lui permettre d'évoluer.

Traduire des langues, parler une langue, parler en temps réel : vous avez sûrement déjà utilisé les traducteurs des moteurs de recherche. Le problème est le contre sens. La difficulté vient de la non connaissance du contexte ou de l'utilisation de vocabulaire technique, qui souvent n'a pas son sens usuel.

2. Représentation des connaissances

Nous présentons, dans ce chapitre, différentes façons de formaliser le discours pour permettre d'abstraire le langage naturel (le français) vers une représentation de l'information sur laquelle l'ordinateur pourra effectuer des traitements. Nous ferons une présentation rapide des concepts pour chacun des formalismes. Pour plus d'informations, le lecteur peut se reporter à la bibliographie.

- Les logiques classiques avec la logique propositionnelle (des propositions) et le calcul des prédicats : [leçon 1](#).
- Les logiques non classiques avec les défauts de Reiter et la logique multi-valuée (logique floue) : [leçon 2](#).
- Les règles de production : [leçon 3](#).
- Les objets structurés avec les réseaux bayésiens, les chaînes de Markov et les réseaux de neurones : [leçon 4](#).

2.1 Les logiques classiques

On entend par logiques classiques, les logiques qui quels que soient le domaine ou le contexte dans lesquels sont réalisés le raisonnement ou l'interprétation, on obtiendra le même résultat au cours du temps.

Il en existe énormément car pour en caractériser une, il suffit de définir :

- Les formules qui la composent : ensemble d'atomes du langage
- Les axiomes : formule toujours vraie quelle que soit la valeur donnée par l'interprétation de ces atomes pour la logique considérée
- Les règles d'inférence : ou de déduction permettant d'obtenir de nouvelles formules à partir d'une formule vraie dans la logique.

Nous allons nous intéresser à deux des plus connues : la logique propositionnelle (ou logique des propositions) et la logique des prédicats. Pour plus d'informations sur celles-ci le lecteur pourra se reporter au cours de logique de 3^{ème} année de licence EAD ou à [\[Del86\]](#) de la bibliographie.

2.1.1 La logique propositionnelle

Cette logique a traversé les âges pour venir d'Aristote et aller jusqu'à Boole.

Structure

Elle est constituée d'un **Ensemble de formules**. Une formule est constituée des atomes du langage :

- Les **variables** ou propositions : Lettres
- Les **connecteurs** ou opérateurs :
 - L'opérateur (unaire) de négation : $\neg$
 - L'opérateur (binaire) d'implication : $\rightarrow$
- Les **parenthèses** pour lever l'ambiguïté

Exemple de formule :

- A
- $\neg A$
- $(A \rightarrow B) \rightarrow \neg A$

Un nombre réduit d'opérateurs suffit pour définir une logique. Par exemple, la négation ($\neg$) et l'implication ($\rightarrow$) permettent d'avoir tous les opérateurs de la logique. Ainsi, on définit le ou comme $A \vee B$ par $\neg A \rightarrow \neg B$

On ajoute les **Axiomes**. Les axiomes sont des formules qui sont toujours vraies dans la logique considérée quelles que soient les formules qui pourront les constituer. Nous utilisons ici trois formules quelconques de la logique F , G et H pour définir les trois axiomes :

1. $F \rightarrow (G \rightarrow F)$
2. $(F \rightarrow (G \rightarrow H)) \rightarrow ((F \rightarrow G) \rightarrow (F \rightarrow H))$
3. $((\neg G) \rightarrow (\neg F)) \rightarrow (F \rightarrow G)$

Le nombre d'axiomes n'est pas borné mais on considère ici un nombre réduit pour pouvoir définir simplement notre logique.

Pour pouvoir raisonner, nous définissons les **règle(s) d'inférence**. La première est le **Modus Ponens**, cette règle dit que si une formule est vraie et qu'il existe une implication de cette formule vers une autre formule alors cette dernière est aussi une formule vraie de la logique : Si F et $(F \rightarrow G)$ alors G .

Cette règle suffit mais on rencontre souvent sa contraposée le **modus Tollens** : Si $(\neg G)$ et $(F \rightarrow G)$ alors $(\neg F)$.

Exemples :

- Maintenant, l'on peut formaliser un raisonnement simple. Posons les propositions suivantes :
 - "il pleut", on note la proposition associée T
 - "je prend mon parapluie", on note la proposition associée P

Prenons comme référence, "s'il pleut alors je prend mon parapluie", on peut ainsi traduire l'information : $T \rightarrow P$.

Si l'information, T ("il pleut") apparaît en utilisant Modus Ponens on en déduit P ("je prend mon parapluie").

Ce mécanisme utilise le formalisme de la logique propositionnelle et la règle d'inférence (Modus Ponens) pour pouvoir déduire de nouvelles informations à partir d'informations connues et de principes simples (notés sous la forme d'implication).

- Montrons que la formule suivante est un théorème : $(A \rightarrow A)$ pour cela, nous avons à notre disposition les axiomes et le Modus Ponens. Ainsi nous pouvons construire la démonstration suivante :

1. $A \rightarrow ((A \rightarrow A) \rightarrow A)$, Axiome type 1 avec $F = A$ et $G = A \rightarrow A$

2. $A \rightarrow ((A \rightarrow A) \rightarrow A) \rightarrow ((A \rightarrow (A \rightarrow A)) \rightarrow (A \rightarrow A))$, Axiome type 2 avec $F = A$, $G = A \rightarrow A$, $H = A$
3. $(A \rightarrow (A \rightarrow A)) \rightarrow (A \rightarrow A)$, Modus Ponens 1 et 2
4. $A \rightarrow (A \rightarrow A)$, Axiome type 1 avec $F = A$ et $G = A$
5. $A \rightarrow A$, Modus Ponens 4 et 3

On vient de montrer que $A \rightarrow A$ est un théorème et on le note $\models (A \rightarrow A)$.

- Faisons maintenant une démonstration avec les hypothèses $A \rightarrow B$ et $B \rightarrow C$ de $A \rightarrow C$, On construit la démonstration suivante :

1. $B \rightarrow C$, Hypothèse 2
2. $(B \rightarrow C) \rightarrow (A \rightarrow (B \rightarrow C))$, Axiome de type 1
3. $A \rightarrow (B \rightarrow C)$, Modus Ponens Hypothèse 2 et 2
4. $(A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$, Axiome de type 2
5. $(A \rightarrow B) \rightarrow (A \rightarrow C)$, Modus Ponens 3 et 4
6. $A \rightarrow B$, Hypothèse 1
7. $A \rightarrow C$, Modus Ponens Hypothèse 1 et 5

on a donc $\{A \rightarrow B, B \rightarrow C\} \models A \rightarrow C$, ceci se lit sous les hypothèses $A \rightarrow B$ et $B \rightarrow C$, $A \rightarrow C$ est un théorème ou encore $A \rightarrow C$ est un conséquence logique des hypothèses.

Interprétation

Cet aspect de la logique est purement syntaxique. Ainsi on considère que la formule est un théorème puisque l'on reconnaît que c'est un axiome ou alors il existe une démonstration. Mais ajoutons un aspect sémantique à notre logique et regardons ce qu'il se passe lorsque nos propositions (ou encore variables propositionnelles) prennent une valeur dans l'algèbre de Boole. Ainsi on définit une fonction σ qui associe aux éléments de l'ensemble des propositions une valeur dans l'algèbre de Boole. $\sigma : \mathcal{P} \rightarrow \mathcal{B}$

Interpréter une formule revient à interpréter chacun des éléments qui la constitue. Ainsi, on associe aux deux opérateurs $\neg$ et $\rightarrow$ les deux lois de compositions de l'algèbre de Boole – et $\Rightarrow$ avec comme règle :

-0	1
-1	0

$0 \Rightarrow 0$	1
$0 \Rightarrow 1$	1
$1 \Rightarrow 0$	0
$1 \Rightarrow 1$	1

Pour connaître l'ensemble des interprétations de la formule $F = ((A \rightarrow B) \rightarrow (A \rightarrow C))$, on utilise une table de vérité :

$\sigma(A)$	$\sigma(B)$	$\sigma(C)$	$\sigma(A \rightarrow B) = \sigma(A) \Rightarrow \sigma(B)$	$\sigma(A \rightarrow C) = \sigma(A) \Rightarrow \sigma(C)$	$\sigma(F)$
O	0	0	1	1	1
0	0	1	1	1	1
0	1	0	1	1	1
0	1	1	1	1	1
1	0	0	0	0	1
1	0	1	0	1	1
1	1	0	1	0	0
1	1	1	1	1	1

La formule a 7 **modèles**, les 7 interprétations telles que $\sigma(F) = 1$ et un **contre-modèle** $\sigma(F) = 0$. On appelle **tautologie** une formule telle que pour toute interprétation la formule soit vraie.

2.1.2 La logique des prédicats

C'est une extension de la logique des propositions proposée par Kleene en 1972.

Structure

Elle est constituée d'un **Ensemble de formules**. On retrouve les éléments des formules de la logique des propositions auxquels on ajoute de nouvelles notions :

- Les constantes : elles représentent les éléments des domaines des variables ou encore appelé le domaine de définition.
- Les variables : éléments du langage permettant de paramétrer les formules. Elles prennent leurs valeurs dans le domaine de définition.
- Les fonctions : éléments du langage permettant d'indiquer une relation sur les variables ou les constantes du langage (fonction du domaine de définition vers le domaine de définition). Le nombre de paramètres de la fonction est donnée par son degré.
- Les prédicats : éléments du langage permettant de vérifier si les éléments précédents sont vrais ou faux (fonction du domaine de définition vers l'algèbre de Boole). Le nombre de paramètres du prédicat est donnée par son poids. Un prédicat de poids 0 est une variable propositionnelle.

Exemples de formules :

- $f(x)$: fonction f de degré 1 ayant une variable x ;
- $P(a,y)$: Prédicat de poids 2 ayant une constante a et une variable y ;
- $(A \rightarrow P(f(x), y)) \rightarrow \neg B$: A et B prédicats de poids 0.

Les axiomes de la logique propositionnelle sont toujours vrais pour la logique des prédicats. On ajoute deux nouveaux axiomes où : x est une variable, t un terme, F et G des formules quelconques :

1. $F \rightarrow (G \rightarrow F)$
2. $(F \rightarrow (G \rightarrow H)) \rightarrow ((F \rightarrow G) \rightarrow (F \rightarrow H))$
3. $((\neg G) \rightarrow (\neg F)) \rightarrow (F \rightarrow G)$
4. L'occurrence de x dans F est liée $\forall x(F \rightarrow G) \rightarrow (F \rightarrow \forall xG)$
5. Les occurrences de x sont liées : $\forall xF(x) \rightarrow F(t)$

Les quantificateurs

La grande différence avec la logique propositionnelle est l'utilisation des **quantificateurs**. Il existe deux types de quantificateurs :

- Universel : $\forall x.P(x)$, quelle que soit la valeur de la variable x , le prédicat P est vrai.
- Existentiel : $\exists x.P(x)$, il existe une valeur de x telle que le prédicat P est vrai.

Le deuxième peut être défini à partir du premier à l'aide de la négation : $\neg\forall x.\neg P(x) = \exists x.P(x)$

Exemple : Traduisons le syllogisme suivant avec la logique des prédicats : "*Tout homme est mortel or Socrate est un homme donc Socrate est mortel*". On pose :

- homme(x) : "*x est un homme*", prédicat de poids 1
- Socrate : une constante
- mortel(x) : "*x est mortel*", prédicat de poids 1

On a ainsi : $\{ \text{homme}(x) \rightarrow \text{mortel}(x), \text{homme}(\text{Socrate}) \} \models \text{mortel}(\text{Socrate})$

La démonstration est immédiate en utilisant le Modus Ponens et en unifiant la variable x avec la valeur Socrate.

Formalisation du discours

On peut chercher à lier les expressions du langage naturel dit encore le monde réel à une représentation informatique du langage. Pour cela, il existe un certain nombre de règles qui permettent de faire plus facilement le lien entre le discours et une représentation de celui-ci à l'aide de la logique (des prédicats). Dans la figure 2.1 ci-dessous, nous proposons un lien entre les éléments du discours et les éléments constituant les formules de la logique.

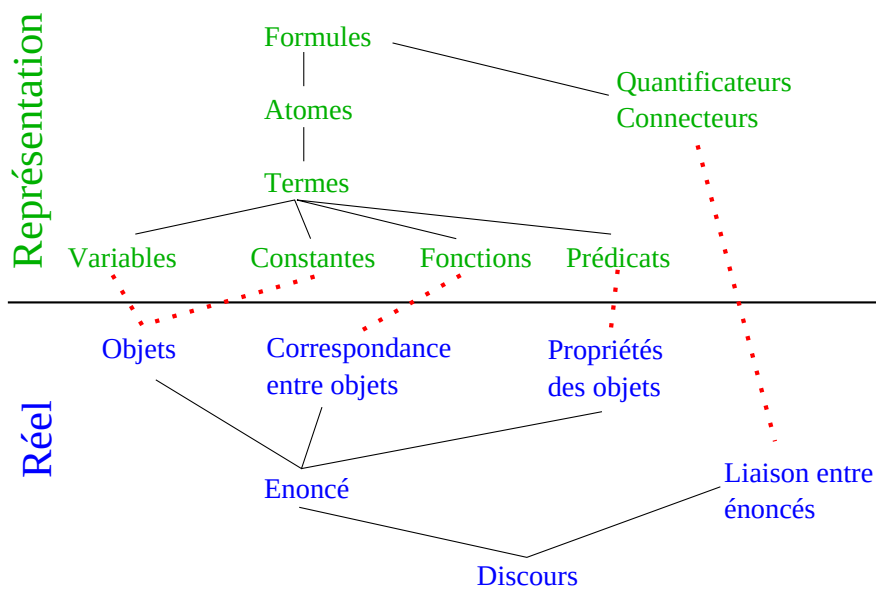


FIGURE 2.1 – Formalisation français et logique

2.2 Les logiques non-classiques

On entend par logiques non-classiques : les logiques qui suivant le domaine ou le contexte dans lequel sont réalisés le raisonnement ou l'interprétation donnent des résultats différents.

Il en existe énormément car comme pour les logiques classiques, il est facile d'en définir une.

Nous allons nous intéresser à deux des plus connues : la logique des défauts et la logique floue (ou multi-valuées). Pour plus d'informations sur celles-ci le lecteur pourra se reporter à la référence [Del86] de la bibliographie.

2.2.1 La logique des défauts

Cette logique a été proposée par Reiter en 1979. Son but est de palier certaines limitations de la logiques des prédicats. Ainsi une théorie des défauts, notée $T = (D, W)$, se compose en deux types de formules :

- W : ensemble de faits, formules closes de la **logique des prédicats**.
- D : ensemble de **défauts**, règles d'inférence à contenu spécifique.

Un **défaut** est une règle d'inférence constituée de trois parties. Ces trois parties sont des formules de la logique des prédicats où la variable x est libre :

- $u(x)$, prérequis
- $v(x)$, la justification On le note : $\frac{u(x) : v(x)}{w(x)}$
- $w(x)$, la conséquence

Exemple : On utilise les défauts pour représenter une information incertaine en opposition avec les logiques classiques qui elles correspondent à des affirmations toujours vraies. Par exemple, pour représenter les affirmation suivantes "*une partie des joueurs ont gagné et tous les gagnants sont des joueurs*", on définit les prédicats suivants :

- $Joueur(x)$: "*x est un joueur*"
- $Gagne(x)$: "*x a gagné*"

On traduit alors la première affirmation avec deux défauts, puisqu'on ne sait pas si le joueur a gagné :

$$R_1 = \frac{Joueur(x):Gagne(x)}{Gagne(x)} \qquad R_2 = \frac{Joueur(x):\neg Gagne(x)}{\neg Gagne(x)}$$

La deuxième affirmation sera traduite en logique des prédicats :

$$W = \forall x. Gagne(x) \rightarrow Joueur(x)$$

Une fois formalisée ces éléments, l'interprétation s'effectue à travers des extensions. Cette notion d'extension permet de prendre en compte les éléments qui pourraient être contradictoire. Ainsi dès que l'on peut utiliser plusieurs défauts antagoniste, il peut être nécessaire de créer des extensions pour permettre de les appliquer.

Maintenant, on sait que Fabrice est un joueur, qu'est-ce que l'on peut déduire sur le fait qu'il ait gagné ?

Le premier travail est de traduire cela : $F = Joueur(Fabrice)$ On crée ainsi une nouvelle théorie $S = (\{R_1, R_2\}, \{W, F\})$. On obtient deux extensions :

- E_1 : Joueur(Fabrice), Gagne(Fabrice) [R_1]
- E_2 : Joueur(Fabrice), $\neg$ Gagne(Fabrice) [R_2]

Pour E_1 Fabrice est un joueur qui gagne et pour E_2 Fabrice est un joueur qui ne gagne pas.

2.2.2 La logique floue

La logique floue ou multivaluée vient d'un constat de Kleene en 1938 sur le fait que tout n'est pas blanc ou noir. Les logiques classiques ne permettent de définir que des choses vraies ou fausses. La question est : que se passe-t-il si un élément n'est pas encore sûr ou presque sûr ? Comment faire pour définir ce type d'information ?

Les logiques multivaluées utilisent les mêmes opérateurs que les logiques classiques mais ils sont redéfinis pour étendre leur domaine d'interprétation à un domaine possédant plus de deux valeurs. L'interprétation σ des opérateurs est la suivante :

- $\sigma(\neg P) = 1 - \sigma(P)$
- $\sigma(P \vee Q) = \text{Max}(\sigma(P), \sigma(Q))$, où Max est une fonction renvoyant la plus grande valeur de ses deux paramètres.
- $\sigma(P \wedge Q) = \text{Min}(\sigma(P), \sigma(Q))$, où Min est une fonction renvoyant la plus petite valeur de ses deux paramètres.
- $\sigma(P \rightarrow Q) = \text{Max}(1 - \sigma(P), \sigma(Q))$
- $\sigma(P \leftrightarrow Q) = (\sigma(P) = \sigma(Q))$

La différence majeure vient du fait que les deux règles suivantes ne sont pas des tautologies :

- Le tiers exclu : $\neg P \vee P$
- Le principe de non contradiction : $\neg(\neg P \wedge P)$

Ceci semble normal puisque le domaine n'est pas réduit aux deux valeurs de la logique de Boole (vrai et faux) comme pour les logiques classiques.

Exemple de table de vérité pour une logique à trois valeurs $(0, \frac{1}{2}, 1)$:

P	Q	$\neg P$	$P \rightarrow Q$	$P \vee Q$	$P \wedge Q$	$P \leftrightarrow Q$
0	0	1	1	0	0	1
0	$\frac{1}{2}$	1	1	$\frac{1}{2}$	0	0
0	1	1	1	1	0	0
$\frac{1}{2}$	0	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	0	0
$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	1
$\frac{1}{2}$	1	$\frac{1}{2}$	1	1	$\frac{1}{2}$	0
1	0	0	0	1	0	0
1	$\frac{1}{2}$	0	$\frac{1}{2}$	1	$\frac{1}{2}$	0
1	1	0	1	1	1	1

2.3 Les règles de Production

Les règles de production sont aussi utilisées comme les règles d'inférence dans les systèmes experts. Elles permettent aussi bien de représenter les connaissances que de pouvoir raisonner.

2.3.1 Qu'est-ce qu'une règle de production ?

C'est une règle de production : $A \xrightarrow{FC} B$

Se décompose en trois parties :

1. La condition d'application : A
2. La déduction obtenue : B
3. Le facteur de certitude : FC

Elle se lit si on a A alors on aura B avec une certitude de FC. Un des problèmes est de pouvoir établir le facteur de certitude associé aux règles initiales. Généralement, c'est des éléments statistiques qui sont utilisés (issus d'étude ou de l'expertise métier).

Calcul d'une transition ?

Pour effectuer la transition de A vers B et ainsi obtenir la valeur du nouveau facteur de certitude de mon chemin, on utilise l'une de ces deux formules :

1. $FC(\text{Source}) = 1$, Source étant l'état initial du chemin
2. $FC(B) = \text{Max}(0, FC(A)) \times FC$

Exemple : Soit les affirmations suivantes "Les gagnants du loto correspondent à 10% des joueurs", "Les gagnants du TacOTac correspondent à 25% des joueurs", "Dans les joueurs intermittents, il y a Fabrice" (50%). "Quelles sont les chances de Fabrice ?".

On va construire le graphe de dépendance entre chacune de ces affirmations comme l'illustre la figure 2.2.



FIGURE 2.2 – Exemple de règles de production

On part de l'information Fabrice, on regarde chaque règle qui ont permis d'obtenir le résultat. Pour cela, il suffit de parcourir le graphe dans le sens inverse des différentes règles :

- $FC(\text{Fabrice}) = 1$,
- $FC(\text{Joueur}) = \text{Max}(0, FC(\text{Fabrice})) \times 0,5 = 0,5$
- $FC(\text{TacOTac}) = \text{Max}(0, FC(\text{Joueur})) \times 0,25 = 0,125$
- $FC(\text{Loto}) = \text{Max}(0, FC(\text{Joueur})) \times 0,1 = 0,05$
- Donc Fabrice à 12,5% de chance de gagner s'il joue au TacOTac et 5% de chance de gagner s'il joue au Loto

2.4 Les objets structurés

Dans cette partie, nous allons voir d'autres formes de représentation à titre informatif. Cette présentation est donc là pour compléter vos connaissances mais n'a pas besoin d'être assimilée pour la suite du module et pour les évaluations.

Les objets structurés se définissent à l'aide d'**Arbre sémantique**. Ces arbres se composent de :

- nœuds qui représentent les concepts
- arcs qui représentent les liens entre les concepts

2.4.1 Comment raisonner ?

On peut rechercher les solutions dans le graphe. Sinon il existe des représentations plus élaborées où l'on ajoute des règles de production ou des opérateurs logiques à la définition des arcs. Nous allons voir trois types d'objets structurés.

2.4.2 Les réseaux bayésiens

Le réseau bayésien est un graphe orienté acyclique :

- Un nœud représente une variable "aléatoire"
- Un arc représente les relations entre les variables auxquelles sont ajoutées des séries de distributions conditionnelles.

Il est basé sur le théorème de Bayes :

$$Prob(A|B) = \frac{Prob(B|A) \times Prob(A)}{Prob(B)}$$

Avec $Prob(A|B)$ la probabilité de A sachant B

L'information est ainsi calculée :

- Rapport de vraisemblance : $l(B|A) = \frac{Prob(B|A)}{Prob(B|\bar{A})}$
- Vraisemblance a priori : $V(A) = \frac{Prob(A)}{Prob(\bar{A})}$
- Vraisemblance a posteriori : $V(A|B) = \frac{Prob(A|B)}{Prob(\bar{A}|B)}$

avec $\bar{A}$ le complémentaire de A

Nous ne rentrerons pas plus dans le détail dans ce cours. Si vous êtes intéressés par cette approche vous pouvez consulter le site <http://marc.bouissou.free.fr/siteRB/default.htm>.

2.4.3 Les chaînes de Markov

Les chaînes de Markov permettent de représenter les informations liées aux transitions d'un système ou de son évolution. Pour cela, on définit une transition entre deux états x et y par une probabilité de déplacement notée $p(x,y)$.

On considère qu'il existe une distribution uniforme $p(x)$, sur l'espace P considéré, donnée par la formule suivante :

$$p(x) = \int_p p(y) \times p(x, y) \delta y$$

Définir $p(x,y)$

Il existe plusieurs approches pour définir la probabilité de transition entre les deux états. Les deux plus connues sont :

- Echantillonneur de Gibbs
- Algorithme de Metropolis-Hastings

Pour fonctionner, cette approche garantit les propriétés suivantes :

- **Réversibilité** : Le calcul peut être fait en avant comme en arrière. Ceci permet d'effectuer des corrections sur le modèle a posteriori.
- **Apériodique** : Il n'existe pas de période, qui pourrait faire diverger le système.
- **Irréductible** : Il n'existe pas de forme plus simple pour représenter la même information.
- **Récurrence** : Elle permet de calculer les éléments les uns en fonction des autres.

Un des exemples d'utilisation de cette approche est la prévision météorologique qui utilise le principe des chaînes de Markov pour faire les prévisions. Vous pouvez retrouver doudou le hamster sur http://fr.wikipedia.org/wiki/Chaîne_de_Markov.

2.4.4 Les réseaux de neurones

L'approche cherche à savoir d'où vient la mémoire d'un être humain ? A priori d'informations stockées dans son cerveau. A partir de cela, les chercheurs ont essayé de modéliser cela avec une approche similaire. Dans la figure 2.3 nous retrouvons un **neurone** avec en son centre (en forme hexagonale ici) le **Soma**. Les ramifications partant du neurone (appelées *Dendrites*) sont connectées aux autres neurones par le biais des **synapses** (forme cylindrique, zoomée dans la figure 2.4). Toutes les informations passent par les **axones** assurant le lien entre les dendrites et le soma.

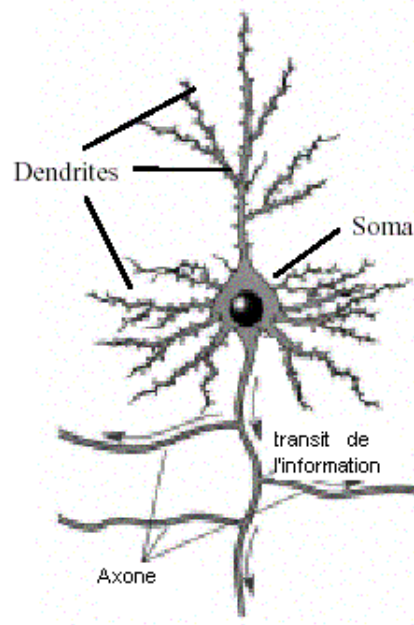


FIGURE 2.3 – Neurone

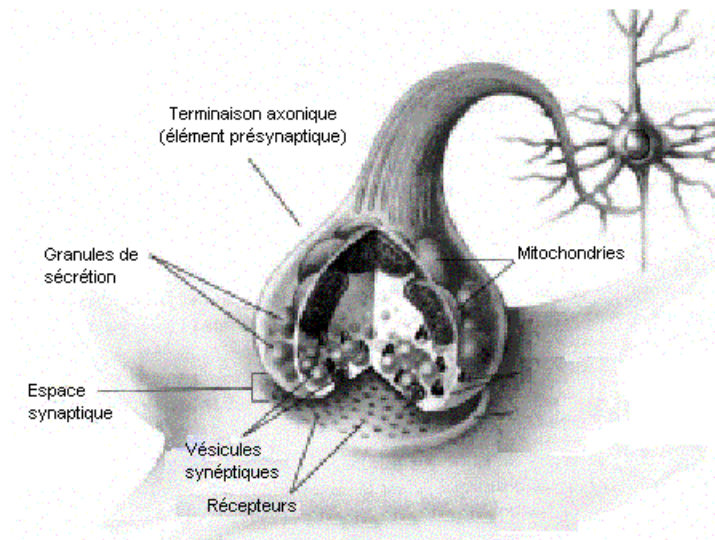


FIGURE 2.4 – Synapse

Une simplification de ce système d'interconnexions entre les neurones peut être vu comme un maillage. Bien sûr, cette vision en deux dimensions ne correspond pas à la réalité de notre cerveau. Nous verrons cela plus en détail dans le [chapitre 8](#).

Mécanisme simplifié

Nous savons aujourd'hui que le cerveau, pour fonctionner, ne se limite pas à l'utilisation de signaux électriques. Un certain nombre de composés chimiques ont aussi un rôle important dans la transmission, le stockage et les performances de notre cerveau. De même, nous savons que celui-ci est en perpétuel remaniement. Par exemple, pour augmenter l'efficacité de zone très souvent sollicitée (activation de cycles neurales), le cerveau va essayer de rapprocher les neurones qui y collaborent.

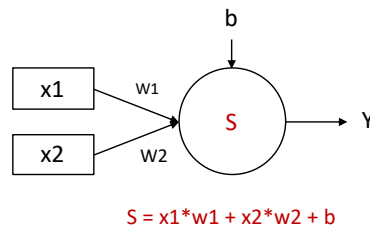


FIGURE 2.5 – Neurone artificiel

Tous ces éléments, qui régissent les réseaux de neurones, ne sont pas pris en compte dans le modèle mathématique simple. Dans la figure 2.5, nous voyons un neurone artificiel. En effet, on utilise une fonction de seuil (S) pour déterminer si le neurone émet à son tour (représenté ici par la variable Y) qui reçoit en entrée les stimuli x_i (qui peuvent être électrique ou chimique), en prenant en compte une pondération sur la liaison (w_i) avec les voisins du neurone et une valeur complémentaire (notée b pour biais et indépendante des voisins) lorsque la somme des éléments atteint un certain seuil H le neurone se met à émettre aussi un stimulus Y à ses voisins. Ainsi $Y = 1$ si $S > H$ et 0 sinon, S est établi de la façon suivante :

$$S = \sum_{i=1}^n (w_i \times x_i) + b$$

2.5 Exercices chapitre 2

Exercice 1 - Traduction

Soient les phrases suivantes :

1. Les étudiants sont jeunes.
2. Les jeunes sont célibataires.
3. Les célibataires sont jeunes.
4. Une partie des jeunes sont étudiants.
5. Les étudiants ayant des enfants sont mariés ou concubins.
6. Les concubins sont jeunes.
7. On ne peut pas être simultanément célibataire, marié ou concubin.

Questions :

1. Traduire les phrases précédentes en **logique propositionnelle**. Qu'est ce que l'on obtient si l'on pose la question : "*Fabrice est jeune ?*"

2. Traduire les phrases précédentes en **règles de production**. Qu'est ce que l'on obtient si l'on pose la question : "*Fabrice est jeune ?*". On associe respectivement à chaque règle (1 à 6) les facteurs de certitude suivants : 0,9 ; 0,8 ; 0,7 ; 0,3 ; 0,95 ; 0,8
3. Traduire les phrases précédentes en **logique des prédicats**. Qu'est ce que l'on obtient si l'on pose la question : "*Fabrice est un étudiant qui vit en concubinage ?*"
4. Traduire les phrases précédentes en **logique des défauts**. Qu'est ce que l'on obtient si l'on pose la question :
 - (a) "*Fabrice est étudiant, quel est son statut marital ?*"
 - (b) "*Fabrice est étudiant et a peut-être des enfants, quel est son statut marital ?*"

Correction.

Exercice 2 - Sportifs

Soient les règles suivantes :

- R_1 : les étudiants sont jeunes.
- R_2 : les jeunes font du foot.
- R_3 : le foot est pratiqué par des jeunes.
- R_4 : une partie des jeunes sont étudiants.
- R_5 : les étudiants qui gagnent font de la belote ou de la pétanque.
- R_6 : les joueurs de belote sont jeunes.
- R_7 : on ne peut pas jouer en même temps à la belote, au foot ou à la pétanque.
- R_8 : une partie des enseignants sont étudiants

Questions :

1. Donner la traduction des règles à l'aide de la logique des défauts.
2. Calculer toutes les extensions de cette théorie des défauts, sachant que "*Fabrice est enseignant*".
3. Calculer toutes les extensions de cette théorie des défauts, sachant que "*Fabrice est enseignant, et qu'il a peut être gagné*".
4. "*Est-ce que l'on peut avoir des enseignants qui sont étudiants et qui jouent au foot ?*" Calculer les extensions de cette théorie des défauts.

Correction.

Exercice 3 - Les pieds Nickelés

Trois suspects ont été arrêtés à la suite du cambriolage de la villa de M. Futay ; ce sont : Croquignol, Ribouldingue et Filochard. Ces trois personnages sont bien connus de l'inspecteur Croquenot pour le caractère très aléatoire de la vérité de leurs affirmations.

Croquignol dit : "*Ribouldingue est coupable et Filochard est innocent*"

Ribouldingue dit : "*Si Croquignol est coupable, alors Filochard l'est aussi*"

Filochard dit : "*Je suis innocent mais l'un au moins des deux autres est coupable*"

C'est une nécessité pour Croquenot d'envisager plusieurs possibilités, alors aidez l'inspecteur à répondre à ces questions :

1. Modéliser le problème sous forme de la logique propositionnelle.
2. Est-il possible que mes trois lascars aient dit la vérité? Alors qui serait coupable?
3. Ils auraient aussi pu mentir tous les trois, je suppose! Alors qui serait coupable?
4. Il me semble que les interprétations de certains témoignages sont inclus dans d'autres, lesquels?
5. Si je suppose que tous sont innocents, qui a menti? et si je les suppose tous coupables qui a menti?
6. Est-il possible qu'il n'y ait qu'un seul faux témoignage? et dans ce cas, qui a menti et qui est coupable?
7. Et si je suppose que l'innocent dit la vérité et le coupable ment, qui alors est innocent, qui est coupable?

Correction.

3. Algorithmes classiques

Après avoir vu comment représenter l'information, nous allons voir comment utiliser celle-ci pour résoudre des problèmes d'IA. Ce chapitre propose les algorithmes de résolutions de problèmes représentables par des graphes :

- Nous explicitons tout d'abord la notion de problème dans la [leçon 1](#).
- Puis nous donnons les algorithmes de base (profondeur, largeur) à la [leçon 2](#).
- Nous proposons pour améliorer ces algorithmes la notion d'heuristique dans la [leçon 3](#).
- Nous finirons ce chapitre avec différentes versions de l'algorithme Best-First [leçon 4](#).

3.1 Définitions

Dans ce chapitre nous cherchons à résoudre des problèmes d'IA. La première action à réaliser est de définir ce qu'est un problème.

3.1.1 Définir un problème?

Tout problème est relatif à un certain ensemble d'objets que l'on peut décrire sous forme de variables d'état du problème.

Une **variable d'état** est une variable qui décrit un élément caractéristique du problème et qui est susceptible de changer au cours du temps.

Un **état du problème** est l'ensemble des valeurs que prennent ses variables d'état à un instant donné.

L'**espace d'états** d'un problème est l'ensemble des états possibles pour le problème considéré.

Ainsi il ne reste plus qu'à représenter l'évolution. On peut alors décrire un problème en quatre parties :

- L'état initial ou initialisation qui donne le(s) premier(s) état(s) du système.
- Les transitions ou règles d'évolution qui vont permettre de décrire comment passer d'un état à un autre.
- L'espace de recherche ou d'états (atteignables) : ensemble des états qui peuvent être générés à partir d'un état initial et en utilisant les règles d'évolution.
- L'état final ou condition d'arrêt.

Ainsi, on cherche une solution pour atteindre l'état final (en [bleu](#)) à partir de l'état initial (en [marron](#)) comme décrit dans la figure [3.1](#).

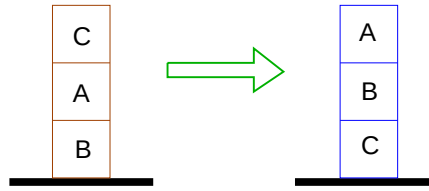


FIGURE 3.1 – Description d'un problème

On devra parcourir l'espace de recherche donné dans la figure 3.2, sachant qu'à chaque transition on ne peut déplacer qu'une lettre :

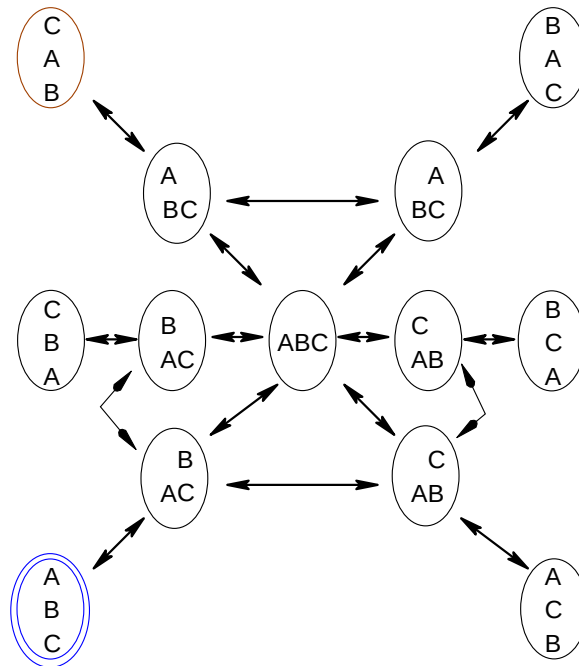


FIGURE 3.2 – Espace d'états

3.1.2 Méthode de résolution

Ainsi, nous nous retrouvons à essayer de résoudre un problème décrit par un graphe (espace de recherche). Nous allons nous intéresser aux algorithmes de parcours de graphe connus pour résoudre notre problème. Pour cela, nous pouvons jouer sur :

- Les stratégies (Largeur, Profondeur, ...)
- Les heuristiques (Best-First, Taboo, Recuit-simulé, Sac à dos, ...)

Dans la mesure du possible, nous ne construirons pas le graphe pour élaborer ensuite les algorithmes sur celui-ci. Ainsi, seulement les sous-chemins utilisés par l'algorithme seront construits par celui-ci pendant le processus de résolution, comme nous le verrons dans les leçons suivantes.

3.2 Les stratégies

Nous allons maintenant voir comment résoudre le problème ?

3.2.1 Parcours en profondeur

But : explorer l'espace de recherche chemin par chemin. On effectue l'appel `Profondeur(Init, )` sur l'algorithme suivant :

Algorithme 1 : `Profondeur(n,Sol)`

Input : un nœud (n)

Output : renvoie Vrai si chemin solution Sol et faux sinon.

$Sol \leftarrow Sol \cup \{n\}$;

if n est solution **then** fin $\leftarrow$ Vrai;

else

 fin $\leftarrow$ Faux; $n_1 \leftarrow$ successeur(n) ;

while non(fin) et n_1 est valide **do**

 fin $\leftarrow$ `Profondeur`(N_1 , Sol) ;

$n_1 \leftarrow$ successeur(n) ;

if non(fin) **then** Sol $\leftarrow$ Sol $\setminus$ {n};

return fin;

Exemple : Le parcours en profondeur va parcourir le graphe en proposant différents chemins jusqu'à trouver une solution. Soit le graphe donné dans la figure 3.3.

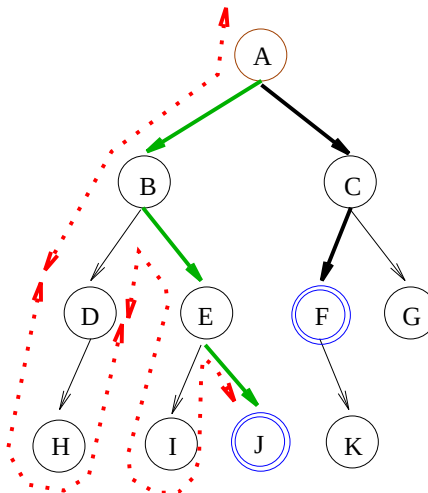


FIGURE 3.3 – Parcours en profondeur

Les réponses obtenues (la partie parcourue est en pointillé rouge sur le dessin) sont :

— A, B, D, H : Faux

- A, B, E, I : Faux
- A, B, E, J : Vrai

Problème : S'il existe un cycle dans le graphe, l'algorithme va boucler puisqu'il est incapable de le détecter comme illustré dans la figure 3.4. Le parcours reste bloqué entre D et H.

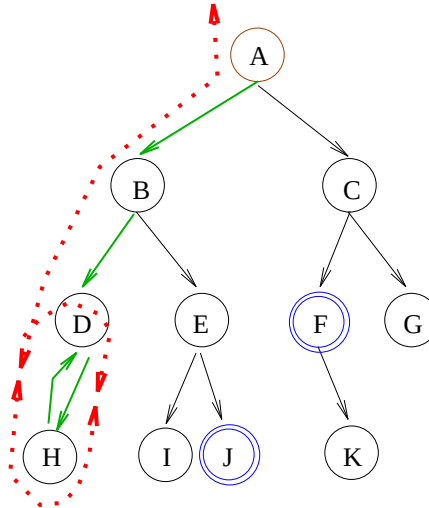


FIGURE 3.4 – Problème du parcours en profondeur

Pour éviter cela, il faudrait ajouter un système de mémorisation pour repérer les nœuds déjà parcourus ou les transitions. On utilise un marqueur ou une table annexe pour sauvegarder cette information.

3.2.2 Profondeur bornée

But : limiter la profondeur de la recherche. On effectue l'appel `ProfondeurB(Init, p, {})` sur l'algorithme suivant :

Exemple :

Le parcours en profondeur bornée va parcourir le graphe comme le parcours en profondeur mais en plus il limite la profondeur de la recherche (il n'ira pas systématiquement jusqu'à une feuille du graphe). Soit dans la figure 3.5 suivant :

Algorithme 2 : ProfondeurBornee(N, p, Sol)

Input : un nœud (N), une profondeur p

Output : renvoie Vrai s'il y a un chemin solution Sol et faux sinon.

$$\text{Sol} \leftarrow \text{Sol} \cup \{n\} ;$$

if n est solution **then** fin $\leftarrow$ Vrai;

else

```
fin ← Faux ;
```

$$n_1 \leftarrow \text{successeur}(n) ;$$
$$p \leftarrow p - 1;$$

while **non(fin)** et n_1 est valide et $p \geq 0$ **do**

$$\text{fin} \leftarrow \text{ProfondeurBornee}(N_1, p, \text{Sol}) ;$$
$$n_1 \leftarrow \text{successeur}(n) ;$$

if non(fin) **then** $\text{Sol} \leftarrow \text{Sol} \setminus \{n\};$

```
return fin;
```

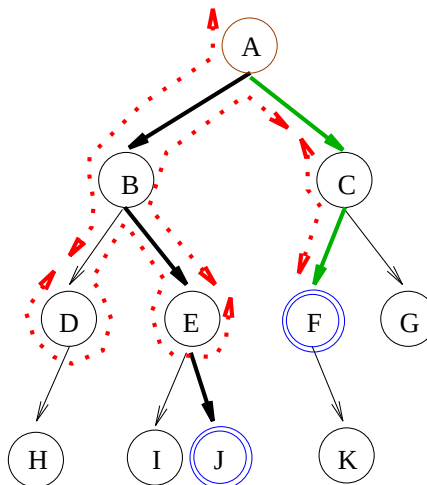


FIGURE 3.5 – Parcours en profondeur bornée

Les réponses obtenues avec $p = 2$ (la partie parcourue est en pointillé rouge sur le dessin) sont :

- A, B, D : Faux
- A, B, E : Faux
- A, C, F : Vrai

Problème : On peut ne pas trouver de solution alors qu'il en existe une à une profondeur plus grande.

3.2.3 Parcours en largeur

But : examiner tous les chemins en même temps. On effectue l'appel `Largeur ( {{Init}}, {} )` sur l'algorithme suivant :

Algorithme 3 : Largeur(Liste, Sol)**Input :** une liste de chemin**Output :** renvoie Vrai s'il y a un chemin solution Sol et faux sinon.

```

Ch ← Premier(Liste); fin ← Faux; while Ch ≠ {} et non(fin) do
  Liste ← Liste \ {Ch}; n ← dernierNoeud(Ch);
  n1 ← successeur(n);
  while non(fin) et n1 est valide do
    if n1 est solution then
      Sol ← Ch ∪f {n1} /* ∪f ajoute à la fin */
      fin ← Vrai;
    else Liste ← Liste ∪f {(Ch ∪f {n1})};
    n1 ← successeur(n);
  Ch ← Premier(Liste);
return fin;

```

Exemple : Le parcours en largeur va parcourir le graphe en proposant les différents chemins jusqu'à trouver une solution. Soit le graphe suivant :

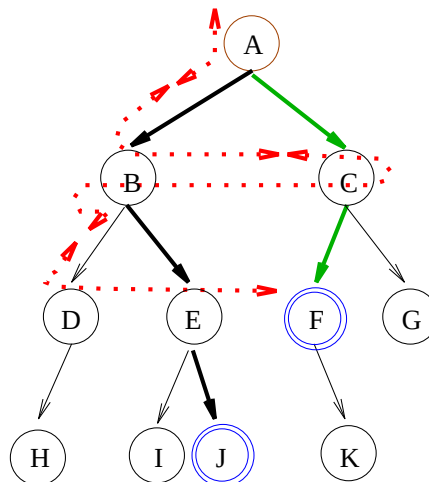


FIGURE 3.6 – Parcours en largeur

On peut aussi travailler avec une version permettant de limiter la profondeur de la largeur en introduisant une limite L à l'algorithme précédent.

Algorithme 4 : LargeurIt(Liste, L, Sol)**Input :** une liste de chemin, L la borne pour la profondeur**Output :** renvoie Vrai si un chemin solution Sol, faux sinon.Ch $\leftarrow$ Premier(Liste); fin $\leftarrow$ Faux; Liste₁ $\leftarrow$ {} ;**while** Ch $\neq$ {} **et non**(fin) **et** L > 0 **do**Liste $\leftarrow$ Liste $\setminus$ {Ch}; n $\leftarrow$ dernierNoeud(Ch);n₁ $\leftarrow$ successeur(n);**while non**(fin) **et** n₁ est valide **do****if** n₁ est solution **then**Sol $\leftarrow$ Ch $\cup_f$ {n₁} /* $\cup_f$ ajoute à la fin */;fin $\leftarrow$ Vrai;**else** Liste₁ $\leftarrow$ Liste₁ $\cup_f$ {(Ch $\cup_f$ {n₁})};n₁ $\leftarrow$ successeur(n);Ch $\leftarrow$ Premier(Liste);**if non**(fin) **et** (L > 0) **then** fin $\leftarrow$ LargeurIt(Liste₁, L-1, Sol);**return** fin;

Les réponses obtenues avec L = 2, l'appel initial est Largeur ({{A}}, 2, {}) (la partie parcourue est en pointillé rouge sur le dessin) sont :

- A : Faux
- A, B : Faux
- A, C : Faux
- A, B, D : Faux
- A, B, E : Faux
- A, C, F : Vrai

Problème : On doit mémoriser l'ensemble des chemins en cours de parcours.

3.2.4 Comparaison des approches

Pour comparer les différentes stratégies nous allons définir les critères sur lesquels nous sommes basés :

- **Complexité en temps** : combien de temps faut-il pour trouver la solution ?
- **Complexité en espace** : quel espace mémoire faut-il pour effectuer la recherche ?
- **Optimalité** : est-ce que la méthode trouve la meilleure solution s'il en existe plusieurs ?
- **Complétude** : est-ce que la méthode garantit de trouver une solution si elle existe ?

Méthode	complexité		Optimal	Complet
	Temps	Espace		
Largeur	$O(b^m)$	$O(b^m)$	oui	oui
Largeur Itératif	$O(b^m)$	$O(b.m)$	oui	oui
Profondeur	$O(b^m)$	$O(m)$	non	oui
Profondeur bornée	$O(b^l)$	$O(l)$	non	oui, si $l \leq p$
Bidirectionnel	$O(b^{\frac{m}{2}})$	$O(b^{\frac{m}{2}})$	oui	oui

- b : facteur de branchement du graphe
- m : profondeur maximal de l'arbre

- p : profondeur de la solution
- l : limite de la profondeur de recherche

Bilan : On vient de voir 3 méthodes pour explorer l'espace de recherche. Il demeure que le parcours en profondeur est plus facile et le moins coûteux en espace mais avec un risque de cycle (limiter recherche, marquage). Le parcours en Largeur est plus difficile à implanter et nécessite une gestion des listes des sous-chemins parcourus.

Nous ne pouvons pas prendre en compte les problèmes d'optimisation où l'on associe un coût aux arcs (transitions). Malgré le fait que le parcours en largeur donne la meilleure solution, on se heurte à l'explosion combinatoire en gardant les sous-chemins en mémoire. Pour tout cela, une nouvelle voie va être explorée avec l'utilisation d'heuristiques.

3.3 Parcours avec heuristiques

Parcours avec heuristique ou comment converger vers une solution ?

3.3.1 Définitions

La première question que nous pouvons nous poser : *"Que signifie heuristique ?"*

Définition du **"Petit Larousse"** :

"Heuristique vient du grec heuriskein, signifiant trouver. Discipline qui se propose de dégager les règles de la recherche scientifique et de la découverte."

Nous utiliserons ce mot pour définir le résultat de l'étude (les règles) permettant d'assurer la convergence de l'algorithme vers une solution pour un type de problème. Nous allons voir, que suivant le type d'heuristique, l'algorithme ne donnera pas toujours la meilleure solution.

Définition de fonction

Nous allons définir une fonction portant sur les nœuds du graphe u et v :

- La fonction $k(u,v)$ correspond au poids sur les arcs du graphe. Elle peut être prise identiquement nulle si on ne se préoccupe pas du rapport de la longueur sur le poids du chemin menant à la solution. Si on cherche le chemin comprenant le moins d'arcs possibles, on prendra $k(u,v) = 1$ pour tout arc (u,v) .
- La fonction $h(u)$ est une heuristique qui estime la distance entre l'état courant et l'état final. De façon formelle, $h(u)$ tente d'estimer le minimum, sur tous les chemins $(u, v_1, \dots, v_n, u_t)$, où u_t est un état terminal, de la somme :

$$k(u, v_1) + k(v_1, v_2) + \dots + k(v_n, u_t)$$

La valeur réelle de ce minimum est notée $h^*(u)$. On dégage plusieurs catégories de fonctions heuristiques h suivant l'approximation qu'elles font.

Catégories d'heuristiques

- Heuristique **parfaite** : une heuristique h est dite parfaite si et seulement si :

$$\forall u, v \quad h(u) = h(v) \leftrightarrow h^*(u) = h^*(v)$$

- Heuristique **presque parfaite** : une heuristique h est dite presque parfaite si et seulement si :

$$\forall u, v \quad h(u) < h(v) \rightarrow h^*(u) < h^*(v)$$

- Heuristique **monotone** : une heuristique h est dite monotone (ou consistante) si et seulement si :

$$\forall u \forall v \text{ descendant de } u \quad h(u) - h(v) \leq k(u, v)$$

- Heuristique **minorante** : une heuristique h est dite minorante (ou admissible) si et seulement si :

$$\forall u \quad h(u) \leq h^*(u)$$

On peut alors donner les propriétés des heuristiques :

- Si l'heuristique est *parfaite* alors l'algorithme *convergera* immédiatement vers le but.
- Une heuristique est propre à la catégorie du problème posée.
- Une heuristique peut définir une relation d'ordre partiel ou une relation d'ordre complet.
- Si l'heuristique est minorante alors l'algorithme A^* (présenté dans la section 3.4.2) trouvera toujours un chemin optimal.
- Une heuristique monotone est non seulement admissible, mais garantit dans le cadre de l'algorithme A^* , que pour tout nœud u développé, le chemin calculé est optimal ($g(u)$ est effectivement égal au coût du plus court chemin menant à u).

3.3.2 Les méthodes à base de gradient

Ces méthodes mélangent l'usage d'heuristique et d'un parcours. Ainsi le gradient est une variation au cours du temps. Il va correspondre à la progression vers la solution. Ainsi, on essaie d'atteindre un optimum global par une succession d'optimums locaux.

Par exemple, pour traverser une ville du nord au sud, on essaiera de minimiser ou de maximiser l'écart entre la position actuelle par rapport à la direction recherchée.

Il existe beaucoup d'algorithmes dans différents domaines qui mettent en œuvre ce principe :

- Algorithme du simplexe en programmation linéaire
- Méthode par approximations successives en analyse numérique
- Min-Max en programmation des jeux
- Algorithme A^* ...

Nous allons nous intéresser dans la leçon suivante à ces algorithmes dans le cas des graphes. On les regroupe sous l'appellation *Best-First*.

3.4 Best-First

Il fait parti des algorithmes effectuant des parcours et une heuristique. L'utilisation des heuristiques est faite pour converger plus rapidement qu'avec des algorithmes de parcours traditionnels. L'idée est de continuer la recherche avec le meilleur candidat à chaque étape.

Il existe un de nombreux algorithmes associés :

- Gourmand (Greedy) ou Escalade (hill-climbing)
- Algorithme A^*
- Variantes de A^* : IDA^* , SMA^* , AO^* ...

3.4.1 Algorithme Gourmand

C'est un algorithme de parcours en profondeur dans lequel on introduit une fonction d'heuristique pour choisir à chaque étape le nœud à générer, au lieu de générer un nœud en prenant une règle de production au hasard. Ici on choisit le nœud qui a la règle de production (arc) la moins coûteuse (dont la valeur sur l'arc est la plus petite). Un exemple est donné dans la figure 3.7. Ainsi, l'algorithme ne remet jamais en question son choix sauf dans le cas où il est bloqué comme avec le nœud U_8 (qui est un puit).

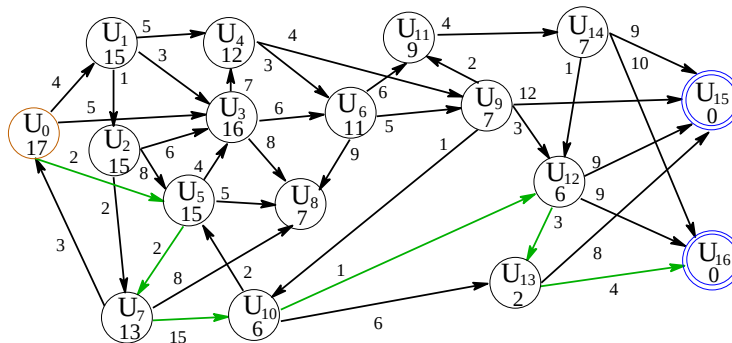


FIGURE 3.7 – Exemple d'algorithme gourmand

- Chemin solution : $\{U_0, U_5, U_7, U_{10}, U_{12}, U_{13}, U_{16}\}$
- Coût : $f(U_{16}) = 2 + 2 + 15 + 1 + 3 + 4 = 27$

3.4.2 Algorithme A^*

But : examiner tous les choix en même temps et faire le meilleur. On a ainsi un parcours entre la profondeur et la largeur.

Les données du problème :

- u_0 : l'état initial.
- T : l'ensemble des états terminaux.
- $P : \{p_1, p_2, \dots, p_n\}$: l'ensemble des règles de production.

- $h(u)$: u est un état, h est une fonction heuristique qui estime le coût de passage de u à l'état final.
- $k(u,v)$: u et v sont des états, k est une fonction qui donne le coût du passage par l'arc (u, v) .
- D : une liste contenant l'ensemble des états déjà développés (un état est développé si on a déjà généré l'ensemble de ses successeurs).
- G : une liste contenant l'ensemble des états générés non encore développés.
- $g(v)$: un tableau indexé par les états qui contient le coût du trajet pour atteindre l'état v .
- $f(v)$: un tableau indexé par les états qui contiendra pour chaque état le coût total estimé.
- $père(v)$: un tableau indexé par les états qui donne, pour chaque état, l'état qui l'a généré.
- $premier(G)$: fonction qui renvoie le premier élément de G .
- $insérer-selon-f(G,v)$: fonction qui insère l'état v dans G dans l'ordre croissant des valeurs f . En cas d'égalité sur f , on va privilégier la partie connue en faisant un sous-tri croissant sur les valeurs de h .

Algorithme 5 : Algorithme : $A^*(Liste, Sol)$

Input : le graphe

Output : $g(u_t)$ et on utilise $père(u_t)$ pour reconstruire le chemin.

$G \leftarrow \{u_0\}$; /* on initialise G avec l'ensemble des états initiaux */

$D \leftarrow \emptyset$;

$g(u_0) \leftarrow 0$;

$père(u_0) \leftarrow \emptyset$;

$u \leftarrow premier(G)$;

while ($G \neq \emptyset$) **et** ($u \notin T$) **do**

$G \leftarrow G \setminus \{u\}$;

$D \leftarrow D \cup \{u\}$;

$v \leftarrow suivant(u)$;

while ($v \neq null$) **do**

if ($(v \notin (D \cup G))$ **ou** ($g(v) > g(u) + k(u,v)$)) **then**

$g(v) \leftarrow g(u) + k(u,v)$;

$f(v) \leftarrow g(v) + h(v)$;

$père(v) \leftarrow u$;

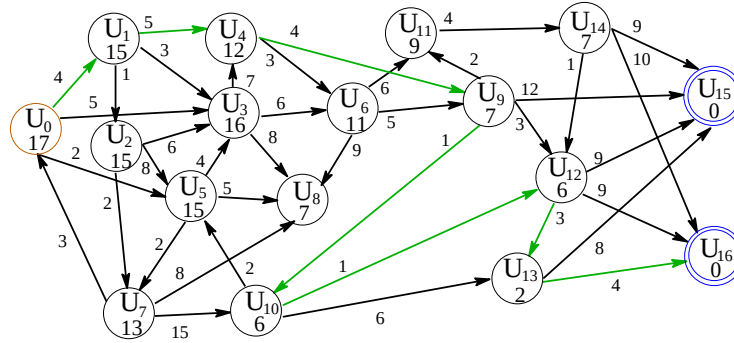
$insérer-selon-f(G,v)$;

$v \leftarrow suivant(u)$;

$u \leftarrow premier(G)$;

Exemple Algorithme A^*

L'algorithme A^* va parcourir le graphe en prenant le chemin correspondant au coût minimal qui lui est donné par son heuristique (valeur donnée dans les nœuds du graphe sous le nom du nœud). En final, l'algorithme produit le chemin illustré dans la figure 3.8. Pour un nœud u donné, $g(u)$ donne le coût réel (valeur des arcs emprunté pour arriver de l'état initial au nœud u et $f(u)$ donne la somme de la valeur heuristique et du coût réel donc $f(u) = g(u) + h(u)$.

FIGURE 3.8 – Exemple de l’algorithmique A^*

u	g(u)	f(u)	père(u)	Ensemble G constitué de nœud u(coût_init : g(u), coût_fin : f(u))
u_0	0	17	$\emptyset$	$u_5(2, 17)u_1(4, 19)u_3(5, 21)$
u_5	2	17	u_0	$u_8(7, 14)u_7(4, 17)u_1(4, 19)u_3(5, 21)$
u_8	7	14	u_5	$u_7(4, 17)u_1(4, 19)u_3(5, 21)$
u_7	4	17	u_5	$u_1(4, 19)u_3(5, 21)u_{10}(19, 25)$
u_1	4	19	u_0	$u_2(5, 20)u_4(9, 21)u_3(5, 21)u_{10}(19, 25)$
u_2	5	20	u_1	$u_4(9, 21)u_3(5, 21)u_{10}(19, 25)$
u_4	9	21	u_1	$u_9(13, 20)u_3(5, 21)u_6(12, 23)u_{10}(19, 25)$
u_9	13	20	u_4	$u_{10}(14, 20)u_3(5, 21)u_{12}(16, 22)u_6(12, 23)u_{11}(15, 24)u_{15}(25, 25)$
u_{10}	14	20	u_9	$u_{12}(15, 21)u_3(5, 21)u_{13}(20, 22)u_6(12, 23)u_{11}(15, 24)u_{15}(25, 25)$
u_{12}	15	21	u_{10}	$u_{13}(18, 20)u_3(5, 21)u_6(12, 23)u_{15}(24, 24)u_{16}(24, 24)u_{11}(15, 24)$
u_{13}	18	20	u_{12}	$u_3(5, 21)u_{16}(22, 22)u_6(12, 23)u_{15}(24, 24)u_{11}(15, 24)$
u_3	5	21	u_0	$u_{16}(22, 22)u_6(11, 22)u_{15}(24, 24)u_{11}(15, 24)$

Le nœud suivant est u_{16} , c’est un nœud terminal. On obtient ainsi la solution en utilisant la fonction père à partir de celui-ci jusqu’à remonter à la ravine u_0 . Pour le coût on utilise la valeur $g(u_{16})$:

- Chemin solution : $\{u_0, u_1, u_4, u_9, u_{10}, u_{12}, u_{13}, u_{16}\}$
- Coût : $f(u_{16}) = 22$

Remarque : La différence avec l’algorithme gourmand vient de la remise en question du chemin courant à chaque nouveau choix pour le nœud suivant. Ainsi l’algorithme peut passer d’un chemin à un autre plusieurs fois avant d’arriver à la solution. Alors que l’algorithme gourmand ne le fera jamais (car il ne remet jamais en question le choix du nœud et par cela même le chemin courant).

Complexité

La complexité en temps de calcul de l’algorithme :

- Dans le pire des cas, la complexité est en 2^N , où N est le nombre de nœuds du graphe, puisque chaque état u sera développé autant de fois qu’il y a de chemins de u_0 à u .
- Si h est une heuristique minorante, on peut, en modifiant légèrement l’algorithme, avoir une complexité en N^2 .

Cependant, il faut remarquer que même une complexité linéaire sur le nombre d'états est clairement inutilisable dans la plupart des cas pratiques. Ainsi, le problème du voyageur de commerce a un nombre d'états en $N = n!$, où n est le nombre de villes.

La complexité en taille (nombre de nœuds que l'on considère en fonction du nombre d'arcs entre l'état initial et un état final, on le note M) :

- Si h est *presque parfaite*, alors la complexité est linéaire en M .
- Si h est *monotone*, avec un arbre dont le facteur de branchement est K , alors la complexité est exponentielle en K^M .
- Pour une heuristique *minorante*, toujours pour un arbre K -aire, on peut déterminer la complexité de l'algorithme en fonction de l'écart entre h et h^* .

Notons qu'un certain nombre de résultats à caractère général montre que si on ne dispose pas d'une "*très bonne*" heuristique (cas le plus fréquent), l'algorithme A^* développera en moyenne un nombre exponentiel d'états en fonction de la longueur optimale du chemin.

Remarque : L'algorithme A^* est extrêmement populaire malgré les résultats de complexité ci-dessus. En effet, il est généralement simple à implémenter et amplement suffisant pour nombre de problèmes de tailles réduites. D'autre part, on sait qu'il est "optimal", c'est-à-dire qu'il fait "au mieux" avec les informations dont il dispose. Pour résoudre les problèmes de consommation mémoire on peut lui ajouter des algorithmes de recherche en profondeur itératif (comme on le verra avec l'algorithme $\alpha - \beta$ présenté dans le chapitre suivant).

3.4.3 Variant A^*

Nous allons nous intéresser à l'utilisation de l'approche de l'algorithme A^* pour une type de graphe différent. En effet, les arcs d'un graphe ne sont plus vus comme des choix indépendants mais comme devant être effectués ensemble. C'est ainsi qu'est apparu le graphe ET-OU.

Ces graphes sont utilisés dans le cas où le problème peut se décomposer en éléments mutuellement indépendants.

Si on prend une formule logique représentant un problème, le graphe ET-OU associé à celle-ci sera donnée par la figure 3.9.

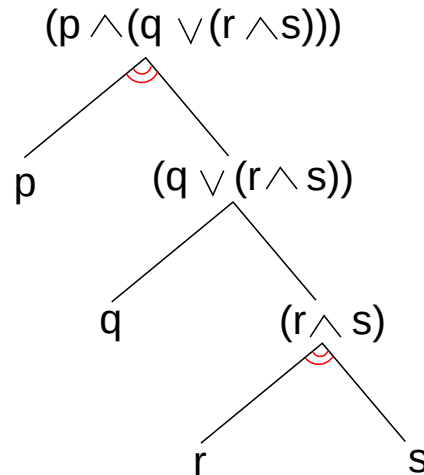


FIGURE 3.9 – Exemple Graphe ET-OU

On redéfinit la façon d'utiliser les heuristiques suivant le type de nœud :

- Pour un nœud ET : $h(u) = \sum_{v \in \text{Suivant}(u)} (k(u, v) + h(v))$
- Pour un nœud OU : $h(u) = \min_{v \in \text{Suivant}(u)} (k(u, v) + h(v))$

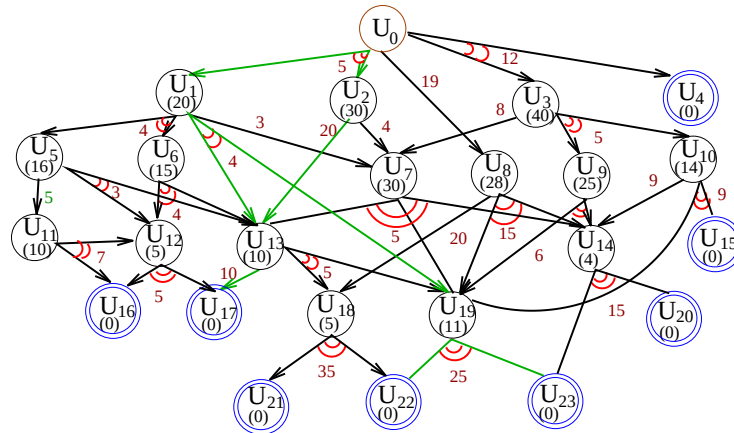
Les données du problème :

- Q : ensemble des états développés (nœuds visités).
- P : ensemble des états pendants (engendrés mais non encore développés).
- $G(u_0)$: meilleure solution partielle connue à l'instant courant.
- A : ensemble des états dont l'estimation f est actualisée.
- $I_0(u)$: sous-ensemble des connecteurs $S_i(u)$ dont l'estimation $f(u)$ devra être mise à jour.

A cela s'ajoute les éléments suivants :

- $I(u)$ sont les groupes de transition issus de u .
- $S_i(u)$ sont le groupe de transitions entre le nœud i et u .
- $f_i(u)$ calcul du coût pour chaque groupe i .
- Définir_Solution permet de donner le graphe qu'il faut étendre pour continuer le parcours. Il reprend à partir des états initiaux chaque groupe de fils qui a été choisi.
- Choisir_Etat permet de déterminer dans le graphe donné par Définir_Solution le nœud qu'il faut étendre avec l'algorithme.

Algorithme 6 : Algorithme : $OA^*(Liste, G(u_0))$ **Input :** le graphe et Liste contenant les nœuds initiaux**Output :** Si $f(u_0) = \infty$ alors il n'y a pas de solution sinon la solution est dans $G(u_0)$ de coût $f(u_0)$. $P \leftarrow \text{Premier(Liste);}$ $Q \leftarrow \emptyset;$ $f(u_0) \leftarrow h(u_0);$ $u \leftarrow u_0;$ **while** ($u \neq \text{null}$) **et** ($f(u_0) \neq \infty$) **do** $P \leftarrow P \setminus \{u\};$ $Q \leftarrow Q \cup \{u\};$ **for** $i \in I(u)$ **do** $v \leftarrow \text{premier}(S_i(u));$ **while** $v \neq \emptyset$ **do** **if** $\text{non}(v \in P \cup Q)$ **then** $P \leftarrow P \cup \{v\};$ $f(v) \leftarrow h(v);$ $v \leftarrow \text{suivant}(S_i(u));$ $f_i(u) \leftarrow k(u, i) + \sum_{v \in S_i(u)} f(v);$ **if** $I(u) = \emptyset$ **then** $f(u) \leftarrow \infty;$ **else** $f(u) \leftarrow \min\{f_i(u) \mid i \in I(u)\};$ $I_{\min}(u) \leftarrow \text{indice_du_minimum};$ $A \leftarrow S^{-1}(u);$ **for** $w \in S^{-1}(u)$ **do** $I_0(w) \leftarrow \{i \in I(w) \mid u \in S_i(w)\};$ **while** $A \neq \emptyset$ **do** Choisir $v \in A$ tel que $A \cap S(v) = \emptyset;$ $A \leftarrow A \setminus \{v\};$ **for** $i \in I_0(v)$ **do** $f_i(v) \leftarrow k(v, i) + \sum_{w \in S_i(v)} f(w);$ **if** $f_{I_{\min}}(v) \neq \min\{f_i(v) \mid i \in I(v)\}$ **then** $I_{\min}(v) \leftarrow \text{indice_du_minimum};$ **if** $f(v) \neq f_{I_{\min}}(v)$ **then** $f(v) \leftarrow f_{I_{\min}}(v);$ $A \leftarrow A \cup S^{-1}(v);$ **for** $w \in S^{-1}(v)$ **do** $I_0(w) \leftarrow I_0(w) \cup \{i \in I(w) \mid v \in S_i(w)\}$ $I_0(v) \leftarrow \emptyset;$ $G(u_0) \leftarrow \text{Définir_Solution}(u_0);$ $u \leftarrow \text{Choisir_Etat}(G(u_0));$ **Exemple :**

FIGURE 3.10 – Exemple de graphe AO^*

Le parcours en profondeur va parcourir le graphe en proposant différents chemins jusqu'à trouver une solution. Soit le graphe donné par la figure 3.10.

Dans le tableau ci-dessous, on note en gras le chemin à développer et en **bleu** les valeurs modifiées. Il faut faire attention à la mise à jour des coûts associés aux nœuds. Par exemple, lorsque u_{19} prend la valeur 25 qui vient remplacer la valeur de l'heuristique, il faut utiliser cette valeur pour tous les nouveaux calculs.

u	h(u)	f(u)	Etats actualisés après développement de u	Arbre à explorer	Etat à explorer après u
u_0		(55,47,52)	$\emptyset$	$u_0 \rightarrow u_8$	u_8
u_8	28	(25,30)	$u_0(55,44,52)$	$u_0 \rightarrow u_8 \rightarrow u_{18}$	u_{18}
u_{18}	5	(35)	$u_8(55,30) u_0(55,49,52)$	$u_0 \rightarrow u_8 \rightarrow u_{19}$ $\searrow$ u_{14}	$u_{19}; u_{14}$
u_{19}	11	(25)	$u_8(55,44) u_0(55,63,52)$	$u_0 \rightarrow u_3$	u_3
u_3	40	(38,44)	$u_0(55,63,50)$	$u_0 \rightarrow u_3 \rightarrow u_7$	u_7
u_7	30	(44)	$u_3(52,44) u_0(55,63,56)$	$u_0 \rightarrow u_1$ $\searrow$ u_2	$u_1; u_2$
u_1	20	(35,39,47)	$u_0(70,63,56)$	$u_0 \rightarrow u_3 \rightarrow u_9$ $\searrow$ u_{10}	$u_9; u_{10}$
u_9	25	(35)	$u_3(52,54) u_0(70,63,64)$	u_{19} $\nearrow$ $u_0 \rightarrow u_8 \rightarrow u_{14}$	u_{14}
u_{14}	4	(15)	$u_9(46) u_8(55,55) u_7(55)$ $u_1(35,39,58) u_3(63,65)$ $u_0(70,74,75)$	u_5 $\nearrow$ $u_0 \rightarrow u_1 \rightarrow u_6$ $\searrow$ u_2	$u_5; u_6; u_2$
u_5	16	(15,18)	$u_1(34,39,58)$ $u_0(69,74,75)$	$u_5 \rightarrow u_{11}$ $\nearrow$ $u_0 \rightarrow u_1 \rightarrow u_6$ $\searrow$ u_2	$u_{11}; u_6; u_2$
u_{11}	10	(12)	$u_5(17,18) u_1(36,39,58)$ $u_0(71,74,75)$	$u_{11} \rightarrow u_{12}$ $\nearrow$ u_5 $\nearrow$ $u_0 \rightarrow u_1 \rightarrow u_6$ $\searrow$ u_2	$u_{12}; u_6; u_2$
u_{12}	5	(5)	$u_{11}(12) u_5(17,18)$ $u_1(36,39,58)$ $u_0(71,74,75)$	$u_{11} \rightarrow u_{12} \rightarrow u_{17}$ $\nearrow$ u_5 $\nearrow$ $u_0 \rightarrow u_1 \rightarrow u_6$ $\searrow$ u_2	$u_6; u_2$
u_6	15	(19)	$u_1(40,39,58)$ $u_0(74,74,75)$	$u_{19} \rightarrow u_{22}$ $\nearrow$ $u_0 \rightarrow u_1 \rightarrow u_{13}$ $\searrow$ u_2	$u_{13}; u_2$
u_{13}	10	(10,65)	$u_1(40,39,58)$ $u_0(74,74,75)$	$u_{19} \rightarrow u_{22}$ $\nearrow$ $u_0 \rightarrow u_1 \rightarrow u_{13}$ $\searrow$ u_2	u_2
u_2	30	(5,30)	$u_0(74,74,75)$	$u_{19} \rightarrow u_{22}$ $\nearrow$ $u_0 \rightarrow u_1 \rightarrow u_{13}$ $\searrow$ u_2	$\emptyset$

En reprenant la liste des nœuds qui ont permis d'avoir chacune des meilleures valeurs :

- Chemin solution : $\{u_0, u_1, u_2, u_{13}, u_{17}, u_{19}, u_{22}, u_{23}\}$
- Coût : $f(u_0) = 74$

3.5 Exercices chapitre 3

Exercice 1 - Parcours en profondeur du graphe d'états

Il s'agit de définir les règles Prolog qui permettent de trouver un chemin dans un labyrinthe en utilisant la stratégie de parcours de l'espace de recherche en profondeur.

Un labyrinthe est représenté par un échiquier dont les cases sont numérotées par ordre croissant de gauche à droite, du haut vers le bas (cf. Figure 3.11).

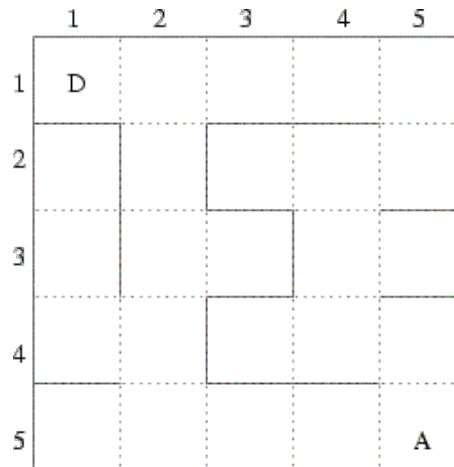


FIGURE 3.11 – Le labyrinthe

Questions :

1. Définir les insertions $\text{frontiere}(C_1, C_2)$ qui permettent d'exprimer qu'il existe une cloison entre deux cases C_1 et C_2 , ainsi que le départ (D) et arrivée (A) qui permettent de distinguer la case de départ D et d'arrivée A .
2. Écrire la règle $\text{caseCorrecte}(C)$ qui renvoie vrai si les coordonnées d'une case C sont valides.
3. Écrire le prédicat $\text{deplacementCorrect}(C_1, C_2)$ qui renvoie vrai si le déplacement de la case C_1 à la case adjacente C_2 est valide.
4. Écrire le prédicat $\text{deplace}(Ch, NvCh)$ qui permet de calculer les positions suivantes dans le labyrinthe. On fera une clause par direction. Ch contient la liste des cases déjà parcourues tandis que $NvCh$ représente cette même liste à laquelle on a ajouté la nouvelle position.
5. Définir le prédicat récursif $\text{parcours}(Ch, Sol)$ qui permet de parcourir le labyrinthe. Ch correspond à la liste de cases déjà parcourues tandis que Sol correspond à la liste solution (celle qui nous amène à l'état final).
6. Définit le prédicat d'appel $\text{labyrinthe}(Sol)$ qui renvoie la liste Sol des cases du labyrinthe par lesquelles il faut passer pour résoudre le problème.

Correction.

Exercice 2 - Parcours en largeur du graphe d'états

Il s'agit de définir les règles Prolog qui permettent de trouver un chemin dans un labyrinthe en utilisant la stratégie de parcours de l'espace de recherche en largeur.

Un labyrinthe est représenté par un échiquier dont les cases sont numérotées par ordre croissant de gauche à droite, du haut vers le bas (cf. Figure 3.11)

Questions : en réutilisant les quatre premières questions de l'exercice 1.

1. Définir le prédicat `etageSuivant (LL, Acc, LLR)` qui calcule, pour une liste de parcours LL de profondeur n tous les parcours suivants de profondeur $n + 1$ dans LLR .
2. Écrire le prédicat `etatFinal (LL, LR)` qui renvoie, à partir d'une liste de parcours LL , un à un les parcours LR qui mènent à la case d'arrivée. S'il n'existe aucun parcours de ce type ou si, le cas échéant, tous les parcours en question ont été retournés, ce prédicat échoue.
3. Définir le prédicat récursif `largeur (LL, LR)` qui permet de parcourir le labyrinthe en largeur. La variable LL correspond à la liste des parcours d'une même profondeur, tandis que LR correspond à une solution potentielle.
4. Définir le prédicat d'appel `labyrinthe (Sol)` qui renvoie la liste Sol des cases du labyrinthe par lesquelles il faut passer pour résoudre le problème.

Correction.

Exercice 3 - Parcours avec Heuristique : Meilleur d'abord

Il s'agit de définir les règles Prolog qui permettent de trouver un chemin dans un labyrinthe en utilisant une heuristique pour un parcours de l'espace de recherche.

Un labyrinthe est représenté par un échiquier dont les cases sont numérotées par ordre croissant de gauche à droite, du haut vers le bas (cf. Figure 3.11)

Questions : en réutilisant les questions des exercices 1 et 2 :

1. Définir la règle `cout (Case, Cout)` qui permet d'exprimer le coût de passage de la case $Case$ vers la sortie sans tenir compte des frontières (`cout (Case, Cout)` sera considéré comme une heuristique).
2. Écrire la règle `size (Ch, G)` qui renvoie le coût du chemin déjà parcouru.
3. Écrire les prédicats `deplace (Ch, NvCh, Cout)` qui permettent de calculer les positions suivantes dans le labyrinthe avec leurs coûts $Cout$ respectifs. Ch contient la liste des cases déjà parcourues. $NvCh$ représente cette même liste à laquelle on a ajouté la nouvelle position. $Cout$ représente le coût réel du nouveau chemin $NvCh$ auquel on a ajouté une estimation de ce qu'il reste à parcourir.
4. Définir le prédicat récursif `parcoursSuivant (X, Acc, R)` qui calcule, pour un parcours X de profondeur n tous les parcours suivants de profondeur $n + 1$ dans R , de la façon suivante :

$$[[Cout_1, Ch_1], [Cout_2, Ch_2], \dots [Cout_m, Ch_m]]$$
5. Définir le prédicat `insere (L1, L2, R)` qui insère les éléments de la liste L_1 dans la liste L_2 avec le résultat R . Cette insertion s'effectue en deux étapes :

- (a) par ordre croissant du coût d'un chemin auquel on a ajouté une heuristique,
- (b) par ordre décroissant du coût réel d'un chemin, si l'ordre précédent donne une égalité.

Définir le prédicat `insereC(L, Ch, R)` correspondant à ces étapes qui insère *Ch* où bon endroit dans la liste *L* pour donner le résultat *R*.

6. Définir le prédicat récursif `heuristique(L, Ch)` qui permet de calculer à l'aide d'une recherche avec heuristique le chemin avec le meilleur coût. La variable *L* correspond à la liste des parcours (accompagnée du coût), tandis que *Ch* correspond à une solution potentielle.
7. Définir le prédicat d'appel `labyrinthe(Sol)` qui renvoie la liste *Sol* des cases du labyrinthe par lesquelles il faut passer pour résoudre le problème.

Correction.

Exercice 4 - Boites

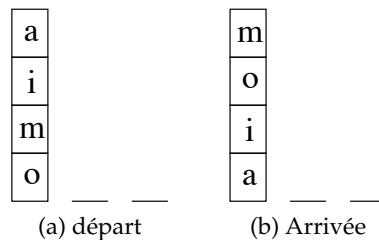


FIGURE 3.12 – Les configurations

En utilisant une technique de parcours avec heuristique, vous devez réaliser le programme prolog qui permet d'effectuer la manipulation des cubes pour passer de la configuration de départ (figure 3.12a) à la configuration d'arrivée (figure 3.12b). Les règles de déplacements sont les suivantes :

- Déplacement d'un cube n'ayant pas de cube au-dessus de lui ;
- On pose un cube au-dessus de l'une des autres colonnes (sans restriction) ;
- Les lettres sont uniques.

A partir des prédicats définis dans le cours, faire les modifications pour pouvoir traiter le problème :

1. `depart/1` : décrit la configuration de départ (on utilisera une liste contenant une liste des cases par colonne).
2. `arrivee/1` : décrit la configuration d'arrivée.
3. `cout(+Conf, ?Cout)` : calcul la différence (hauteur et nombre de colonne) entre la configuration `Conf` avec la configuration d'arrivée. Pour cela, on définira un prédicat `place(+X, +Conf, ?Cout)` qui pour la lettre *X* calcule la différence par rapport à l'arrivée et la configuration `Conf`, résultat dans `Cout`. On utilisera le prédicat `hauteur(+X, +C, ?Acc, ?H, ?R)`, qui pour une lettre *X* et une colonne *C*, calcule la hauteur où la lettre se situe dans celle-ci (on utilisera *Acc* comme accumulateur), si elle n'est pas présente alors *H* vaut 0 et *R* aussi, sinon *H* donne cette hauteur et *R* vaut 1. Par exemple :

- `cout ([[o,a],[i],[m]], Cout)` . donne $Cout = 1+0+2+5 = 8$
 - `place(m, [[o,a],[i],[m]], Cout)` . donne $Cout = \text{abs}(1-(1*0+2*0+3*1))+\text{abs}(4-1) = 5$
 - `hauteur(m, [o,a], _, H,R)` . donne $H = 0, R = 0$
 - `hauteur(m, [m], _, H,R)` . donne $H = 1, R = 1$
4. `size(+Confs, ?Dis)` : Calcule le nombre de configurations réalisés dans `Confs` depuis l'état initial et met le résultat dans `Dis`.
 5. `deplace(+Confs, -NConfs, -Cout)` : permet de calculer les configurations suivantes avec leurs coûts `Cout` respectifs. `Confs` contient la liste des configurations déjà calculées. `NConfs` représente cette même liste à laquelle on a ajouté la nouvelle configuration. `Cout` représente la coût réel du nouveau chemin `NConfs` auquel on a ajouté une estimation de ce qu'il reste à parcourir. On fera les déplacements dans l'ordre des colonnes ($1 \rightarrow 2, 1 \rightarrow 3, 2 \rightarrow 1, 2 \rightarrow 3, 3 \rightarrow 1, 3 \rightarrow 2$).
 6. `supprime(+CoutConfs, +SetCConfs, ?NCConfs, ?NSetCConfs)` : vérifie si le dernier élément inséré de la configuration de `CoutConfs` (liste de couple [cout,configuration]) est identique à l'un des derniers éléments de chacune des listes de configurations contenue dans `SetCConfs`. Si oui alors :
 - Si le coût de `CoutConfs` est inférieur alors on supprime cette liste de configurations de `SetCConfs` et on renvoie le résultat de cette suppression dans `NSetCConfs` et `NCConfs` est égal à `CoutConfs`.
 - Si le coût de `CoutConfs` est supérieur alors on supprime cette liste de configurations. On renvoie la liste vide pour `NCConfs` et `NSetCConfs` est égal à `SetCConfs`.
 7. `supprimeAll(+SetCConfs1, +SetCConfs2, ?NSetCConfs1, ?NSetCConfs2)` : prend chaque liste de configurations de `SetCConfs1` et supprime les doublons avec `SetCConfs2` à l'aide du prédicat `supprime`, ne sachant pas où la suppression aura lieu en renvoie `NSetCConfs1` la liste issue de `SetCConfs1` et `NSetCConfs2` la liste issue de `SetCConfs2` après les suppressions.
 8. `insere` : ajoutez dans le traitement de ce prédicat le fait que le prédicat `supprimeAll` fait apparaître des listes de configurations vide.
 9. `heuristique` : modifier le prédicat `heuristique` pour qu'il prenne en compte `supprimeAll`.
- Les autres prédicats restent inchangés.

Questions :

- 1..9. Donner les prédicats définis dans les 9 points précédents nécessaires pour résoudre le problème.
10. Donner une solution.

[Correction.](#)

4. Programmation par contraintes

Nous présentons dans ce chapitre la programmation par contraintes. Ce type de programmation est apparu dans les années 1970, mais n'a pas rencontré à cette époque un franc succès. Il faudra attendre les années 1980 pour voir apparaître les premiers algorithmes efficaces pour la gestion et la propagation des informations sur les contraintes. Dans les années 1990, on voit apparaître les premiers langages (Prolog et différentes bibliothèques C, C++, Java) permettant l'écriture de problèmes sous forme de contraintes numériques à valeurs entières. Puis les solveurs de contraintes ont évolué pour gérer les flottants, les ensembles abstraits, les booléens, ...

- Dans la [leçon 1](#), nous allons définir quel est l'intérêt des contraintes et comment on les définit.
- Dans la [leçon 2](#), nous présenterons les algorithmes pour effectuer la résolution d'un système de contraintes.
- Dans la [leçon 3](#), nous présenterons les systèmes pour rendre plus efficace la résolution, en utilisant les notions de consistance pour maintenir la cohérence dans le système et les stratégies de traitements.

Les différentes approches de consistance et de propagation sont données à fin de bien comprendre les mécanismes mis en /oeuvre. Dans le cadre de cet enseignement, seul l'aspect utilisation de ces techniques nous intéresse. Ainsi, il faut comprendre l'usage (la transformation d'un problème sous la forme de contraintes) et l'impact de l'ajout des contraintes sur le système pour obtenir ainsi la/les solution(s) au problème.

4.1 Définitions des contraintes

On peut voir les contraintes comme une forme de systèmes d'équations. Cette approche a été proposée pour compléter les approches classiques, comme nous avons pu le voir dans le chapitre précédent, car elles se heurtent souvent à des problèmes comme :

- Comment déduire des informations pertinentes.
- Comment éviter les boucles infinies.
- Comment trouver la bonne heuristique.
- Comment faire la bonne abstraction.

Par exemple, comment faire pour résoudre des problèmes aussi variés que des équations, le problème du placement des reines ou l'ordonnancement des tâches.

Pour cela il faut :

- Définir les contraintes par rapport aux variables du système.
- Chercher une solution satisfaisant toutes les contraintes du problème.

Finalement, qu'est-ce qu'une contrainte à quoi cela sert-il :

- Définir et réduire le domaine des variables



- Donner des relations entre les inconnues (variables)

Les contraintes sont caractérisées par :

- **Information partielle** (on ne donne qu'une borne à la variable) : $X > 2$
- **Hétérogénéité**, on peut mélanger les types des éléments manipulés (des entiers avec des ensembles par exemple) : $N = \text{Cardinalité_de}(S)$
- **Indirectionnelle**, l'inconnue peut être à droite ou à gauche de l'égalité :
L'équation $X = Y + 2$ peut se réécrire sous deux formes :
 1. $X \leftarrow Y + 2$
 2. $Y \leftarrow X - 2$
- **Déclaration Additive**, l'ordre d'ajout des contraintes n'a pas d'importance. On obtient le même résultat en posant :
 - $X > 2$ puis $X < 5$
 - Ou $X < 5$ puis $X > 2$
- Rarement **indépendantes**, les variables apparaissent dans différentes contraintes : $A + B = 5$; $A - C = 10$

4.1.1 Constraints Satisfaction Problem

Les problèmes qui utilisent les contraintes sont appelés **P**roblème de **S**atisfaction de **C**ontraintes ou **CSP** en anglais. Il est défini par un quadruplet (X, D, C, R) :

- $X = \{x_1, \dots, x_n\}$, un ensemble de n variables
- $D = \{D_1, \dots, D_n\}$, un ensemble de domaines avec $x_i \in D_i$
- $C = \{C_1, \dots, C_m\}$, un ensemble de m contraintes
- A chaque contrainte C_i est associée une relation R_i entre les variables de la contrainte.

On cherche alors à répondre à différents types de questions ;

- Le système est-il **satisfiable** (A-t-il une solution) ?
- Le système est-il **non-satisfiable** (Il n'y a pas de solution) ?
- Donner les **solutions** du système (Enumérer les valeurs compatibles de l'ensemble des variables) ?
- Les contraintes sont-elles **équivalentes** (Ont-elles le même ensemble de solutions) ?
- La contrainte C_1 implique la contrainte C_2 (Les solutions de C_1 constituent un sous-ensemble des solutions de C_2) ?

4.2 Méthodes de résolution

Le but de la **résolution** est de trouver les solutions aux systèmes. Pour cela, il existe deux méthodes pour la résolution :

- La méthode dite brutale : **Générer et Tester**.
- Une méthode avec réflexion : **Retour en arrière (Backtracking)** en anglais).

Les méthodes de résolution doivent :

- Enumérer les valeurs à partir du domaine des variables.
- Vérifier la compatibilité de ces valeurs avec les contraintes.

4.2.1 Générer et Tester

Le principe de l'approche Générer et Tester est simple. Ceci lui permet d'être une méthode de résolution générale et systématique. Il est basé sur cet algorithme :

- Génération de valeurs (Labeling) pour toutes les variables
- Test de la satisfaction

Exemple

Soient les contraintes suivantes :

- Définition des domaines : $X \in \{1, 2\}$, $Y \in \{1, 2\}$, $Z \in \{1, 2\}$
- Relations entre variables : $X = Y$, $X \neq Z$, $Y > Z$

Résultat obtenu (on s'arrête à la première solution) :

Enumération des valeurs			Test de
X	Y	Z	consistance
1	1	1	Fail
1	1	2	Fail
1	2	1	Fail
1	2	2	Fail
2	1	1	Fail
2	1	2	Fail
2	2	1	Ok

Bilan

Il possède les inconvénients suivants :

- Génération aveugle et systématique
- Inconsistance découverte au dernier moment (lorsqu'on teste la satisfaction)

Les améliorations possibles sont :

- Générateur avec de l'intelligence : **Recherche locale** pour savoir quelles sont les valeurs encore possible pour les autres variables
- Tester l'inconsistance dans le générateur : **Backtracking**, on essaye une valeur choisie pour une variable au plus tôt.

4.2.2 Backtracking

Le principe est d'étendre incrémentalement une solution partielle en solution complète.

Algorithme 7 : Algorithme : BT(V,A)

Input : V une liste de variables à instancier

Output : A une instantiation des variables, initialisée à vide.

if $V = \emptyset$ **then** A est une solution;

else

 Soit $x_i \in V$;

for $v \in D_i$ **do**

if $A \cup \{x_i = v\}$ est consistant avec le problème **then**

 BT($V \setminus \{x_i\}, A \cup \{x_i = v\}$);

L'intérêt de cette méthode est de réduire l'énumération des valeurs des autres variables à partir du moment on l'algorithme détecte une inconsistance. Ainsi on évite un travail inutile et redondant. Le problème est qu'il est possible que la détection de l'inconsistance reste tardive.

Par exemple, soient les contraintes suivantes :

- Définition des domaines : $A \in \{1, 2\}, B \in \{1, 2\}, C \in \{1, 2\}, D \in \{1, 2\}$
- Relations entre variables : $A = D, B \neq D, A + C < 4$

Ainsi, le choix de l'ordre des variables pour la décision des valeurs est important car tant que D n'a pas pris de valeur, l'inconsistance entre les valeurs de B et de D ne peut être détectée comme le montre la figure 4.1.

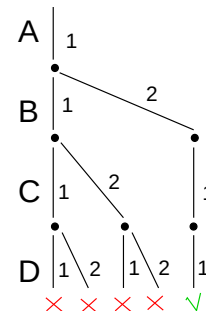


FIGURE 4.1 – Exemple backtracking

Exemple

Soient les contraintes suivantes :

- Définition des domaines : $X \in \{1, 2\}, Y \in \{1, 2\}, Z \in \{1, 2\}$
- Relations entre variables : $X = Y, X \neq Z, Y > Z$

Résultat obtenu, on met en gras les valeurs des variables qui permettent de conclure (Fail ou Ok) :

Enumération des valeurs			Test de
X	Y	Z	consistance
1	1	1	Fail
1	1	2	Fail
1	2		Fail
2	1		Fail
2	2	1	Ok

Bilan

Le choix des variables reste crucial même si le backtracking semble le plus efficace. Généralement les algorithmes utilisent, pour choisir la variable à instancier, des heuristiques du type :

- Explorer une variable de plus petit domaine.
- Explorer une variable portant le plus de contraintes.
- Explorer une variable appartenant à la contrainte la plus difficile à satisfaire.

4.3 Consistances - Enumérations

Nous allons voir les différentes techniques de consistances qui permettent de filtrer le domaines des variables. Puis, nous présenterons comment énumérer les valeurs restantes dans les domaines.

4.3.1 Techniques de consistance

Le but de ces techniques est de garder les valeurs cohérentes des contraintes. Pour cela, on retire les valeurs inconsistantes des domaines des variables. Le CSP est représenté sous la forme d'un graphe (cf. figure 4.2), où :

- les nœuds sont les variables
- les arêtes sont les contraintes

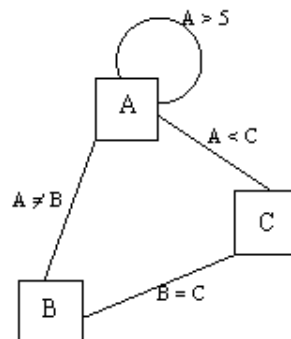


FIGURE 4.2 – Exemple de CSP

Les types de consistance sont nombreuses, mais nous nous intéresserons seulement aux trois premières :

- Consistance de nœuds (NC)
- Consistance d'arêtes (AC)
- Consistance de chemin (PC)
- K-Consistance

Consistance de nœuds (NC)

C'est la consistance la plus facile à mettre en œuvre. Elle correspond au traitement des arêtes réflexives sur le graphe. Elle permet ainsi de réduire le domaine de la variable par rapport aux contraintes qui ne portent que sur elle-même. Dans l'exemple de la figure 4.2 ci-dessus, elle correspond à la contrainte $A > 5$.

Consistance d'arêtes (AC)

La consistance d'arêtes ou *Arc Consistency* en anglais. Le terme anglais fait référence aux arcs. En français, on utilise le terme d'arêtes. Ce choix vient du fait que les contraintes sont orientées dans le graphe. c'est la Technique la plus utilisée car la moins coûteuse d'un point de vue de la complexité des algorithmes mis en œuvre. Les algorithmes travaillent sur les contraintes binaires, mettant en relation deux variables. Cela veut dire, qu'il existe une contrainte mettant en relation au moins ces deux variables.

Exemple, Soient les contraintes suivantes :

- Définition des domaines : $X \in \{1, 2\}, Y \in \{1, 2\}, Z \in \{1, 2\}$
- Relations entre variables : $X = Y, X \neq Z, Y > Z$

Nous voyons dans la figure 4.3 ci-dessous, le résultat obtenu après l'application de la consistance d'arêtes. Les valeurs des domaines des variables qui n'ont plus de support (contrainte) avec les autres sont supprimées.

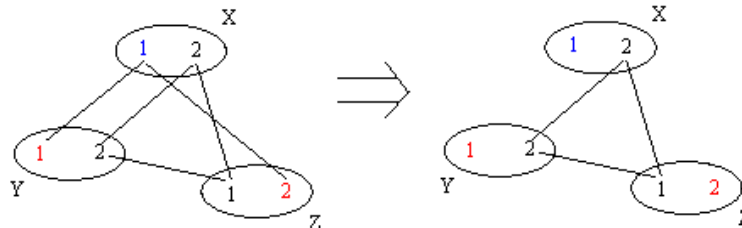


FIGURE 4.3 – Exemple de consistance d'arcs

Intérêts

Dans quels cas AC est suffisante :

- Lorsqu'un domaine est vide, on n'a pas de solution
- Si la cardinalité de tous les domaines est réduite à 1, alors on a une solution au CSP (équivalent).
- Bon rapport : simplification / performance

Exemple : soient les contraintes suivantes :

- Définition des domaines : $X \in \{1, 2\}, Y \in \{1, 2\}, Z \in \{1, 2\}$
- Relations entre variables : $X \neq Y, X \neq Z, Y \neq Z$

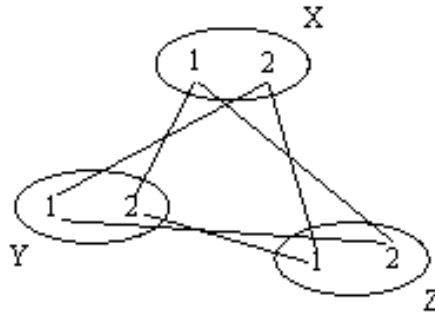


FIGURE 4.4 – Exemple d’insuffisance pour consistance d’arcs

Dans ce cas là, comme le montre la figure 4.4, la consistance d’arête n’aura aucun effet alors qu’il n’existe pas de solution à ce système de contraintes.

Exemple :

Soit le problème suivant, posé initialement par Lewis Carroll : Cinq maisons consécutives, de couleurs différentes, sont habitées par des hommes de différentes nationalités. Chacun possède un animal différent, a une boisson préférée différente et fume des cigarettes différentes. De plus, on sait que :

1. Le norvégien habite la première maison,
2. La maison à coté de celle du norvégien est bleue,
3. L’habitant de la troisième maison boit du lait,
4. L’anglais habite la maison rouge,
5. L’habitant de la maison verte boit du café,
6. L’habitant de la maison jaune fume des kools,
7. La maison blanche se trouve juste après la verte,
8. L’espagnol a un chien,
9. L’ukrainien boit du thé,
10. Le japonais fume des cravens,
11. Le fumeur de old golds a un escargot,
12. Le fumeur de gitanes boit du vin,
13. Le voisin du fumeur de Chesterfields a un renard,
14. Le voisin du fumeur de kools a un cheval.

Qui boit de l’eau ? A qui appartient le zèbre ?

Pour modéliser ce problème sous la forme d’un CSP, il s’agit d’identifier les variables (les inconnues du problème), les domaines de valeur de ces variables, et les contraintes existant entre ces variables.

Ici, il s’agit de déterminer, pour chaque caractéristique (couleurs, animaux, nationalités, cigarettes et boissons) à quelle maison elle appartient. Pour cela, on peut numéroté les maisons de 1 à 5, de la gauche vers la droite, et chercher, pour chaque caractéristique, à quel numéro de maison elle appartient. Ainsi pour chacune des variables du problèmes, elles prendront une

valeur dans le domaine de 1 à 5. Ainsi, on pourra situer chaque caractéristique ou entité avec le numéro de maison où elle est.

Il faut ensuite définir une contrainte pour chacune des assertions de l'énoncé. Il faut également ajouter des contraintes pour spécifier qu'on ne peut avoir plusieurs maisons de la même couleur, ni plusieurs maisons avec le même animal, ...

— On pose tout d'abord une contrainte pour chaque assertion de l'énoncé :

1. norvégien = 1,
2. bleue = norvégien + 1,
3. lait = 3,
4. anglais = rouge,
5. verte = café,
6. jaune = kools,
7. blanche = verte + 1,
8. espagnol = chien,
9. ukrainien = thé,
10. japonais = cravens,
11. old_golds = escargot,
12. gitanes = vin,
13. (chesterfields = renard + 1) ou (chesterfields = renard - 1),
14. (kools = cheval + 1) ou (kools = cheval - 1)

— De plus, toutes les variables de même "type" doivent avoir des valeurs différentes (il ne peut pas y avoir plusieurs maisons qui ont la même couleur, ou un même animal, ...) :

- blanche $\neq$ rouge $\neq$ verte $\neq$ jaune $\neq$ bleue,
- thé $\neq$ eau $\neq$ lait $\neq$ café $\neq$ vin,
- norvégien $\neq$ anglais $\neq$ ukrainien $\neq$ japonais $\neq$ espagnol,
- cheval $\neq$ renard $\neq$ zèbre $\neq$ escargot $\neq$ chien,
- kools $\neq$ chesterfields $\neq$ old_golds $\neq$ cravens $\neq$ gitanes

Les contraintes de différence donne le graphe de la figure 4.5 :

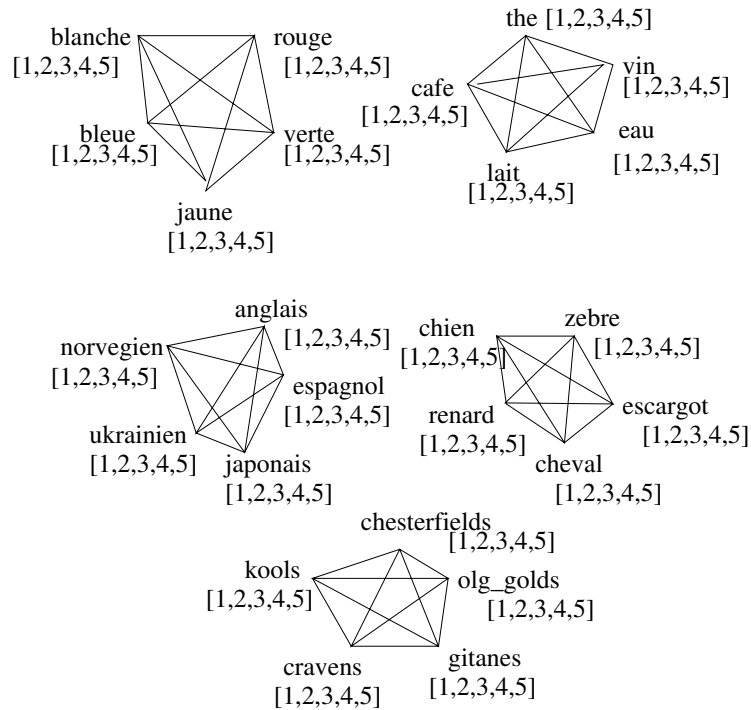


FIGURE 4.5 – Exemple zèbre - contraintes différences

On ajoute les 6 premières contraintes (on barre avec la couleur de la contrainte posée les valeurs réduites dans le domaine par arc consistance) comme dans la figure 4.6 :

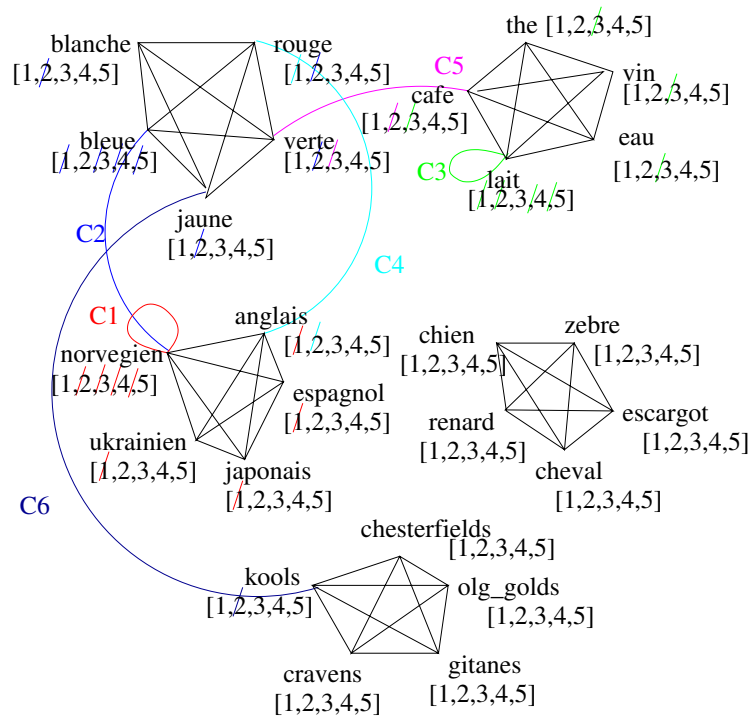


FIGURE 4.6 – Exemple zèbre - après 6 contraintes

On ajoute les 13 premières contraintes pour obtenir la figure 4.7 :

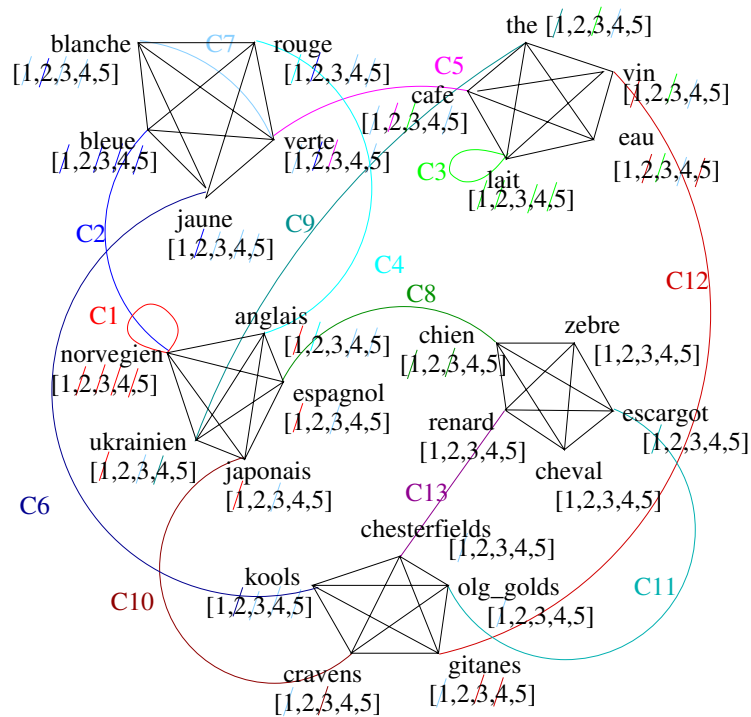


FIGURE 4.7 – Exemple zèbre - après 13 contraintes

On ajoute la 14^{ème} et dernière contrainte pour obtenir la figure 4.8 :

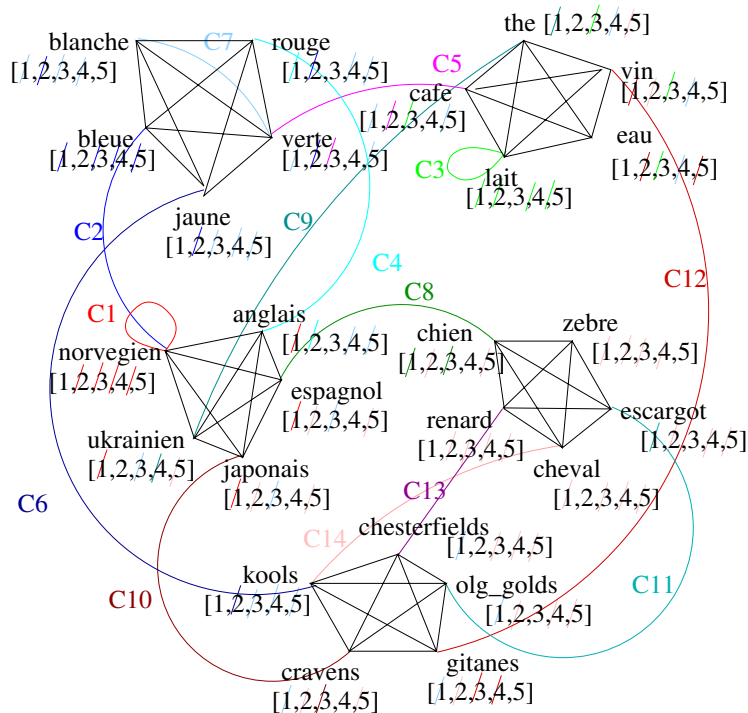


FIGURE 4.8 – Exemple zèbre - après 14 contraintes

On obtient ainsi une solution partielle où il faut effectuer une valuation pour obtenir la solution suivante. Avec une méthode de consistance de chemin on aurait obtenu directement la solution.

La solution : le zèbre est dans la maison 4 et appartient au japonais et le norvégien en maison 1 boit de l'eau.

On peut résoudre ce problème à l'aide du programme prolog suivant :

```
:-use_module(library(clpfd)).

solution(S) :-
    S = [[Norvegien, Anglais, Espagnol, Ukrainien, Japonais],
         [Blanche, Rouge, Bleue, Verte, Jaune],
         [The, Vin, Eau, Cafe, Lait],
         [Chien, Cheval, Zebre, Renard, Escargot ],
         [Chesterfield, Kools, Old, Cravens, Gitane]],
    % Definition des domaines
    [Norvegien, Anglais, Espagnol, Ukrainien, Japonais] ins 1..5,
    all_different([Norvegien, Anglais, Espagnol, Ukrainien, Japonais]),
    [Blanche, Rouge, Bleue, Verte, Jaune] ins 1..5,
    all_different([Blanche, Rouge, Bleue, Verte, Jaune]),
    [The, Vin, Eau, Cafe, Lait] ins 1..5,
    all_different([The, Vin, Eau, Cafe, Lait]),
    [Chien, Cheval, Zebre, Renard, Escargot ] ins 1..5,
```

```

all_different([Chien, Cheval, Zebre, Renard, Escargot ]),
[Chesterfield, Kools, Old, Cravens, Gitane] ins 1..5,
all_different([Chesterfield, Kools, Old, Cravens, Gitane]),
% Phrase 1
Norvegien #= 1,
% Phrase 2
Bleue #= Norvegien + 1,
% Phrase 3
Lait #= 3,
% Phrase 4
Anglais #= Rouge,
% Phrase 5
Verte #= Cafe,
% Phrase 6
Jaune #= Kools,
% Phrase 7
Blanche #= Verte + 1,
% Phrase 8
Espagnol #= Chien,
% Phrase 9
Ukrainien #= The,
% Phrase 10
Japonais #= Cravens,
% Phrase 11
Old #= Escargot,
% Phrase 12
Gitane #= Vin,
% Phrase 13
% Naturellement on aurait écrit :
%((Chesterfield #= Renard + 1) #\ (Chesterfield #= Renard - 1)),
abs(Chesterfield - Renard) #= 1,
% Phrase 14
%((Kools #= Cheval + 1) #\ (Kools #= Cheval -1)),
abs(Kools - Cheval) #= 1,
labeling([], [Norvegien, Anglais, Espagnol, Ukrainien, Japonais,
    Blanche, Rouge, Bleue, Verte, Jaune,
    The, Vin, Eau, Cafe, Lait,
    Chien, Cheval, Zebre, Renard, Escargot,
    Chesterfield, Kools, Old, Cravens, Gitane]).
    
```

Il est à noter que l'utilisation de l'opérateur `#\` engendre l'apparition de points de choix dans les contraintes les rendant ainsi moins efficaces. C'est pourquoi dans ce cas, on va privilégier la forme avec la valeur absolue (`abs`). Si on veut retrouver les solutions partielles, il suffit de retirer l'appel au prédicat `labeling`.

Le résultat est :

```
| ?- solution(S).
```

$S = [[1, 3, 5, 2, 4], [5, 3, 2, 4, 1], [2, 5, 1, 4, 3], [5, 2, 4, 1, 3], [2, 1, 3, 4, 5]] ? ;$
 no

Soit en clair :

Maison	Nationalité	Couleur	Boisson	Animal	Cigarette
1	Norvégien	Jaune	Eau	Renard	Kools
2	Ukrainien	Bleue	The	Cheval	Chesterfield
3	Anglais	Rouge	Lait	Escargot	Old
4	Japonais	Verte	Cafe	Zebre	Cravens
5	Espagnol	Blanche	Vin	Chien	Gitane

Consistance de chemin (PC)

La consistance de chemin ou *Path Consistance* en anglais permet de tester trois variables ayant des contraintes en commun et ainsi de détecter plus d'inconsistance que la consistance d'arêtes.

Si on reprend l'exemple précédent avec les contraintes suivantes :

- Définition des domaines : $X \in \{1, 2\}, Y \in \{1, 2\}, Z \in \{1, 2\}$
- Relations entre variables : $X \neq Y, X \neq Z, Y \neq Z$

Nous aurons bien vidé les domaines de chacune des variables et ainsi détecté l'inconsistance.

Généralisation : K-Consistance

Définition : Un CSP est dit K-consistant si pour tout n-uplet de k variables $(x_1, \dots, x_k)$, pour toute instanciation A consistante des k - 1 variables $(x_1, \dots, x_{k-1})$, il existe une valeur $v \in d_k$ telle que l'instanciation $A \cup \{x_k = v\}$ soit consistante.

On a ainsi comme propriétés de K-Consistance :

- Consistance de (K-1) variables pour la $k^{\text{ième}}$ variable
- On parle de K-Consistance forte, lorsque $\forall j, j < k$ on a la J-Consistance

On a deux exemples de K-Consistance forte : AC (2-Consistance forte) et PC (3-Consistance forte)

Bilan

Généralement, seule l'AC est utilisée à chaque étape. Malgré le faible coût (linéaire dans le produit du nombre de contraintes par la taille du plus grand domaine dans le meilleur des cas), on utilise des algorithmes permettant de définir un sous-ensemble de l'AC. On considère que les n variables du CSP sont ordonnées et une variable x_i donnée :

- **Check backward** : on fait l'AC seulement aux couples (x_i, x_j) tels que $1 \leq j < i$.
- **Check forward** : on fait l'AC seulement aux couples (x_j, x_i) tels que $i < j \leq n$.
- **Partial look future** : on fait l'AC seulement aux couples (x_j, x_k) tels que $i \leq j < k \leq n$.
- **Full look future** : on fait l'AC seulement aux couples (x_j, x_k) tels que $i \leq j \leq n, i \leq k \leq n$ et $j \neq k$.

4.3.2 Propagation de contraintes

Comment pouvons nous faire pour choisir l'ordre des contraintes à traiter ou les variables ? Dans les pages précédentes, on a présenté deux approches mais qui possèdent chacune des limites :

- les recherches systématiques (Générer Tester & Backtracking) seules ne sont pas efficaces,
- l'utilisation du maintien de la consistance seule est incomplet.

Nous allons donc voir une approche combinée des recherches (Backtracking) avec les techniques de consistance.

Pour l'approche par propagation, il y a deux grandes familles comme le montre la figure 4.9 :

- Retour Arrière guidé (Look back) : Revenir avant le conflit.
- Avancement guidé (Look Ahead) : Prévenir les conflits.

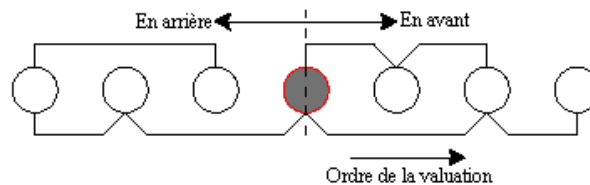


FIGURE 4.9 – Les deux approches pour la propagation

Retour en arrière guidé

L'idée directrice est d'exploiter les violations de contraintes constatées pendant l'exploration de l'arborescence pour construire, plus ou moins localement, une instanciation "contenue" dans l'instanciation courante et non globalement consistante. On pourra alors reconsidérer les liaisons de plusieurs variables plutôt que de n'en reconsidérer qu'une (retour à l'état précédent) comme le fait l'algorithme de backtracking. Pour cela, on utilise deux approches :

- **Back-jumping** : on saute ailleurs, là où la valeur instanciée peut changer la suite.
- **Back-marking** : on note les états impossibles afin de ne pas les répéter.

Problème :

- Parcours en aveugle de l'espace de recherche.
- Redécouverte répétée des mêmes inconsistances locales.
- Non conservation de solutions partielles.

En avant guidée

Les méthodes prospectives sont, en général, les méthodes les plus efficaces. L'algorithme de backtracking est simplement modifié en remplaçant la vérification de la consistance de l'instanciation partielle courante par l'établissement d'une propriété de consistance locale. Il y a retour-arrière dès que la propriété de consistance locale n'est plus vérifiée. Il est alors nécessaire de "défaire" les effets du filtrage effectué précédemment. Chaque instanciation peut donc

supprimer des valeurs des domaines d'autres variables. Il y a retour-arrière dès qu'un domaine est vide. Selon le niveau d'AC utilisé, on aboutit à différents algorithmes :

- **Backtrack** ou **Générer & Test** : emploi d'un check backward après chaque instantiation.
- **Forward-checking** : emploi d'un check forward après chaque instantiation.
- **Partial look-ahead** : emploi d'un partial look future après chaque instantiation.
- **Full look-ahead** : emploi d'un look future après chaque instantiation.
- **Real full look-ahead** : emploi de la consistance d'arête AC complète à chaque instantiation.

L'algorithme forward-checking reste le plus simple à réaliser et propose l'un des meilleurs rendus.

4.3.3 Branch and Bound

L'algorithme de Branch and Bound permet pour chaque branchement du graphe d'évaluer le poids/coût pour atteindre le nœud et ainsi déterminer si il est pertinent d'explorer la transition suivante. On utilise le backtracking pour effectuer la totalité de la recherche avec comme condition pour arrêter l'exploration du sous-graphe le fait d'être supérieur au meilleur coût courant. Ainsi, on peut élaguer des branches qui ont un coût plus élevé que le coût courant. Pour cela, on va garder à chaque nœud le coût le plus faible qui a mené à une solution.

Algorithme 8 : Algorithme : Branch&Bound($n, \text{Cout}, \text{Sol}$)

```

Input : Un nœuds  $n$  (l'état initial ou la racine d'un graphe).
Data :  $\text{Pere}$  indique le nœud permettant d'en atteindre un autre
 $\text{Val}$  donne la valeur pour atteindre ce nœud
Output : renvoie la solution  $\text{Sol}$  et le  $\text{Cout}$ 
 $\forall i \text{ Val}(i) \leftarrow +\infty, \text{Pere}(i) \leftarrow \perp$  /* Initialisation table */
// Initialisation avec une "bonne" solution ou par défaut :
 $\text{Cout} \leftarrow +\infty; \text{Sol} \leftarrow \emptyset$ ;
 $V \leftarrow \{n\}; \text{Val}(n) \leftarrow 0; \text{Pere}(n) \leftarrow \perp$  /* Initialisation des nœuds à traiter */
while ( $V \neq \emptyset$ ) do
     $n \leftarrow$  prendre un élément de  $V; V \leftarrow V \setminus \{n\}$ ;
    if ( $\text{Val}(n) < \text{Cout}$ ) then
        foreach fils  $f$  de  $n$  do
            if ( $\text{val}(f) > \text{val}(n) + k(f, N)$ ) then
                 $\text{Val}(f) \leftarrow \text{Val}(n) + k(f, n); \text{Pere}(f) \leftarrow n$ ;
                if ( $\text{val}(f) < \text{Cout}$ ) then
                    if  $f$  est un nœud terminal then
                         $\text{Cout} \leftarrow \text{val}(f)$ ;
                         $\text{Sol} \leftarrow$  chemin menant à  $f$ ;
                    else  $V \leftarrow V \cup \{f\}$ ;
    
```

Remarques :

- Il est complet si on n'a pas mis une limite initiale trop basse.

- Il est optimal car il trouvera la meilleure solution.
- Sa complexité en temps peut être importante puisque dans le pire des cas on va parcourir l'ensemble du graphe plusieurs fois.
- Il consomme en mémoire légèrement plus qu'un parcours en profondeur mais beaucoup moins qu'un parcours en largeur car il ne garde pas en mémoire les chemins partiels (sa complexité mémoire est en $O(n)$, n étant le nombre de nœuds).
- L'évaluation de la limite initiale (le premier coût) pour lancer l'algorithme, peut être une difficulté. Le but étant d'être le plus proche du résultat pour élaguer un maximum de branches sans faire disparaître toutes les solutions. Par défaut comme proposé l'algorithme, on peut démarrer celui-ci sans solution et obtenir la solution lors du premier chemin.

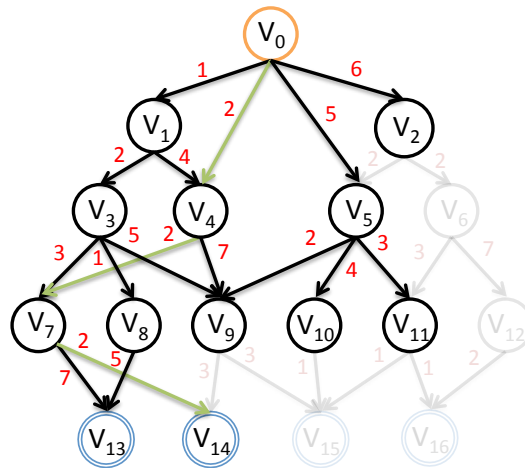


FIGURE 4.10 – Exemple de Branch and Bound

Exemple : Soit le graphe donné dans la figure 4.10, l'exécution de l'algorithme sur celui-ci donne le tableau suivant :

n	val(n)	Cout	f	Val(f)	Pere(f)	V	Remarque
V_0	0	$+\infty$	V_1	1	V_0	$\{V_1\}$	
		$+\infty$	V_4	2	V_0	$\{V_1, V_4\}$	
		$+\infty$	V_5	5	V_0	$\{V_1, V_4, V_5\}$	
		$+\infty$	V_2	6	V_0	$\{V_1, V_4, V_5, V_2\}$	
V_1	1	$+\infty$	V_3	3	V_1	$\{V_4, V_3, V_5, V_2\}$	$\text{Val}(V_3) = \text{Val}(V_1) + 2$
		$+\infty$	V_4	2	V_0	$\{V_4, V_3, V_5, V_2\}$	Val et Pere de V_4 inchangées
V_4	2	$+\infty$	V_7	4	V_4	$\{V_3, V_7, V_5, V_2\}$	
		$+\infty$	V_9	9	V_4	$\{V_3, V_7, V_5, V_2, V_9\}$	
V_3	3	$+\infty$	V_7	4	V_4	$\{V_7, V_5, V_2, V_9\}$	Val et Pere de V_7 inchangées
		$+\infty$	V_8	4	V_3	$\{V_7, V_8, V_5, V_2, V_9\}$	
		$+\infty$	V_9	8	V_3	$\{V_7, V_8, V_5, V_2, V_9\}$	$\text{Val}(V_9) = \text{Val}(V_3) + 5$
V_7	4	11	V_{13}	11	V_7	$\{V_8, V_5, V_2, V_9\}$	$\text{Sol} = \{V_0, V_4, V_7, V_{13}\}$
		6	V_{14}	6	V_7	$\{V_8, V_5, V_2, V_9\}$	$\text{Sol} = \{V_0, V_4, V_7, V_{14}\}$
V_8	4	6	V_{13}	9	V_8	$\{V_5, V_2, V_9\}$	$\text{Val}(V_{13}) \geq \text{Cout}$
V_5	5	6	V_9	7	V_5	$\{V_2, V_9\}$	$\text{Val}(V_9) \geq \text{Cout}$
		6	V_{10}	9	V_5	$\{V_2, V_9\}$	$\text{Val}(V_{10}) \geq \text{Cout}$
		6	V_{11}	8	V_5	$\{V_2, V_9\}$	$\text{Val}(V_{11}) \geq \text{Cout}$
V_2	6	6	$\text{Val}(V_2) \geq \text{Cout}$, on prend un autre nœud dans $V = \{V_9\}$				
V_9	8	6	$\text{Val}(V_9) \geq \text{Cout}$, on prend un autre nœud dans $V = \{\}$				

La solution avec un coût de 6 passe par les nœuds : $\{V_0, V_4, V_7, V_{14}\}$ en vert dans la figure 4.10. Ainsi toute une partie du graphe n'a pas été visité comme en témoigne la partie grisée.

Il existe plusieurs variantes que nous ne présentons pas ici. Par exemple, une ajoute la notion de fils et le coût le plus faible pour atteindre un nœud terminal à chaque nœud. Ceci permet ainsi d'éviter de reparcourir les sous-graphes si la mise à jour du nœud ne permet pas d'améliorer le coût de la solution courante.

4.4 Exercices chapitre 4

Nous avons pu voir l'utilisation de la programmation logique, nous allons nous intéresser à l'utilisation de la programmation avec contraintes. A partir des exercices suivants, nous allons comparer les deux types d'approche (avec et sans contrainte). Sicstus met à disposition un solveur de contraintes sur domaines finis. Pour pouvoir l'utiliser, il suffit d'ajouter en entête de votre programme l'appel à la librairie clpfd (`:-use_module(library(clpfd))`). Nous nous intéresserons ici à l'utilisation des contraintes dans le cadre de manipulations d'entiers.

Exercice 1 - Crypto-arithmétique

Soit le problème dont le but consiste à trouver les valeurs des 8 variables D, E, M, N, O, R, S et Y telles que :

- toutes ces variables soient des chiffres entre 0 et 9,
- toutes ces variables soient différentes,
- l'équation précédente soit vérifiée, ainsi $M \neq 0$.

$$\begin{array}{rcccccc}
 & & S & E & N & D \\
 + & & M & O & R & E \\
 \hline
 & M & O & N & E & Y
 \end{array}$$

Questions :

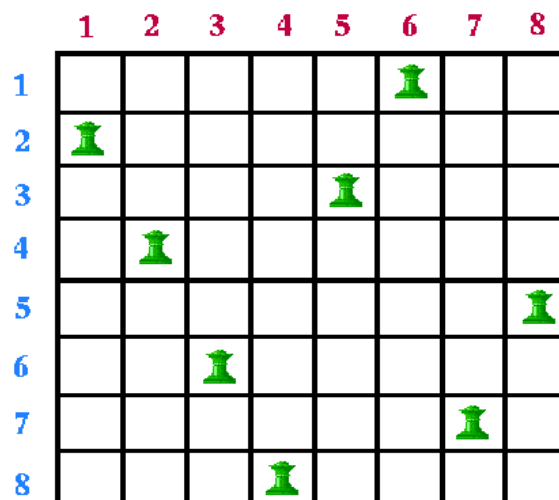
Réaliser un programme prolog permettant de résoudre le problème :

1. Avec la méthode du "Générer & Tester"
 - définir un état mémoire,
 - tester sa validité
2. Avec la méthode de Backtracking
 - définir une valeur à une des variables,
 - vérifier si elle n'est pas en conflit :
 - continuer si elle n'est pas en conflit
 - backtracker si elle est en conflit
3. avec le paradigme de la programmation avec contraintes :
 - définir les domaines,
 - poser des contraintes,
 - demander la résolution.
4. Donner une solution

[Correction.](#)

Exercice 2 - Les reines

Le problème des N-reines consiste à trouver toutes les configurations permettant de placer N reines sur un échiquier de taille N sans qu'aucune des reines ne soit en prise. Aucune reine ne doit être sur la même ligne, colonne ou diagonales qu'une autre reine.


Questions :

1. Réaliser un programme prolog permettant de résoudre le problème : Avec la méthode du "Générer & Tester" (il faut définir un état mémoire et tester sa validité ensuite).

2. Avec la méthode de "**Backtracking**" (définir une valeur à une des variables et vérifier si elle n'est pas en conflit).
3. Avec le paradigme de la **programmation avec contrainte**.
4. Comparez les temps d'exécution pour les reines 4 (2 solutions sans permutations), 6 (4 solutions), 8 (92 solutions), 10 (sans générer / tester, 724 solutions) et 12 (14200 solutions).
Pour cela, on utilisera `statistics, findall(R, reines(4, R), T), length(T, L), statistics`.

Correction.

Exercice 3 - Les amazones

Le problème des amazones est une variante de celui des reines. En plus des positions de prises des reines, on rajoute celles des cavaliers. Pour mémoire une pièce est en prise avec un cavalier si celle-ci est à deux cases verticales et une case horizontale ou deux cases horizontales et une case verticale. La figure ci-dessous illustre les cas (le cavalier est représenté par © et les cases en prise par un point cerclé ⊙) :

		⊙		⊙		
	⊙				⊙	
			©			
	⊙				⊙	
		⊙		⊙		

Questions :

1. Réaliser le prédicat `amazones(N, R)` qui est satisfait si il existe une répartition R de N amazones pour un damier de taille $N \times N$ à l'aide de l'approche Générer & Tester
2. Avec la méthode de "**Backtracking**".
3. Avec le paradigme de la **PLC** - programmation avec contrainte.
4. Ce problème n'ayant des solutions qu'à partir d'un damier de taille 10, il devient long de le faire résoudre avec la méthode générer / tester. Nous vous proposons de comparez les temps d'exécution pour les amazones 10 (4 solutions), 12 (156 solutions), 14 (5180 solutions) pour les deux approches Backtracking et PLC.
Pour obtenir le temps, on peut utiliser le prédicat suivant, où N est le nombre d'amazones, L est le nombre de solution et T le temps total.

```

chrono(N, L, T) :-
    statistics(runtime, [T0|_]),
    findall(R, amazones(N, R), LR),
    statistics(runtime, [T1|_]),
    length(LR, L),
    T is T1 - T0.
    
```

Correction.

Exercice 4 - Carré magique

Des nombres de 1 à n^2 sont initialement répartis aléatoirement sur une grille carrée de dimension $n \times n$. Le but est de trouver un arrangement dans lequel toutes les sommes des colonnes, des lignes et des deux diagonales principales donnent le même résultat S , appelé la somme magique, un tel arrangement est appelé un carré magique. Ci-contre un exemple, avec $n = 4$ et comme valeur de la somme magique 34.

16	3	2	13
5	10	11	8
9	6	7	12
4	15	14	1

Questions :

1. Donner le programme prolog qui résoud le problème avec la méthode du Backtracking. Vous pouvez le tester avec $n = 3$ et la somme sera 15. Il y a 8 permutations possibles.
2. Donner le programme prolog qui résoud le problème avec la PLC.

[Correction.](#)

Exercice 5 - Graphe de nombres

On cherche à placer neuf nombres parmi ceux de 2 à 12 dans les cercles de la figure 4.11 ci-contre.

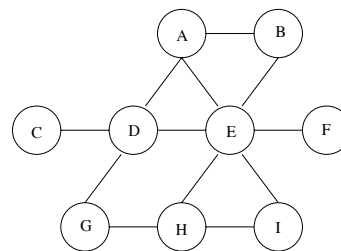


FIGURE 4.11 – Graphe de nombres

Question : Réaliser un programme prolog qui permet de placer ces 9 nombres dans cette figure pour que la somme des cercles reliés par une même droite soit égale à 20 :

1. Avec une approche de type Backtracking ;
2. Avec une approche de type programmation logique avec contrainte.
3. Donner une solution dans le cas où 6 et 11 sont exclus.

[Correction.](#)

Exercice 6 - Soyons logique

Soit le problème suivant :

Fabrice est un inconditionnel de l'équipe de football de sa ville. L'an dernier, à cause de sa nomination dans l'est de la France, il n'a pu assister qu'à cinq rencontres à domicile. Réaliser le prédicat `solution(S)` en Prolog qui permet de retrouver le prénom de l'ami avec qui il s'est rendu au stade, le mois, l'équipe adverse et le score du match sachant que :

1. Les cinq amis sont Arnaud, Christian, Hubert, Julien et Stéphane.
2. Les cinq mois sont septembre, octobre, décembre, janvier et mars.
3. Les cinq équipes visiteuses sont Bordeaux, Lens, Nantes, Saint-Etienne et Lyon.

4. Les cinq résultats sont 1-0, 1-1, 2-1, 2-3 et 3-2.
5. Julien a assisté au match contre Lyon.
6. Arnaud n'a vu évoluer ni l'équipe de Lens ni celle de Nantes.
7. Ni Arnaud ni Hubert n'a vu l'équipe de Bordeaux.
8. Stéphane a vu gagner l'équipe visiteuse.
9. Lyon a perdu son match.
10. Nantes a perdu son match.
11. Le match contre Bordeaux s'est terminé par une victoire sur le score de 2-1.
12. Arnaud s'est rendu au stade un mois après Julien.
13. Le match contre Bordeaux a eu lieu deux mois après le résultat de 1-0.
14. Le match perdu s'est déroulé un mois avant le match contre Nantes.

Sachant que l'on veut utiliser le prédicat `solution(S)` avec comme résultat une liste de la forme: `S=[ami(arnaud,M1,S1,V1), ami(christian,M2,S2,V2), ami(hubert,M3,S3,V3), ami(julien,M4,S4,V4), ami(stephane,M5,S5,V5)]` où `ami` est un terme constituant un quadruplet correspondant respectivement au nom de l'ami, du mois, du score et de la ville de l'adversaire du match.

Questions

1. Définir le prédicat `unMoisApres(X, Y)` : "X est le mois après Y".
2. Définir le prédicat `deuxMoisApres(X, Y)` : "X est le deuxième mois après Y".
3. Définir le prédicat `membre(X, L)` : "X appartient à la liste L".
4. Définir le prédicat `membreRetire(X, L, L1)` : "Retire X de la liste L et renvoie la liste, privée de X, dans L1".
5. Définir le prédicat `element_membre(X, L)` : "Pour chaque élément de X on lui affecte une valeur de la liste L, non encore attribuée à un autre élément de X".
6. Écrire le problème en prolog à l'aide des éléments précédents pour le résoudre avec la méthode du Générer et Tester.
7. Même question à l'aide de la méthode du backtracking.
8. Même question à l'aide de la PLC.
9. Donner la solution du problème.

[Correction.](#)

5. Systèmes experts

Nous présentons dans ce chapitre les concepts de système expert. Cette approche a été une révolution dans l'Intelligence Artificielle et, pendant longtemps, on a pensé pouvoir transférer la connaissance humaine dans la machine avec ce type d'approche. Mais une des difficultés a été de pouvoir faire évoluer la connaissance et de pouvoir raisonner à partir de celle-ci. Après un très fort engouement dans les années 70, les systèmes experts ont vite montré leurs limites pour revenir, aujourd'hui encore, avec des systèmes d'aides intelligents utilisés dans les hot-lines téléphoniques ou tout simplement dans les assistances intégrées dans les systèmes d'exploitation.

- Dans la [leçon 1](#), nous présenterons les principes généraux et le mode de fonctionnement des systèmes experts.
- Dans la [leçon 2](#), nous présenterons deux approches pour la résolution et le traitement des requêtes faites au système.

Les compétences à acquérir dans ce chapitre sont la traduction d'un problème sous la forme de règle pour pouvoir ensuite utiliser les raisonnements par chaînage avant ou arrière pour résoudre le problème.

5.1 Présentation

Un système expert est un logiciel exploitant les connaissances acquises dans un domaine spécialisé et simulant le comportement humain pour aider à résoudre des problèmes qui relèvent de ce domaine.

5.1.1 But

On cherche à recréer, dans la machine, le rôle et la connaissance d'un **expert humain**. Pour cela, on cherche à le **modéliser**. Ainsi, la machine pourra effectuer des tâche de **résolution de problème** comme l'expert le ferait. Elle pourra aussi donner les **explications sur le raisonnement** qui lui a permis de donner son avis.

5.1.2 Les éléments clefs

Pour effectuer cette tâche, le système va devoir posséder une **base de connaissances**. Elle renferme le savoir de l'expert. Pour lui permettre de raisonner et de faire évoluer ses connaissances, le système doit être doté d'un **moteur d'inférence**.

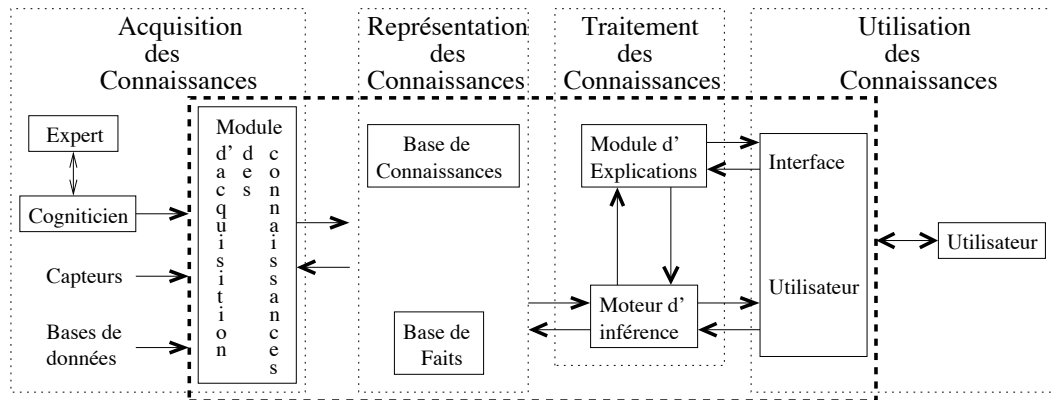


FIGURE 5.1 – Composant d'un Système expert

Dans la figure 5.1, nous voyons les différents composants du système expert :

- **Base de connaissances** : regroupe l'ensemble des données du système.
- **Moteur d'inférence** : le mécanisme permettant de raisonner et faire évoluer le système.
- **Base de faits** : les nouvelles informations arrivant dans le système (ou par des éléments externes ou par le moteur d'inférence).
- **Système de consultation** : il permet d'utiliser les connaissances en assurant la communication entre l'utilisateur et le système.
- **Module d'explications** : il permet de transcrire le raisonnement qui a conduit le système à la solution proposée.
- **Module d'acquisition des connaissances** : ensemble des moyens de communications avec le monde extérieur (utilisateur, capteur, base de connaissances, ...).

5.1.3 Cycle du moteur d'inférence

- **Phase de sélection** : elle permet de choisir un sous-ensemble de la base de faits (et de la base de règles) qui mérite le "plus d'attention" que le reste de la base. Le "plus d'attention" est un élément souvent difficile à déterminer. Cette phase permet une économie sur le temps de calcul pour la phase suivante. Ce choix est l'expression d'une stratégie dans le moteur d'inférence qui fait préférer tel groupe de règles, comme les plus courtes, les plus souvent utilisées, ...
- **Phase de filtrage** : elle permet au moteur d'inférence de comparer la partie prémisses des règles sélectionnées avec les faits de la base de faits pour déterminer l'ensemble des règles applicables. Le bon déroulement de cette phase est lié au nombre de règles à traiter venant directement de la phase de sélection.
- **Phase de résolution de conflits** : elle permet d'effectuer le choix de la règle qui va être appliquée effectivement. Cette phase cruciale exprime également une stratégie qui peut être très simple et sans rapport avec le contexte (comme première règle de la liste, la moins longue, la moins utilisée, ...) ou plus complexe car tenant compte du contexte (comme la plus prometteuse, la plus fiable, le moins coûteuse, ...). De la qualité de ce choix dépendent les performances du moteur d'inférence qui va déduire plus ou moins rapidement la solution.

- **Phase d'exécution** : elle consiste à appliquer la règle choisie précédemment. Cette action correspond généralement à ajouter un ou plusieurs faits à la base de faits, modifier la base de faits, questionner l'utilisateur ou appeler des procédures externes.

Chacune des ces phases sera ensuite utilisée suivant le mode de raisonnement comme nous allons le voir dans les parties suivantes.

5.2 Mode de raisonnement

Il y a deux modes de raisonnement auquel s'ajoute un mode hybride.

5.2.1 Chaînage avant

Il existe plusieurs noms dans la littérature pour définir ce type d'approche "forward chaining", "pattern directed" (recherche guidée par les règles) et "data driven" (recherche guidée par les faits).

Le moteur d'inférence part des faits pour arriver au but, c'est-à-dire qu'il ne sélectionne que les règles dont les conditions (partie gauche de la règle) sont vérifiées (dans la phase de filtrage).

Exemple : Soit la base de règles :

- R_1 : si B et D et E alors F
- R_2 : si D et G alors A
- R_3 : si C et F alors A
- R_4 : si B alors X
- R_5 : si D alors E
- R_6 : si A et X alors H
- R_7 : si C alors D
- R_8 : si X et C alors A
- R_9 : si X et B alors D
- La base de faits contient : B, C
- Le but à prouver : H

On définit comme stratégie de choisir la première règle applicable dans la base. On obtient le graphe de la figure 5.2, où les flèches correspondent à l'application des règles avec leur numéro (en pointillés pour indiquer qu'elles peuvent s'appliquer mais n'ont pas été appliquées à ce niveau, on n'affiche plus les règles déjà utilisées). Le résultat obtenu correspond aux faits présents dans la base après l'application de la règle.

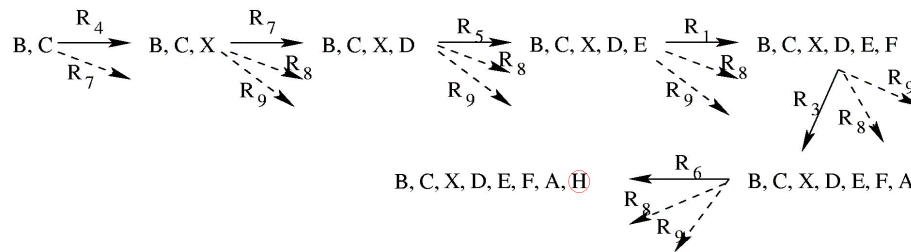


FIGURE 5.2 – Exemple chainage avant

Avec le même exemple, on peut utiliser une autre stratégie dans laquelle on sélectionne, parmi les règles applicables, celle qui comporte le plus grand nombre de conditions. En cas d'égalité, c'est la règle de numéro le plus faible qui est utilisée. Le résultat est donné dans la figure 5.3.

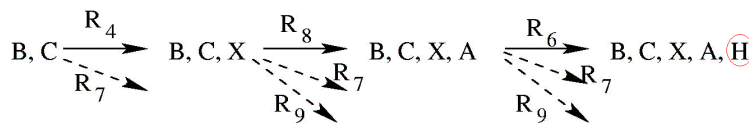


FIGURE 5.3 – Exemple chainage avant suite

Inconvénients : Nombre de problèmes peuvent rendre le système inefficace :

- Le système déclenche toutes les règles applicables même si certaines n'offrent aucun intérêt.
- La base de faits doit contenir suffisamment de faits initiaux pour que le système puisse arriver au but.
- Le processus n'est pas interactif (on ne peut pas compléter les faits si nécessaire).
- Le problème de l'explosion combinatoire est omniprésent : ceci est dû au nombre de règles et de faits qui est potentiellement important. Dans le cas où, le but à atteindre n'est pas connu, il faudra appliquer toutes les règles pour déduire tout qui peut l'être. Le nombre d'éléments à gérer devient impossible d'autant plus si l'algorithme utilisé est un parcours en largeur et non en profondeur.

5.2.2 Chaînage arrière

Il existe plusieurs noms dans la littérature pour définir ce type d'approche "backward chaining" et "goal driving" (recherche guidée par le but). Le moteur d'inférence part du but (ou d'une hypothèse du but) et essaie de remonter aux faits pour le démontrer :

- Les règles sélectionnées sont celles dont la partie droite (la conséquence) correspond au but recherché.
- Les conditions inconnues (partie gauche) de ces règles deviennent autant de sous-buts à démontrer.

Exemple : Soit la base de règles :

- R_1 : si B et D et E alors F
- R_2 : si D et G alors A
- R_3 : si C et F alors A
- R_4 : si B alors X
- R_5 : si D alors E
- R_6 : si A et X alors H
- R_7 : si C alors D
- R_8 : si X et C alors A
- R_9 : si X et B alors D
- La base de faits contient : B, C
- Le but à prouver : H

On définit comme stratégie de choisir la première règle applicable dans la base. On obtient le graphe donné dans la figure 5.4, où les flèches correspondent à l'application des règles avec leurs numéros (en pointillés pour indiquer qu'elles peuvent s'appliquer mais n'ont pas été appliquées à ce niveau). Un but est vérifié si c'est un fait ou s'il a déjà été vérifié. Si plusieurs règles s'appliquent alors le nœud est de type **ou**, s'il y a plusieurs sous-buts alors le nœud est de type **et** :

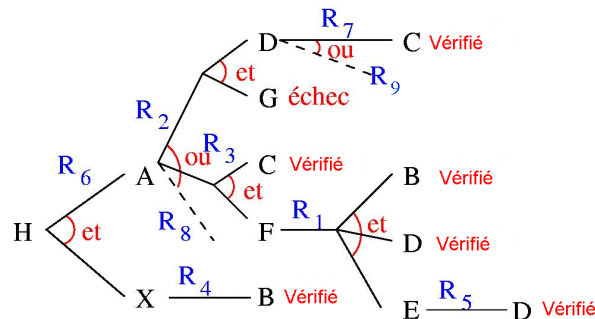


FIGURE 5.4 – Exemple de chainage Arrière

Avantages : Il existe plusieurs avantages de cette méthode par rapport au chaînage avant :

- Le système pose des questions uniquement quand cela est nécessaire.
- L'arbre de recherche est souvent plus petit qu'en chaînage avant.
- Le processus est interactif.

Mais il demeure un problème : par exemple, pour démontrer A il faut démontrer B et pour démontrer B il faut démontrer A, ... Il peut être résolu mais, bien sûr, l'interdépendance peut être plus éloignée.

5.3 Exercices chapitre 5

Exercice 1 - Les animaux

Soient les règles suivantes :

- R_1 : Si l'animal a des poils alors c'est un mammifère
- R_2 : Si l'animal donne du lait alors c'est un mammifère
- R_3 : Si l'animal a des plumes c'est un oiseau
- R_4 : Si l'animal vole et pond des œufs alors c'est un oiseau
- R_5 : Si l'animal mange de la viande alors c'est un carnivore
- R_6 : Si l'animal a des dents acérées, a des griffes et a des yeux perçants alors c'est un carnivore
- R_7 : Si l'animal est un mammifère et a des sabots alors c'est un ongulé
- R_8 : Si l'animal est un mammifère et rumine alors c'est un ongulé
- R_9 : Si l'animal est un mammifère, est carnivore, est de couleur fauve et a des taches sombres alors c'est un guépard
- R_{10} : Si l'animal est un mammifère, est carnivore, est de couleur fauve et a des stries noires alors c'est un tigre
- R_{11} : Si l'animal est un ongulé, a un long cou, a de longues jambes, et a des taches sombres alors c'est une girafe
- R_{12} : Si l'animal est un ongulé et a des stries noires, alors c'est un zèbre
- R_{13} : Si l'animal est un oiseau, ne vole pas, a un long cou, a des longues jambes et est blanc et noir alors c'est une autruche
- R_{14} : Si l'animal est un oiseau, ne vole pas, nage et est blanc et noir alors c'est un pingouin
- R_{15} : Si l'animal est un oiseau et vole alors c'est albatros
- R_{16} : Un animal ne peut pas être en même temps un mammifère et un oiseau
- R_{17} : Si l'animal est mammifère alors il est dans un des ces deux cas, carnivore ou ongulé

Questions :

1. Donner la traduction des règles en logique propositionnelle.
2. Donner le graphe correspondant au schéma de résolution du système par chaînage avant.
On utilise :
 - Le critère de choix est la règle de plus petit numéro
 - Les faits initiaux :
 - l'animal a des poils,
 - l'animal a des dents acérées, a des griffes et a des yeux perçants
 - l'animal est de couleur fauve
 - l'animal a des stries noires
 - Le but est de trouver "*Quel est l'animal observé ?*"
3. Donner le graphe correspondant au schéma de résolution du système par chaînage arrière ainsi que les questions posées par le système pour montrer l'hypothèse suivante : "*l'animal observé est un guépard*" en réutilisant les mêmes faits que la question 2.

Correction.**Exercice 2 - Inférence**

Soit le système expert suivant qui utilise comme règles d'inférences, le Modus ponens et les axiomes suivants :

Axiome 1 : $A \rightarrow (B \rightarrow A)$

Axiome 2 : $(A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$

Axiome 3 : $(\neg A \rightarrow \neg B) \rightarrow (B \rightarrow A)$

Questions :

Donner le graphe de résolution par chaînage avant pour ces différents systèmes :

1. Pour le fait : $\{X \rightarrow Y\}$ et le but $(Z \rightarrow X) \rightarrow (Z \rightarrow Y)$
2. Pour les faits $\{X \rightarrow Y, Z \rightarrow X, Z\}$ et le but $(Z \rightarrow Y)$
3. Pour les faits $\{X \rightarrow Y, Y \rightarrow Z\}$ et le but $(X \rightarrow Z)$

[Correction.](#)

6. Algorithmes des jeux

Nous présentons dans ce chapitre deux algorithmes se rapportant à l'univers des jeux.

- [Leçon 1](#) présente les deux algorithmes qui sont minimax et $\alpha - \beta$.
- [Leçon 2](#) présente trois exemples d'applications de ces algorithmes avec les problèmes inhérents à leur utilisation dans le contexte des exemples.

Les compétences à acquérir dans ce chapitre concerne l'utilisation de l'algorithme $\alpha - \beta$ et la prise de conscience du travail à effectuer d'analyse sur le problème à traiter. Cette analyse permet d'établir les différentes stratégies à mettre en œuvre pendant la durée de la partie et la méthode d'évaluation de l'état du système.

6.1 Algorithmes

Les informations associées aux problèmes peuvent être représentées à l'aide d'arbres ou de graphes (avec des états et des transitions) comme précédemment. La différence réside dans le fait qu'il y a deux joueurs. Ainsi, on ne cherche pas à obtenir le meilleur résultat (suivant le même critère) à chaque pas de l'algorithme. On ne peut donc pas utiliser les algorithmes de type best-first comme A^* . Il faut donc créer de nouveaux algorithmes dédiés.

Pour cela, nous allons partir sur les mêmes bases que dans les chapitres précédents avec une représentation sous forme d'arbre. Chaque nœud représente une configuration de jeu et ses fils les évolutions par rapport au coup joué par le joueur dont c'est le tour.

La difficulté est d'avoir une bonne fonction d'évaluation pour avoir un algorithme de parcours de graphe qui soit efficace. Un problème persiste car le nombre de possibilités étant vaste, il n'est donc pas envisageable de calculer l'ensemble du graphe. Ainsi, il faudra borner la profondeur de recherche pour obtenir des résultats corrects dans un temps raisonnable. Nous pourrons ainsi simuler l'intelligence par la profondeur du calcul.

6.1.1 Algorithme Minimax

Les caractéristiques de cet algorithme sont :

- C'est un parcours en profondeur d'abord
- Il intègre la vision double pour permettre ainsi de gérer deux joueurs :
 - Les coûts de l'adversaire sont minimisés (étape minimisant), puisque l'on considère qu'il joue un coup me rapportant le moins de point
 - Mes coûts sont maximisés (étape maximisant) puisque je cherche à joueur le meilleur coup

- L'évaluation s'effectue par les feuilles puis en remontant. L'influence du choix de la profondeur s'en ressent d'autant plus. Ainsi à chaque étape on prendra le min ou le max suivant quel est le joueur considéré comme illustré dans la figure 6.1.

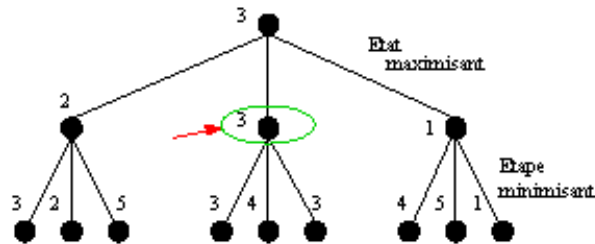


FIGURE 6.1 – Exemple de graphe Minmax

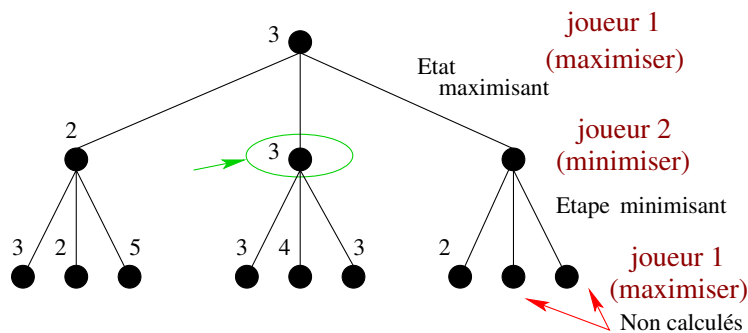
L'algorithme possède les propriétés suivantes :

- Il est **complet** si l'arbre de jeu est fini.
- Il est **optimal** si l'adversaire joue aussi de façon optimale.
- En prenant b le nombre de branchements en moyenne par coup et m le nombre de coups (profondeur de recherche), on obtient en :
 - complexité en temps : $O(b^m)$
 - complexité en espace : $O(b^m)$

6.1.2 Algorithme $\alpha - \beta$

Cet algorithme permet d'optimiser l'algorithme Minimax. Pour cela, on utilise la notion de seuil associée aux deux variables α et β pour éviter l'exploration de certaines branches :

- α : Pour un nœud Min n , valeur la plus grande de tous les nœuds Max ancêtres de n
Si $\text{val}(n) < \alpha$ alors l'exploration est inutile
- β : Pour un nœud Max n , valeur la plus petite de tous les nœuds Min ancêtres de n
Si $\text{val}(n) > \beta$ alors l'exploration est inutile

FIGURE 6.2 – Exemple de graphe $\alpha - \beta$

Algorithme 9 : Algorithme : AlphaBeta($n, p, \alpha, \beta, \text{MaxJoueur}$)

```

Input :  $n$  nœud
Input :  $p$  profondeur
Input :  $[\alpha, \beta]$  borne
Input : MaxJoueur Vrai si nœud type max
if  $p = 0$  ou  $n$  est terminal then return  $h(n)$ ;
else
  if MaxJoueur then
     $f \leftarrow \text{PremierFils}(n)$ ;
    while  $(f \neq \perp)$  et  $(\alpha < \beta)$  do
       $\alpha \leftarrow \max(\alpha, \text{AlphaBeta}(f, p - 1, \alpha, \beta, \text{faux}))$ ;
       $f \leftarrow \text{FilsSuivant}(n)$ ;
    return  $\alpha$ ;
  else
     $f \leftarrow \text{PremierFils}(n)$ ;
    while  $(f \neq \perp)$  et  $(\alpha < \beta)$  do
       $\beta \leftarrow \min(\beta, \text{AlphaBeta}(f, p - 1, \alpha, \beta, \text{vrai}))$ ;
       $f \leftarrow \text{FilsSuivant}(n)$ ;
    return  $\beta$ ;

```

Dans la figure 6.2, les deux nœuds en bas à droite ne sont pas calculés puisque le premier nœud renvoie la valeur 2 et donc que le nœud au niveau supérieur (qui correspond au niveau où l'on cherche à minimiser α) rendra à minimum la valeur 2 même si les deux autres fils donnent des valeurs plus grande. C'est pourquoi, on n'effectue pas le calcul pour les 2 autres nœuds.

L'algorithme possède les propriétés suivantes :

- Il est **complet** si l'arbre de jeu est fini.
- Il est **optimal** si l'adversaire joue aussi de façon optimale.
- L'appel s'effectue sur la racine root : $\text{AlphaBeta}(\text{root}, -\infty, +\infty)$
- Au mieux, on parcourt $\sqrt{n}$ nœuds contre n pour Minimax.

6.2 Exemples

Nous allons présenter trois exemples de jeux pour illustrer les évolutions et l'application des méthodes présentées.

6.2.1 Othello

Le jeu othello connu également sous le nom de reversi, correspond à une table de jeu de 8x8 cases où sont placés, au début du jeu, 4 pions (deux noirs et deux blancs). Chaque joueur joue à tour de rôle sauf s'il ne peut pas jouer, à ce moment là, il passe son tour. Un joueur peut jouer à partir du moment où il construit une ligne (horizontale, verticale ou diagonale) entre

un pion à lui présent sur le plateau et son nouveau pion contenant au moins un pion adverse entre ses deux pions. Les blancs commencent la partie et on a donc quatre positions possibles (cercle blanc vide) comme illustré dans la Figure 1 de la figure 6.3 ci-dessous.

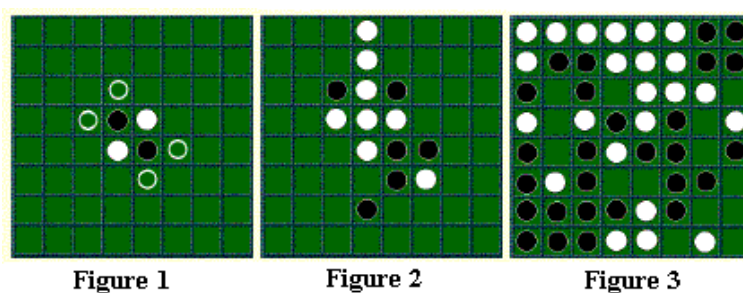


FIGURE 6.3 – Exemple du jeu Othello

Les phases de jeu se déroulent en une partie de 60 demi-coups répartis en trois phases :

- **Début** (Figure 1) environ 4 demi-coups : Ouverture pré définie à partir de bibliothèque
- **Milieu** (Figure 2) : on utilise des valeurs tactiques variables pour chacune des cases. Ainsi les algorithmes Minimax ou AlphaBeta vont pouvoir calculer la valeur des coups par rapport aux cases occupées. Par exemple en début de partie les valeurs sont données par le tableau suivant :

500	-150	30	10	10	30	-150	500
-150	-200	0	0	0	0	-200	-150
30	0	1	2	2	1	0	30
10	0	2	16	16	2	0	10
10	0	2	16	16	2	0	10
30	0	1	2	2	1	0	30
-150	-200	0	0	0	0	-200	-150
500	-150	30	10	10	30	-150	500

Mais une des difficultés est de faire évoluer ces valeurs au cours de la partie car elles impactent directement sur l'efficacité des coups proposés par l'algorithme.

- **Fin** (Figure 3) environ 15 demi-coups : Calcul par différence

6.2.2 Les échecs

Ce jeu n'est plus à présenter. Mais il est à noter qu'il est sans nul doute, le jeu le plus étudié dans de nombreux ouvrages sur des méthodes d'apprentissage, d'ouverture ou d'analyse des parties des grands maîtres (plus grand joueur d'échec).

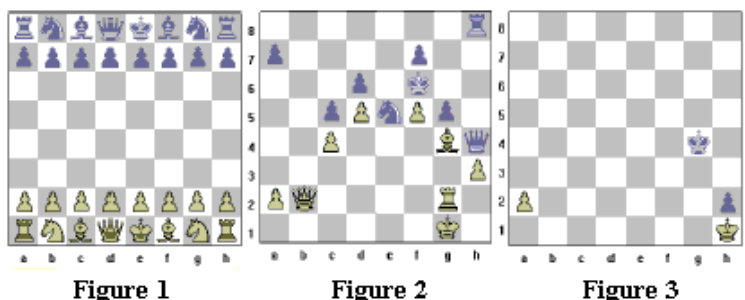


FIGURE 6.4 – Exemple du jeu d'échec

Les phases de jeu se déroulent en une partie entre 10 et +100 demi-coups répartis en trois phases :

- **Début** (Figure 1) environ 6 demi-coups.
- **Milieu** (Figure 2) : Valeur tactique.

Dame	Tour	Fou	Cavalier	Pion
9	5	3	3	1

- **Fin** (Figure 3) lorsqu'il ne reste plus que 8 pièces sur l'échiquier.

Pour optimiser les calculs, on peut être tenté de découper l'échiquier en sous parties pour ne s'intéresser qu'à une partie. Même si l'approche est généralement bénéfique, elle peut entraîner des erreurs dues à l'**effet de bloc**. Dans la figure 6.5 ci-dessous, nous voyons comment les noirs manœuvrent pour mettre les blancs en échec et mat mais le problème est qu'ils ont complètement oublié les pièces blanches en bas à gauche. Ainsi la dame en b2 interdit le mouvement du cavalier e5 pour conclure la partie car le mouvement est illégal car mettant le roi noir en échec.

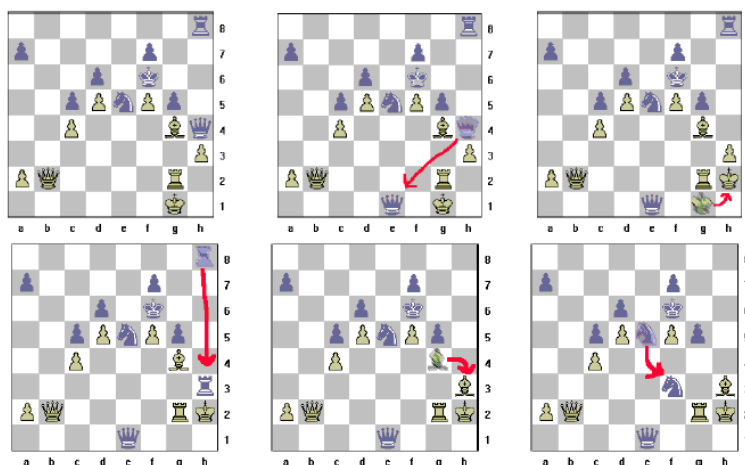


FIGURE 6.5 – Exemple effet Bloc

Un autre problème lié au calcul du meilleur coup par rapport au point qu'il permet de gagner est ce que l'on appelle l'**effet d'horizon**. Celui-ci est juste là pour rappeler que la méthode

est efficace par rapport à une profondeur donnée et donc limite ainsi la vision de la partie. Dans l'exemple donné par la figure 6.6 ci-dessous. Les blancs peuvent jouer de deux façons. La première permet de prendre le pion noir et ne présente que cet intérêt. La seconde permet d'avancer le pion pour le transformer en une autre pièce une fois le bord adverse atteint. Mais, pour cela, il doit être avancé dès ce coup car sinon le roi noir pourra l'intercepter et la partie se finira en pat.

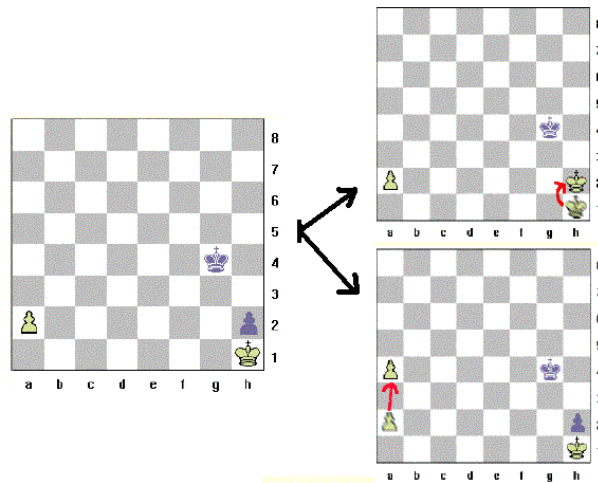


FIGURE 6.6 – Exemple effet d’horizon

6.2.3 La contrée

La contrée est le jeu de cartes basé sur les annonces le plus populaire, comme peut l’être le bridge. On le trouve sous d’autres appellations comme la belote contrée ou la coinche suivant les régions.



FIGURE 6.7 – Exemple jeu de la contrée

Les phases de jeu se déroulent en une partie se déroule en deux temps :

- **Annonce** : elle permet d’identifier et d’échanger des informations entre les joueurs. Elles sont généralement codifiées de la façon suivante :

Annonce	80	90	100	110	120	130	...
Jeu	Valet ou 9 et 2 As	Valet et 9 et 2As	Maître aux Atouts	MA et reste 3 plis	MA et reste 2 plis	1 pli	...

Ajout : +10 pour le 9, +20 pour le Valet, +10 par As, +10 par dix second ou filante,...

— Partie se décompose suivant deux principes :

- Règles du type : **Si** allé **alors** Jouer atout **sinon** Jouer ses as
- Probabilité : Déduction sur les mains de chacun, lors des annonces, puis les coefficients sont mis à jour pendant le jeu suivant les cartes jouées.

6.3 Exercices chapitre 6

Exercice 1 - Algorithmes des jeux

Le jeu du *Tic-Tac-Toe* se joue sur une grille 3×3 , chaque joueur coche à tour de rôle une case, celui qui en aligne 3 a gagné.

Remarque : on notera la case vide par 0, la case du joueur A (max) par 1 et la case du joueur B (min) par 2.

Questions :

1. Réaliser un prédicat `successeurs(C, Liste)` qui calcule toutes les coups possibles à partir de cette case et forme la Liste solution. On distinguera les deux cas suivant le joueur qui doit jouer (max, min).
2. Réaliser un prédicat `gain_possible(A, B, C, V)` qui calcule la valeur V de la configuration des trois cases A, B, C.
3. Réaliser un prédicat `h(C, V)` qui joue le rôle de l'heuristique en calculant la valeur du coût V de la configuration C. On prendra comme valeur max pour un coup gagnant 999 ou -999 suivant si on est dans la configuration max ou min.
4. Réaliser un prédicat `gagnant(C, V)` qui renvoie la valeur V égale à 999 ou à -999 suivant si la configuration C contient une position gagnante pour max ou min.
5. Réaliser deux prédicats `max_children(L, Alpha, Beta, Val, S, P)` et le deuxième `min_children(L, Alpha, Beta, Val, S, P)` qui calculent suivant une liste L de coups possibles le meilleur coup possible pour une profondeur P en utilisant le principe Alpha-Beta. La valeur du meilleur coup est renvoyée dans Val et le chemin sera S.
6. Réaliser le prédicat `alphaBeta(L, Alpha, Beta, Val, S, P)` qui calcule suivant une configuration C le meilleur coup possible pour une profondeur P en utilisant le principe Alpha-Beta. La valeur du meilleur coup, dont le chemin sera S, est renvoyée dans Val.

[Correction.](#)

7. Planification

Nous présentons dans ce chapitre les notions liées à la planification.

- La [leçon 1](#) correspond à une présentation de la problématique et à un cas particulier qui est l'ordonnancement des tâches sur des machines.
- La [leçon 2](#) présente sur un exemple comment effectuer la planification de tâches en général.

Compétence à acquérir dans ce chapitre est la décomposition d'un problème de planification ou d'ordonnancement. Comment le traiter avec une approche simple comme le Job-shop ou de façon plus générale avec le graphe de potentiels et le calcul de chemin critique.

7.1 Contextes

Ce chapitre concerne une problématique liée à la recherche d'optimisation. On peut classiquement utiliser les méthodes vu précédemment mais pour être plus efficace dans ce domaine il a été proposé plusieurs algorithmes spécifiques. Ainsi la planification est devenue une branche d'étude à part entière.

Qu'est-ce que la planification? C'est la science qui a pour objet l'établissement de programmes économiques.

7.1.1 Problématique

Le but est de fixer les dates de démarrage des tâches de façon à respecter les contraintes selon un critère objectif à optimiser.

Les données du problème :

- Ensemble de tâches : $I = \{t_1, t_2, \dots, t_n\}$.
- Chaque tâche t_i possède une durée d_i .
- Un ensemble de contraintes C sur I .

Les types de contraintes :

- Contrainte de **précédence** : $\text{prec}(i, j)$
la tâche i doit être terminée avant de pouvoir commencer l'exécution de la tâche j .
- Contrainte de **ressource** :
 - ressource **consommable** : la ressource disparaît lors de son utilisation
 - ressource **renouvelable** : elle se décompose en deux grandes familles. La première est les disjonctives dont le renouvellement s'effectue indépendamment du système, comme l'électricité qui est renouvelée mais toujours disponible de la vision du système. La seconde est cumulative car elle nécessite du temps pour son renouvellement, comme le changement du tonner dans une imprimante.

Les critères d'optimisation :

- minimisation de la durée totale
- minimisation du nombre de tâches en retard
- minimisation des consommables ...

7.1.2 Job-Shop

C'est un ordonnancement de tâches sur des machines.

Les données sont :

- nombre de machines : m
- Séquences d'opérations (= jobs) : n

Les contraintes sont :

- Les ressources sont les machines (contrainte disjonctive)
- Les contraintes de précédence sont entre les jobs

Les problèmes que l'on cherche à résoudre :

- On cherche un ordre d'exécution des opérations (des jobs) sur chaque machine.
- On suppose qu'un job ne passe qu'une fois par machine.
- Nombre de solutions possibles : $(n!)^m$

On va chercher une solution approximative en utilisant des heuristiques pour obtenir une solution. Ensuite, on pourra affiner cette solution en utilisant un algorithme de type Branch and Bound.

Exemple : Soient deux machines M_1 et M_2 et 3 jobs s'exécutant suivant le temps suivant :

Machine	Job 1	Job 2	Job 3
M_1	2	2	4
M_2	3	5	5

Sans ordre sur les machines, l'heuristique la plus efficace est de mettre en premier la tâche de durée **maximale**. Le Gantt de la figure 7.1 suivante illustre cette approche :

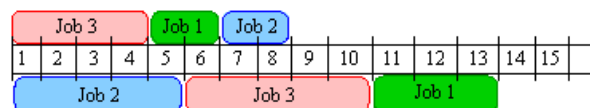


FIGURE 7.1 – Exemple Job shop sans ordre avec heuristique

Pour mieux se rendre compte de l'effet de l'heuristique, nous présentons dans la figure 7.2 la même approche mais sans avoir respecté l'heuristique.

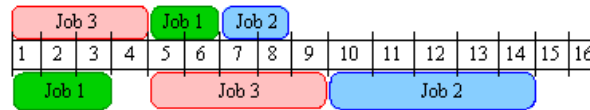


FIGURE 7.2 – Exemple Job shop sans ordre

Avec un ordre sur les machines, l'heuristique la plus efficace est de mettre en premier la tâche de durée **minimale**. Le Gantt de la figure 7.1 suivante illustre cette approche :

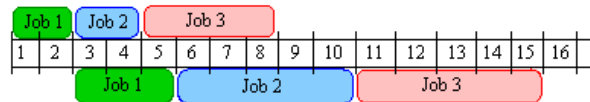


FIGURE 7.3 – Exemple Job shop avec ordre et heuristique

Pour mieux se rendre compte de l'effet de l'heuristique, nous présentons dans la figure 7.4 la même approche mais sans avoir respecté l'heuristique.

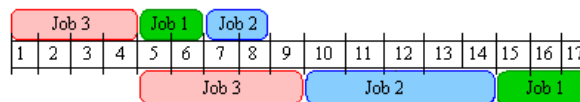


FIGURE 7.4 – Exemple Job shop avec ordre

7.1.3 Les autres types de machines

- **Machine unique** : l'ensemble des tâches à réaliser est fait par une seule machine. Les tâches correspondent à une seule opération qui nécessite la même machine. L'une des situations intéressantes pour ce type d'approche est le cas où on est devant un système de production comprenant une machine goulot qui influence l'ensemble du processus. L'étude peut alors être restreinte à l'étude de cette machine.
- **Machines parallèles** : on dispose d'un ensemble de machines identiques pour réaliser les tâches. Les tâches se composent d'une seule opération et un travail exige une seule machine. L'ordonnancement s'effectue en deux phases : la première phase consiste à affecter les travaux aux machines et la deuxième phase consiste à établir la séquence de réalisation sur chaque machine.
- Ateliers à **cheminement unique (Flow Shop)**. Un atelier à cheminement unique est un atelier où le processus d'élaboration de produits est dit « linéaire », c'est-à-dire que les étapes de transformation sont identiques pour tous les produits fabriqués. Selon les types de produits élaborés, on distingue la production continue et la production discrète. La production continue correspond à la fluidité du processus et à l'élimination des problèmes d'approvisionnements et de stockages. La production discrète de masse s'applique principalement aux produits de grande consommation fabriqués à la chaîne. Dans

les deux cas, les machines peuvent être dédiées à une opération précise. Elles sont implantées en fonction de leur séquence d'intervention dans la gamme de production.

— Les autres configurations connues sont :

- les ateliers de type **flow shop hybride** : il s'agit d'ateliers flow shop dans lesquels un « étage » donné de la fabrication peut être assuré par plusieurs machines en parallèle. Dans ce genre d'ateliers, tout travail passe par chaque étage et l'ordre de passage sur les étages est le même pour chaque travail. Ce type d'ateliers est également appelé « atelier à cheminement unique avec machines en exemplaires multiples » ;
- les ateliers à cheminement libre (**open shop**) : chaque produit à traiter doit subir un ensemble d'opérations sur un ensemble de machines, mais dans un ordre totalement libre ;
- les ateliers **flexibles** : ces ateliers sont caractérisés par un niveau d'automatisation élevé, cherchant par là un compromis entre flexibilité et productivité. Ils sont à la base des ateliers à cheminement multiples où les principales tâches (stockage, traitement de pièces, manutention...) sont automatisées.

Soit l'ordonnancement mono-machine (machine unique) suivant avec 5 tâches à réaliser de façon indépendante :

Identification	T_1	T_2	T_3	T_4	T_5
Temps de traitement	5	12	21	3	16
Date promise	54	26	31	8	27

Nous allons présenter quatre approches qui sont le plus souvent utilisées pour effectuer un ordonnancement.

Premier arrivée, premier servi C'est une méthodologie d'ordonnancement simple puisque les tâches sont traitées dans l'ordre de commandes (ici leur identification) :

Ordre	Date actuelle	Temps de traitement	Date promise	Retard
T_1	0	5	54	0
T_2	5	12	26	0
T_3	17	21	31	7
T_4	38	3	8	33
T_5	41	16	27	30
	57			

Ainsi il y aura 3 tâches en retard et un temps de retard moyen de $(7+33+30)/5 = 70/5 = 14$

Temps de traitement le plus court C'est une méthodologie qui consiste à faire passer en priorité les tâches qui ne nécessitent pas un traitement trop long pouvant ainsi servir les clients le plus rapidement possible :

Ordre	Date actuelle	Temps de traitement	Date promise	Retard
T_4	0	3	8	0
T_1	3	5	54	0
T_2	8	12	26	0
T_5	20	16	27	9
T_3	36	21	31	26
	57			

Ainsi il y aura 2 tâches en retard et un temps de retard moyen de $35/5 = 7$

Date promise le plus tôt C'est une méthodologie qui a comme objectif d'essayer de tenir ses engagements et effectuant les tâches par rapport à la date de livraison initialement prévue :

Ordre	Date actuelle	Temps de traitement	Date promise	Retard
T_4	0	3	8	0
T_2	3	12	26	0
T_5	15	16	27	4
T_3	31	21	31	21
T_1	52	5	54	3
	57			

Ainsi il y aura 3 tâches en retard et un temps de retard moyen de $28/5 = 5,6$

Ratio critique C'est une méthodologie qui a comme objectif de traiter les tâches les plus proches de leur échéance. Ainsi il sera choisi la tâche ayant le plus haut ratio critique à chaque date actuelle. Il est à noter que les tâches en retard auront un score négatif et donc ne seront pas privilégiées. La formule pour calculer un ratio critique est (Temps de traitement / (date promise - date actuelle)). Nous allons effectuer un tableau calculant les ratios critiques pour chaque tâche puis nous donnerons l'ordonnancement obtenu :

Date Actuelle \ Tâches	T_1	T_2	T_3	T_4	T_5	
0	0,09	0,46	0,67	0,37	0,59	On choisit T_3
21	0,15	2,4	–	-0,23	2,66	On choisit T_5
37	0,29	-1,09	–	-0,1	–	On choisit T_1
42	–	-0,75	–	-0,08	–	On choisit T_4
45	–	-0,63	–	–	–	On choisit T_2

Ordre	Date actuelle	Temps de traitement	Date promise	Retard
T_3	0	21	31	0
T_5	21	16	27	10
T_1	37	5	54	0
T_4	42	3	8	37
T_2	45	12	26	31
	57			

Ainsi il y aura 3 tâches en retard et un temps de retard moyen de $78/5 = 15,6$

7.2 Graphe de potentiels

C'est une méthode générale, pour résoudre le problème de façon générale, nous utilisons une représentation à l'aide de graphe de potentiels. Il est aussi connu sous le nom de diagramme de PERT (pour "Program Evaluation and Review Technic") Il se compose en les éléments suivants :

- Les sommets représentent les tâches.
- Les arcs correspondent aux contraintes de précédence.
- La valuation des arcs (p_i) correspondent à la durée des tâches.

Propriétés :

- Le graphe est sans circuit et peut se décomposer par niveau permettant ainsi d'avoir un ordre partiel entre les sommets donné par la précédence des tâches.
- On effectue le calcul en trois phases pour résoudre le problème en calculant un chemin de durée maximale :
 - Construction du graphe de potentiels
 - Date de démarrage au plus tôt
 - Date de démarrage au plus tard

Exemple de graphe de potentiels, soit le problème constitué par les tâches suivantes avec leurs durées et leurs précédences :

Tâche(t_i)	A	B	C	D	E	F	G	H	I	J
Durée(p_i)	7	3	1	8	2	1	1	3	2	1
Précédence		A	B	A	C, D	C, D	C, D	F	H	E, I, G

On ajoute deux sommets α et ω :

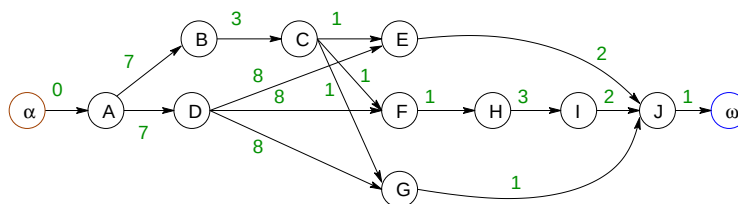


FIGURE 7.5 – Exemple de graphe de potentiel

Date de démarrage au plus tôt

Le calcul de la date au plus tôt t_i pour chaque nœud se calcule avec la formule suivante :

- $t_\alpha = 0$
- $t_i = \max_{j \in \text{précédent}(i)} (t_j + p_j)$

On obtient dans la figure 7.6, le résultat écrit en rouge à partir de l'exemple précédent.

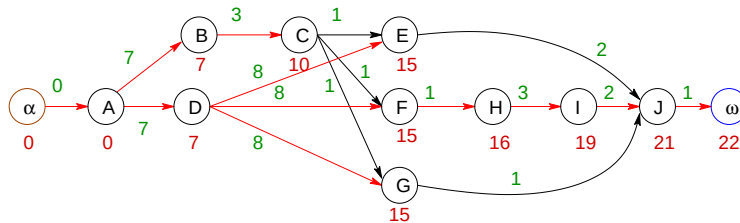


FIGURE 7.6 – Exemple de graphe au plus tôt

Date de démarrage au plus tard

Le calcul de la date au plus tard T_i pour chaque nœud se calcule avec la formule suivante :

— $T_\omega = t_\omega$

— $T_i = \min_{j \in \text{suivant}(i)} (T_j - p_i)$

On obtient le résultat écrit en bleu à partir de l'exemple précédent

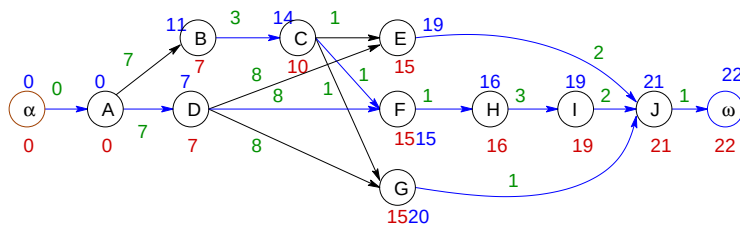


FIGURE 7.7 – Exemple de graphe au plus tard

On obtient ainsi l'enchaînement des tâches sachant que celui-ci prendra 22 unités de temps. Une autre information intéressante est donnée par l'encadrement au plus tôt et au plus tard sur les tâches pour savoir celles sur lesquelles on peut prendre du retard sans avoir d'impact sur le reste. Par exemple, la tâche C doit commencer dans 10 unités de temps au minimum et au maximum 14 unités, ceci permet d'avoir une marge de 4 unités de temps.

Pour plus d'information sur l'ordonnancement, vous pouvez vous référer à [GOT04].

7.3 Exercices chapitre 7

Exercice 1 - Ordonnancement mono-machine

Une usine doit réaliser les 5 tâches suivantes :

Identification	T_1	T_2	T_3	T_4	T_5
Temps de traitement	11	29	31	1	2
Date promise	61	45	31	33	32

Questions : Donner l'ordre, le retard, le retard moyen et le nombre de tâches en retard en utilisant la stratégie suivante :

1. Premier arrivée premier servi.
2. Temps de traitement le plus court.
3. Date promise la plus tôt
4. Ratio critique (Temps de traitement / (date promise - date actuelle))

[Correction.](#)

Exercice 2 - Ordonnancement multi-machine

Une usine doit réaliser les 5 tâches suivantes sur deux machines A et B :

Identification	T_1	T_2	T_3	T_4	T_5
Machine A	5	1	9	3	10
Machine B	2	6	7	8	4

Questions :

1. En utilisant la règle de Johnson : "Un ordonnancement, ord , des tâches T_i est un ordonnancement de Johnson si et seulement si pour toute i on a $\min(A_{ord(i)}, B_{ord(i+1)}) < \min(A_{ord(i+1)}, B_{ord(i)})$ ", donner l'ordre des tâches et le temps nécessaire à la réalisation de celles-ci.
2. Donner les prédicats nécessaires pour implanter cette règle. Sachant que l'on appelle le processus de résolution avec le prédicat `johnson/4` où le premier argument est la liste des traitements sur la machine A, le deuxième sur la machine B, le troisième donne la solution (la liste des tâches dans l'ordre), le quatrième donne le temps total de traitement.

[Correction.](#)

8. Apprentissage

Nous présentons dans ce chapitre les notions d'apprentissage et d'évolution de la connaissance. Nous nous limiterons à des exemples simples sans rentrer dans la problématique de la remise en question de la connaissance acquise.

- La [leçon 1](#) présente la notion d'apprentissage et l'intérêt pour l'Intelligence Artificielle.
- La [leçon 2](#) présente deux types d'algorithmes associés à l'apprentissage
- La [leçon 3](#) présente les réseaux de neurones comme une illustration de/pour l'apprentissage.

8.1 Présentation

L'apprentissage c'est l'action d'apprendre. Mais c'est aussi l'ensemble des méthodes permettant d'établir chez les êtres vivants des connexions entre certains stimuli et certaines réponses. Finalement, on peut se poser la question : est-ce qu'apprendre est-il un signe d'intelligence et que permet-il vraiment ?

- capacité d'association : on peut ainsi mémoriser des informations issues de nos sens et les associer avec des concepts comme les couleurs, les odeurs, les souvenirs, les mots et les sentiments. Chacun pourra être réutilisé pour communiquer, appréhender son environnement ou encore apprendre des concepts plus complexes.
- capacité d'abstraction : il y a plusieurs niveaux dans l'apprentissage. Les deux plus connus sont pour le premier le bachotage ou le par cœur et pour le second est la compréhension où le sujet est suffisamment dominé pour en comprendre les concepts. Ainsi on ne retient pas les différents exemples, mais bien les règles sous-jacentes.
- explicabilité : en apprenant comment les choses fonctionnent, on peut comprendre les tenants et les aboutissants. On a ainsi un comportement rationnel, car on peut expliquer chaque choix.
- capacité de prévision : puisque l'on peut expliquer, on peut aussi anticiper, car à priori, une cause doit avoir un effet. Celui-ci étant, à priori, similaire au précédent dans les mêmes conditions. Ainsi on a un plus grand contrôle sur ce qui peut se passer.
- répliquabilité : à partir du moment où l'on peut prévoir, on peut ainsi planifier et organiser. Ainsi les cycles de la vie peuvent être prévus et engendrent moins d'aléatoires dans le comportement.
- capacité de révision : mais il ne faut pas oublier que le monde évolue et que l'information est aussi dynamique ou ponctuelle. Il faut se tenir informé de ses évolutions pour pouvoir réviser ses connaissances ou son jugement. On parle aussi de capacité d'adaptation.

Il n'existe pas d'algorithme précis ou universel. Par exemple, la reconnaissance des visages est très difficile même si on arrive à reconnaître des objets. Tout cela est dû au fait qu'il y a beaucoup de données à traiter ou d'éléments discriminants. On trouve cela dans le marketing avec le "market analysis" pour savoir qui consomme quoi et comment, plus généralement dans

le "data mining" qui est le traitement de l'information dans le but d'identifier des tendances ou des motifs présents dans les données. Un dernier point est l'évolution de cette information, comme la transformation avec le temps d'un visage. Mais le plus connu est la prévision météo. Elle essaie avec une modélisation, à base de chaîne de Markov, de prédire par rapport aux données des années précédentes et des informations actuelles de deviner le temps pour les jours à venir.

Exemple d'applications :

- Analyse de **données** : des marchés financiers, banques
- Analyse de **comportements** : de traitements médicaux, bio-informatique
- **Contrôle et optimisation** de réseaux de distribution : d'électricité, d'eau, de services de (télé-)communications, de trafic urbain (aérien), de dépenses énergétiques d'un immeuble ou d'une habitation
- **Personnalisation** de services : personnalisation des interfaces (TV interactive)
- **Identification** d'individus : écriture (signature), voix, empreintes digitales ou rétinienne, visage ...

La révolution de ces dernières années est l'apparition des Large Language Models (LLM) qui ont révolutionnés le traitement à l'aide des IA Générative. Les points clés sont l'utilisation des Transformer qui ont permis de mieux cibler les données dans le modèle [VSP⁺23]. Ceci a donné le nom à l'un des plus connus créés par Open AI Generative Pre-trained Transformer (GPT).

Définition : T. Mitchell, Machine Learning McGraw Hill, 1997

"Un programme d'ordinateur est capable d'apprendre à partir d'une expérience E et par rapport à un ensemble T de tâches et selon une mesure de performances P, si sa performance à effectuer une tâche T mesurée par P s'améliore avec l'expérience E."

Nous allons utiliser l'exemple des échecs pour illustrer la définition de l'apprentissage de T. Mitchell :

- T : jouer aux échecs et gagner
- P : % de victoires
- E : possibilités de jouer contre quelqu'un

8.1.1 Apprentissage et inférence

Il y a l'apprentissage "*par cœur*", où l'on répète comme un perroquet ce que l'on vous a dit. Ou il existe d'autres approches où l'on va essayer d'aller plus loin avec l'information reçue en inférant de nouvelles informations :

- Inférence déductive : à partir de A, et de $A \rightarrow B$, inférer B.
- Inférence abductive : à partir de B, et de $A \rightarrow B$, inférer A.
- Inférence inductive :
 - "j'ai été malade après avoir mangé trop de gâteaux de Noël"
 - "j'ai été malade après avoir mangé trop de gâteaux de Pâques"
 - donc "je serai malade si je mange trop de gâteaux"
- Attention cependant aux conclusions hâtives, par exemple :
 - "Oncle Jean a été hospitalisé et il est mort"

- "Grand-mère Marie a été hospitalisée et elle est morte"
- Il ne faut pas conclure "Quiconque se fait hospitaliser ne reviendra pas chez lui vivant".

Il y a trois types d'apprentissage :

- **Supervisé** : l'apprenant reçoit des exemples d'apprentissage comprenant à la fois les valeurs d'entrée et de sortie.
- **Non-supervisé** : l'apprenant reçoit des exemples d'apprentissage ne comprenant que les valeurs d'entrée (pas de valeurs de sortie). C'est lui qui va inférer des relations entre les éléments et les organiser.
- **Par renforcement** : l'apprenant reçoit des exemples d'apprentissage ne comprenant que les valeurs d'entrée (pas de valeurs de sortie). L'apprenant doit recevoir une indication en retour qui l'aide à déterminer s'il est dans la bonne direction, mais cette indication est souvent vague, ambiguë ou tardive.

L'apprentissage peut être réalisé suivant deux manières :

- **"Par paquets"** ou **"batch"** : processus dans lequel des exemples d'apprentissage sont pris en compte tous à la fois.
- **"En ligne"** : processus dans lequel des exemples d'apprentissage sont pris en compte un à un.

On peut alors définir la fonction d'apprentissage f de la façon suivante :

- Soit X l'ensemble de tous les exemples possibles.
- Une classe C est un sous-ensemble de X .
- L'ensemble d'exemples d'apprentissage est $\{(x, y)\}$, où $x \in X$ et $y \in \{0,1\}$.
- Trouver une fonction f telle que :
 - $f(x) = 1$, si $x \in C$
 - $f(x) = 0$, si $x \notin C$
- On appelle espace d'hypothèses H , l'ensemble de toutes les fonctions f possibles.
- On peut généraliser de $H = \{f : X \rightarrow \{0,1\}\}$ à $H = \{f : X \rightarrow Y\}$ où X et Y sont des espaces de valeurs d'entrée et de sortie arbitraires.

8.1.2 Les phases pour faire de l'apprentissage

Les techniques d'apprentissages (ou "*machine learning*") se décomposent en sept étapes :

1. La **récolte** de données : cette phase consiste à récupérer les informations concernant le système à étudier. Il permet d'identifier les variables et leurs domaines de définition qui caractérisent le système. La volumétrie des données doit être importante pour permettre de couvrir le plus grand nombre de représentants du système.
2. La **préparation** de données : cette phase permet de normaliser les données et vérifier la complétude. En effet, il est possible que certaines variables n'aient pas de valeur, par exemple un capteur défaillant pendant une partie de l'échantillonnage. Cette phase est importante puisqu'elle permettra ensuite d'utiliser les algorithmes d'apprentissage associés au choix du modèle. Il permet de déterminer comment décomposer le corpus en deux : l'un pour l'apprentissage (corpus d'étude) et l'un pour l'évaluation du modèle (corpus de validation).

3. Le **choix d'un modèle** : cette phase permet de choisir les algorithmes d'apprentissage. Ces algorithmes vont construire un modèle sur les données d'entrée.
4. L'**entraînement** : cette phase consiste à faire traiter les données d'apprentissage par l'algorithme retenu pour construire le modèle à partir du corpus d'étude.
5. La validation du **modèle** : cette phase permet d'évaluer le modèle obtenu au regard de critères (voir la section 8.1.4 sur l'évaluation). Lorsqu'on est en apprentissage supervisé, il permet de vérifier que le modèle établi à l'aide du corpus d'étude donne de bons résultats sur le corpus de validation.
6. Le **réglages** des (méta-)paramètres : cette phase permet d'ajuster les paramètres associés aux algorithmes aux regards des évaluations pour permettre d'obtenir de meilleurs résultats.
7. L'**application** : cette phase concerne l'utilisation de modèle sur de nouvelles données. il permet de prédire les informations complémentaires pour :
 - **classifier** les éléments : donner la valeur concernant l'attribut sur lequel on souhaite faire la classification.
 - faire une **régression** : pour prédire une quantité à partir d'autres.
 - **détecter une anomalie** : un élément dont les valeurs des attributs ne sont pas cohérentes avec les autres
 - faire des **partitionnement** ("*clustering*") : pouvoir détecter des schémas ("*patterns*") dans les données et ainsi regrouper les éléments par rapport à ces schémas.

Pour être sûr d'avoir bien compris les différents types d'applications, vous pouvez faire le quiz proposé en [exercice 1](#).

8.1.3 Traitement(s) des données

Le traitement des données est un travail important puisqu'il nécessite différentes étapes avant que les données soient utilisables dans les algorithmes d'apprentissage. C'est le travail de la science des données (*data science* en anglais).

Naturellement, le traitement auquel on pense est la préparation des données. En effet, c'est une étape clé dans les méthodes d'apprentissage. Elle possède comme première (sous-)étape la vérification des données. On trouve aussi dans la littérature les termes : nettoyage, assainissement ("*Data cleaning, cleansing, scurbbing*" en anglais), cela consiste à :

- Définir et déterminer les données inutilisables (incomplètes, incohérentes);
- Chercher et identifier les doublons (pour les corriger, supprimer ou maintenir);

Dans les faits, la réalité est beaucoup plus complexe puisque pour préparer les données, il faut déjà les avoir collectées. la figure 8.1 reprend ainsi les différentes étapes avant d'arriver à l'exploitation de ces données par les algorithmes d'apprentissage. Chacune de ces six étapes va faire intervenir des compétences ou des métiers différents :

- La première étape (collecter) est généralement faite par les développeurs, par les testeurs ou directement générés par le système réel.
- L'ingénieur de données va intervenir sur les étapes 2 et 3 pour en assurer la préservation et le stockage.
- L'analyste de données ("*Data Analyst*") va intervenir sur les étapes 3, 4 et 5 pour vérifier la cohérence des informations entre elles.

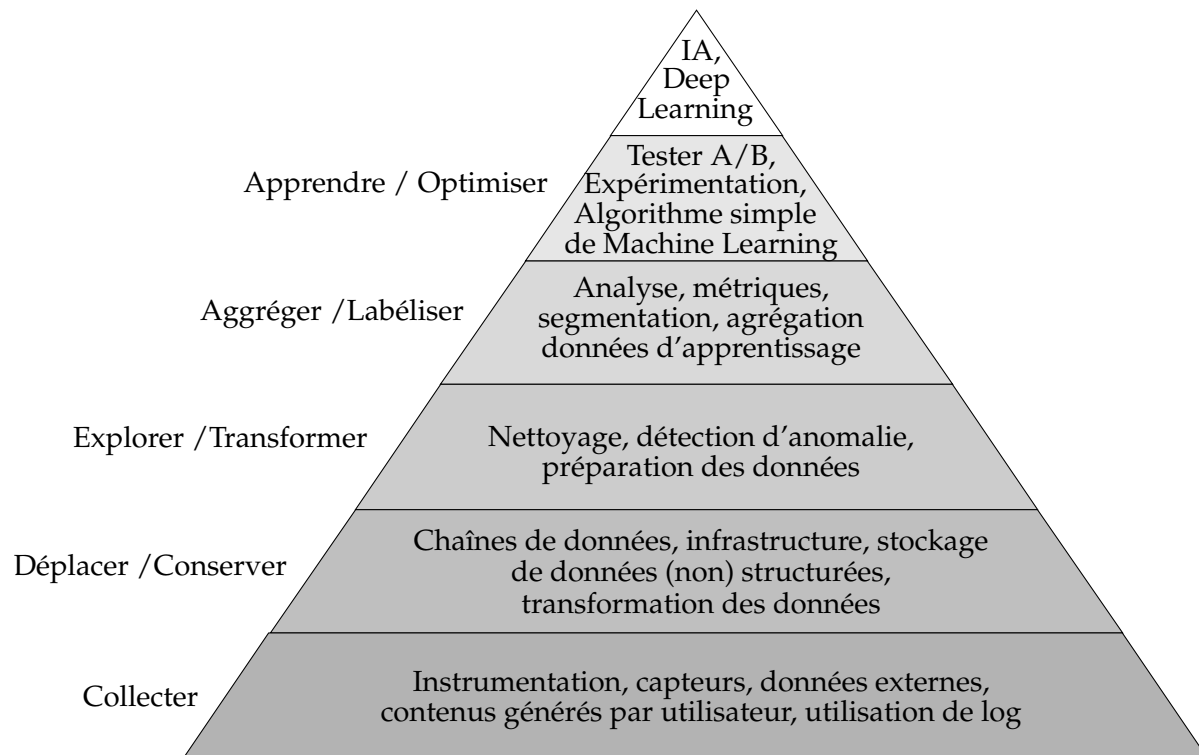


FIGURE 8.1 – Schéma du scientifique des données

- Le scientifique des données (*"Data Scientist"*) va intervenir sur les étapes 5 et 6 amenant une partie du savoir métier et ainsi voir la convergence entre les informations réelles et celles obtenues par les méthodes d'apprentissage.

Une fois ce premier travail effectué et suivant les algorithmes qui seront utilisés, il faut transformer ces données pour pouvoir les traiter de façon cohérente, plus exactement sans que certaines données masquent ou absorbent les autres. On appelle cela la normalisation. La normalisation revient à appliquer une fonction sur la donnée. Il y a plusieurs techniques pour normaliser les variables. Nous présentons ici deux des plus utilisées pour la transformation des données :

1. La **normalisation** à l'écart-type a été utilisée lorsque les données seront traitées indépendamment. Elle permet de ramener les domaines de définition de la variable sur un intervalle uniforme pour éviter les effets liés à la distribution des valeurs. Pour obtenir la nouvelle donnée x_n , on divise la donnée x par leur écart-type (et) : $x_n = \frac{x}{et}$
2. La **standardisation**. Elle permet de ramener les domaines de définition des différentes variables dans le même intervalle et de le centrer sur 0.

Pour obtenir la nouvelle donnée x_n , on soustrait à la donnée x et la moyenne des données (m) que l'on divise par leur écart-type (et) : $x_n = \frac{x-m}{et}$

En effet, il faut savoir que les algorithmes vont comparer les éléments entre eux. Ainsi dans le cas de données :

- numérique, ils calculeront un indice de distance à l'aide d'une :
 - distance euclidienne usuelle
 - distance du Chi2

- distance de Manhattan
- non numériques, on utilise un des indices de (dis)similarité. Ils se construisent de diverses manières à partir des quatre nombres suivants (entre deux entités i et j) :
 - a = nombre de caractéristiques communes ;
 - b = nombre de caractéristiques possédées par i et pas par j ;
 - c = nombre de caractéristiques possédées par j et pas par i ;
 - d = nombre de caractéristiques que ne possèdent ni i , ni j /

Les indices les plus courants sont :

- Jaccard : $\frac{a}{a+b+c}$
- Dice : $\frac{2a}{2a+b+c}$
- Concordance : $\frac{a+d}{a+b+c+d}$

8.1.4 L'évaluation

Pour pouvoir connaître la qualité du modèle obtenu, nous devons l'évaluer pour savoir si celui-ci est meilleur que les précédents. Nous proposons ici sept types d'évaluation que l'on retrouve pour les différents types de modèles.

Précision de la classification

La précision de la classification est aussi appelée exactitude ("*accuracy*"). Cela correspond au rapport entre le nombre de prédictions correctes sur le nombre total d'échantillons d'entrée.

Cette évaluation fonctionne bien que si chaque classe a un nombre égal d'éléments dans l'échantillon. Par exemple, si on a 98% d'éléments de la classe A et 2% de la classe B dans notre corpus d'entraînement. Ainsi, notre modèle peut facilement obtenir une précision à 98% simplement en mettant tous les échantillons dans la classe I. Maintenant, lorsqu'on teste le même modèle sur un ensemble comprenant 60% d'échantillons de la classe A et 40% d'échantillons de la classe B, la précision chute à 60%. La précision de la classification reste bonne (puisque à 60%) même si elle est toute relative car il est incapable de repérer les éléments de la classe B. Le véritable problème se pose si l'on analyse le coût de la classification erronée des échantillons de la classe minoritaire. Par exemple, si cette classe correspond à une maladie rare, mais mortelle, le fait de ne pas diagnostiquer la maladie d'un malade est beaucoup plus coûteux que celui d'envoyer une personne en bonne santé faire des tests complémentaires pour affiner la classification, le pronostique dans ce cas.

Perte logarithmique ou Log Loos

La perte logarithmique ou perte de journal, fonctionne en pénalisant les fausses classifications. Cela fonctionne bien pour la classification multi-classes. Pour calculer la perte logarithmique "*Log Loss*", on doit attribuer une probabilité à chaque classe pour tous les échantillons. Supposons qu'il y a N échantillons appartenant à M classes, la perte (LL) est calculée comme suit :

$$LL = \frac{-1}{N} \sum_{i=1}^N \sum_{j=1}^M (y_{ij} * \log(p_{ij}))$$

où :

- y_{ij} : indique si l'échantillon i appartient ou pas à la classe j
- p_{ij} : indique la probabilité que l'échantillon i appartienne à la classe j

La perte n'a pas de limite supérieure et il existe dans l'intervalle $[0, +\infty[$. Une perte proche de 0 indique une meilleure classification. Si la perte est loin de 0, elle indique une moins bonne précision. En général, minimiser la perte de journal montre une meilleure précision au classificateur.

Matrice de confusion

La matrice de confusion fournit une matrice en sortie pour décrire les performances complètes du modèle. Supposons que nous ayons un problème de classification binaire (à deux valeurs). Ainsi, nous avons des échantillons qui appartiennent à deux classes : **Oui** ou **Non**. Notre modèle donne une prédiction pour chaque élément du corpus. En testant notre modèle sur 100 échantillons, nous obtenons la matrice suivante :

	Prédit : Non	Prédit : Oui
Réel : Non	30	6
Réel : Oui	3	61

Il y a 4 termes importants :

- **Vrai positif** : ce sont les cas dans lesquels nous avons prédit **Oui** et le résultat réel était également **Oui**.
- **Vrai négatif** : ce sont les cas dans lesquels nous avons prédit **Non** et dont le résultat réel était **Non**.
- **Faux positif** : ce sont les cas dans lesquels nous avons prédit **Oui** et dont le résultat réel était **Non**.
- **Faux négatif** : ce sont les cas dans lesquels nous avons prédit **Non** et dont le résultat réel était **Oui**.

La précision de la matrice est calculée en prenant la moyenne des valeurs se trouvant sur la "**diagonale principale**", à savoir : $precision = \frac{(\text{Vrai positif} + \text{Vrai Négatif})}{\text{Nombre totale d'échantillons}}$

Soit pour l'exemple : $precision = \frac{30+61}{100} = 0,91$

Pour établir l'erreur on prendra la "**diagonale négative**", à savoir $erreur = \frac{(\text{Faux Positif} + \text{Faux Négatif})}{\text{Nombre totale d'échantillons}}$

Soit pour l'exemple : $erreur = \frac{3+6}{100} = 0,09$

La matrice de confusion va nous servir pour établir les types de métriques suivants.

Espace sous la courbe

L'espace ou l'aire sous la courbe (AUC, "*Area under Curve*") ou encore noté AUC-ROC (espace sous la courbe de caractéristiques de fonctionnement du récepteur, "*Receiver Operating Characteristics*") est l'un des paramètres d'évaluation les plus largement utilisés. Il s'utilise pour la classification à deux valeurs. L'AUC permet d'évaluer la performance de classification du modèle à différent réglage de seuil. Pour cela, nous avons besoin de définir les deux taux suivants :

- **Taux de vrais positifs (TVP)** (Sensibilité, "*sensitivity*") : cela correspond au nombre d'éléments bien classés pour les éléments positifs. Ce taux est défini comme le rapport du nombre de vrais positifs (VP) sur la somme du nombre de faux négatifs (FN) et de vrais positifs : $TVP = \frac{VP}{(FN+VP)}$.
- **Taux de vrais négatifs (TVN)** (spécificité, "*Specificity*") : cela correspond au nombre d'éléments classés éléments négatifs. Le taux est défini comme le rapport du nombre de vrai négatif (VN) sur la somme du nombre de faux positifs (FP) et de vrais négatifs (VN) : $TVN = \frac{VN}{(FP+VN)}$.

Ensuite, l'AUC est calculé en traçant la courbe pour chaque seuil en prenant le TVN en axe des abscisses et le TVP en axe des ordonnées. La courbe mesure la performance pour un problème de classification à différents réglages de seuil. C'est une probabilité qui représente le degré ou la mesure de la séparabilité. Il indique si les modèles sont capables de distinguer les classes. Ainsi, plus la valeur de l'AUC est élevée (proche de 1), meilleur est le modèle.

Score F1

Le score F1 est la moyenne harmonique entre la précision (PRE) et le rappel (RAP, qui correspond à la définition du taux de vrais positifs précédent). Son domaine de définition est sur l'intervalle $[0, 1]$. Il indique la précision du classificateur (combien d'instances il classifie correctement), ainsi que sa robustesse (combien il en a mal classé). Lorsque la précision est grande et que le rappel est petit, la précision est bonne, mais il manque un grand nombre d'instances qui sont difficiles à classer. Ainsi, lorsque le score F1 est élevé, il indique une bonne performance du modèle. Il se calcule de la façon suivante :

$$F1 = 2 * \frac{1}{\frac{1}{PRE} + \frac{1}{RAP}}$$

- Précision (PRE) : il s'agit du nombre de résultats positifs corrects (vrai positif, VP) divisé par le nombre de résultats positifs total (vrai positif et faux positif, FP) : $PRE = \frac{VP}{FP+VP}$
- Rappel (RAP, "*Recall*" en anglais) : il s'agit du nombre de résultats positifs corrects divisé par le nombre de tous les échantillons positifs. Il est donc équivalent au tau de vrai positif : $RAP = \frac{VP}{FN+VP}$

Moyenne des erreurs absolues

La moyenne des erreurs absolues (MAE, "*Mean Absolute Error*") est la moyenne de la différence entre les valeurs d'origine (y_i) et les valeurs prédites (p_i). Cela donne la mesure de la distance entre les prévisions et la production réelle. Cependant, elle ne nous donne aucune idée du sens de l'erreur, c'est-à-dire si nous sous-prédisons les données ou si nous les sur-prédisons. Elle se calcule ainsi :

$$MAE = \frac{1}{N} \sum_{j=1}^N |y_i - p_i|$$

Moyenne de l'erreur quadratique

La moyenne de l'erreur quadratique (MSE, "Mean Squared Error") est assez similaire à la moyenne des erreurs absolues, la seule différence est que le MSE prend la moyenne du carré de la différence entre les valeurs d'origine et les valeurs prédites. L'avantage de MSE vient du fait qu'il devient plus facile de calculer le gradient. Elle se calcule ainsi :

$$MSE = \frac{1}{N} \sum_{j=1}^N (y_i - p_i)^2$$

8.1.5 Les dilemmes

Pour la mise au point des modèles d'apprentissage, nous sommes souvent confrontés à un dilemme entre le biais et la variance. Nous allons voir les concepts associés à ces deux notions. Sachant que de toute façon, nous serons toujours face à l'erreur irréductible qui est due au bruit dans le problème lui-même. Par exemple, si l'on veut effectuer un apprentissage sur les chiffres 4, 7 et 9 sur la base des échantillons donnés dans la figure 8.2, même un humain a du mal à déterminer quel symbole correspond à quel chiffre.

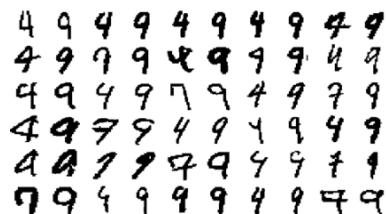


FIGURE 8.2 – Exemple d'erreur irréductible

Prenons comme exemple l'étude de la courbe de poids des lapins par rapport avec leur taille. On voit que les lapins légers ont tendance à être petits et les plus lourds ont tendance à être plus grands, mais après un certain poids, les lapins ne grandissent plus, ils deviennent obèses. L'idée est de pouvoir mettre en place un modèle pour prédire la taille d'un lapin à partir de son poids. On décompose les données en deux ensembles (cf. Figure 8.3a). Nous avons en bleu, les données d'entraînement et en vert les données d'évaluation. Nous proposons deux modèles pour cela. Le premier est une approche à base de régression linéaire qui va couper les données avec une ligne droite (cf. Figure 8.3b). Le second utilise une approche avec des splines qui passent par chaque point des données d'apprentissage (cf. Figure 8.3c).

Biais

Le biais est l'erreur provenant d'hypothèses erronées dans la représentation ou l'algorithme d'apprentissage. En reprenant l'exemple des lapins, on voit que la ligne droite n'est pas assez flexible pour répliquer la vraie relation alors que les splines sont très flexibles et suivent avec les arcs la vraie relation. Lorsqu'on compare les deux modèles en mesurant l'erreur moyenne, les splines l'emportent et ils ont le biais le plus faible. Ainsi, l'incapacité pour un modèle de

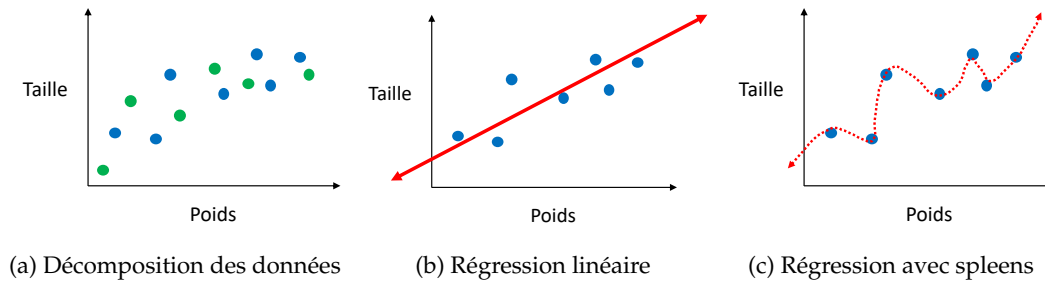


FIGURE 8.3 – Exemple d'apprentissage pour les lapins

machine learning à capturer la vraie relation s'appelle le biais, mais alors d'où peut venir ce biais?

Apprendre nécessite de représenter l'information et de la traiter. Ainsi toute modification de l'information engendre un **biais**. On peut les regrouper en trois grandes catégories :

- Le biais **inductif** qui fournit à l'apprenant une base pour choisir parmi les fonctions d'évaluation ou le modèle d'apprentissage qu'il peut utiliser.
- Le biais **restrictif** qui choisit un ensemble restreint de solutions ou des données :
 - l'ensemble des fonctions linéaires de Réel dans Réel,
 - Représenter l'absence ou la présence d'une valeur et non l'arité,
 - l'ensemble des conjonctions de N variables logiques.
- Le biais de **préférence** qui détermine un ordre sur l'ensemble des solutions possibles :
 - préférer les formules logiques courtes,
 - préférer les polynômes de faibles degrés.

Variance

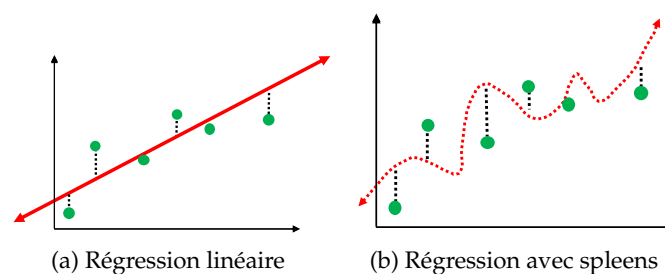


FIGURE 8.4 – Exemple de variance pour les lapins

La variance est l'erreur due à la sensibilité des petites fluctuations de l'échantillon d'apprentissage. En reprenant l'exemple du Lapin, nous allons nous intéresser aux données de validation (les points verts). Si l'on calcule l'erreur moyenne entre les deux régressions (droite - Figure 8.4a et spline - Figure 8.4b), c'est la ligne droite qui gagne. La différence de performance entre les différents ensembles de données (apprentissage et validation) s'appelle la variance.

Cette variance est souvent liée à un corpus complexe qui fait que le modèle qui a été construit lors de la phase d'apprentissage n'a pas bien pu prendre en compte l'ensemble des spécificités du problème. Ainsi lors de l'utilisation de nouvelles données ou dans la phase de validation, les résultats obtenus sont éloignés de ceux prédits par le modèle.

Généralement un bon modèle est celui qui a ni un grand biais et ni une grande variance.

Sur-apprentissage (*overfitting* en anglais) ou sous-apprentissage (*underfitting* en anglais)

Le sur-apprentissage consiste en la "*Mémorisation de tout l'ensemble d'apprentissage plutôt que d'induire un concept général*". On parle aussi d'un biais faible sur les données d'entraînement, mais élevé sur les nouvelles données (ou données de validation). C'est bien le cas sur notre exemple des lapins pour le modèle de régression des spleens.

L'**avantage** est : plus on apprend, plus on "colle" aux données. Mais le risque est qu'à la fin tous les exemples sont appris par cœur, y compris le bruit (les informations contenues dans l'exemple ne servant pas dans l'étude). C'est le phénomène de **sur-apprentissage**. C'est lui qui va conduire à avoir une variance importante.

Pour éviter ce piège, le critère d'arrêt de l'apprentissage doit être défini. Soit $\text{erreur}_{\text{app}}(h)$ l'erreur commise par l'hypothèse h sur les données d'apprentissage, et $\text{erreur}_D(h)$ l'erreur commise par h sur la distribution totale des données.

Définition : On dira que h sur-apprend les données d'apprentissage s'il existe une hypothèse h' telle que :

$$\text{erreur}_{\text{app}}(h) < \text{erreur}_{\text{app}}(h') \text{ et } \text{erreur}_D(h) > \text{erreur}_D(h')$$

Pour corriger le sur-apprentissage, il faut arrêter l'apprentissage aussi tôt que possible, soit :

- cesser d'étendre lorsque le partitionnement n'est plus significatif,
- tester si la partition produite améliore la précision.

Sinon on peut effectuer un élagage, a posteriori, des éléments inutiles dans le graphe de concepts calculés :

- construire l'arbre entier,
- supprimer les nœuds inutiles (test approprié).

Le sous-apprentissage est lorsqu'on a un modèle général, mais qui est incapable de fournir des prédictions précises. On parle ainsi de biais fort sur les données d'entraînement et qui reste élevé sur les données de validation.

8.2 Les approches

Dans les sections suivantes, nous allons voir deux approches d'apprentissages supervisés (version space et arbre de décisions) permettant de faire de la classification à partir d'un corpus d'étude. Puis deux approches de partitionnement (non supervisés, K-Means et MeanShift) et une méthode de classification.

8.2.1 Régression

Il est possible de faire de l'apprentissage supervisé en utilisant des outils mathématiques. On cherche ainsi à décomposer les espaces entre les individus à classer à l'aide d'objets mathématiques.

Régression linéaire

Pour émettre une prédiction, il faut définir le meilleur modèle possible de la variable y en utilisant plusieurs variables indépendantes x avec θ_n : le n ème paramètre de la valeur de la variable indépendante n .

$$y = \theta_0 + \theta_1 \times x_1 + \theta_2 \times x_2 + \dots + \theta_n \times x_n \quad (8.1)$$

Entraîner un système de régression linéaire c'est définir ces paramètres avec pour but de trouver les valeurs des paramètres qui minimisent l'erreur quadratique moyenne (MSE). Pour ce faire, nous pouvons utiliser ;

L'équation normale, rapide même sur des données de grande taille (algorithme en $O(m)$).

La descente de gradient qui corrige petit à petit les paramètres. On initialise les paramètres de façon aléatoire et à chaque itération on réduit le coût de MSE. Cela nécessite de définir un nombre d'itérations et un pas de descente (en hyper-paramètres) un petit pas et/ou un grand nombre d'itérations donne une meilleure précision et à l'inverse un grand pas et/ou un petit nombre d'itérations un gain de temps. Il peut être utile de mettre en place une tolérance pour arrêter le système et d'éviter de faire des itérations inutilement si nous sommes satisfait du résultat. Il faut aussi faire attention, car le pas peut nous faire sauter d'un côté à l'autre du minimum sans jamais le toucher.

La courbe représente le coût pour les différents paramètres. La descente de gradient nécessite de connaître la pente de cette courbe. Pour cela, on utilise la dérivé avec comme objectif de descendre (minimiser) ce coût.

Deux résultats sont alors possibles, tendre vers :

- **Un minimum global** : on a le meilleur résultat possible,
- **Un minimum local** : on est dans une cuvette qui fait croire que l'algorithme a trouvé le minimum global.

Dans la figure 8.5, si l'algorithme de descente de gradient part de 1 ou 4 le résultat va tendre vers le minimum global alors que s'il part de 5, il ira dans le minimum local en pensant que c'est la valeur minimum. Si l'on utilise MSE avec des variables possédant la même échelle, il n'y a pas de minimum local.

La descente de gradient est très lente, elle utilise le jeu d'entraînement complet à chaque itération. Il existe quelques variantes comme :

Descente de gradient stochastique est proche de la descente de gradient sauf que le calcul de la pente se fait sur une observation aléatoire. Les points négatifs de cette méthode sont son irrégularité (l'observation peut faire croître ou décroître) même si globalement la moyenne

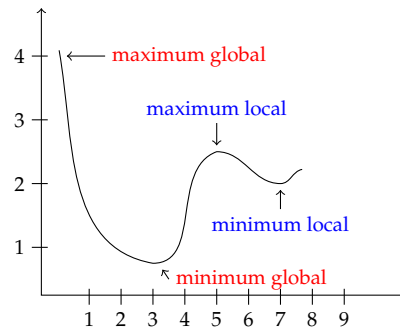


FIGURE 8.5 – Courbe avec explication du minimum global et local

décroît, et la solution n'est pas obligatoirement optimale, mais on en sera proche à la fin. Cependant elle reste plus rapide et ne se bloque pas dans le minimum local.

Descente de gradient par mini-lots ("*mini batch gradient descent*" en anglais) identique à la descente de gradient mais avec plusieurs lancements différents et aléatoires.

Cet algorithme est plus régulier que le stochastique mais moins que le gradient simple.

Pour ces différents problèmes, il est possible de réduire progressivement le pas d'apprentissage, mais il faut alors faire attention à la vitesse de réduction ("*learning schedule*"), ni trop rapide ni trop lente et mettre en place des cycles ("*epoch*") de m itérations entre chaque réduction.

De prime à bord, il y a peu de différence entre ces 3 classificateurs, mais il peut avoir de grosses différences sur les résultats obtenus.

Régression polynomiale

La Régression polynomiale est utilisée pour ajuster des données non-linéaire (voir figure 8.6). Dans notre équation, le degré est signe de précision. S'il n'y a pas assez de degrés, cela amène au sous-ajustement mais il est d'autant plus vrai que s'il y a trop de degrés, cela amène au sur-ajustement. Pour combattre ces deux problèmes, on peut dans le cas où un modèle sous-ajuste changer le modèle ou les variables. Dans le cas où le modèle sur-ajuste, on peut augmenter le nombre d'observations d'entraînement.

Nous pouvons citer les algorithmes basés sur la méthode des moindres carrés de régressions : **ridge** qui maintient les coefficients bas, l'algorithme de **régression lasso** qui supprime les poids des variables moins importantes ou **elastic net** qui est un compromis des deux algorithmes précédents.

Il existe d'autres méthodes de régression comme la **régression logistique** qui prédit en répondant oui ou non, celle-ci pourra être approfondie dans le cadre de la cybersécurité.

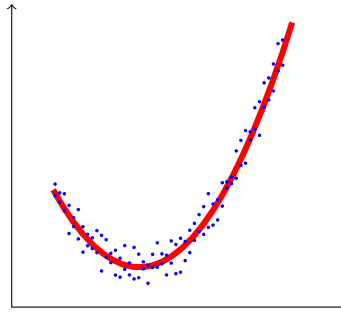


FIGURE 8.6 – Régression polynomiale

8.2.2 Machine à vecteur de support (SVM)

Le but d'une SVM est de trouver la meilleure fonction $f(x)$ de classification pour distinguer deux classes dans les données d'apprentissage. On l'appelle **hyperplan de séparation** (voir figure 8.7 page 130). Les SVM garantissent en plus que l'hyperplan optimal est trouvé en maximisant la marge entre les deux classes. La **marge** correspond à la distance la plus courte entre les points de données les plus proches d'un point de l'hyperplan. L'hyperplan sera alors centré sur la marge, pour ce faire de façon efficace il faut de préférence utiliser des échelles identiques.

Il existe la **classification à marge rigide**, rigide impose qu'une classification doit contenir toutes ces données du bon côté de la marge de séparation. Cela fonctionne uniquement si les données sont séparables. S'il y a une donnée d'une classe au milieu de l'autre, la classification échouera.

Contrairement à l'autre marge, la **classification à marge souple** prend en paramètre la largeur de la marge maximum et va ajouter pour chaque élément mal classé ou en empiètement¹, un ϵ_i , appelé variable d'ajustement ou d'écart, pour traiter. Ainsi, on complète le problème d'optimisation avec ces nouveaux termes et une constante C : $\sum \epsilon_i + C$. Cette constante C identifie la pénalité sur le modèle. Ainsi, l'utilisation de la notion de marge souple permet de trouver une solution au problème non linéairement séparable (à cause des éléments mal-classés). La diminution de l'importance de la pénalisation C , se traduit par une diminution de la marge de confiance (marge maximum) et donc des risques pour l'utilisation du classifieur. Ainsi il faut établir une marge avec le moins d'empiètement de frontière sans sur-ajuster le modèle, car en cas de sur-ajustement, il faudra réduire C .

Par exemple pour un ensemble de données linéairement séparable (SVM linéaire), l'hyperplan de séparation qui traverse le milieu des deux classes en les séparant en deux, sera une droite si nous sommes sur deux axes sinon se sera un plan.

Pour connaître la classification il suffit alors d'utiliser la fonction $f(x_n)$ en regardant le signe de retour, x_n appartient à la classe positive si $f(x_n) > 0$.

1. Si une donnée se retrouve sur la marge ou dans l'autre classification on dit qu'elle empiète.

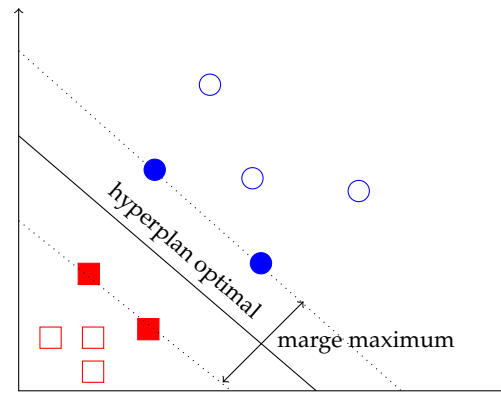


FIGURE 8.7 – SVM linéaire à marge rigide

8.2.3 Version space

La Version space ou généralisation/spécialisation est une méthode d'apprentissage basée sur la généralisation des concepts.

Cette technique est le fruit des travaux de Mitchell. Par l'utilisation d'exemples positifs et négatifs d'un concept (ou objet), on réalise une version space qui permet de créer un modèle de description de ce concept. Certaines conditions sont nécessaires au bon fonctionnement du mécanisme :

- Il existe un nombre fixe d'attributs pour définir les exemples présentés au système.
- Le modèle de description d'un exemple est une combinaison des valeurs de ces attributs.
- En aucune façon, les exemples positifs et négatifs ne doivent être erronés.

Avec les techniques précédentes (apprentissage par analyse des différences), le modèle de description était modifié à chaque fois qu'un nouvel exemple était présenté au système. Pour éviter une mauvaise modification, chaque nouvel exemple ne devait être que légèrement différent du modèle. Ainsi, on évitait de faire évoluer le modèle vers des interprétations incertaines. Avec la "version space", on explore chaque interprétation possible, et ce, tant qu'elle reste viable.

Une version space est une représentation qui enregistre toutes les informations utiles des exemples fournis au système sans conserver un seul de ces exemples. Une version space est une représentation dans laquelle :

- Il y a un arbre de spécialisation (noté S) et un arbre de généralisation (noté G).
- Chaque nœud correspond à un modèle de description.
- La racine de l'arbre de généralisation est un modèle qui accepte tous les exemples.
- La racine de l'arbre de spécialisation est un modèle qui accepte un unique exemple.
- Les liens entre les nœuds dénotent des relations de généralisation et de spécialisation.

L'idée de l'apprentissage par version space est que la généralisation des modèles spécifiques et la spécialisation des modèles généraux conduisent à un modèle assurément correct qui accepte tous les exemples positifs présentés au système et rejette tous ceux qui sont négatifs.

Exemple : Les concepts du problème sont :

- Taille : grand, petit, moyen

Algorithme 10 : Algorithme : Version_Space(LExemple, S,G)

Input : une liste d'exemples positifs et négatifs LExemple
Output : Si le programme c'est bien terminé et les deux graphes S et G
 Initialiser G au niveau de généralisation maximum;
 Initialiser S au premier exemple positif;
 Ex $\leftarrow$ prendreUnElement(LExemple);
 PasFini $\leftarrow$ Vrai;
while Ex $\neq$ null **et** PasFini **do**
 if Ex est positif **then**
 Supprimer de G toutes les branches qui n'acceptent pas Ex;
 while Chaque feuille fs de S n'acceptant pas Ex **do**
 Supprimer fs ;
 Ajouter à S toutes les généralisations minimal de fs à partir de G qui
 acceptent Ex
 Supprimer de S les feuilles plus générales qu'une branche déjà dans S.
 if Ex est négatif **then**
 Supprimer de S toutes les branches acceptant Ex;
 while Chaque feuille fg de G acceptant Ex **do**
 Supprimer fg ;
 Ajouter à G toutes les spécialisations minimales de fg qui n'acceptent plus
 Ex et qui généralisent au moins un élément de S
 Supprimer de G les feuilles moins générales qu'une branche déjà dans G.
 if S = G **then**
 PasFini $\leftarrow$ Faux;
 else
 PasFini = consistant(S,G);
return pasFini, S, G;

- Forme : cercle, triangle, carré, octogone
- Couleur : blanc, bleu, rouge, vert

Dans les trois exemples de la figure 8.8, nous voyons comment s'effectue la généralisation par batch. Le problème est dans le dernier cas, où il n'existe aucune généralisation entre les exemples.

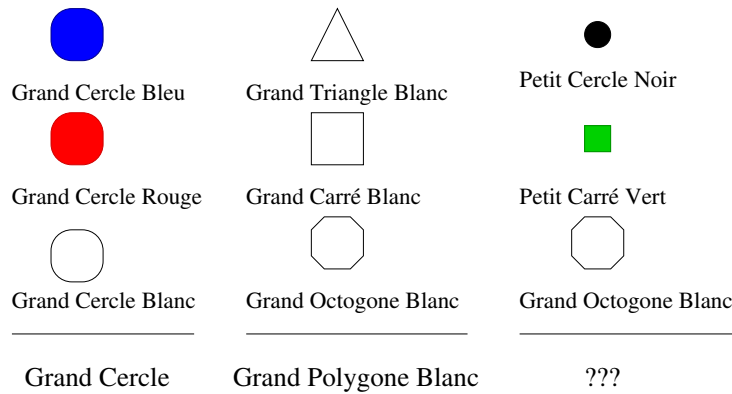


FIGURE 8.8 – Exemple de méthode de généralisation

Pour permettre de mieux organiser le raisonnement, l'approche en version space propose de travailler sur deux graphes. L'un permet de travailler sur la généralisation des concepts rencontrés pour ainsi accepter les exemples positifs et le second est l'arbre de spécialisation pour permettre de refuser les exemples négatifs. On reprenant la colonne de gauche de la figure on obtient :

$$1. \textcircled{G} = \langle ?, ?, ? \rangle$$

$$\textcircled{S} = \langle \text{grand}, \text{cercle}, \text{bleu} \rangle$$

$$2. \textcircled{G} = \langle ?, ?, ? \rangle$$

$$\langle \text{grand}, \text{cercle}, ? \rangle$$

$$\textcircled{S} = \langle \text{grand}, \text{cercle}, \text{bleu} \rangle$$

$$3. \textcircled{G} = \langle ?, ?, ? \rangle$$

$$\langle \text{grand}, \text{cercle}, ? \rangle$$

$$\textcircled{S} = \langle \text{grand}, \text{cercle}, \text{bleu} \rangle$$

Si maintenant on ajoute comme contre-exemple le petit cercle noir, on obtient :

$$1. \textcircled{G} = \langle ?, ?, ? \rangle$$

$$\begin{array}{cc} / & \backslash \\ \langle \text{grand}, ?, ? \rangle & \langle ?, \text{cercle}, ? \rangle \end{array}$$

$$\langle \text{grand}, \text{cercle}, ? \rangle$$

$$\textcircled{S} = \langle \text{grand}, \text{cercle}, \text{bleu} \rangle$$

8.2.4 Arbre de décisions

Un arbre de décisions est composé de :

- Un nœud non-terminal qui représente un test sur un attribut.
- Une branche qui correspond à une valeur possible pour un attribut.

- Un nœud terminal qui correspond à une classification possible.

Le paradigme associé à ces arbres regroupe tous les d'**algorithmes d'entraînements** basés sur utilisation des arbres décisionnels. Ils permettent de représenter les informations par un arbre dont les branches vont guider les résultats possibles d'une série de choix. C'est la méthode d'apprentissage la plus utilisée. Des centaines d'applications couvrant des domaines comme le diagnostic médical ou encore le contrôle de processus utilisent les arbres de décision. Quelques propriétés de ce type d'apprentissage :

- Les instances doivent être représentées par des couples <attributs,valeurs>
- Les valeurs sont discrètes.
- Les données pour l'apprentissage (instances) peuvent contenir des erreurs.

En tout état de cause, cette méthode d'apprentissage concerne essentiellement les systèmes de classification. Nous présenterons principal l'algorithme **ID3** (pour *Iterative Dichotomiser 3*). Il crée un arbre avec plusieurs séparations, en trouvant pour chaque nœud d'attributs, dont les valeurs sont **catégoriques**, produisant le gain d'information le plus important. Ces arbres sont développés jusqu'à leur taille maximale, puis une étape d'élagage est généralement appliquée afin d'améliorer la capacité de l'arbre à se généraliser à des données inédites [Qui86].

Nous allons voir l'algorithme pour la classification par arbre de décision. On utilise le corpus d'études en le découpant en deux parties. La première sert pour engendrer l'arbre et la seconde valide les règles qui ont été établies. On utilise :

- Corpus d'études C_e : ensemble des individus
- Le nombre de valeurs d'un Attribut : $\text{Card}(\text{Attribut})$
- La $i^{\text{ème}}$ valeur de l'Attribut : $\text{Attribut}(i)$
- Un individu Ind a comme valeur pour l'attribut Att : $\text{Valeur}(Ind, Att)$
- Une fonction-objectif F_{ob} qui est un attribut particulier puisque c'est lui qui sert d'élément discriminant
- L'ensemble d'individus du Corpus qui ont la valeur Val pour l'Attribut A : $\text{set}(\text{Corpus}, A, Val)$

Nous allons voir comme obtenir l'attribut le plus discriminant. Il existe plusieurs critères correspondant à la notion "de plus discriminant" :

- Notion d'**entropie** : nombre de bits nécessaires pour coder la classification de membres du corpus C_e . On se sert donc de cette notion pour connaître quel est l'attribut A qui donne le résultat le plus petit avec la fonction objectif (on note $\text{set}(A, i) = \text{set}(C_e, A, A(i))$) :

$$\text{Entropie}(A(i)) =$$

$$\sum_{j=1}^{\text{Card}(F_{ob})} -\log_2\left(\frac{\text{Card}(\text{set}(\text{set}(A, i), F_{ob}, F_{ob}(j)))}{\text{Card}(\text{set}(A, i))}\right) \times \frac{\text{Card}(\text{set}(\text{set}(A, i), F_{ob}, F_{ob}(j)))}{\text{Card}(\text{set}(A, i))}$$

- **Table de contingence**, pour un corpus C , les colonnes sont étiquetées par les valeurs de F_{ob} et les lignes par les valeurs de l'attribut A :

$$\begin{aligned} \text{Table}(i,j) &= \text{Card}(\text{set}(\text{set}(C, A, A(i)), F_{ob}, F_{ob}(j))) \\ \text{Ligne}(i) &= \sum_{j=1}^{\text{Card}(F_{ob})} \text{Card}(\text{set}(\text{set}(C, A, A(i)), F_{ob}, F_{ob}(j))) \\ \text{Colonne}(j) &= \sum_{i=1}^{\text{Card}(A)} \text{Card}(\text{set}(\text{set}(C, F_{ob}, F_{ob}(j)), A, A(i))) \end{aligned}$$

Algorithme 11 : Algorithme : ID3(Ce, F_{ob} , Attributs)**Input :** une partie du corpus d'études Ce (entre 40 et 70 %)Une fonction-objectif F_{ob}

La liste des attributs à traiter Attributs

Output : un arbre de décisionCréer un nœud n **if** F_{ob} est égale à une seule valeur **then** Etiquette(n) $\leftarrow$ valeur;**else if** Attributs = $\emptyset$ **then**| Etiquette(n) $\leftarrow$ valeur_{max} de F_{ob} **else**| Soit $A \in$ Attributs **et** A l'Attribut le plus discriminant;| Etiquette(n) $\leftarrow A$;| $i \leftarrow 0$;| **for** chaque Valeur possible de A **do**| | **if** set(Ce,A,Valeur) $\neq \emptyset$ **then**| | | Etiquette_arc(n,i) $\leftarrow$ Valeur ;| | | Fils(n,i) $\leftarrow$ ID3(set(Ce,A,Valeur), F_{ob} ,Attributs $\setminus A$);| | | $i \leftarrow i+1$;**return** n ;— Critère global (CG) de A :

$$CG(A) = \sum_{i=1}^{Card(A)} \text{MAX}_{j \in 1..Card(F_{ob})} Table(i, j)$$

— Il existe d'autres critères utilisables comme l'information mutuelle, tau ou Xchi².Finalement, pour effectuer le choix de l'attribut A le **plus discriminant**, on calcule pour :

$$\text{— l'entropie : } \text{Min}_A \left(\sum_{i=1}^{Card(A)} \frac{Card(\text{set}(\text{Corpus}, A, A(i)))}{Card(\text{Corpus})} \times \text{Entropie}(A(i)) \right)$$

$$\text{— le critère majoritaire : } \text{Max}_A(CG(A))$$

Exemple : En utilisant le critère d'entropie, nous allons construire l'arbre de décision associé à ces données, le but est de réaliser la bonne classification par rapport à l'attribut **Crédit** :

Individu	Salarié	Situation Familiale	Période de travail	Crédit
I_1	Plein-temps	Marié	Jour	Oui
I_2	Plein-temps	Marié	Jour	Oui
I_3	Plein-temps	Marié	Jour	Oui
I_4	Plein-temps	Marié	Jour	Oui
I_5	Plein-temps	Marié	Nuit	Oui
I_6	Plein-temps	Marié	Nuit	Oui
I_7	Plein-temps	Marié	Nuit	Oui
I_8	Plein-temps	Marié	Nuit	Oui
I_9	Plein-temps	Concubin	Nuit	Non
I_{10}	Plein-temps	Concubin	Jour	Non
I_{11}	Plein-temps	Concubin	Jour	Non
I_{12}	Plein-temps	Concubin	Jour	Non
I_{13}	Plein-temps	Célibataire	Jour	Oui
I_{14}	Plein-temps	Célibataire	Nuit	Oui
I_{15}	Plein-temps	Célibataire	Nuit	Oui
I_{16}	Plein-temps	Célibataire	Jour	Oui
I_{17}	Non	Marié	Jour	Non
I_{18}	Non	Marié	Nuit	Non
I_{19}	Mi-temps	Marié	Nuit	Non
I_{20}	Mi-temps	Marié	Jour	Non
I_{21}	Mi-temps	Marié	Jour	Oui
I_{22}	Mi-temps	Marié	Jour	Oui
I_{23}	Mi-temps	Marié	Jour	Oui
I_{24}	Mi-temps	Marié	Jour	Oui
I_{25}	Mi-temps	Marié	Jour	Oui
I_{26}	Mi-temps	Marié	Jour	Oui

La première étape est de réaliser la table de contingence. Cela correspond aux quatre premières colonnes du tableau ci-dessous. Les deux suivantes correspondent aux calculs des critères majoritaire et entropie.

		Crédit		Critères	
		Oui	Non	Majoritaire	Entropie
Salarié	Mi-temps	6	2	6	$-\frac{6}{8} * \log_2(\frac{6}{8}) - \frac{2}{8} * \log_2(\frac{2}{8}) = 0,811$
	Non	0	2	2	$-\frac{0}{2} * \log_2(\frac{0}{2}) - \frac{2}{2} * \log_2(\frac{2}{2}) = 0$
	Plein-Temps	12	4	12	$-\frac{12}{16} * \log_2(\frac{12}{16}) - \frac{4}{16} * \log_2(\frac{4}{16}) = 0,811$
Bilan		18	8	$6 + 2 + 12 = 20$	$0,811 * \frac{8}{26} + 0 * \frac{2}{26} + 0,811 * \frac{16}{26} = 0,748$
Sit. Fam.	Marié	14	4	14	$-\frac{14}{18} * \log_2(\frac{14}{18}) - \frac{4}{18} * \log_2(\frac{4}{18}) = 0,764$
	Concubin	0	4	4	$-\frac{0}{4} * \log_2(\frac{0}{4}) - \frac{4}{4} * \log_2(\frac{4}{4}) = 0$
	Célibataire	4	0	4	$-\frac{4}{4} * \log_2(\frac{4}{4}) - \frac{0}{4} * \log_2(\frac{0}{4}) = 0$
Bilan		18	8	$14 + 4 + 4 = 22$	$0,764 * \frac{18}{26} + 0 * \frac{4}{26} + 0 * \frac{4}{26} = 0,529$
Période	Jour	12	5	12	$-\frac{12}{17} * \log_2(\frac{12}{17}) - \frac{5}{17} * \log_2(\frac{5}{17}) = 0,874$
	Nuit	6	3	6	$-\frac{6}{9} * \log_2(\frac{6}{9}) - \frac{3}{9} * \log_2(\frac{3}{9}) = 0,918$
Bilan		18	8	$12 + 6 = 18$	$0,874 * \frac{17}{26} + 0,918 * \frac{9}{26} = 0,889$

Il n'y a pas d'ambiguïté puisque quelques soit le critère choisi, majoritaire en prenant la plus forte valeur (22) ou l'entropie avec la plus faible (0,529), c'est toujours l'attribut situation maritale qui donne le meilleur résultat. La répartition se fait de la façon suivante :

- Situation = Marié, il reste à discriminer les individus $\{I_1 \text{ à } I_8, I_{17} \text{ à } I_{26}\}$
- Situation = Concubin, le crédit est non
- Situation = Célibataire, le crédit est oui.

Il faut donc refaire une table de contingence pour les individus correspondants à la situation familiale = Marié. En effectuant les calculs pour chaque étape, nous obtenons finalement le graphe suivant (cf. Figure 8.9). Sur celui-ci, on retrouve entre parenthèses (en bleu) la répartition entre (Oui/Non) pour l'attribution du crédit, les chiffres en dessous correspondent à la valeur de l'entropie et du critère majoritaire et entre accolades la population restant à traiter. Il est à noter qu'une fois l'application du traitement la règle pour faire la classification : Situation_maritale = marié $\wedge$ Salarié = Mi-temps $\wedge$ Période = Jour $\rightarrow$ Crédit = Oui, laisse un individu mal classé. Nous ne pouvons rien faire pour celui-ci, car nous n'avons plus d'attributs. Il fera partie du pourcentage d'erreur dans l'apprentissage.

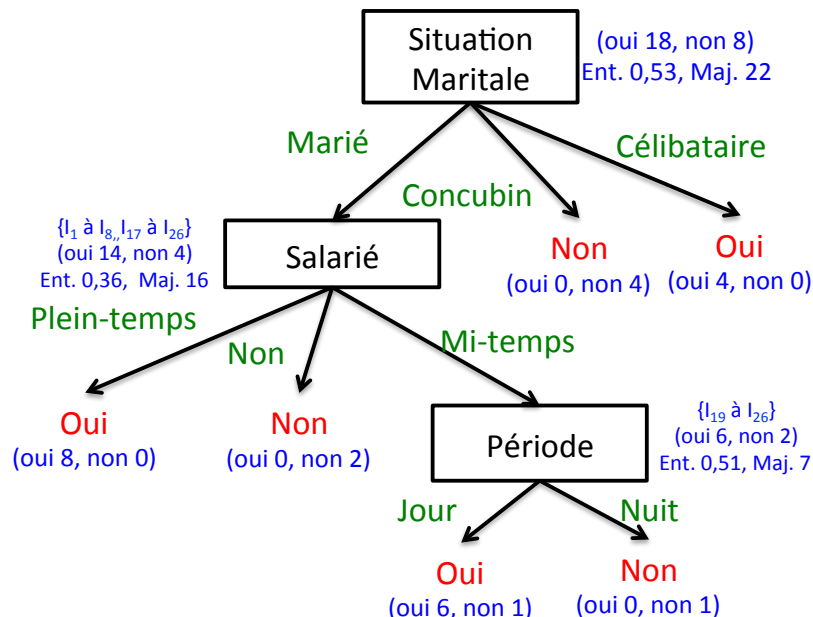


FIGURE 8.9 – Exemple de la méthode ID3

Comme nous venons de le voir, nous nous intéressons exclusivement aux deux critères : entropie et majoritaire. De nombreux travaux sur cet algorithme ont été proposés pour améliorer les performances (temps, taille de l'arbre, mémoire...). Ainsi l'algorithme **C4.5** [Qui93] améliore **ID3** en supprimant la restriction des valeurs **catégoriques** pour les attributs en définissant dynamiquement un attribut discret (basé sur des variables numériques) qui divise la valeur de l'attribut continu en un ensemble discret d'intervalles. On peut aussi citer l'algorithme **C5.0** qui est une amélioration de **C4.5** utilisant moins de mémoire et construit des jeux de règles plus petits tout en étant plus précise [Qui96]. On trouve souvent dans les bibliothèques un autre algorithme **CART** [?] (pour *Classification and Regression Trees*) qui est similaire à **C4.5**

8.2.5 K-Means

La méthode K-means fait partie des algorithmes d'apprentissage non-supervisés et il permet de réaliser des partitionnements ("*clustering*" en anglais) non hiérarchiques des données. Le K représente le nombre de clusters qui vont être utilisés pour distinguer les individus issus des données d'apprentissage. Il est à noter qu'une donnée ne peut appartenir qu'à une seule partition. Pour calculer la notion de similarité entre les données, il est souvent utilisé une distance. On retrouve ainsi les deux calculs de distance classiques comme la distance euclidienne et de Manhattan.

Les données sont :

- valeur(Ind,Att) : représente la valeur de l'individu Ind pour l'attribut Att
- A : nombre d'attributs
- D : l'ensemble des données
- K : le nombre de partitions
- I : représente l'ensemble des individus
- c_k : représente le centroïde de la partition k .
- l_k : liste des individus dans la partition k .
- L : la liste des listes des individus dans les partitions.

L'algorithme 12 donne l'algorithme K-means.

Prenons un exemple : On cherche à réaliser un partitionnement en deux des sujets suivants :

Sujet	1	2	3	4	5	6	7
Att ₁	1	1,5	3	5	3,5	4,5	3,5
Att ₂	1	2	4	7	5	5	4,5

Dans ce cas, nous ne normalisons pas les données puisqu'elles sont de mêmes grandeurs, mais ce traitement devrait être fait systématiquement. On prend comme centroïde deux sujets : 1 (1;1) et 4 (5;7) On calcule la distance de chaque sujet avec les centroïdes (1;1) et (5;7) :

Sujet	1	2	3	4	5	6	7
Distance avec 1	0	$\sqrt{(1,5-1)^2 + (1-2)^2} = 1,11$	3,6	7,21	4,71	5,31	4,3
Distance avec 4	7,21	6,1	3,6	0	2,5	2,06	2,91

Pour choisir le partitionnement, on prend la distance la plus petite concernant le centroïde du partitionnement. En cas d'ex æquo comme dans le cas du sujet 3, on garde le premier. On obtient le partitionnement suivant et on calcule le nouveau centroïde associé à chaque partition.

- Partition₁ : Sujets : {1,2,3} et nouveau centroïde : ($\frac{1+1,5+3}{3} = 1,83$ et $\frac{1+2+4}{3} = 2,33$)
- Partition₂ : Sujets : {4,5,6,7} et nouveau centroïde : (4,12 et 5,37)

On calcule la distance de chaque sujet avec les centroïdes (1,83;2,33) et (4,12;5,37) :

Sujet	1	2	3	4	5	6	7
Distance Partition ₁	1,57	0,47	2,03	5,64	3,14	3,77	2,73
Distance Partition ₂	5,37	4,37	1,77	1,84	0,72	0,53	1,07

On obtient le partitionnement suivant et on calcule le nouveau centroïde associé à chaque partition.

- Partition₁ : Sujets : {1,2} et nouveau centroïde : ($\frac{1+1,5}{2} = 1,25$ et $\frac{1+2}{2} = 1,5$)

Algorithme 12 : Algorithme : K-Means(D, K)**Entrées :** Le nombre de partitions K et l'ensemble des données D**Sorties :** Renvoi la liste des partitions.**entier** $p \leftarrow 0$;**entier** $\min \leftarrow 0$;**booléen** $\text{fin} \leftarrow \text{faux}$;**individu** $t \leftarrow \emptyset$;

/* on initialise les centroïdes et les partitions */

pour $k=1$ à K **faire** $c_k \leftarrow \text{copier_un_individu}(I)$;**tant que non fin faire**

/* on affecte chaque individu dans les partitions */

pour $k=1$ à K **faire** $l_k \leftarrow \emptyset$;**pour chaque** $\text{ind} \in I$ **faire** $\min \leftarrow +\infty$; **pour** $k=1$ à K **faire** **si** $\min > \text{distanceEuclidienne}(c_k, \text{ind})$ **alors** $\min = \text{distanceEuclidienne}(c_k, \text{ind})$; $p = k$ $l_p \leftarrow l_p \cup \{\text{ind}\}$;

/* on réajuste les coordonnées du centroïde */

 $\text{fin} \leftarrow \text{vrai}$;**pour** $k=1$ à K **faire** **pour** $a=1$ à A **faire** $\text{valeur}(t, a) \leftarrow 0$; **pour chaque** ($\text{ind} \in l_k$) **faire** **pour** $a=1$ à A **faire** $\text{valeur}(t, a) += \text{valeur}(\text{ind}, a) / \text{card}(l_k)$;

/* on regarde si le centroïde s'est déplacé */

si $t \neq c_k$ **alors** $c_k \leftarrow t$; $\text{fin} = \text{faux}$ **retourner** L — Partition₂ : Sujets : {3,4,5,6,7} et nouveau centroïde : (3,9 et 5,1)

On calcule la distance de chaque sujet avec les centroïdes (1,25;1,5) et (3,9;5,1) :

Sujet	1	2	3	4	5	6	7
Distance Partition ₁	0,56	0,56	3,05	6,65	4,16	4,77	3,75
Distance Partition ₂	5,02	3,92	1,42	2,19	0,41	0,61	0,72

Le partitionnement n'a pas changé suite à la dernière itération, nous obtenons donc la répartition des sujets suivants les deux partitions suivantes : {1,2} et {3,4,5,6,7}.

Une des difficultés est d'établir le nombre de partitions nécessaire, soit déterminer le k dans l'approche k-means. En effet, un k trop grand engendrera une fragmentation des données qui empêchera de découvrir des schémas intéressants dans les données. Un k trop petit engendrera peu de partitions qui seront trop généralistes et contenant beaucoup de types de données différents.

Pour cela, il n'existe pas de méthode sûre pour trouver le k , mais on peut lancer plusieurs fois le k-means avec différentes valeurs de k et de calculer la performance de chaque modèle. Par exemple, on peut calculer la variance ou la somme de l'erreur quadratique des différentes partitions. La variance va calculer la distance entre le centroïde et les éléments de chacune des classes. On parle de minimiser la distance intra-classe.

Sur l'exemple la variance (donnée dans le tableau précédent) sera : $0,56 + 0,56 + 1,42 + 2,19 + 0,41 + 0,61 + 0,72 = 6,47$

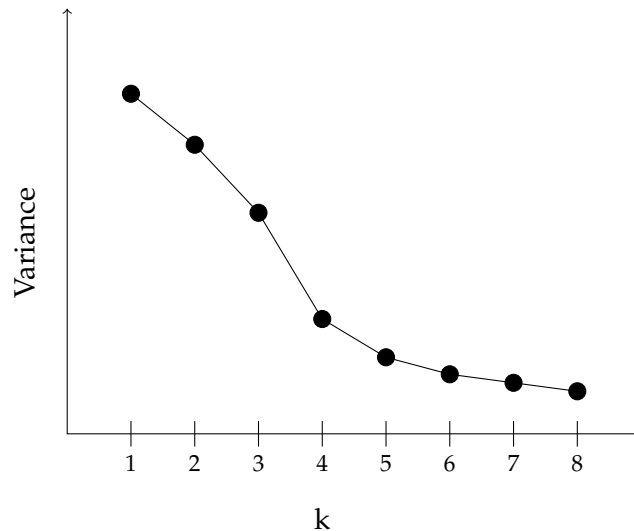


FIGURE 8.10 – Exemple graphe de la méthode elbow

La figure 8.10 présente une figure typique de l'impact du nombre de partitions sur la variance. La courbe obtenue ressemble à un bras avec l'épaule au niveau du point $k=1$ et la main vers le droite de la courbe. Ainsi le meilleur k correspond au coude de la courbe ("elbow" en anglais), il se situe entre 3 et 4. Il reste ensuite à choisir l'un des deux puisque c'est eux qui donnent la meilleure réduction de la variance. En effet, la baisse de la courbe de variance (ou distorsion) entre les partitions 1 à 4 est significativement plus grande que celle entre les partitions 4 à 9.

8.2.6 MeanShift

L'approche MeanShift est proche de la méthode précédente dans son mécanisme. Cependant, elle diffère sur deux aspects concernant les méta-paramètres. Le premier concerne les centroïdes. En effet, ces derniers ne sont pas fixés en fonction du k comme pour K-means mais leurs nombres vont être choisis pour couvrir l'espace de recherche. Le deuxième concerne le voisinage de chacun des centroïdes qui sera pris en compte pour faire évoluer les coordonnées du centroïde. Ce n'est pas en fonction des individus appartenant au cluster, mais celui inclus dans un cercle (sphère ou ovoïde suivant la dimension de l'espace) autour du centroïde.

L'algorithme est le suivant :

1. Création des centroïdes (répartis dans l'espace).
2. Calcule des nouvelles coordonnées du centroïde en prenant les individus dans sa sphère de proximité.

Algorithme 13 : Algorithme : MeanShift(D)

```

Création des centroïdes;
while Centroides pas modifiés do
    On place chaque individu dans la sphère des centroïdes;
    On retire les centroïde qui n'ont pas d'individus dans leur sphère;
    Calcule des nouvelles coordonnées du centroïde;
    On fusionne les centroïdes aux mêmes coordonnées;
si plusieurs centroïdes sont dans une même sphère alors
    on les remplace par un nouveau centroïde prenant en compte ces derniers;
On crée autant de clusters que de centroïdes restants;
On calcule pour chaque individu sa distance à chaque centroïde et on l'affecte dans le cluster associé au centroïde le
plus proche de l'individu.

```

3. On fusionne les centroïdes aux mêmes coordonnées ou ceux qui n'ont pas d'individus dans leur sphère.
4. On répète le point 2. tant qu'il y a des centroïdes modifiés.
5. Si plusieurs centroïdes sont dans une même sphère on les remplace par un nouveau centroïde prenant en compte ces derniers.
6. On crée autant de clusters que de centroïdes restants.
7. On calcule pour chaque individu sa distance à chaque centroïde et on l'affecte dans le cluster associé au centroïde le plus proche de l'individu.

L'avantage de cette méthode est que les clusters sont établis automatiquement et on n'en présume pas le nombre. L'inconvénient est qu'il faut avoir suffisamment mis de centroïdes initiaux pour avoir un bon partitionnement.

Reprenons l'exemple précédent avec les 7 individus :

Sujet	1	2	3	4	5	6	7
Att ₁	1	1,5	3	5	3,5	4,5	3,5
Att ₂	1	2	4	7	5	5	4,5

Nous avons deux attributs, on regarde le domaine de chacun soient [1,5] et [1,7] on peut envisager 3 partitions de chaque domaine pour avoir une répartition homogène ce qui donne respectivement comme valeurs (1,66; 3; 4,32) et (2; 4; 6) et ainsi avoir 9 centroïdes. On peut prendre comme distance 0,66 pour le premier et 1 pour le second pour que les sphères se touche sans se recouvrir. Si vous vous posez la question, comment nous avons obtenu ces chiffres, on prend les bornes de l'intervalle 1 et 5, on fait la soustraction pour avoir la valeur à couvrir soit $(5-1) = 4$. Ensuite, il faut déterminer 3 points pivots permettant de réaliser les 4 partitions. Ainsi, pour couvrir l'intervalle on résout l'équation suivante $3 \times X = 4$, on a ainsi le diamètre du cercle qui $4/3 = 1,3333$. Pour finalement obtenir le rayon, on divise par 2 soit $1,3333 / 2 = 0,666$. On place donc le premier point à l'abscisse correspondant à la borne inférieure de l'intervalle plus le rayon du cercle : $1+0,666 = 1,6666$, le second point à l'abscisse du précédent point plus 2 rayons pour éviter le chevauchement : $1,6666+1,3333 = 3$ et idem pour le troisième à $3+1,3333 = 4,33$. On fait la même chose pour les ordonnées.

— Itération 1 :

Centroïde	C1 (1,66;2)	C2 (1,66;4)	C3 (1,66;6)	C4 (3;2)	C5 (3;4)	C6 (3;6)	C7 (4,33;2)	C8 (4,33;4)	C9 (4,33;6)
Intervalle sphère	[1;1] à [2,32;3]	[1;3] à [2,32;5]	[1;5] à [2,32;7]	[2,33;1] à [3,66;3]	[2,33;3] à [3,66;5]	[2,33;5] à [3,66;7]	[3,67;1] à [5;3]	[3,67;3] à [5;5]	[3,67;5] à [5;7]
Individu dans sphère	1, 2	0	0	0	3, 5, 7	5	0	6	4, 6

On ne garde par les centroïdes sans voisinage soit C2, C3, C4 et C7. On recalcule les coordonnées des autres.

— **Itération 2 :**

Centroïde	C1 (1,25;1,5)	C5(3,33;4,5)	C6(3,5;5)	C8(4,5;5)	C9(4,75;6)
Intervalle sphère	[0,59;0,5] à [1,91;2,5]	[2,66;3,5] à [4,5;5]	[2,84;4] à [4,16;6]	[3,84;4] à [5,16;6]	[4,09;5] à [5,42;7]
Individu dans sphère	1, 2	3, 5, 7	3, 5, 7	6	4, 6

Le centroïde C1 n'est pas mis à jour, on fusionne C5 / C6 et on laisse C8 et C9, car les centroïdes sont différents.

— **Itération 3 :**

Centroïde	C1 (1,25;1,5)	C5(3,33;4,5)	C8(4,5;5)	C9(4,75;6)
Intervalle sphère	[0,59;0,5] à [1,91;2,5]	[2,66;3,5] à [4,5;5]	[3,84;4] à [5,16;6]	[4,09;5] à [5,42;7]
Individu dans sphère	1, 2	3, 5, 7	6	4, 6

— Après cette fusion, on atteint un état stable et on sort de la boucle. Nous avons les centroïdes C8 et C9 qui sont dans la même sphère donc on les fusionne et on calcule le nouveau centroïde C8 (milieu des centroïdes) :

Centroïde	C1(1,25;1,5)	C5(3,33;4,5)	C8(4,625;5,5)
Intervalle sphère	[0,59;0,5] à [1,91;2,5]	[2,66;3,5] à [4,5;5]	[3,95;4,5] à [5,29;6,5]
Individu dans sphère	1, 2	3, 5, 7	4, 6

Il n'y a pas de centroïde recouvert par la sphère d'un autre. On peut affecter dans les 3 clusters les individus, en fonction de la distance des individus au centroïde :

Individu	1	2	3	4	5	6	7
Distance C1	0,55	0,55	3,05	6,65	4,16	4,77	3,75
Distance C5	4,2	3,09	0,6	3	0,52	1,26	0,167
Distance C8	5,77	4,69	2,21	1,54	1,23	0,51	1,50
Cluster	1	1	2	3	2	3	2

8.2.7 Classification ascendante hiérarchique

L'approche Classification Ascendante Hiérarchique (CAH) cherche à regrouper des classes d'individus ayant la plus faible distance (ou dissemblance). L'algorithme démarre avec des classes (singleton) constituées d'un seul individu et par itération va regrouper les classes ayant la plus petite distance à chaque itération jusqu'à n'avoir plus qu'une seule classe. La distance entre deux classes A et B notée $\text{distWard}(A, B)$ est calculée à l'aide de la formule de Ward pour distance euclidienne. Elle capture l'inertie entre les classes et elle aide ainsi à regrouper dans une nouvelle classe celles qui auront l'inertie la plus faible. Pour cela, elle prend en compte le nombre d'éléments de chaque classe considérée A et B (aussi appelé poids) qui sont notés respectivement p_A et p_B et la distance euclidienne entre les centroïdes des classes $d(g_A, g_B)$:

$$\text{distWard}(A, B) = \frac{p_A \times p_B}{p_A + p_B} d(g_A, g_B)^2$$

L'algorithme 14 donne l'algorithme de la classification ascendante hiérarchique.

Reprenons l'exemple précédent avec les 7 individus :

Sujet	1	2	3	4	5	6	7
Att ₁	1	1,5	3	5	3,5	4,5	3,5
Att ₂	1	2	4	7	5	5	4,5

Algorithme 14 : Algorithme : CAH(D)

```

Entrées : L'ensemble des données X
Sorties : Renvoie le tableau T (distance, centroïde, liste individus) et le nombre de partitions i.
set L, L1  $\leftarrow \emptyset, \emptyset$ ;
entier min  $\leftarrow +\infty$ ;
entier i  $\leftarrow 0$ ;
/* on initialise les premières partitions avec un élément par partition */
pour chaque ind  $\in I$  faire
    L  $\leftarrow L \cup \{i\}$ ;
    T[i]  $\leftarrow \{0, ind, \{ind\}\}$ ;
    i++;
/* On itère jusqu'à avoir fusionné toutes les partitions */
tant que card(L) > 1 faire
    /* On calcule la distance entre chaque partition non encore fusionnée */
    min  $\leftarrow +\infty$ ;
    L1  $\leftarrow L$ ;
    pour chaque part  $\in L$  faire
        L1  $\leftarrow L1 \setminus \{part\}$ ;
        pour chaque part1  $\in L1$  faire
            dist(part, part1)  $\leftarrow$  distWard(part, part1);
            si dist(part, part1) < min alors
                min  $\leftarrow$  dist(part, part1);
                couple  $\leftarrow$  (part, part1)
            sinon
                dist(part, part1)  $\leftarrow +\infty$ 
        /* Pour la nouvelle partition obtenue de la fusion des plus proche : */
        /* - On crée la liste contenant les individus associés */
        Lind  $\leftarrow$  T[couple.1].individu  $\cup$  T[couple.2].individu ;
        /* - On calcule pour le nouveau centroïde i la valeur de chaque attribut a */
        /* - On initialise l'attribut a de i à 0 */
        pour a=1 à A faire
            valeur(i,a)  $\leftarrow 0$ ;
            /* - On calcule la valeur de l'attribut à partir de chaque individu de la partition */
            pour chaque ind  $\in$  Lind faire
                valeur(i,a) += valeur(ind,a) / card(Lind)
        /* - On ajoute à T : la distance, le centroïde et la liste des individus */
        T[i]  $\leftarrow$  {min, i, Lind};
        /* On retire les 2 ensembles fusionnés de la liste L */
        L  $\leftarrow L \setminus \{couple.1, couple.2\}$ ;
        /* On ajoute à L la nouvelle partition créée */
        L  $\leftarrow L \cup \{i\}$ ;
        i++;
retourner (T,i)

```

Nous avons les 7 premières partitions représentant chaque individu avec la distance entre les individus de la partition, le centroïde de la partition et la liste des individus dans la partition :

1. : {0; (1;1); {1}}
2. : {0; (1,5;2); {2}}
3. : {0; (3;4); {3}}
4. : {0; (5;7); {4}}
5. : {0; (3,5;5); {5}}
6. : {0; (4,5;5); {6}}
7. : {0; (3,5;4,5); {7}}

Nous effectuons le calcul de la distance de Ward avec la matrice suivante. Nous avons représenté ici l'ensemble des partitions qui seront construites par l'algorithme.

	2	3	4	5	6	7
1	$\frac{1*1}{1+1}((1-1,5)^2 + (1-2)^2) = 0,625$	6,5	26	11,125	14,125	9,25
2		3,125	18,625	6,5	9	5,125
3			6,5	0,625	1,625	0,25
4				3,125	2,125	4,25
5					0,5	0,125
6						0,625

La nouvelle partition n°8 va se composer des éléments 5 et 7 avec la plus petite distance de Ward de 0,125 et le nouveau centroïde est (3,5; 4,75). On met à jour le tableau de distance :

	2	3	4	6	8
1	0,625	6,5	26	14,125	$\frac{1*2}{1+2}((1-3,5)^2 + (1-4,75)^2) = 13,54$
2		3,125	18,625	9	7,21
3			6,5	1,625	0,54
4				2,125	4,875
6					0,71

La nouvelle partition n°9 va se composer des éléments 3, 5 et 7 avec la plus petite distance de Ward de 0,54 et le nouveau centroïde est (3,333; 4,5). On met à jour le tableau de distance :

	2	4	6	9
1	0,625	26	14,125	$\frac{1*3}{1+3}((1-3,333)^2 + (1-4,5)^2) = 13,27$
2		18,625	9	7,21
4			2,125	6,77
6				1,21

La nouvelle partition n°10 va se composer des éléments 1 et 2 avec la plus petite distance de Ward de 0,625 et le nouveau centroïde est (1,25; 1,5). On met à jour le tableau de distance :

	6	9	10
4	2,125	6,77	$\frac{1*2}{1+2}((5-1,25)^2 + (7-1,5)^2) = 29,54$
6		1,21	15,21
9			$\frac{3*2}{3+2}((3,333-1,25)^2 + (4,5-1,5)^2) = 16,01$

La nouvelle partition n°11 va se composer des éléments 3, 5, 6 et 7 avec la plus petite distance de Ward de 1,21 et le nouveau centroïde est (3,625; 4,625). On met à jour le tableau de distance :

	10	11
4	29,54	$\frac{1*4}{1+4}((5-3,625)^2 + (7-4,625)^2) = 6,025$
10		$\frac{2*4}{2+4}((1,25-3,625)^2 + (1,5-4,625)^2) = 20,54$

La nouvelle partition n°12 va se composer des éléments 3, 4, 5, 6 et 7 avec la plus petite distance de Ward de 6,025 et le nouveau centroïde est (3,9; 5,1). On calcule la distance entre les partitions 10 et 12 : $\frac{2*5}{2+5}((1,25-3,9)^2 + (1,5-5,1)^2) = 28,54$.

La nouvelle partition n°13 va se composer de l'ensemble des éléments 1, 2, 3, 4, 5, 6 et 7 avec la distance de Ward de 28,54 et le nouveau centroïde est (3,14; 4,07).

Une façon graphique de représenter le résultat est d'utiliser un dendrogramme qui permet de visualiser les distances entre partitions. La figure 8.11 représente les partitions sur l'exemple, on peut ainsi se rendre compte que le regroupement des deux dernières partitions donne une inertie très importante. On peut aussi se poser la question sur les 3 dernières qui passent d'une inertie de 1,21 à plus du 6. Une bonne pratique est de considérer qu'un bon partitionnement est proche de la somme des écart-types des données. Ici l'écart-type sur les coordonnées est de 1,46 pour les abscisses et de 2 pour les ordonnées donc au maximum 3,46.

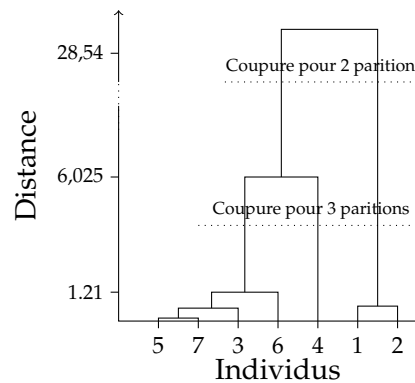


FIGURE 8.11 – Dendrogramme de l'exemple

8.3 Réseau de neurones

Cette section est donnée à titre informatif dans le cadre de cet enseignement. Il n'est pas nécessaire d'en comprendre les détails.

L'étude des réseaux de neurones a beaucoup fluctué depuis 1906 où Santiago Ramón y Cajal & Camilo Golgi reçoivent le prix Nobel pour la découverte du **neurone** comme l'élément de structure du cerveau. Il se passe peu de choses jusqu'en 1943 où J. McCulloch & W. Pitts proposent un modèle neuronal simple capable de produire une machine de Turing. Dans la foulée, D. Hebb propose une règle d'apprentissage pour des réseaux de neurones (1948). En suite il faudra 10 ans pour que F. Rosenblatt propose le modèle du *Perceptron* et démontre son théorème de convergence. En 1969, M. Minsky & S. Papert démontrent les limitations du modèle du *Perceptron*, ceci va mettre un coup d'arrêt aux réseaux de neurones. Il faudra attendre 1985 pour que les travaux sur l'apprentissage par rétro-propagation et donc l'utilisation des réseaux multi-couches pour dépasser la limitation du perceptron pour que l'approche neuronale soit relancée.

Aujourd'hui, il existe beaucoup de travaux effectués sur les réseaux de neurones. Cela s'explique en grande partie par l'aspect anthropomorphique. En effet, il y a une forte similitude entre la représentation du cerveau (sous forme de matrice 3D) et la méthode d'apprentissage (pondération sur les arcs). Ainsi on comprend mieux le fonctionnement du cerveau en mettant au point des techniques d'apprentissage pour les réseaux de neurones artificiels. Ainsi, on peut aussi utiliser ces méthodes d'apprentissage pour aider les cerveaux ayant subis des altérations. Nous pouvons donner quelques chiffres pour le cerveau humain :

- nombre de neurones dans le cerveau : environ 10^{11}
- nombre de connexions par neurone : entre 10^4 et 10^5
- temps de cycle (switching time) : de l'ordre de 10^{-3} seconde
- temps moyen d'une activité cognitive : de l'ordre de 0,1 seconde

Finalement l'**intelligence**, c'est un millier de connexions pour chacun des 100 milliards de neurones du cerveau humain.

Mais : le cerveau sait faire des traitements distribués et du parallélisme massif. De plus, il existe beaucoup d'interconnexions entre unités. Le cerveau n'est pas figé, car pendant toute sa durée de vie, il va changer en ajustant automatiquement les connexions et leurs poids. De plus, le travail dans le cerveau s'effectue sous la forme de traitement sous-symbolique : intelligence

comme propriété émergente. Une idée ou une reconnaissance s'effectue par activation d'un groupe de neurones par un système de cycle.

8.3.1 Apprentissage

Dans le cas des réseaux de neurones, toute la difficulté va être de déterminer la valeur des connexions entre les neurones et donc l'apprentissage va chercher à modifier ce poids pour correspondre aux données d'apprentissage. Ainsi la modification du poids s'exprime par :

$$\Delta W_{ij} = \eta(T_j - O_j)I_i = \eta\delta_j I_i$$

avec :

- η : représente le coefficient d'apprentissage
- T : le vecteur théorique en sortie
- O : le vecteur calculé par le réseau en sortie
- I : le vecteur en entrée.

8.3.2 Le perceptron

Tous les amateurs de science-fiction ont déjà entendu parler du **perceptron**. C'est le nom donné généralement au cerveau des androïdes ou des robots. En IA, c'est le nom qui a été donné au modèle le plus simple de réseau de neurones qui ne contient qu'un seul niveau (couche) de neurones artificiels (présentés en section 2.4.4). L'intérêt est qu'il possède de bonnes propriétés.

Théorème de représentation : Un réseau "feedforward" à une seule couche (Perceptron) peut uniquement représenter des fonctions linéairement séparables. C'est-à-dire celles pour lesquelles la surface de décision séparant les cas positifs des cas négatifs est un (hyper-)plan.

Théorème d'apprentissage : Étant donné suffisamment d'exemples d'apprentissage, il existe un algorithme qui apprendra n'importe quelle fonction linéairement séparable.

La **convergence** est assurée si l'erreur E a une forme quadratique dans l'espace des poids. Ainsi, elle possède un seul minimum global et la descente du gradient garantit de la trouver.

Sinon il faut avoir des données d'apprentissage qui soient linéairement séparables où le taux d'apprentissage soit suffisamment petit.

Prenons par exemple la fonction **xor**, dans la table ci-dessous, nous avons la table de vérité associée et à gauche la représentation dans l'espace associée. Nous voyons que pour effectuer la classification par rapport à la fonction xor, il faut utiliser deux droites pour séparer l'espace (comme dans la figure 8.12 à droite). Ainsi, il faudra un réseau multi-couches pour arriver à cela.

Entrée (X1)	Entrée (X2)	Sortie (Y)
0	0	0
0	1	1
1	0	1
1	1	0

TABLE 8.1 – Table de vérité du xor

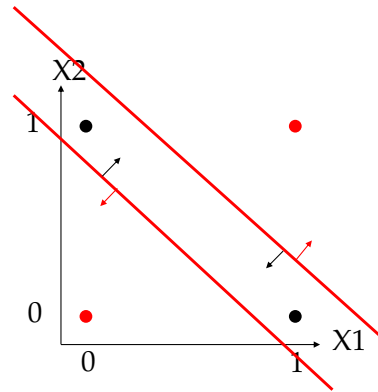
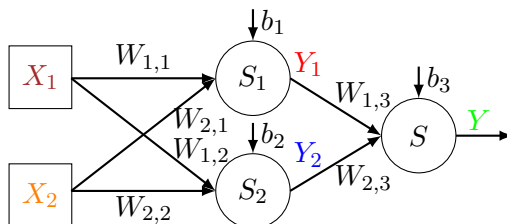


FIGURE 8.12 – Espace représentant la fonction xor

8.3.3 Les réseaux multi-couches

Ils apparaissent à partir de 1986 et ils permettent de représenter des fonctions non linéairement séparables. Ils utilisent la théorie de la régularisation statistique qui permet d'établir les différents paramètres des neurones (pondération, valeur complémentaire et seuil). Cette dernière permet de réduire les problèmes liés au sur-apprentissage. Par exemple, un réseau à 2 couches (une couche cachée composée de S_1, S_2 et une couche de sortie S) peut représenter la fonction logique xor.



On choisit pour $W_{1,1} = 20, W_{1,2} = -20, W_{2,1} = 20, W_{2,2} = -20, W_{1,3} = 20, W_{2,3} = 20$ pour les biais $b_1 = -10, b_2 = 30, b_3 = -30$ et la fonction de seuil $\varphi(x) = \frac{1}{1+e^{-x}}$ (appelée softmax)

X_1	X_2	S_1	Y_1	S_2	Y_2	S_3	Y
0	0	$20*0+20*0-10$	0	$-20*0-20*0+30$	1	$20*0+20*1-30$	0
0	1	$20*0+20*1-10$	1	$-20*0-20*1+30$	1	$20*1+20*1-30$	1
1	0	$20*1+20*0-10$	1	$-20*1-20*0+30$	1	$20*1+20*1-30$	1
1	1	$20*1+20*1-10$	1	$-20*1-20*1+30$	0	$20*1+20*0-30$	0

Les réseaux multi-couches "feedforward" peuvent être entraînés par rétro-propagation pour autant que la fonction de transition des unités soit différentiable.

Différents types de fonctions :

Seuil

$$\varphi(v) = \begin{cases} 1, & \text{si } v \geq 0 \\ 0, & \text{si } v < 0 \end{cases}$$

linéaire par partie

$$\varphi(v) = \begin{cases} 1, & \text{si } v \geq \alpha \\ v, & \text{si } \alpha > v \geq \beta \\ 0, & \text{si } v < \beta \end{cases}$$

sigmoïde

$$\varphi(v) = \frac{1}{1+e^{-\alpha v}}$$

$$\varphi(v) = \frac{1-e^{-v}}{1+e^{-v}}$$

Nous représenterons dans les figures qui suivent différents types de réseaux de neurones. Pour cela, nous dessinerons en **marron** la couche de neurones pour les entrées, en **bleu** la couche de sortie et en **vert** les couches intermédiaires. Nous rappelons que les w correspondants au poids des transitions sont calculés avec un algorithme d'apprentissage associé à un corpus.

Réseau à une couche

Dans la figure 8.13, nous donnons un exemple de réseau mono-couche. Le calcul des sorties est effectué en fonction des entrées de la façon suivante :

- $O_i = 1$ si $\sum_k (w_{ik} I_k) > 0$
- $O_i = 0$ sinon

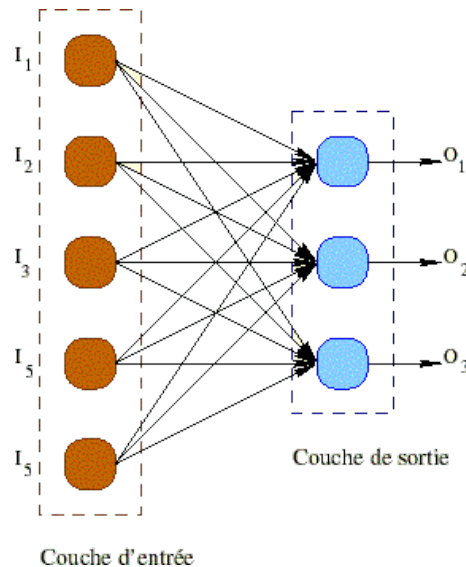


FIGURE 8.13 – Exemple de réseau une couche

"Feedforward" ou multi-couches

Dans la figure 8.14, nous donnons un exemple de réseau multi-couches. Le calcul des différentes couches est effectué en fonction des entrées ou des couches intermédiaires (cachées) de la façon suivante :

- Couche cachée :
 - $C_j = 1$ si $\sigma_k(w_{jk} I_k) > 0$
 - $C_j = 0$ sinon
- Couche de sortie :
 - $O_i = 1$ si $\sigma_k(w_{ik} C_k) > 0$
 - $O_i = 0$ sinon

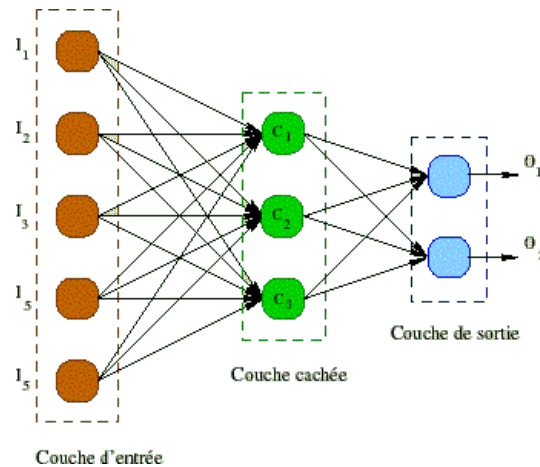


FIGURE 8.14 – Exemple de réseau multi-couches

Dès qu'il y a plus d'une couche cachée, on parlera de réseau de neurones **profond** (ou "*deep learning*").

Réseau récurrent ou de Hopfield

Dans la figure 8.15, nous donnons un exemple de réseau de Hopfield. Le calcul des sorties est effectué en fonction des entrées car chaque unité i est connectée à chaque autre unité j par un poids w_{ij} . Les poids sont supposés symétriques : $w_{ij} = w_{ji}$.

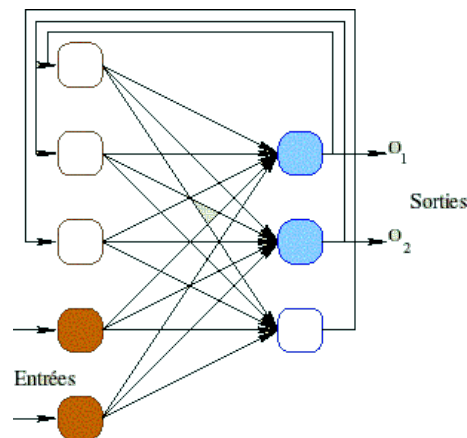


FIGURE 8.15 – Exemple de réseau Hopfield

Réseau de treillis

Dans la figure 8.16, nous donnons un exemple de réseau en treillis bi-dimensionnelle 3×3 .

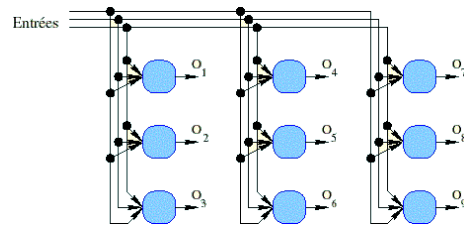


FIGURE 8.16 – Exemple de réseau en treillis

Pour plus d'information sur les réseaux de neurones, vous pouvez vous référer à [SYOS97] pour la partie utilisation ou sinon pour une présentation plus théorique à [GMJM⁺04].

8.4 Exercices chapitre 8

Exercice 1 - Quiz application

Question : Retrouver le type d'application pour le modèle d'IA établi dans ce cadre :

1. Isoler les comportements indiquant des mal-fonctions d'un site internet ?
2. Prédire le salaire d'une personne en fonction de ses années d'expérience et ses performances ?
3. Une entreprise de carte bancaire reçoit des milliers de demandes de nouvelles cartes. Les demandes contiennent des informations sur le client comme son salaire annuel, de grosses dettes le cas échéant, l'âge, etc. Qui a un bon profil, un mauvais profil, ou un profil nécessitant plus d'analyse ?
4. Trouver des groupes de clients ayant des comportements similaires dans les magasins à partir d'une large base de données incluant leurs précédents achats et leur fréquence de venue ?

[Correction.](#)

Exercice 2 - Villes

Nom	Latitude	Longitude
Autun	46,951	4,298
Auxerre	47,796	3,57
Avallon	47,488	3,907
Beaune	47,023	4,838
Belfort	47,637	6,862
Besançon	47,238	6,024
Chalon sur Saône	46,781	4,853
Dijon	47,321	5,041
Dole	47,092	5,492
Lons le Saunier	46,674	5,553
Lure	47,686	6,494
Mâcon	46,306	4,831
Montbard	47,624	4,337
Montbéliard	47,51	6,797
Nevers	46,987	3,157
Pontarlier	46,904	6,354
Sens	48,197	3,282
Vesoul	47,619	6,154

Questions :

1. Normaliser les données.
2. Utiliser la méthode de classification ascendante hiérarchique pour partitionner les villes. Quel pourrait être le bon partitionnement ?
3. Utiliser le nombre de partitions correspondant au partitionnement trouvé à la question précédente pour faire un partitionnement par k-means sur les données.
4. Combien de partition nous obtiendrons avec meanShift ?

[Correction.](#)

Exercice 3 - Maison bretonne

Nous avons obtenu les résultats suivants, suite à une étude sur des maisons pour déterminer les maisons bretonnes ou non :

n°	Toit	Murs	Cheminée	Portes	Maison bretonne
1	Ardoises	Pierres	Une	Petites	OUI
2	Tuiles	Briques	Une	Grandes	NON
3	Ardoises	Parpaings	Deux	Petites	OUI
4	Chaume	Pierres	Une	Petites	NON
5	Ardoises	Pierres	Une	Moyennes	NON

Question : Construire les graphes de généralisation/spécialisation pour chacun des exemples pour obtenir le concept de "maison bretonne".

[Correction.](#)

Exercice 4 - Gagnant

Id	Animal	Couleur	Marque	Age	Gagnant
1	Chien	Bleu	Oui	30	Oui
2	Chat	Rouge	Oui	40	Non
3	Chien	Vert	Non	30	Oui
4	Poisson	Bleu	Oui	30	Non
5	Chien	Bleu	Oui	50	Non

Question : Construire les graphes de généralisation/spécialisation pour chacun des exemples pour obtenir le concept de "Gagnant". On utilisera l'arbre de la figure 8.17 pour généraliser les couleurs.

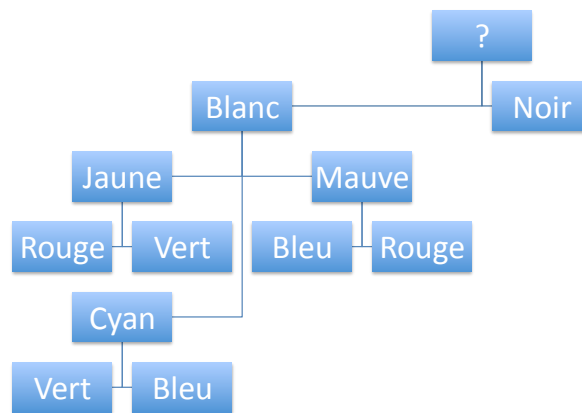


FIGURE 8.17 – Arbre de hiérarchie de couleur

Correction.

Exercice 5 - Id3 météo

On désire savoir en fonction d'un certain nombre d'attributs si on peut ou non jouer au tennis à l'extérieur. A partir d'observations (instances) qui constituent les données de l'apprentissage, on établit le tableau suivant :

Jour	Météo	Temp.	Humidité	Vent	Tennis
J1	soleil	chaude	élevée	faible	non
J2	soleil	chaude	élevée	fort	non
J3	couvert	chaude	élevée	faible	oui
J4	pluie	idéale	élevée	faible	oui
J5	pluie	froide	normale	faible	oui
J6	pluie	froide	normale	fort	non
J7	couvert	froide	normale	fort	oui
J8	soleil	idéale	élevée	faible	non
J9	soleil	froide	normale	faible	oui
J10	pluie	idéale	normale	faible	oui
J11	soleil	idéale	normale	fort	oui
J12	couvert	idéale	élevée	fort	oui
J13	couvert	chaude	normale	faible	oui
J14	pluie	idéale	élevée	fort	non

Question : Construire les tableaux et les graphes associés pour les deux critères le majoritaire et l'entropie en détaillant les calculs pour déterminer l'attribut choisi à chaque étape.

[Correction.](#)

Exercice 6 - Id3 Sensible

Pour faire des diagnostics sur la sensibilité au soleil, un médecin se base sur une analyse effectuée sur 8 patients :

Numéro	Cheveux	Taille	Poids	Crème	Résultat
1	blond	moyen	léger	non	sensible
2	blond	grand	moyen	non	non
3	brun	petit	moyen	oui	non
4	blond	petit	moyen	non	non
5	blond	moyen	lourd	non	sensible
6	brun	grand	lourd	non	non
7	brun	moyen	lourd	non	non
8	blond	petit	léger	oui	non

Question : Construire l'arbre de décision en appliquant l'algorithme ID3 à l'aide du calcul d'entropie. Donner cet arbre en indiquant vos calculs.

[Correction.](#)

Exercice 7 - Utilisation d'un outil

Pour permettre d'utiliser les algorithmes du chapitre, il existe plusieurs bibliothèques et outils. Nous vous proposons à travers l'activité "*Chapitre 8 - Knime*" sur moodle de mettre en œuvre cela avec l'environnement knime. Il met à disposition un grand nombre d'algorithmes

et de traitements des données. Tout cela se fait graphiquement en indiquant l'enchaînement des différentes étapes.

[Correction.](#)

9. Méthode incomplète

Nous présentons dans ce chapitre une approche complémentaire à celles vues précédemment. En effet, les méthodes incomplètes ne garantissent pas d'obtenir une solution mais permettent d'en avoir une plus rapidement. Ce chapitre ne contient qu'une seule [leçon](#).

9.1 Intérêt

Les Méthodes incomplètes sont une méthode de résolution ne garantissant pas de trouver une solution. Même si les méthodes incomplètes ne permettent pas de trouver systématiquement une solution, elles restent intéressantes car elles sont rapides et permettent, en général, de trouver une solution même si ce n'est pas toujours la meilleure. Ensuite, on peut utiliser des algorithmes classiques comme le Branch & Bound pour trouver une meilleure solution en utilisant comme borne la solution trouvée avec la méthode incomplète.

9.2 Fonctionnement

Elles sont généralement basées sur une fonction de génération aléatoire qui permet de donner une première instance à chaque variable du problème et ensuite par correction locale, on transforme les valeurs de variables qui rendent le système inconsistant pour trouver une solution.

9.3 Exemples

Nous allons voir deux exemples pour illustrer l'approches des méthodes incomplètes.

9.3.1 Les reines

Dans ce premier exemple, nous reparlons du problème des reines que nous avons déjà vu dans les chapitres précédents. Si on fait une synthèse rapide on sait résoudre le problème (dans un temps raisonnable) :

- Placer les reines simplement, on arrive à 20 reines
- Avec une heuristique : < 2000 reines
en partant du placement simple et en choisissant
 - Colonnes les plus contraintes on arrive à 100 reines
 - Lignes les plus contraintes on arrive à 500 reines
 - Choix des colonnes du milieu à instancier en premier pour le 1^{er} et 3^{ème} tiers, le bord pour le 2^{ème}.

— En effectuant un placement aléatoire (méthode incomplète) : 1 million de reines en 5mn

9.3.2 3-Sat

Dans cet exemple, nous parlons du problème 3-Sat. Celui-ci est l'archétype des problèmes NP-Complet. Il sert de référence pour ces types de problèmes. Ce problème consiste à trouver un modèle (une interprétation à chaque variable propositionnelle, constituant le problème). Puisque toute formule de la logique propositionnelle peut se réécrire sous la forme clausale et que l'on peut ramener toute clause à une clause de taille 3 (constituée de trois variables propositionnelles) on s'intéresse à ce type de problème : 3-Sat.

Il faut étudier plus finement le comportement de la satisfiabilité du système. En effet, il a été montré et prouvé qu'il existe un phénomène de seuil (voir figure 9.1) où le problème passe de l'état satisfiable à non satisfiable. Ce niveau correspond au rapport nombre de variables sur nombre de clauses à 4,3. Ainsi, si un problème est en-dessous du seuil, on peut chercher une solution avec la méthode incomplète et au-dessus du seuil, il faudra utiliser une autre forme d'approche. Il est à noter que le même travail a été fait pour 2-Sat (problème quadratique pour la résolution) et le seuil est à 1.

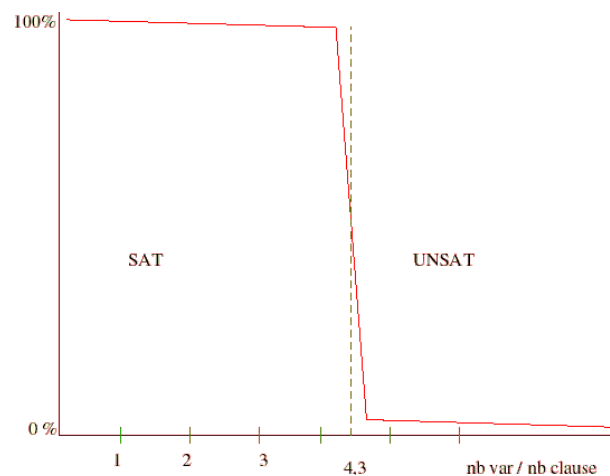


FIGURE 9.1 – Seuil pour les problèmes 3-Sat

9.4 Exercices chapitre 9

Exercice 1 - Reines

Nous allons essayer de réaliser un programme qui résout le problème des reines de taille 100, 500 puis 1000 avec la méthode incomplète.

Questions :

1. Réaliser un prédicat `(N, S)` qui permet de générer aléatoirement une instance de N reines dans S sur un damier $N \times N$, sans se soucier si elles sont en prises. On utilisera la bibliothèque "*Random*" de Sicstus et le prédicat `random(1, N, Nb)` qui tire au hasard un nombre entre 1 et N et met le résultat dans Nb .
2. Réaliser un prédicat `test_en_prise11(L, X)` qui permet de vérifier si une reine X est en prise avec une liste de reines L .
3. Réaliser un prédicat `permutation(N, X, L, S)` qui permet d'échanger la position d'une reine X avec une des reines de L qui est en prise et met le résultat dans S . On limitera le nombre de permutation maximum à N .
4. Réaliser un prédicat `partition(L, Ok, SOk, Ko, SKo)` qui permet de partitionner une liste de reines L en deux groupes : en prises SOk et non prises SKo .
5. Réaliser un prédicat `reparation(N, Ok, Ko, S)` qui permet d'effectuer une permutation entre les reines en prise Ko . La permutation est effectuée si elle réduit le nombre de reines en prises N . Si il n'y a plus de reines en prises alors on a trouvé une solution S .
6. Réaliser le prédicat `test(N, L, R, S)` qui permet de tester si les reines placées sur le damier sont correctement positionnées. Sinon il effectue un certain nombre de permutations (inférieur ou égal au nombre de reines en prises). Si cette tentative n'a pas donnée satisfaction, il essaye avec un placement des reines différents.
7. Réaliser le prédicat d'appel `reine(N, S)` qui permet de résoudre le problème des reines N , la solution sera dans S .

[Correction.](#)

10. Linguistique

Nous présentons dans ce dernier chapitre les problèmes liés aux traitements du langage (naturel). Il est constitué d'une seule [leçon](#).

10.1 Sensibilisation

La linguistique est une science qui a pour objet l'étude du langage et des langues. L'intérêt de la linguistique de l'IA est double. La première est que le langage est le vecteur de communication naturel de l'individu. Il semble naturel que l'ordinateur soit capable de le comprendre et de s'exprimer via celui-ci. Un autre élément vient dans l'aide à la création de phrases pour les personnes en difficulté physiquement ou mentalement pour qu'elles ne se retrouvent pas isolées par leur handicap. Un exemple de la vie courante est l'intégration des correcteurs orthographiques et grammaticaux dans les traitements de texte.

Nous allons présenter dans ce qui suit les problèmes inhérents aux langages et donc les pièges possibles pour une IA ou les concepteurs.

10.1.1 Problème : Homonyme et polysémie

1. "Y a-t-il un porte-monnaie dans la **pièce** voisine?"

"Si oui, y a-t-il des **pièces** dans ce porte-monnaie?"

Dans cet exemple, la **pièce** est employée deux fois mais avec des sens différents c'est la **polysémie**. L'ambiguïté ne peut être levée quand, connaissant le contexte d'emploi du mot et le dimensionnement des objets référant. Ainsi on peut savoir si on parle de la pièce de monnaie ou du lieu lorsque l'on fait le lien avec le porte-monnaie qui contient ou qui est contenu.

2. "La belle ferme la porte."

Dans ce texte, l'ambiguïté vient d'un manque de ponctuation. Si le discours est oral, éventuellement l'intonation permettrait de lever cette ambiguïté. Typiquement cette phrase a au moins 2 interprétations :

- (a) La belle (une personne) ferme la porte
- (b) La belle ferme (une bâtisse) la (une cheminée) porte

Ici, on ne parle que de la version **homographique** mais dans le discours l'ambiguïté est encore plus grande avec l'**homophonie**, comme conter ou compter.

10.1.2 Problème : Synonyme

Il y a deux types de synonymie :

- **paradigme** : il existe plusieurs mots pour définir le même concept comme pour abdomen, ventre et bidon.
- **syntagme** : c'est donc des groupes de mots qui représentent les mêmes concepts : connaître le succès et être réputé.

10.1.3 Problème : Contexte et Figure de style

1. *"Le dictateur promet aux manifestants de s'en aller".*
"Le dictateur hurle aux manifestants de s'en aller".

Le fait de changer le verbe de la phrase en change complètement le sens. Mais si on change les groupes nominaux comme dans la phrase suivante on trouve encore un autre sens. Il faut alors faire attention aux effets de style dans l'écriture, qui souvent, donnent un sens complètement différent.

"Les manifestants hurlent au dictateur de s'en aller".

2. *"Le capitaine propose au second de partir en préretraite."*

Là, l'effet de style vient de la hiérarchie qui existe entre les deux individus. Si on prend la phrase en inversant les groupes nominaux comme dans la phrase suivante, on obtient le même sens. Même si l'ambiguïté peut demeurer, la bienséance (ou la culture du lieu) fait que l'on suppose que le second ne peut se permettre de faire une telle proposition à son supérieur.

"Le second propose au capitaine de partir en préretraite."

10.1.4 Solution

Au niveau de l'analyse, on peut définir des règles de grammaire qui vont permettre de structurer le discours. Pour établir le lien entre les idées, il faut pouvoir identifier les synonymes mais aussi les expressions équivalentes. Le dernier point important est de connaître le contexte et les éléments auxquels il est fait référence (comme avec l'utilisation des pronoms) ou pour connaître la progression des idées dans le texte.

Par exemple, si on a les deux phrases suivantes :

1. Toto va à la banque
2. Toto prend de l'argent

Nous avons deux interprétations possibles suivant l'ordre :

- 1 et 2 : pour un retrait
- 2 et 1 : pour un dépôt

Il existe plusieurs générateurs de phrases, utilisant ces principes à l'adresse suivante : <https://pragmatics.net/lesite/spip.php?article137>.

Dans les exercices qui suivent, nous verrons comment faire la génération à l'aide de règles. Il existe aujourd'hui de plus en plus de système utilisant une approche sur l'analyse de corpus de données appelée simplement l'analyse du langage naturel (en anglais Natural Language Processing - NLP). C'est ces méthodes qui sont utilisées dans les outils de traduction comme google translate ou deepl.

10.2 Exercices chapitre 10

Exercice 1 - Génération de phrase

Nous allons nous intéresser à la génération automatique de phrases. Pour cela, nous allons définir une grammaire simple du français :

- `phrase` $\rightarrow$ `groupe_nominal` `groupe_verbal`
- `groupe_nominal` $\rightarrow$ `article` `nom`
- `groupe_verbal` $\rightarrow$ `verbe` `groupe_nominal`

Cette grammaire réduite nous permet de définir des phrases simples composées d'un sujet d'un verbe et d'un complément. Nous allons maintenant définir les terminaux de notre grammaire ou encore appelés notre vocabulaire :

- `article` $\rightarrow$ `le`
- `nom` $\rightarrow$ `chat` | `souris`
- `verbe` $\rightarrow$ `mange` | `effraye`

Questions :

1. Réaliser un programme prolog permettant de générer les phrases correspondantes à ces règles de grammaire et ce vocabulaire. Il est à noter que nous utiliserons les listes pour simplifier l'écriture de la phrase à construire.

Remarque : On remarque que le système effectue toutes les combinaisons possibles et bien sûr ne tient pas compte des accords de genre entre l'article et le nom. Prolog en plus d'être une application de la logique des prédicats permet aussi d'utiliser les grammaires holomorphiques ou dépendantes du contexte. Nous allons ainsi pouvoir passer entre les règles les accords à effectuer entre les éléments des groupes nominaux mais aussi avec le verbe. Remplaçons notre vocabulaire avec les éléments suivant :

- `article` $\rightarrow$ `le` | `la` | `les`
 - `nom` $\rightarrow$ `chat` | `souris`
 - `verbe` $\rightarrow$ `mange` | `effraye` | `effrayent`
2. Réaliser un programme prolog permettant de générer les phrases correspondantes à ces règles de grammaire et ce vocabulaire.

Remarque : Maintenant que nous savons écrire des phrases grammaticalement correctes aux sens du français, nous allons essayer d'écrire des phrases conceptuellement correctes. Pour cela, nous allons définir un ordre sur nos trois concepts (`souris` au singulier, `chat` et `souris` au pluriel). Il suffit en suite de définir au niveau des verbes les règles conceptuelles autorisées :

- Il faut être plus grand pour effrayer.
 - Il faut être plus grand pour manger.
 - Le cannibalisme n'existe pas
3. Réaliser un programme prolog permettant de générer les phrases correspondantes à ces règles de grammaire et ce vocabulaire.

Félicitation, vous venez de réaliser votre premier générateur de phrases corrects conceptuellement et grammaticalement. Il ne vous reste plus qu'à enrichir son vocabulaire...

[Correction.](#)

11. Correction des exercices

Ce chapitre donne les différentes corrections des exercices proposés en fin des chapitres zéro à dix. On peut y accéder directement à l'aide du lien correction après l'énoncé de l'exercice ou ici en accédant à la section correspondant au chapitre :

- Correction des exercices du [chapitre 0](#)
- Correction des exercices du [chapitre 2](#)
- Correction des exercices du [chapitre 3](#)
- Correction des exercices du [chapitre 4](#)
- Correction des exercices du [chapitre 5](#)
- Correction des exercices du [chapitre 6](#)
- Correction des exercices du [chapitre 7](#)
- Correction des exercices du [chapitre 8](#)
- Correction des exercices du [chapitre 9](#)
- Correction des exercices du [chapitre 10](#)

11.1 Exercices chapitre 0

Exercice 1

[Retour à l'énoncé.](#)

```
% Le prédicat premier : première solution
premier1(X,L):-
    L=[X|_].

% Le prédicat premier : solution équivalente
premier2(X,[X|_]).

%%% Test
:-begin_tests(premier).
test(premier):- premier1(1,[1,2,3,4]).
test('premier(X,[1,2,3,4]).',[true(X==1)]):- premier1(X,[1,2,3,4]).
test('premier(2,[1,2,3,4]).',[fail]):- premier1(2,[1,2,3,4]).
:-end_tests(premier).
```

Exercice 2

[Retour à l'énoncé.](#)

```
% Le prédicat dernier
dernier(X, [X]) :-!.
dernier(X, [_|L]) :-
    dernier(X, L).

%%% Test
:-begin_tests(dernier).
test('dernier(4, [1,2,3,4]).') :- dernier(4, [1,2,3,4]).
test('dernier(X, [1,2,3,4]).', [true(X==4)]) :- dernier(X, [1,2,3,4]).
test('dernier(1, [1]).') :- dernier(1, [1]).
test('dernier(3, [1,2,3,4]).', fail) :- dernier(3, [1,2,3,4]).
:-end_tests(dernier).
```

Exercice 3

[Retour à l'énoncé.](#)

```
% Le prédicat ajout\_debut
ajout_debut(X, L, [X|L]).

%%% Test
:-begin_tests(ajoutDebut).
test('ajoutDebut(1, [2,3,4], [1,2,3,4]).') :- ajout_debut(1, [2,3,4], [1,2,3,4]).
test('ajoutDebut(1, [], [1]).') :- ajout_debut(1, [], [1]).
:-end_tests(ajoutDebut).
```

Exercice 4

[Retour à l'énoncé.](#) *% Le prédicat ajout_fin*

```
ajout_fin(X, [], [X]) :-!.
ajout_fin(X, [Y|L], [Y|R]) :-
    ajout_fin(X, L, R).

%%% Test
:-begin_tests(ajoutFin).
test('ajoutFin(4, [1,2,3], [1,2,3,4]).') :- ajout_fin(4, [1,2,3], [1,2,3,4]).
test('ajoutFin(1, [], [1]).') :- ajout_fin(1, [], [1]).
:-end_tests(ajoutFin).
```

Exercice 5

[Retour à l'énoncé.](#)

```
% Le prédicat inclus

inclus([],_).
inclus([X|L],M):-
    ote_element(X,M,M1),
    inclus(L,M1).

% Le prédicat appartient_et_ote

ote_element(X,[X|L],L).
ote_element(X,[Y|L],[Y|R]):-
    ote_element(X,L,R).

%%% Test
:-begin_tests(inclus).
test('inclus([], [1,2,3]).'):- inclus([], [1,2,3]).
test('inclus([1], [1,2,3]).', [nondet,true]):- inclus([1], [1,2,3]).
test('inclus([2], [1,2,3]).', [nondet,true]):- inclus([2], [1,2,3]).
test('inclus([3], [1,2,3]).', [nondet,true]):- inclus([3], [1,2,3]).
test('inclus([1,3], [1,2,3]).', [nondet,true]):- inclus([1,3], [1,2,3]).
test('inclus([1,2,3], [1,2,3]).', [nondet,true]):- inclus([1,2,3], [1,2,3]).
test('inclus([5,3,2], [1,2,3,4,5]).', [nondet,true]):- inclus([5,3,2], [1,2,3,4,5]).
test('inclus(R, [1,2]).', all(R=[[], [1], [1,2], [2], [2,1]])):-inclus(R, [1,2]).
:-end_tests(inclus).
```

Exercice 6

[Retour à l'énoncé.](#)

```
% Le prédicat renverse_liste
% sans accumulateur
renverse_liste1([],[]).
renverse_liste1([X|L],R):-
    renverse_liste1(L,R1),
    ajout_fin(X,R1,R).

%%% Test
:-begin_tests(renverseListe).
test('renverseListe1([],[]).'):- renverse_liste1([],[]).
test('renverseListe1([1],[1]).'):- renverse_liste1([1],[1]).
test('renverseListe1([1,2,3],[3,2,1]).'):- renverse_liste1([1,2,3],[3,2,1]).
:-end_tests(renverseListe).

% avec accumulateur (argument 2 de renverse_liste2/3)
renverse_liste2(X,R):-
    renverse_liste2(X,[],R).
```



```
renverse_liste2([],R,R).
renverse_liste2([X|L],R1,R):-
    renverse_liste2(L,[X|R1],R).

%%% Test
:-begin_tests(renverseListe2).
test('renverseListe2([],[]).'):- renverse_liste2([],[],[]).
test('renverseListe2([1],[1]).'):- renverse_liste2([1],[],[1]).
test('renverseListe2([1,2,3],[3,2,1]).'):- renverse_liste2([1,2,3],[],[3,2,1]).
:-end_tests(renverseListe2).
```

Exercice 7

[Retour à l'énoncé.](#)

```
% Le prédicat sous_liste_droite
sous_liste_droite(L,L).
sous_liste_droite([_|L],K):-
    sous_liste_droite(L,K).

%%% Test
:-begin_tests(sousListeDroite).
test('sousListeDroite([1,2,3],[1,2,3]).',[nondet,true]):- sous_liste_droite([1,2,3],[1,2,3]).
test('sousListeDroite([1,2,3],[3]).',[nondet,true]):- sous_liste_droite([1,2,3],[3]).
test('sousListeDroite([1,2,3],[ ]).',true):- sous_liste_droite([1,2,3],[ ]).
test('sousListeDroite([1,2,3],R).',[all(R==[1,2,3],[2,3],[3],[ ]]):- sous_liste_droite([1,2,3],R).
:-end_tests(sousListeDroite).
```

Exercice 8

[Retour à l'énoncé.](#)

```
% Le prédicat sous_liste_gauche
sous_liste_gauche([X|L],[X|R]):-
    sous_liste_gauche(L,R).
sous_liste_gauche(_,[]).

%%% Test
:-begin_tests(sousListeGauche).
test('sousListeGauche([1,2,3],[1,2,3]).'):- sous_liste_gauche([1,2,3],[1,2,3]).
test('sousListeGauche([1,2,3],[3]).'):- sous_liste_gauche([1,2,3],[1]).
test('sousListeGauche([1,2,3]).'):- sous_liste_gauche([1,2,3],[ ]).
test('sousListeGauche([1,2,3]).',[all(R==[1,2,3],[1,2],[1],[ ]]):- sous_liste_gauche([1,2,3],R).
:-end_tests(sousListeGauche).
```

Exercice 9

[Retour à l'énoncé.](#)

```
% Le prédicat longueur
longueur([],0).
longueur(_|L,A):-
    longueur(L,A1),
    A is A1+1.

:-begin_tests(longueur).
test('longueur([],0).'):- longueur([],0).
test('longueur([3],1).'):- longueur([3],1).
test('longueur([1,2,3,4],4).'):- longueur([1,2,3,4],4).
:-end_tests(longueur).
```

Exercice 10

[Retour à l'énoncé.](#)

```
% Le prédicat somme
somme([],0).
somme([X|L],S):-
    somme(L,S1),
    S is X+S1.

%%% Test
:-begin_tests(somme).
test('somme([],0).'):- somme([],0).
test('somme([1],1).'):- somme([1],1).
test('somme([1,2,3,4],10).'):- somme([1,2,3,4],10).
:-end_tests(somme).
```

Exercice 11

[Retour à l'énoncé.](#)

```
% 1ère question
homme(luc,grand,blond,mur).
homme(gaston,moyen,blond,mur).
homme(kevin,grand,brun,jeune).
homme(remi,moyen,blond,vieux).
femme(martine,petit,blond,mur).
femme(odile,moyen,blond,jeune).
femme(lea,petit,brun,mur).
```

```
femme(elsa,grand,brun,jeune).
femme(sylvie,moyen,blond,jeune).
femme(julie,petit,blond,mur).
femme(mathilde,grand,blond,mur).
femme(anne,grand,blond,mur).

gout(luc,pop,policier,tennis).
gout(gaston,classique,aventure,natation).
gout(kevin,jazz,sciences_fiction,jogging).
gout(remi,classique,aventure,jogging).
gout(martine,classique,aventure,natation).
gout(odile,classique,sciences_fiction,natation).
gout(lea,classique,policier,tennis).
gout(elsa,jazz,sciences_fiction,jogging).
gout(sylvie,classique,sciences_fiction,natation).
gout(julie,pop,policier,tennis).
gout(mathilde,classique,policier,tennis).
gout(anne,classique,aventure,natation).

recherche(luc,moyen,roux,jeune).
recherche(luc,moyen,roux,mur).
recherche(gaston,petit,blond,jeune).
recherche(gaston,petit,blond,mur).
recherche(kevin,moyen,roux,jeune).
recherche(remi,petit,blond,mur).
recherche(martine,moyen,blond,mur).
recherche(martine,moyen,blond,vieux).
recherche(odile,grand,brun,jeune).
recherche(lea,moyen,brun,mur).
recherche(lea,moyen,brun,vieux).
recherche(elsa,grand,brun,mur).
recherche(elsa,grand,roux,mur).
recherche(sylvie,grand,_,mur).
recherche(julie,petit,roux,jeune).
recherche(julie,moyen,roux,jeune).
recherche(mathilde,grand,brun,mur).
recherche(mathilde,grand,brun,vieux).
recherche(anne,grand,brun,vieux).
recherche(anne,grand,blond,vieux).
```

% 2ème question

```
convient_physiquement(X,Y):-
    homme(X,_,_,_),
    recherche(X,TY,CY,AY),
    femme(Y,TY,CY,AY).

convient_physiquement(X,Y):-
    femme(X,_,_,_),
```

```

    recherche(X,TY,CY,AY),
    homme(Y,TY,CY,AY).

ont_memes_gouts(X,Y):-
    homme(X,_,_,_),
    femme(Y,_,_,_),
    gout(X,M,L,S),
    gout(Y,M,L,S).

% 3ème question
assortis(X,Y):-
    convient_physiquement(X,Y),
    convient_physiquement(Y,X),
    ont_memes_gouts(X,Y).

% 4eme question
% Si symétrique ont n'aurait pas appeler 2 fois convient physiquement
assortis4(X,Y):-
    convient_physiquement(X,Y),
    ont_memes_gouts(X,Y).

```

Exercice 12

[Retour à l'énoncé.](#)

```

% INITIALISATION DES ADJACENCES
% Le système d'adjacence défini par la carte est traduit sous forme de fait par le prédicat
% adjacence/2 (un fait par région) dont le premier argument est une région et le second
% argument la liste de ses régions adjacentes.

adjacence(alsace,[lorraine,franche_comte]).
adjacence(aquitaine,[poitou_charentes,limousin,midi_pyrennees]).
adjacence(auvergne,[languedoc_roussillon,midi_pyrennees,limousin,centre,bourgogne,rhone_alpes]).
adjacence(bourgogne,[champagne_ardennes,franche_comte,rhone_alpes,auvergne,
    centre,ile_de_france]).
adjacence(bretagne,[pays_de_la_loire,basse_normandie]).
adjacence(centre,[ile_de_france,bourgogne,pays_de_la_loire,basse_normandie,
    haute_normandie,auvergne,limousin,poitou_charentes]).
adjacence(champagne_ardennes,[nord_pas_de_calais,picardie,ile_de_france,bourgogne,
    lorraine,franche_comte]).
adjacence(corse,[provence_alpes_cote_d_azur]).
adjacence(franche_comte,[alsace,lorraine,champagne_ardennes,bourgogne,rhone_alpes]).
adjacence(ile_de_france,[picardie,haute_normandie,centre,bourgogne,
    champagne_ardennes]).
adjacence(languedoc_roussillon,[auvergne,rhone_alpes,provence_alpes_cote_d_azur,
    midi_pyrennees]).
adjacence(limousin,[aquitaine,midi_pyrennees,auvergne,centre,poitou_charentes]).
adjacence(lorraine,[champagne_ardennes,franche_comte,alsace]).
adjacence(midi_pyrennees,[aquitaine,limousin,auvergne,languedoc_roussillon]).
adjacence(nord_pas_de_calais,[champagne_ardennes,picardie]).
adjacence(basse_normandie,[haute_normandie,centre,pays_de_la_loire,bretagne]).
adjacence(haute_normandie,[basse_normandie,centre,ile_de_france,picardie]).

```

```

adjacence(pays_de_la_loire, [basse_normandie, bretagne, centre, poitou_charentes]).
adjacence(picardie, [nord_pas_de_calais, haute_normandie, ile_de_france,
                    champagne_ardennes]).
adjacence(poitou_charentes, [pays_de_la_loire, centre, limousin, aquitaine]).
adjacence(provence_alpes_cote_d_azur, [corse, rhone_alpes, languedoc_roussillon]).
adjacence(rhone_alpes, [franche_comte, auvergne, bourgogne, languedoc_roussillon]).

%PROCEDURE DE COLORIAGE
% Le prédicat d'appel de la résolution est le prédicat coloriage/2 dont le premier argument
% est la liste des couleurs à utiliser, le second argument étant la liste-réponse des régions
% et de leur couleur respective.
% Le prédicat coloriage/2 permet d'appeler le prédicat colorie/4, et évite de devoir à
% chaque appel depuis la console SICStus de taper la liste de toutes les régions !

coloriage(Couleur, Rep) :-
    colorie([alsace, aquitaine, auvergne, bourgogne, bretagne, centre,
            champagne_ardennes, corse, franche_comte, ile_de_france,
            languedoc_roussillon, limousin, lorraine, midi_pyrennees,
            nord_pas_de_calais, basse_normandie, haute_normandie,
            pays_de_la_loire, picardie, poitou_charentes,
            provence_alpes_cote_d_azur, rhone_alpes], Couleur, [], Rep),
    nl,
    write('La reponse est :'), % permet de d'afficher du texte dans la console
    nl,
    write(Rep), % permet de d'afficher de la liste complète dans la console
    nl.

% Le prédicat colorie/4 présente les quatre arguments suivants : le premier argument est
% constitué des par la liste des régions pas encore coloriées, le second argument est la
% liste des couleurs possibles, le troisième argument est une liste de listes [region, couleur]
% permettant d'accumuler les régions (et leur couleur) déjà coloriées (liste vide à l'appel),
% le quatrième argument constitue la réponse au problème posé.

colorie([], _, Rep, Rep). % Lorsque toutes les régions ont été coloriées (Argument 1 = vide),
                        % l'accumulateur (Argument 3) constitue la réponse (Argument 4)
colorie([R|Region], Couleur, Acc, Rep) :- % Pour chaque région à colorier
    adjacence(R, Adj), % on récupère les régions adjacentes
    member(C, Couleur), % on choisit une couleur
    verifie_couleur(Adj, C, Acc), % on teste la couleur choisie en
                                % fonction des régions déjà coloriées
    colorie(Region, Couleur, [[R, C]|Acc], Rep). % on lance récursivement le
                                                % traitement sur les autres régions
                                                % en ajoutant à l'accumulateur le
                                                % nouveau couple région-couleur

% Le prédicat verifie_couleur/3 présente trois arguments : le premier est la liste des
% régions adjacentes à une région donnée, le deuxième argument est la couleur que l'on
% veut octroyer à cette région donnée, tandis que le troisième argument est la liste des
% régions déjà coloriées.
% Ce prédicat permet de vérifier que la couleur pressentie (Argument 2) pour une région
% donnée n'a pas été déjà donné à une de ses régions adjacentes ([A, C] n'appartient pas
% à Acc). Si une région adjacente est déjà coloriée de la même couleur, ce prédicat
% échoue.

verifie_couleur([], _, _).
verifie_couleur([A|Adj], C, Acc) :-
    \+member([A, C], Acc),

```

```
verifie_couleur(Adj,C,Acc).
```

Exercice 13

[Retour à l'énoncé.](#)

```
% PRE-REQUIS
:-use_module(library(lists)).

% ote\_element/3 permet d'ôter un élément (Argument 1) d'une liste (Argument 2)
% pour obtenir une nouvelle liste amputée de cet élément (Argument 3).
% Si l'élément (Argument 1) n'appartient à la liste (Argument 2), ce prédicat échoue.
% Il faut mieux privilégier le prédicat natif select/3 de la bibliothèque lists.

ote_element(X,[X|L],L).
ote_element(X,[Y|L],[Y|R]):-
    ote_element(X,L,R).

% renverse/3 permet à partir d'une liste (Argument 1) d'obtenir sa liste renversée
% (Argument 3). L'Argument 2 permet de construire la réponse au cours des appels
% récursifs (sa valeur est à vide à l'appel de ce prédicat).
% Il faut mieux privilégier le prédicat natif reverse/2 de la bibliothèque lists.

renverse([],Acc,Acc).
renverse([E|L],Acc,R):-
    renverse(L,[E|Acc],R).

% INITIALISATION DES COUPLES INTERDITS
% Ces faits permettent de définir tous les couples interdits

couple_interdit([[chevre,chou],_,2]).
couple_interdit([[chevre,loup],_,2]).
couple_interdit([[chevre,chou,loup],_,2]).
couple_interdit([_,A,1]):-
    couple_interdit([A,_,2]).

% INITIALISATION DEPART ET ARRIVEE
% Ces faits permettent de définir respectivement l'état de départ et l'état de fin

depart([[chevre,chou,loup],[],1]).      % A l'état initial, les trois animaux sont sur la berge 1
                                         % tout comme le passeur
arrivee([[],[chevre,chou,loup],2]).     % A l'état final, les trois animaux sont sur la berge 2
                                         % tout comme le passeur

% TRI DE LA LISTE
% Le prédicat tri_liste/2 permet de trier la liste en premier argument : la liste triée est
% renvoyée dans le second argument.

tri_liste([],[]).                       % initialisation de la réponse à vide (Argument 2) lorsque tous les
                                         % éléments de la liste à trier (Argument 1) ont été traités
tri_liste(L,[chevre|R]):-               % ajout de chèvre en première place de la réponse
    ote_element(chevre,L,LL),           % si elle est élément de la liste à trier
    !,                                  % puisque chèvre est élément, on ne veut pas
```

```

                                % pouvoir trouver un autre élément en première
                                % place de la réponse pour cet appel d'où le cut
    tri_liste(LL,R).             % lancement récursif sur le reste de la liste
tri_liste(L,[chou|R]):-         % ajout de chou en première place de la réponse
    ote_element(chou,L,LL),     % si il est élément de la liste à trier
    !,                           % puisque chou est élément, on ne veut pas
                                % pouvoir trouver un autre élément en première
                                % place de la réponse pour cet appel d'où le cut
    tri_liste(LL,R).           % lancement récursif sur le reste de la liste
tri_liste(L,[loup|R]):-        % ajout de loup en première place de la réponse
    ote_element(loup,L,LL),     % si il est élément de la liste à trier
    tri_liste(LL,R).           % lancement récursif sur le reste de la liste

% CALCUL DE LA SOLUTION
% appel principal dont l'objectif est d'appeler le prédicat traverse/2 avec
% comme premier argument une liste comprenant l'état actuel c-à-d l'état de départ.

solution(S):-
    depart(L),                  % initialisation de l'état initial
    traverse([L],S),            % appel de la résolution
    nl,                         % permet d'effectuer un saut de ligne dans la console
    write('La reponse est :'),  % permet de d'afficher du texte dans la console
    nl,
    write(S),                   % permet de d'afficher de la solution dans la console
    nl.

% Le prédicat traverse/2 a simplement deux arguments : le premier est une liste
% d'états rangés dans le sens contraire de leur chronologie (le premier état est
% l'état courant, le dernier l'état initial). Ce premier argument va donc accumuler
% les états traversés en les empilant par l'avant. Le second argument est une liste
% d'états solution.

traverse([E|L],R):-            % prédicat d'arrêt
    arrivee(E),                 % si la l'état courant est l'état final
    renverse([E|L],[],R).       % on calcule la réponse en retournant la liste des
                                % états traversés

traverse([[R1,R2,1]|L],R):-    % traversée du bord 1 au bord 2
                                % l'état courant est alors [R1,R2,1]
    ote_element(X,R1,R1_new),   % prélèvement d'un animal (Argument 1) de la
                                % berge 1 (Argument 2); les animaux restant sur
                                % la berge 1 sont identifiés par la liste de
                                % l'argument 3.
    tri_liste([X|R2],R2_new),    % ajout de cet animal sur la berge 2 (Argument 1)
                                % et tri de la nouvelle liste (Argument 2)
    \+couple_interdit([R1_new,R2_new,2]), % absence de couples interdits
    \+member([R1_new,R2_new,2],L), % on n'est pas en train de calculer un état
                                % dans lequel on est déjà passé
    traverse([R1_new,R2_new,2],[R1,R2,1]|L),R). % appel récursif en ajoutant
                                % le nouvel état calculé à la
                                % liste des états déjà traversés

traverse([[R1,R2,2]|L],R):-    % traversée du bord 2 au bord 1 (même principe que 1 -> 2)
    ote_element(X,R2,R2_new),
    tri_liste([X|R1],R1_new),
    \+couple_interdit([R1_new,R2_new,1]),
    \+member([R1_new,R2_new,1],L),
    traverse([R1_new,R2_new,1],[R1,R2,2]|L),R).

```

```
traverse([R1,R2,B]|L],R):- % traversée à vide
    B1 is mod(B,2),          % calcul de la nouvelle berge pour le passeur :
    B_new is B1+1,          % modulo(1,2)+1=2 et modulo(2,2)+1=1
    \+couple_interdit([R1,R2,B_new]), % absence de couples interdits
    \+member([R1,R2,B_new],L), % on n'est pas en train de calculer un état
                                % dans lequel on est déjà passé
    traverse([R1,R2,B_new],[R1,R2,B]|L],R). % appel récursif en ajoutant
                                            % le nouvel état calculé à la
                                            % liste des états déjà traversés
```


11.2 Exercices chapitre 2

Exercice 1

[Retour à l'énoncé.](#)

— **Question 1 :** Traduire les phrases précédentes en **logique propositionnelle** :

1. étudiant $\rightarrow$ jeune
2. jeune $\rightarrow$ célibataire
3. célibataire $\rightarrow$ jeune
4. jeune $\rightarrow$ étudiant
5. (étudiant $\wedge$ enfant) $\rightarrow$ (marié $\vee$ concubin)
6. concubin $\rightarrow$ jeune
- 7.a célibataire $\rightarrow \neg$ marié $\wedge \neg$ concubin
- 7.b marié $\rightarrow \neg$ célibataire $\wedge \neg$ concubin
- 7.c concubin $\rightarrow \neg$ célibataire $\wedge \neg$ marié

Remarque :

La première difficulté est de pouvoir traduire le "une partie" de l'affirmation 4.

Qu'est ce que l'on obtient si l'on pose la question : "Fabrice est jeune ?"

- On ajoute cette nouvelle affirmation : Fabrice $\wedge$ Jeune.
- Avec l'affirmation 2, par Modus ponens on obtient : célibataire
- Avec l'affirmation 7a, par Modus ponens on obtient : $\neg$ marié
- Avec l'affirmation 7a, par Modus ponens on obtient : $\neg$ concubin
- Avec l'affirmation 4, par Modus ponens on obtient : étudiant
- Avec l'affirmation 5, par Modus Tollens on obtient : $\neg$ enfant puisqu'on a étudiant.

Remarque :

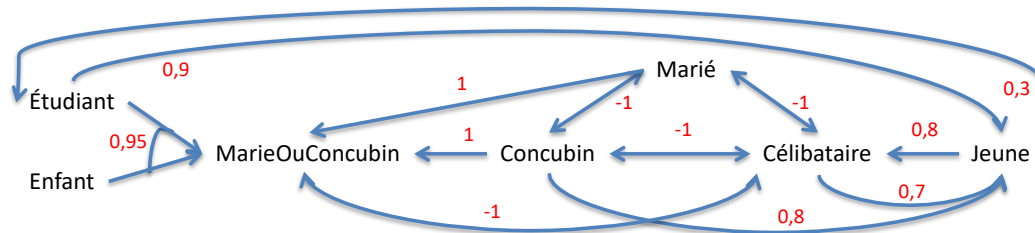
On peut se demander l'intérêt de savoir que l'individu jeune est Fabrice car cette information n'est pas exploitée en logique propositionnelle.

— **Question 2 :** Traduire les phrases précédentes en **règles de production** :

1. Si étudiant alors jeune (0,9)
2. Si jeune alors célibataire (0,8)
3. Si célibataire alors jeune (0,7)
4. Si jeune alors étudiant (0,3)
5. Si (étudiant $\wedge$ enfant) alors (mariéOuConcubin) (0,95)
6. Si concubin alors jeune (0,8)
7. Si célibataire alors marié (-1)
 Si célibataire alors concubin (-1)
 Si concubin alors marié (-1)
 Si concubin alors célibataire (-1)
 Si marié alors concubin (-1)
 Si marié alors célibataire (-1)

8. On ajoute les nouvelles règles :
- Si concubin alors mariéOuConcubin (1)
 - Si marié alors mariéOuConcubin (1)
 - Si mariéOuConcubin alors célibataire (-1)

Qu'est ce que l'on obtient si l'on pose la question : "Fabrice est jeune?"



Le fait que "Fabrice est jeune" permet de construire le graphe ci-dessus et d'obtenir les informations suivantes :

- étudiant avec un facteur de certitude de 0,9.
- concubin avec un facteur de certitude de 0,8.
- ou célibataire avec un facteur de certitude de 0,7

— **Question 3 :** Traduire les phrases précédentes en **logique des prédicats** :

1. $\forall x (\text{étudiant}(x) \rightarrow \text{jeune}(x))$
2. $\forall x (\text{jeune}(x) \rightarrow \text{célibataire}(x))$
3. $\forall x (\text{célibataire}(x) \rightarrow \text{jeune}(x))$
4. $\exists x (\text{jeune}(x) \rightarrow \text{étudiant}(x))$
5. $\forall x (\text{étudiant}(x) \wedge \text{enfant}(x) \rightarrow \text{marié}(x) \vee \text{concubin}(x))$
6. $\forall x (\text{concubin}(x) \rightarrow \text{jeune}(x))$
- 7.a $\forall x (\text{célibataire}(x) \rightarrow \neg \text{marié}(x) \wedge \neg \text{concubin}(x))$
- 7.b $\forall x (\text{marié}(x) \rightarrow \neg \text{célibataire}(x) \wedge \neg \text{concubin}(x))$
- 7.c $\forall x (\text{concubin}(x) \rightarrow \neg \text{célibataire}(x) \wedge \neg \text{marié}(x))$

Traduction de la phrase : "Fabrice est un étudiant qui vit en concubinage?"

- $\text{étudiant}(\text{Fabrice}) \wedge \text{concubin}(\text{Fabrice})$.
- Avec l'affirmation 1, on obtient : $\text{jeune}(\text{Fabrice})$
- Avec l'affirmation 2, on obtient : $\text{célibataire}(\text{Fabrice})$
- Avec l'affirmation 7a, on obtient : $\neg \text{marié}(\text{Fabrice})$
- Avec l'affirmation 7a, on obtient : $\neg \text{concubin}(\text{Fabrice})$

Ceci est contradictoire donc il faut gérer le problème pour cela, il existe plusieurs solutions pour modéliser cette exception :

1. On peut réécrire les affirmations pour y incorporer l'information sur l'individu déviant :
 - 1) $\forall x ((\text{étudiant}(x) \wedge x \neq \text{Fabrice}) \rightarrow \text{jeune}(x))$
 - 6) $\forall x ((\text{concubin}(x) \wedge x \neq \text{Fabrice}) \rightarrow \text{jeune}(x))$
 - 2) $\forall x ((\text{jeune}(x) \wedge x \neq \text{Fabrice}) \rightarrow \text{célibataire}(x))$

2. ou plus général 2) $\forall ((\text{Jeune}(X) \wedge \neg \text{concubin}(x)) \longrightarrow \text{célibataire}(x))$

— **Question 4 :** Traduire les phrases précédentes en **logique des défauts** :

1.
$$\frac{\text{étudiant}(x) : \text{jeune}(x)}{\text{jeune}(x)}$$
2.
$$\frac{\text{jeune}(x) : \text{célibataire}(x)}{\text{célibataire}(x)}$$
3.
$$\frac{\text{célibataire}(x) : \text{jeune}(x)}{\text{jeune}(x)}$$
4.
$$\frac{\text{jeune}(x) : \text{étudiant}(x)}{\text{étudiant}(x)} \quad \frac{\text{jeune}(x) : \neg \text{étudiant}(x)}{\neg \text{étudiant}(x)}$$
5.
$$\frac{\text{étudiant}(x) \wedge \text{enfant}(x) : \text{marié}(x) \vee \text{concubin}(x)}{\text{marié}(x) \vee \text{concubin}(x)}$$

On peut aussi travailler en décomposant en deux le défaut :

- $$\frac{\text{étudiant}(x) \wedge \text{enfant}(x) : \text{marié}(x)}{\text{marié}(x)} \quad \frac{\text{étudiant}(x) \wedge \text{enfant}(x) : \text{concubin}(x)}{\text{concubin}(x)}$$
6.
$$\frac{\text{concubin}(x) : \text{jeune}(x)}{\text{jeune}(x)}$$
 - 7.a $\forall x (\text{célibataire}(x) \longrightarrow \neg \text{marié}(x) \wedge \neg \text{concubin}(x))$
 - 7.b $\forall x (\text{marié}(x) \longrightarrow \neg \text{célibataire}(x) \wedge \neg \text{concubin}(x))$
 - 7.c $\forall x (\text{concubin}(x) \longrightarrow \neg \text{célibataire}(x) \wedge \neg \text{marié}(x))$

Qu'est ce que l'on obtient si l'on pose la question :

On traduit l'information "*Fabrice est étudiant, quel est son statut marital ?*", en logique des défauts :

8. $\text{étudiant}(\text{Fabrice})$

On crée ainsi une nouvelle théorie $S1=(D, W \cup \{8\})$, avec $D = \{1,2,3,4,5,6\}$ et $W = \{7a, 7b, 7c\}$. On obtient une extension :

— $E_1 : \text{étudiant}(\text{Fabrice}) [8], \text{Jeune}(\text{Fabrice}) [R_1], \text{célibataire}(\text{Fabrice}) [2], \neg \text{marié}(\text{Fabrice}) [7a], \neg \text{concubin}(\text{Fabrice}) [7b]$. On conclut à partir de 1 et 2 que s'il est étudiant alors il est célibataire.

On traduit l'information : "*Fabrice est étudiant et a peut-être des enfants, quel est son statut marital ?*", en logique des défauts, ceci nous donne deux défauts :

- 9a.
$$\frac{}{\text{enfant}(x)}$$
- 9b.
$$\frac{}{\neg \text{enfant}(x)}$$

On crée ainsi une nouvelle théorie $S2=(D \cup \{9,10\}, W \cup \{8\})$, on obtient quatre extensions :

- $E_1 : \text{étudiant}(\text{Fabrice}) [8], \text{enfant}(\text{Fabrice}) [9a], \text{jeune}(\text{Fabrice}) [1], \text{concubin}(\text{Fabrice}) [5b], \neg \text{célibataire}(\text{Fabrice}) [7b], \neg \text{marié}(\text{Fabrice}) [7c]$.
- $E_2 : \text{étudiant}(\text{Fabrice}) [8], \text{enfant}(\text{Fabrice}) [9a], \text{jeune}(\text{Fabrice}) [1], \text{marié}(\text{Fabrice}) [5a], \neg \text{concubin}(\text{Fabrice}) [7c], \neg \text{célibataire}(\text{Fabrice}) [7a]$.
- $E_3 : \text{étudiant}(\text{Fabrice}) [8], \text{enfant}(\text{Fabrice}) [9a], \text{jeune}(\text{Fabrice}) [1], \text{célibataire}(\text{Fabrice}) [2], \neg \text{marié}(\text{Fabrice}) [7a], \neg \text{concubin}(\text{Fabrice}) [7b]$.
- $E_4 : \text{étudiant}(\text{Fabrice}) [8], \neg \text{enfant}(\text{Fabrice}) [9b], \text{jeune}(\text{Fabrice}) [1], \text{célibataire}(\text{Fabrice}) [2], \neg \text{marié}(\text{Fabrice}) [7a], \neg \text{concubin}(\text{Fabrice}) [7b]$.

Exercice 2

[Retour à l'énoncé.](#)

— **Question 1 :** Traduire les phrases précédentes en **logique des défauts** :

$$R_1 : \frac{\text{étudiant}(x) : \text{jeune}(x)}{\text{jeune}(x)}$$

$$R_2 : \frac{\text{jeune}(x) : \text{foot}(x)}{\text{foot}(x)}$$

$$R_3 : \frac{\text{foot}(x) : \text{jeune}(x)}{\text{jeune}(x)}$$

$$R_{4a} : \frac{\text{jeune}(x) : \text{étudiant}(x)}{\text{étudiant}(x)}$$

$$R_{4b} : \frac{\text{jeune}(x) : \neg \text{étudiant}(x)}{\neg \text{étudiant}(x)}$$

$$R_{5a} : \frac{\text{étudiant}(x) \wedge \text{gagne}(x) : \text{belote}(x)}{\text{belote}(x)}$$

$$R_{5b} : \frac{\text{étudiant}(x) \wedge \text{gagne}(x) : \text{pétanque}(x)}{\text{pétanque}(x)}$$

$$R_6 : \frac{\text{belote}(x) : \text{jeune}(x)}{\text{jeune}(x)}$$

$$R_{7a} : \forall x (\neg \text{belote}(x) \vee \neg \text{pétanque}(x)) \wedge$$

$$R_{7b} : \forall x (\neg \text{belote}(x) \vee \neg \text{foot}(x)) \wedge$$

$$R_{7c} : \forall x (\neg \text{foot}(x) \vee \neg \text{pétanque}(x))$$

$$R_{8a} : \frac{\text{enseignant}(x) : \text{étudiant}(x)}{\text{étudiant}(x)}$$

$$R_{8b} : \frac{\text{enseignant}(x) : \neg \text{étudiant}(x)}{\neg \text{étudiant}(x)}$$

— **Question 2 :** Calculer toutes les extensions de cette théorie des défauts, sachant que "Fabrice est un enseignant ?" :

$$R_9 : \text{enseignant}(\text{Fabrice})$$

On crée ainsi une nouvelle théorie $S1 = (D, W \cup \{R_9\})$, avec $D = \{R_1, R_2, R_3, R_4, R_5, R_6, R_8\}$ et $W = \{R_{7a}, R_{7b}, R_{7c}\}$. On obtient deux extensions :

— E_1 : $\text{enseignant}(\text{Fabrice}) [R_9]$, $\neg \text{étudiant}(\text{Fabrice}) [R_{8b}]$. Elle peut être complétée par toutes autres valeurs pour les autres prédicats soit $2^3 = 8$ solutions complètes.

— E_2 : $\text{enseignant}(\text{Fabrice}) [R_9]$, $\text{étudiant}(\text{Fabrice}) [R_{8a}]$, $\text{jeune}(\text{Fabrice}) [R_1]$, $\text{foot}(\text{Fabrice}) [R_2]$, $\neg \text{belote}(\text{Fabrice}) [R_{7b}]$, $\neg \text{pétanque}(\text{Fabrice}) [R_{7c}]$.

— **Question 3 :** Calculer toutes les extensions de cette théorie des défauts, sachant que "Fabrice est un enseignant, et qu'il a peut être gagné" :

$$R_{10a} : \frac{: \text{gagne}(x)}{\text{gagne}(x)}$$

$$R_{10b} : \frac{: \neg \text{gagne}(x)}{\neg \text{gagne}(x)}$$

On crée ainsi une nouvelle théorie $S2 = (D \cup \{R_{10}\}, W \cup \{R_9\})$, On obtient six extensions :

— E_3 : $\text{enseignant}(\text{Fabrice}) [R_9]$, $\neg \text{étudiant}(\text{Fabrice}) [R_{8b}]$, $\text{gagne}(\text{Fabrice}) [R_{10a}]$.

— E_4 : $\text{enseignant}(\text{Fabrice}) [R_9]$, $\neg \text{étudiant}(\text{Fabrice}) [R_{8b}]$, $\neg \text{gagne}(\text{Fabrice}) [R_{10b}]$.

— E_5 : $\text{enseignant}(\text{Fabrice}) [R_9]$, $\text{étudiant}(\text{Fabrice}) [R_{8a}]$, $\text{jeune}(\text{Fabrice}) [R_1]$, $\text{foot}(\text{Fabrice}) [R_2]$, $\neg \text{belote}(\text{Fabrice}) [R_{7b}]$, $\neg \text{pétanque}(\text{Fabrice}) [R_{7c}]$, $\text{gagne}(\text{Fabrice}) [R_{10a}]$.

— E_6 : $\text{enseignant}(\text{Fabrice}) [R_9]$, $\text{étudiant}(\text{Fabrice}) [R_{8a}]$, $\text{jeune}(\text{Fabrice}) [R_1]$, $\text{foot}(\text{Fabrice}) [R_2]$, $\neg \text{belote}(\text{Fabrice}) [R_{7b}]$, $\neg \text{pétanque}(\text{Fabrice}) [R_{7c}]$, $\neg \text{gagne}(\text{Fabrice}) [R_{10b}]$.

- E_7 : $\text{enseignant}(\text{Fabrice}) [R_9]$, $\text{gagne}(\text{Fabrice}) [R_{10a}]$, $\text{étudiant}(\text{Fabrice}) [R_{8a}]$, $\text{jeune}(\text{Fabrice}) [R_1]$, $\text{belote}(\text{Fabrice}) [R_{5a}]$, $\neg \text{pétanque}(\text{Fabrice}) [R_{7a}]$, $\neg \text{foot}(\text{Fabrice}) [R_{7b}]$.
- E_8 : $\text{enseignant}(\text{Fabrice}) [R_9]$, $\text{gagne}(\text{Fabrice}) [R_{10a}]$, $\text{étudiant}(\text{Fabrice}) [R_{8a}]$, $\text{jeune}(\text{Fabrice}) [R_1]$, $\text{pétanque}(\text{Fabrice}) [R_{5b}]$, $\neg \text{belote}(\text{Fabrice}) [R_{7a}]$, $\neg \text{foot}(\text{Fabrice}) [R_{7c}]$.
- **Question 4** : "Est-ce que l'on peut avoir des enseignants qui sont étudiants et qui jouent au foot ?", calculer les extensions de cette théorie des défauts :
 - On peut directement dire que Fabrice correspond à ces éléments et reprendre l'extension E_2 de la question 2.
 - On rajoute une nouvelle information R_{11} : " $\forall x \text{ enseignant}(x) \wedge \text{étudiant}(x) \wedge \text{foot}(x)$ ". On crée ainsi une nouvelle théorie $S3=(D, W \cup 11)$, avec $D = 1,2,3,4,5,6$ et $W = 7a, 7b, 7c$. On obtient une extension :
 - E_9 : $\text{enseignant}(x) [R_{11}]$, $\text{étudiant}(x) [R_{11}]$, $\text{foot}(x) [R_{11}]$, $\neg \text{belote}(x) [R_{7b}]$, $\neg \text{pétanque}(x) [R_{7c}]$, $\text{jeune}(x) [R_1]$. Il y a juste un problème ce que l'on n'a pas d'individu identifié correspondant au profil. Mais on a trouvé un modèle à notre théorie qui autorise des individus avec ce profil.

Exercice 3

[Retour à l'énoncé.](#)

On définit les variables propositionnelles suivantes :

- RC : Ribouldingue est coupable
- FC : Filochard est coupable
- CC : Croquignol est coupable

1. Les affirmations se traduisent de la façon suivante en logique propositionnelle :

- (a) $RC \wedge \neg FC$
- (b) $(CC \rightarrow FC)$, on peut l'écrire $\neg CC \vee FC$
- (c) $\neg FC \wedge (CC \vee RC)$

2. On analyse phrase par phrase les affirmations

- (a) $RC \wedge \neg FC$, pour que la formule soit satisfaite il faut que les deux variables soient interprétées respectivement à vraie et à faux donc Ribouldingue est coupable et Filochard non
- (b) $\neg CC \vee FC$, pour que la formule soit satisfaite, il faut que CC soit interprété à faux donc Croquignol est non coupable
- (c) $\neg FC \wedge (CC \vee RC)$, cette formule est aussi satisfaite.

donc Ribouldingue est coupable, Filochard non et croquignol non.

Autre solution : utiliser une table de vérité, alors on prend la ligne 5 où les affirmations 1,2,3 valent 1 :

RC	FC	CC	1	2	3
0	0	0	0	1	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	0
1	0	0	1	1	1
1	0	1	1	0	1
1	1	0	0	1	0
1	1	1	0	1	0

3. On prend la négation des affirmations et on obtient phrase par phrase :

- (a) $\neg RC \vee FC$, pour que la formule soit satisfaite il faut que ou RC soit interprété à Faux ou FC à vraie
- (b) $CC \wedge \neg FC$, pour que la formule soit satisfaite, il faut que CC soit interprété à vraie et FC à faux donc par rapport à la première affirmation il faut que RC soit à Faux.
- (c) $FC \vee (\neg CC \wedge \neg RC)$, cette formule n'est pas satisfaite, donc ils ne peuvent pas avoir tous mentis. En effet, on trouve aucune ligne dans la table où toutes les affirmations sont à 0.

4. En reprenant les affirmations :

- (a) $RC \wedge \neg FC$
- (b) $\neg CC \vee FC$
- (c) $\neg FC \wedge (CC \vee RC)$, peut se réécrire en $(\neg FC \wedge CC) \vee (\neg FC \wedge RC)$

Pour répondre à la question on va chercher quelle affirmation implique quelle autre. Pour cela on peut faire une table de vérité comme ci-dessous, ce qui nous donne que la 3^{ème} affirmation est incluse (impliquée) par la 1^{ière}. Cette information est utile pour la suite car on sera que si Croquignol dit la vérité Filochard aussi et si Filochard ment alors Croquignol aussi.

RC	FC	CC	1	2	3	$1 \rightarrow 2$	$1 \rightarrow 3$	$2 \rightarrow 1$	$2 \rightarrow 3$	$3 \rightarrow 1$	$3 \rightarrow 2$
0	0	0	0	1	0	1	1	0	0	1	1
0	0	1	0	0	1	1	1	1	1	0	0
0	1	0	0	1	0	1	1	0	0	1	1
0	1	1	0	1	0	1	1	0	0	1	1
1	0	0	1	1	1	1	1	1	1	1	1
1	0	1	1	0	1	0	1	1	1	1	0
1	1	0	0	1	0	1	1	0	0	1	1
1	1	1	0	1	0	1	1	0	0	1	1

5. a. En reprenant les affirmations, en ajoutant comme information $\neg RC \wedge \neg FC \wedge \neg CC$ (ligne 1 de la table de vérité) :

- (a) $RC \wedge \neg FC$, cette formule n'est pas satisfaite donc Croquignol a menti
 - (b) $\neg CC \vee FC$, cette formule est satisfaite donc Riboudingue n'a pas menti
 - (c) $(\neg FC \wedge CC) \vee (\neg FC \wedge RC)$, cette formule n'est pas satisfaite donc Filochard a menti
- b. En reprenant les affirmations, en ajoutant comme information $RC \wedge FC \wedge CC$ (ligne 8 de la table de vérité) :
- (a) $RC \wedge \neg FC$, cette formule n'est pas satisfaite donc Croquignol a menti

- (b) $\neg CC \vee FC$, cette formule est satisfaisable donc Ribouldingue n'a pas menti
 - (c) $(\neg FC \wedge CC) \vee (\neg FC \wedge RC)$, cette formule n'est pas satisfaite donc Filochard a menti
6. En se référant à la table de vérité on a une possibilité avec un seul faux témoignage (une seule phrase à 0), en ligne 6 : le menteur est Ribouldingue (qui a donné l'affirmation), Ribouldingue et Croquignol sont coupables (variables à la valeur 1 sur la ligne).
7. En se référant à la table de vérité on a une possibilité avec comme information :
- (a) L'affirmation 1 vaut 0 lorsque CC vaut 1 (ou 1 pour CC = 0)
 - (b) L'affirmation 2 vaut 0 lorsque RC vaut 1 (ou 1 pour RC = 0)
 - (c) L'affirmation 3 vaut 0 lorsque FC vaut 1 (ou 1 pour FC = 0)
- Ligne 4, Filochard et Croquignol sont coupables et Ribouldingue est innocent.

11.3 Exercices chapitre 3

Exercice 1

[Retour à l'énoncé.](#)

1. Question 1 :

```
:-use_module(library(lists)).

depart([1,1]).
arrivee([5,5]).

frontiere([1,1],[1,2]).
frontiere([1,2],[2,2]).
frontiere([1,3],[2,3]).
frontiere([1,4],[1,5]).
frontiere([2,2],[3,2]).
frontiere([2,4],[3,4]).
frontiere([3,1],[3,2]).
frontiere([3,2],[3,3]).
frontiere([3,3],[3,4]).
frontiere([3,3],[4,3]).
frontiere([3,4],[3,5]).
frontiere([4,1],[4,2]).
frontiere([4,4],[4,5]).
frontiere([5,2],[5,3]).
frontiere([5,3],[5,4]).

%Pour éviter de définir les murs dans l'autre sens, on définit
% le prédicat front qui effectue le test dans les deux sens X->Y et Y->X
front(X,Y):- frontiere(X,Y).
front(X,Y):- frontiere(Y,X).
```

2. Question 2 :

```
%Une case est correct si ses coordonnées sont dans le labyrinthe
caseCorrecte([X,Y]):-
    X > 0,
    Y > 0,
    X =< 5,
    Y =< 5.
```

3. Question 3 :

```
%Un déplacement est correct si il n'y a pas de frontière et
% que la case où l'on veut aller est correct.
deplacementCorrect(C1,C2):-
    \+front(C1,C2),
    caseCorrecte(C2).
```


4. Question 4 :

```
% à droite
deplace([C_ini|Parcours],[C_fin,C_ini|Parcours]):-
    C_ini = [X_ini,Y_ini],
    X_fin is X_ini+1,
    C_fin = [X_fin,Y_ini],
    deplacementCorrect(C_ini,C_fin),
    \+ member(C_fin,Parcours).
```

```
% à gauche
deplace([C_ini|Parcours],[C_fin,C_ini|Parcours]):-
    C_ini = [X_ini,Y_ini],
    X_fin is X_ini-1,
    C_fin = [X_fin,Y_ini],
    deplacementCorrect(C_ini,C_fin),
    \+ member(C_fin,Parcours).
```

```
% en haut
deplace([C_ini|Parcours],[C_fin,C_ini|Parcours]):-
    C_ini = [X_ini,Y_ini],
    Y_fin is Y_ini+1,
    C_fin = [X_ini,Y_fin],
    deplacementCorrect(C_ini,C_fin),
    \+ member(C_fin,Parcours).
```

```
% en bas
deplace([C_ini|Parcours],[C_fin,C_ini|Parcours]):-
    C_ini = [X_ini,Y_ini],
    Y_fin is Y_ini-1,
    C_fin = [X_ini,Y_fin],
    deplacementCorrect(C_ini,C_fin),
    \+ member(C_fin,Parcours).
```

5. Question 5 :

```
% Si la première case du parcours courant est l'arrivée alors on a
% trouvé un chemin
parcours([C|Parcours],[C|Parcours]):-
    arrivee(C),!.
```

```
% Sinon on effectue les déplacements possibles à partir de la
% dernière case du chemin courant et on voit si le parcours est correct.
parcours(Parcours,R):-
    deplace(Parcours,Suivant),
    parcours(Suivant,R).
```

6. Question 6 :

```

labyrinthe(R):-
    depart(C),
    parcours([C],R1),
    reverse(R1,R).

```

Exercice 2

[Retour à l'énoncé.](#)

```

% PRE-REQUIS
% On ajoute une liste d'élément L1 entête d'une liste L2
% le résultat est donné dans L3
% La liste L1 est vide on recopie directement L2 dans L3
ajout([],L2,L2):-!.
% On privilégiera l'utilisation du prédicat append/3 de la bibliothèque lists.
% Cas général, on ajoute un élément X de L1 dans L3 en
% retour de l'appel récursif
ajout([X|L1],L2,[X|L3]):-
    ajout(L1,L2,L3).

```

1. Question 1 :

```

% Identifier si le chemin a atteint l'arrivée
contientEtatFinal([A|_]):-
    arrivee(A).

% Prédicat calculant la liste des états suivants
% à partir d'un état initial

parcoursSuivant(X,N_PC,R):-
    deplace(X,Parcours),
    \+ member(Parcours,N_PC),
    parcoursSuivant(X,[Parcours|N_PC],R),!.

parcoursSuivant(_,R,R).

% On a traité tous les chemins de la liste
etageSuivant([],NV_L,NV_L):-!.

% On n'étend pas un chemin qui a déjà atteint l'arrivée
etageSuivant([X|L],NV_L,R):-
    contientEtatFinal(X),!,
    etageSuivant(L,NV_L,R).

%On récupère un chemin X pour lequel on calcule tous les déplacements,
%on le rajoute à la liste des chemins et on recommence
% avec les chemins restant L.

```

```

etageSuivant ([X|L], NV_L, R) :-
    parcoursSuivant (X, [], PC),
    ajout (PC, NV_L, NV_L1),
    etageSuivant (L, NV_L1, R) .

```

2. Question 2 :

```

% On est dans l'état final, si le chemin R contient comme
% dernière case l'arrivée A
etatFinal ([R|_], R) :-
    contientEtatFinal (R) .

% Sinon on regarde les autres chemins
etatFinal ([_|N_L], R) :-
    etatFinal (N_L, R) .

```

3. Question 3 :

```

% Il n'y a plus aucun chemin a etendre
largeur ([], _) :- !, fail.
% Permet de tester si on a trouve une solution
largeur (N_L, R) :-
    etatFinal (N_L, R) .
% On etend la liste des chemins d'un pas
largeur (L, R) :-
    etageSuivant (L, [], N_L),
    largeur (N_L, R) .

```

4. Question 4 :

```

labyrinthe (R) :-
    depart (C),
    largeur ([C], R1),
    reverse (R1, R) .

```

Exercice 3

[Retour à l'énoncé.](#)

1. Question 1 :

```

% Le calcul du coût est égal à la distance (norme euclidienne)
% entre la case courante et l'arrivée
cout ([XC, YC], Cout) :-
    arrivee ([XA, YA]),
    X is XC-XA,
    Y is YC-YA,
    Cout is sqrt ((X*X)+(Y*Y)) .

```

2. Question 2 :

```
size([],0).
```

```
size([_|L],G):-  
    size(L,G1),  
    G is G1+1.
```

3. Question 3 :

```
% La difficulté est sur la forme de la liste qui en plus de contenir  
% des chemins comme pour la largeur, elle contient aussi le coût  
% (heuristique + distance déjà parcourue)
```

```
% à droite
```

```
deplace([C_ini|Parcours],[C_fin,C_ini|Parcours],Cout):-  
    C_ini = [X_ini,Y_ini],  
    X_fin is X_ini+1,  
    C_fin = [X_fin,Y_ini],  
    deplacementCorrect(C_ini,C_fin),  
    \+ member(C_fin,Parcours),  
    cout(C_fin,Cout1),  
    size([C_fin,C_ini|Parcours],Cout2),  
    Cout is Cout1+Cout2.
```

```
% à gauche
```

```
deplace([C_ini|Parcours],[C_fin,C_ini|Parcours],Cout):-  
    C_ini = [X_ini,Y_ini],  
    X_fin is X_ini-1,  
    C_fin = [X_fin,Y_ini],  
    deplacementCorrect(C_ini,C_fin),  
    \+ member(C_fin,Parcours),  
    cout(C_fin,Cout1),  
    size([C_fin,C_ini|Parcours],Cout2),  
    Cout is Cout1+Cout2.
```

```
% en haut
```

```
deplace([C_ini|Parcours],[C_fin,C_ini|Parcours],Cout):-  
    C_ini = [X_ini,Y_ini],  
    Y_fin is Y_ini+1,  
    C_fin = [X_ini,Y_fin],  
    deplacementCorrect(C_ini,C_fin),  
    \+ member(C_fin,Parcours),  
    cout(C_fin,Cout1),  
    size([C_fin,C_ini|Parcours],Cout2),  
    Cout is Cout1+Cout2.
```

```
% en bas
```

```

deplace([C_ini|Parcours],[C_fin,C_ini|Parcours],Cout):-
    C_ini = [X_ini,Y_ini],
    Y_fin is Y_ini-1,
    C_fin = [X_ini,Y_fin],
    deplacementCorrect(C_ini,C_fin),
    \+ member(C_fin,Parcours),
    cout(C_fin,Cout1),
    size([C_fin,C_ini|Parcours],Cout2),
    Cout is Cout1+Cout2.

```

4. Question 4 :

```

parcoursSuivant(X,N_PC,R):-
    deplace(X,Parcours,Cout),
    \+ member([Cout,Parcours],N_PC),!,
    parcoursSuivant(X,[Cout,Parcours]|N_PC,R).

```

```

parcoursSuivant(_,R,R).

```

5. Question 5 :

```

insere([],R,R).

```

```

insere([X|LC2],LC,R):-
    insereC(LC,X,LCX),
    insere(LC2,LCX,R).

```

```

insereC([],X,[X]).

```

```

insereC([Cout,L]|LC,[CoutX,LX],[CoutX,LX],[Cout,L]|LC):-
    Cout > CoutX.

```

```

insereC([Cout,L]|LC,[CoutX,LX],[Cout,L]|LCX):-
    Cout < CoutX,
    insereC(LC,[CoutX,LX],LCX).

```

```

insereC([Cout,L]|LC,[CoutX,LX],[Cout,L]|LCX):-
    Cout = CoutX,
    size(L,SizeL),
    size(LX,SizeLX),
    SizeL > SizeLX,
    insereC(LC,[CoutX,LX],LCX).

```

```

insereC([Cout,L]|LC,[CoutX,LX],[CoutX,LX],[Cout,L]|LC):-
    Cout = CoutX,
    size(L,SizeL),
    size(LX,SizeLX),
    SizeL =< SizeLX.

```

6. Question 6 :

```

etatFinal([[_ , R] | _], R) :-
    contientEtatFinal(R) .

heuristique(N_L, R) :-
    etatFinal(N_L, R) .

heuristique([[_ , L] | LR], R) :-
    parcoursSuivant(L, [], NL) ,
    insere(LR, NL, N_L) ,
    heuristique(N_L, R) .

```

7. Question 7 :

```

labyrinthe(R) :-
    depart(A) ,
    cout(A, Cout1) ,
    size([A], Cout2) ,
    Cout is Cout1+Cout2,
    heuristique([[Cout, [A]]], R1) ,
    reverse(R1, R) .

```

Exercice 4

[Retour à l'énoncé.](#)

```

1. depart([[a, i, m, o], [], []]) .
2. arrivee([[m, o, i, a], [], []]) .
3. position(m, 3, 1) .
   position(o, 2, 1) .
   position(i, 1, 1) .
   position(a, 0, 1) .

hauteur(_X, [], 0, 0, 0) .

hauteur(X, [X|L], R, R, 1) :-!,
    hauteur(X, L, R, _R, _P) .

hauteur(X, [_|L], T1, R, P) :-
    hauteur(X, L, T, R, P) ,
    T1 is T + 1 .

place(X, [C1, C2, C3], P) :-
    position(X, H, C) ,
    hauteur(X, C1, _, H1, R1) ,
    hauteur(X, C2, _, H2, R2) ,

```

```

    hauteur(X,C3,_,H3,R3),
    P is (abs(H - (H1 + H2 + H3)) + abs(C - (R1 + 2 * R2 + 3 * R3))).

cout(L,Cout):-
    place(m,L,Pm),
    place(o,L,Po),
    place(i,L,Pi),
    place(a,L,Pa),
    Cout is Pm + Po + Pi + Pa.
4. size(L,G):-
    length(L,G).
5. deplaceColonne([X|L],L1,L,[X|L1]).

%calcul cout
calcul_cout(EtatFin, Parcours,Cout):-
    \+ member(EtatFin,Parcours),
    cout(EtatFin,Cout1),
    size([EtatFin|Parcours],Cout2),
    Cout is Cout1+Cout2.

%prédicat de déplacement (1 par colonne)
% 1 -> 2
deplace([EtatInit|Parcours],[EtatFin, EtatInit|Parcours],Cout):-
    EtatInit=[C1,C2,C3],
    deplaceColonne(C1,C2,C1New,C2New),
    EtatFin = [C1New, C2New, C3],
    calcul_cout(EtatFin,[EtatInit|Parcours],Cout).
% 1 -> 3
deplace([EtatInit|Parcours],[EtatFin, EtatInit|Parcours],Cout):-
    EtatInit=[C1,C2,C3],
    deplaceColonne(C1,C3,C1New,C3New),
    EtatFin = [C1New, C2, C3New],
    calcul_cout(EtatFin,[EtatInit|Parcours],Cout).
%2 -> 1
deplace([EtatInit|Parcours],[EtatFin, EtatInit|Parcours],Cout):-
    EtatInit = [C1,C2,C3],
    deplaceColonne(C2,C1, C2New, C1New),
    EtatFin = [C1New, C2New, C3],
    calcul_cout(EtatFin,[EtatInit|Parcours],Cout).
%2 -> 3
deplace([EtatInit|Parcours],[EtatFin, EtatInit|Parcours],Cout):-
    EtatInit = [C1,C2,C3],
    deplaceColonne(C2,C3, C2New, C3New),
    EtatFin = [C1, C2New, C3New],
    calcul_cout(EtatFin,[EtatInit|Parcours],Cout).
%3 -> 1
deplace([EtatInit|Parcours],[EtatFin, EtatInit|Parcours],Cout):-
    EtatInit = [C1,C2,C3],

```

```

        deplaceColonne(C3,C1, C3New, C1New),
        EtatFin = [C1New, C2, C3New],
        calcul_cout(EtatFin,[EtatInit|Parcours],Cout).
%3 -> 2
deplace([EtatInit|Parcours],[EtatFin, EtatInit|Parcours],Cout):-
    EtatInit = [C1,C2,C3],
    deplaceColonne(C3,C2, C3New, C2New),
    EtatFin = [C1, C2New, C3New],
    calcul_cout(EtatFin,[EtatInit|Parcours],Cout).
6. supprime(L,[],L,[]).

supprime([CL,[X|L]], [[CL1,[X|_L1]]|LR],[CL,[X|L]],LR):-
    CL =< CL1,!.

supprime([CL,[X|_L]], [[CL1,[X|L1]]|LR],[[], [[CL1,[X|L1]]|LR]]:-
    CL > CL1,!.

supprime(L,[X|LR],NL,[X|NLR]):-
    supprime(L,LR,NL,NLR).
7. supprimeAll([],LR,[],LR).

supprimeAll([X|L],LR,[NX|NL],NNLR):-
    supprime(X,LR,NX, NLR),
    supprimeAll(L,NLR,NL,NNLR).
8. insere(R,[],R).

insere(LC,[X|LC2],R):-
    insereC(LC,X,LCX),
    insere(LCX,LC2,R).

%% Le seul nouveau prédicat
%%
insereC(L,[],L).

%% Le reste est inchangé
%%
insereC([],X,[X]).

insereC([[Cout,L]|LC],[CoutX,LX],[[CoutX,LX],[Cout,L]|LC]):-
    Cout > CoutX,!.

insereC([[Cout,L]|LC],[CoutX,LX],[[Cout,L]|LCX]):-
    Cout =< CoutX,
    insereC(LC,[CoutX,LX],LCX).

insereC([[Cout,L]|LC],[Cout,LX],[[Cout,L]|LCX]):-
    size(L,SizeL),
    size(LX,SizeLX),

```



```

        SizeL > SizeLX,
        insereC(LC, [Cout, LX], LCX) .

insereC([ [Cout, L] | LC], [Cout, LX], [ [Cout, LX], [Cout, L] | LC]) :-
    size(L, SizeL),
    size(LX, SizeLX),
    SizeL =< SizeLX.
9. heuristique(N_L, R) :-
    etat_final(N_L, R) .

heuristique([[_ , L] | LR], R) :-
    parcoursSuivant(L, [], NL),
    supprimeAll(NL, LR, NNL, NLR),
    insere(NLR, NNL, N_L),
    heuristique(N_L, R) .
10. % Solutions :

S = [10, [[m,o,i,a],[],[[]],[[o,i,a],[m],[[]],[[i,a],[o,m],[[]],
[[a],[i,o,m],[[]],[[]],[i,o,m],[a]],[[]],[o,m],[i,a]],[[o],[m],[i,a]],
[[m,o],[[]],[i,a]],[[i,m,o],[[]],[a]],[[a,i,m,o],[[]],[[]]]]

```

11.4 Exercices chapitre 4

Exercice 1

[Retour à l'énoncé.](#)

```
% Pré-requis :
:-use_module(library(lists)).
```

1. Question 1 :

```
crypto1([D,E,M,N,O,R,S,Y]):-
    member(D,[0,1,2,3,4,5,6,7,8,9]),
    member(E,[0,1,2,3,4,5,6,7,8,9]),
    member(M,[0,1,2,3,4,5,6,7,8,9]),
    member(N,[0,1,2,3,4,5,6,7,8,9]),
    member(O,[0,1,2,3,4,5,6,7,8,9]),
    member(R,[0,1,2,3,4,5,6,7,8,9]),
    member(S,[0,1,2,3,4,5,6,7,8,9]),
    member(Y,[0,1,2,3,4,5,6,7,8,9]),
    dif(M,0),
    dif(D,E),dif(D,M),dif(D,N),dif(D,O),dif(D,R),dif(D,S),dif(D,Y),
    dif(E,M),dif(E,N),dif(E,O),dif(E,R),dif(E,S),dif(E,Y),
    dif(M,N),dif(M,O),dif(M,R),dif(M,S),dif(M,Y),
    dif(N,O),dif(N,R),dif(N,S),dif(N,Y),
    dif(O,R),dif(O,S),dif(O,Y),
    dif(R,S),dif(R,Y),
    dif(S,Y),
    Somme is (1000*(S+M))+(100*(E+O))+(10*(N+R))+(D+E),
    Somme is (10000*M)+(1000*O)+(100*N)+(10*E)+Y.
```

Dans cette approche, nous définissons les domaines de chacune des variables avec le prédicat `member`. Le système va ainsi effectuer toutes les combinaisons possibles de valeurs pour les variables. On supprime les combinaisons pour lesquelles les variables ne sont pas toutes différentes, avec les `dif`. Puis, on cherche la solution correspondant à l'addition tel que `SEND+MORE` est la même somme que `MONEY`. Une version moins naïve serait d'utiliser le `appartient_dif` ou le `select` à la place du `appartient/member`.

2. Question 2 :

```
crypto2([D,E,M,N,O,R,S,Y]):-
    select(D,[0,1,2,3,4,5,6,7,8,9],Nv_Dom1),
    select(E,Nv_Dom1,Nv_Dom2),
    select(Y,Nv_Dom2,Nv_Dom3),
    calc_retenue(D,E,0,Y,Ret1),
    select(N,Nv_Dom3,Nv_Dom4),
    select(R,Nv_Dom4,Nv_Dom5),
```

```

    calc_retenue(N,R,Ret1,E,Ret2),
    select(O,Nv_Dom5,Nv_Dom6),
    calc_retenue(E,O,Ret2,N,Ret3),
    select(S,Nv_Dom6,Nv_Dom7),
    select(M,Nv_Dom7,_),
    calc_retenue(S,M,Ret3,O,M),
    dif(M,0).

calc_retenue(X,Y,Retenue,Somme,N_Retenue):-
    Add is X+Y+Retenue,
    Somme is mod(Add,10),
    N_Retenue is (Add-Somme)//10.

appartient_dif(X,[X|L],L).
appartient_dif(D,[X|L],[X|NL]):-
    appartient_dif(D,L,NL).

```

Dans cette approche, on utilise le prédicat `select` pour proposer à la variable `X` un domaine ne contenant que des valeurs non encore utilisées. L'idée est de pouvoir poser le plus rapidement les additions, pour cela on donne un domaine aux trois variables correspondant aux unités (`D,E,Y`) et on pose l'addition avec le prédicat `calc_retenue`. Ainsi on peut faire les dizaines en utilisant l'information sur la propagation éventuelle d'une retenue venant des unités, puis on continue avec (`N,R`) et le calcul de la retenue des dizaines pour calculer les centaines et etc.

3. Question 3 :

```

:- use_module(library(clpfd)).

send([S,E,N,D,M,O,R,Y],Type):-
    domaines(S,E,N,D,M,O,R,Y),
    contraintes(S,E,N,D,M,O,R,Y),
    somme(S,E,N,D,M,O,R,Y),
    labeling(Type,[S,E,N,D,M,O,R,Y]).

domaines(S,E,N,D,M,O,R,Y):-
    [S,E,N,D,M,O,R,Y] ins 0..9.

contraintes(S,E,N,D,M,O,R,Y):-
    M#>0,
    all_different([S,E,N,D,M,O,R,Y]).

somme(S,E,N,D,M,O,R,Y):-
    (1000*(S+M))+(100*(E+O))+(10*(N+R))+(D+E) #=
    (10000*M)+(1000*O)+(100*N)+(10*E)+Y.

```

Cette dernière approche est basé sur la PLC est fait appelle aux primitives de la bibliothèque CLP(FD). Elle est très simple puisqu'elle permet de définir les variables avec leurs domaines (`domaines/8`). En suite, on pose les contraintes (`contraintes/8`) où toutes les

valeurs des variables doivent être différentes et M ne peut être égal à zéro et la somme (somme/8) que l'on veut représenter. En final, pour avoir nos solutions, on utilise la procédure de labelling qui va énumérer les solutions possibles du système.

4. Question 4 : S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2

Exercice 2

[Retour à l'énoncé.](#)

1. Question 1 :

```
:- use_module(library(lists)).

reines(N,R):-
    construit_liste(N,L_Nb),
    placer1(N,L_Nb,[],R),
    test_nen_prise1(R).

%Construit une liste de N entiers
construit_liste(1,[1]):- !.
construit_liste(N,[N|L_Nb]):-
    N1 is N-1,
    construit_liste(N1,L_Nb).

% Construit toutes les permutations en mettant une liste candidate
% dans le 4eme argument
placer1(0,_,R,R):- !.
placer1(N,L_Nb,L,R):-
    member(Y,L_Nb),
    N1 is N-1,
    placer1(N1,L_Nb,[N,Y]|L,R).

%Regarde si la liste contient des reines en prises
test_nen_prise1([]).
test_nen_prise1([C1|R]):-
    test_nen_prise11(R,C1),
    test_nen_prise1(R).

% Regarde si une case (2eme argument) est en prise avec un des
% éléments de la liste (1er argument).
test_nen_prise11([],_).
test_nen_prise11([X1,Y1]|R,[X,Y]):-
    X =\= X1, % Colonne
    Y =\= Y1, % Ligne
    abs(X-X1) =\= abs(Y-Y1), % Diagonale
    test_nen_prise11(R,[X,Y]).
```

2. Question 2 :

```

reinesBT(N,R):-
    construit_liste(N,L_Nb),
    placer2(N,L_Nb,[],R).

placer2(0,_,R,R):-!.
placer2(Nb,L_Nb,L,R):-
    select(Y,L_Nb,L_Nb1),
    test_nen_prisell(L,[Nb,Y]),
    Nb1 is Nb-1,
    placer2(Nb1,L_Nb1,[ [Nb,Y] | L],R).

```

3. Question 3 :

```

:- use_module(library(clpfd)).

reinesPLC(N,Type,Sol):-
    length(LV,N),
    LV ins 1..N,
    all_different(LV),
    constraint(N,LV,Los),
    labeling(Type,LV),
    %% on peut faire un reverse pour avoir la liste comme les autres
    reverse(Los,Sol).

constraint(0,_,[]):-!.
constraint(I,[Y|LV],[[I,Y]|R]):-
    I1 is I-1,
    constraint(I1,LV,R),
    constraint_one([I,Y],R).

constraint_one(_,[]):-!.
constraint_one([A,B],[[C,D]|L]):-
    A #\= C, % inutile par construction du prédicat constraint
    B #\= D, % inutile avec le all_different
    abs(A-C) #\= abs(B-D),
    constraint_one([A,B],L).

```

4. Question 4 : Comparaison des résultats :

Algorithme	4	6	8	10	12	14
nombre de solutions	2	4	92	724	14 200	365 596
Générer & Tester	0s	0,9s	315s	X	X	X
Backtracking	0s	0s	0,2s	2s	69s	2684s
Contrainte	0s	0s	0s	0,5s	7s	218s

Exercice 3

[Retour à l'énoncé.](#)

1. **Question 1 : Prédicat amazones Générer & Tester** : on définit le prédicat comme pour les reines, on ne fait qu'ajouter les prédicats liés aux déplacements de type cavalier.

```
amazonesGT(N,R):-
    construit_liste(N,L_Nb),
    placerGT(N,L_Nb,[],R),
    test_nen_priseGT(R).

placerGT(0,_,R,R):-!.
placerGT(Nb,L_Nb,L,R):-
    select(Y,L_Nb,L_Nb1),
    Nb1 is Nb-1,
    placerGT(Nb1,L_Nb1,[[Nb,Y]|L],R).

test_nen_priseGT([]).
test_nen_priseGT([C1|R]):-
    non_en_priseRGT(R,C1),
    non_en_priseCGT(R,C1),
    test_nen_priseGT(R).

% Regarde si une case (2eme argument) est en prise avec un des
% éléments de la liste (1er argument).
non_en_priseRGT([],_).
non_en_priseRGT([[X1,Y1]|R],[X,Y]):-
    X =\= X1, % Colonne
    Y =\= Y1, % Ligne
    abs(X-X1) =\= abs(Y-Y1), % Diagonale
    non_en_priseRGT(R,[X,Y]).

non_en_priseCGT([],_).
non_en_priseCGT([[X1,Y1]|L],[X,Y]):-
    3 =\= abs(Y-Y1)+abs(X-X1),
    non_en_priseCGT(L,[X,Y]).
```

2. **Question 2 : Prédicat amazones Backtracking** : on définit le prédicat comme pour les reines, on ne fait qu'ajouter l'appel au prédicat des déplacements de type cavalier défini précédemment pour le Générer & Tester.

```
amazonesBT(N,R):-
    construit_liste(N,L_Nb),
    placerBT(N,L_Nb,[],R).

placerBT(0,_,R,R):-!.
placerBT(Nb,L_Nb,L,R):-
    select(Y,L_Nb,L_Nb1),
    non_en_priseRGT(L,[Nb,Y]),
    non_en_priseCGT(L,[Nb,Y]),
    Nb1 is Nb-1,
    placerBT(Nb1,L_Nb1,[[Nb,Y]|L],R).
```

3. **Question 3 : Prédicat amazones contraintes** : on définit le prédicat comme pour les reines, on ne fait qu'ajouter la définition des contraintes liées aux déplacements de type cavalier.

```
:-use_module(library(clpfd)).

amazonesPLC(N,Type,Sol):-
    length(LV,N),
    LV ins 1..N,
    all_different(LV),
    contraintes_amazones(N,LV,Los),
```

```

labeling(Type,LV),
%% on peut faire un reverse pour avoir la liste comme les autres
reverse(Los,Sol).

contraintes_amazones(0,_,[]):-!.
contraintes_amazones(I,[Y|LV],[[I,Y]|R):-
    I1 is I-1,
    contraintes_amazones(I1,LV,R),
    contraintes_reine(R,[I,Y]),
    contraintes_cavalier(R,[I,Y]).

contraintes_reine([],_):-!.
contraintes_reine([[X1,Y1]|L],[X,Y):-
    abs(X1-X) #\= abs(Y1-Y),% inutile de vérifier X1\=X et Y1\=Y
    contraintes_reine(L,[X,Y]).

contraintes_cavalier([],_).
contraintes_cavalier([[X1,Y1]|R],[X,Y):-
    3 #\= abs(Y-Y1)+abs(X-X1),
    contraintes_cavalier(R,[X,Y]).

```

4. Question 4 : Comparaison des résultats :

Algorithme	10	12	14	16
nombre de solutions	4	156	5180	202 900
Backtracking	0,2s	2,6s	105s	5 702s
Contrainte	0s	0,6s	12s	486s

Exercice 4

[Retour à l'énoncé.](#)

1. Question 1 : Méthode du backtracking

```

:- use_module(library(lists)).

% calcule le carre magique de dimension N*N et renvoi le nombre magique dans R
% et le carré dans S, sous forme de matrice (liste de listes de longueur N).
carreMagique(N,R,S):-
    N2 is N*N,
    construit_liste(N2,LS),
    placerBT(N,R,LS,S).

%Construit une liste de N entiers
construit_liste(1,[1]):-!.
construit_liste(N,[N|L_Nb]):-
    N1 is N-1,
    construit_liste(N1,L_Nb).

% Prédicat placerBT(-N,+Ref,-Ls,+S)
% Place N*N nombres en vérifiant que la somme des lignes est égale à Ref ainsi
% que la somme des colonnes et des diagonales principales et renvoi le résultat dans S.
placerBT(N,Ref,LS,S):-
    placer_les_lignes(N,N,LS,[],S,Ref),
    tester_colonnes(N,S,Ref),
    test_diagonaleM(S,1,0,Ref),

```

```

test_diagonaleD(S,N,0,Ref).

% Prédicat placer_les_lignes(-N,-Ls,+Acc,+S,+Ref)
% Place N lignes en prenant les nombres dans Ls et renvoi le résultat dans S
% La partie de Backtracking qui cherche à placer les nombres tel que la somme
% des lignes correspond au même nombre Ref (le nombre magique).
placer_les_lignes(0,_,_,S,S,_Ref):-!.
placer_les_lignes(N,Nb,Ls,Acc,S,Ref):-
    placer_ligne(Nb,Ls,Lred,[],Lnew,Ref),
    N1 is N-1,
    placer_les_lignes(N1,Nb,Lred,[Lnew|Acc],S,Ref).
% placer_ligne(-Nb,-Ls,+Lred,-L,+Lnew,?Ref)
% On a Nb nombre à placer dans la ligne. On doit les prendre dans Ls et
% on retourne la liste Lred des nombres non encore utilisés.
% Si la somme est correct on construit une liste dans Lnew.
% Ainsi Place une ligne de nombre en vérifiant si leur somme est égale à Ref.
placer_ligne(0,LS,LS,L,L,R):-
    test_ligne(L,0,R),!.
placer_ligne(Nb,LS,Lred,L,Lnew,R):-
    select(Y,LS,LSred),
    Nb1 is Nb-1,
    placer_ligne(Nb1,LSred,Lred,[Y|L],Lnew,R).

% Prédicat tester_colonnes(N,M,Ref)
% Vérifie si pour les N colonnes de M la somme des éléments de chacun d'elle est égale à Ref.
tester_colonnes(0,_M,_Ref):-!.
tester_colonnes(I,M,Ref):-
    test_colonne(M,I,0,Ref),
    I1 is I-1,
    tester_colonnes(I1,M,Ref).

% Prédicat test_ligne(L,S,Ref)
% Vérifie si pour la ligne L la somme S de ses éléments est égale à Ref.
test_ligne([],S,S).
test_ligne([X|L],Som,Ref):-
    Som1 is Som+X,
    test_ligne(L,Som1,Ref).

% Prédicat test_ligne(M,I,S,Ref)
% Vérifie si pour la colonne I de M la somme S de ses éléments est égale à Ref.
test_colonne([],_,S,S).
test_colonne([L|Lliste],I,Som,Ref):-
    nth1(I,L,X),
    Som1 is Som+X,
    test_colonne(Lliste,I,Som1,Ref).

% Prédicat test_diagonaleD(M,I,S,Ref)
% Vérifie si pour la diagonale négative de M la somme S de ses élément est égale à Ref.
test_diagonaleD([],_,S,S).
test_diagonaleD([L|Lliste],I,Som,Ref):-
    nth1(I,L,X),
    Som1 is Som+X,
    I1 is I-1,
    test_diagonaleD(Lliste,I1,Som1,Ref).

% Prédicat test_diagonaleM(M,I,S,Ref)
% Vérifie si pour la diagonale positive de M la somme S de ses éléments est égale à Ref.
test_diagonaleM([],_,S,S).
test_diagonaleM([L|Lliste],I,Som,Ref):-

```



```

nth1(I,L,X),
Som1 is Som+X,
I1 is I+1,
test_diagonaleM(Lliste,I1,Som1,Ref).

```

2. Question 2 : Méthode PLC

```

:- use_module(library(clpfd)).
:- use_module(library(lists)).

% calcul le carre magique de dimension N*N et renvoi le nombre magique dans R et
% le carré dans S, sous forme de matrice (liste de listes de longueur N).
magicSquare(N,R,S):-
    N2 is N*N,
    creer_Matrice(N,N,N2,L),
    contraintes_LC(N,L,L,R),
    contraintes_diagonales(N,L,R),
    recupere_Y(L,S),
    all_different(S), % all_different est un prédicat de la biblio clpfd qui dit ttes les
    labeling([],S). % variables de la liste doivent avoir une valeur différente.

% Le prédicat creer_Matrice(I,N,N2,M)
% Créé une matrice M (liste de listes) de dimension N*N,
% soit N listes de longueur N, de variables ayant comme domaine l'intervalle [1,N2].
creer_Matrice(0,_,_,[]):-!.
creer_Matrice(I,N,N2,[L|M]):-
    creer_Ligne(N,N2,L),
    I1 is I-1,
    creer_Matrice(I1,N,N2,M).

% Le prédicat creer_Ligne(-I,-N,+L)
% Créé une liste L de I variables dont le domaine est compris entre 1 et N
creer_Ligne(0,_,[]):-!.
creer_Ligne(I,N2,[Y|L]):-
    Y in 1..N2,
    I1 is I-1,
    creer_Ligne(I1,N2,L).

% Pose les contraintes sur les lignes et les colonnes
contraintes_LC(0,_,_,_):-!.
contraintes_LC(I,[L|Lliste],M,R):-
    contraintes_ligne(L,0,R),
    contraintes_colonne(M,I,0,R),
    I1 is I-1,
    contraintes_LC(I1,Lliste,M,R).

% Pose les contraintes sur la somme des lignes
contraintes_ligne([],S,S).
contraintes_ligne([X|L],Som,Ref):-
    Som1 #= Som+X,
    contraintes_ligne(L,Som1,Ref).

% Pose les contraintes sur la somme des colonnes
contraintes_colonne([],_,S,S).
contraintes_colonne([L|Lliste],I,Som,Ref):-
    nth1(I,L,X), % nth1 prédicat de la biblio lists qui renvoie X, le Ieme élément de L
    Som1 #= Som+X,
    contraintes_colonne(Lliste,I,Som1,Ref).

```

```

% Pose les contraintes concernant les diagonales.
contraintes_diagonales(N,S,R):-
    contraintes_diagonaleD(S,N,0,R),
    contraintes_diagonaleM(S,1,0,R).

% le prédicat contraintes_diagonaleD(-L,-I,-Som,-Ref) pose les contraintes
% qui vérifient que la somme des éléments de la diagonale négative est égale à Ref.
% I est égal à 1 et Som à 0 au premier appel.
% Le cas terminal, une fois que l'on a sommé tous les éléments de la diagonale,
% vérifie que cette somme est égale à la somme déjà calculée.
contraintes_diagonaleD([],_,S,S).
contraintes_diagonaleD([L|Lliste],I,Som,Ref):-
    nth1(I,L,X),
    Som1 #= Som+X,
    I1 is I-1,
    contraintes_diagonaleD(Lliste,I1,Som1,Ref).

% le prédicat contraintes_diagonaleM(-L,-I,-Som,-Ref) pose les contraintes
% qui vérifient que la somme des éléments de la diagonale positive est égale à Ref.
% I est égal à 1 et Som à 0 au premier appel.
% Le cas terminal, une fois que l'on a sommé tous les éléments de la diagonale,
% vérifie que cette somme est égale à la somme déjà calculée.
contraintes_diagonaleM([],_,S,S).
contraintes_diagonaleM([L|Lliste],I,Som,Ref):-
    nth1(I,L,X),
    Som1 #= Som+X,
    I1 is I+1,
    contraintes_diagonaleM(Lliste,I1,Som1,Ref).

% Le prédicat recupere_y(-M,+L), met à plat la matrice (liste de listes) M en une liste L.
% On récupère la liste L des variables pour pouvoir faire le labeling
recupere_Y([],[]) :- !.
recupere_Y([L|R],M):-
    recupere_Y(R,L1),
    append(L,L1,M). % le append est défini dans la librairie(list)

```

Exercice 5

[Retour à l'énoncé.](#)

1. Question 1 :

```

% On code le prédicat solution
solution([A,B,C,D,E,F,G,H,I]):-
    % On effectue le traitement ligne par ligne
    % ligne entre A et B
    select(A,[2,3,4,5,6,7,8,9,10,11,12],LA),
    select(B,LA,LB),
    20 is A + B,
    % Ligne entre A, D et G
    select(D,LB,LD),
    select(G,LD,LG),
    20 is A+D+G,
    % Ligne entre B, E et H
    select(E,LG,LE),
    select(H,LE,LH),

```

```

20 is B+E+H,
% Lignes entre G, H et I et A, E et I
select(I,LH,LI),
20 is G+H+I,
20 is A+E+I,
% Dernière ligne entre C, D, E et F
select(C,LI,LC),
select(F,LC,_),
20 is C+D+E+F.

```

2. Question 2 :

```

% L'utilisation de la plc se fait au travers de la librairie CLP(FD)
:-use_module(library(clpfd)).

```

```

% Définition du prédicat solution_clp
solution_clp([A,B,C,D,E,F,G,H,I]):-
    % Définition du domaine pour chaque noeud
    domains([A,B,C,D,E,F,G,H,I],2,12),
    % Chaque noeud possède une valeur différente
    all_different([A,B,C,D,E,F,G,H,I]),
    % Définition de chacune des lignes
    A+B #= 20,
    A+D+G #= 20,
    B+E+H #= 20,
    G+H+I #= 20,
    A+E+I #= 20,
    C+D+E+F #= 20,
    % Recherche des solutions
    labeling([], [A,B,C,D,E,F,G,H,I]).

domains(L,Min,Max):-
    L ins Min..Max.

```

3. Question 3 : par rapport au code précédent :

- pour le backtracking, il suffit de partir avec la liste [2,3,4,5,7,8,9,10,12]
- pour la plc, on peut changer le prédicat domains :

```

domains(L,T):-
    L ins T.

```

```

avec comme appel:domains([A,B,C,D,E,F,G,H,I],(2..5) \ (7..10) \ {12}),

```

- On a deux solutions :

- (a) L = [8,12,4,2,5,9,10,3,7]
- (b) L = [8,12,9,2,5,4,10,3,7]

Exercice 6

[Retour à l'énoncé.](#)

1. Question 1 :

```

% Le plus simple est de prendre les seuls mois proposés et de faire la liste

```

```
unMoisApres(octobre,septembre).
unMoisApres(janvier,decembre).
```

2. Question 2 :

```
% Idem question 1
deuxMoisApres(decembre,octobre).
deuxMoisApres(mars,janvier).
```

3. Question 3 :

```
% Cas terminal, on a trouvé l'élément dans la liste
membre(X,[X|_L]):-!.
% Sinon on cherche dans le reste de la liste
membre(X,[_|L]):-
    membre(X,L).
```

4. Question 4 :

```
% Cas terminal, on retire le premier élément de la liste
membreRetire(X,[X|L],L).
% Sinon on prend un autre élément de la liste
membreRetire(X,[Y|L],[Y|L1]):-
    membreRetire(X,L,L1).
```

5. Question 5 :

```
% Cas terminal, on a associé tous les éléments de chaque liste
element_membre([],_).
% Sinon on prend un élément de la première liste et on l'associe
% avec un élément de la seconde via le prédicat membreRetire
% puis on continue avec les autres éléments
element_membre([X|Y],L2):-
    membreRetire(X,L2,L),
    element_membre(Y,L).
```

6. Question 6 :

```
solution(S):-
% Phrase 1
    S=[ami(arnaud,M1,S1,V1), ami(christian,M2,S2,V2), ami(hubert,M3,S3,V3),
    ami(julien,M4,S4,V4), ami(stephane,M5,S5,V5) ],
% Phrase 2
    element_membre([M1,M2,M3,M4,M5],[septembre,octobre,decembre,janvier,mars]),
% Phrase 3
    element_membre([V1,V2,V3,V4,V5],[bordeaux,lens,nantes,saint_Etienne,lyon]),
% Phrase 4
    element_membre([S1,S2,S3,S4,S5],[sc(1,0),sc(1,1),sc(2,1),sc(2,3),sc(3,2)]),
% Phrase 5
    V4 = lyon,
% Phrase 6
```

```

    V1 \= lens, V1 \=nantes,
% Phrase 7
    V1 \= bordeaux, V3 \= bordeaux,
% Phrase 8
    S5 = sc(ScD,ScE), ScD < ScE,
% Phrase 9
    membre(ami(_,_,sc(ScD1,ScE1),lyon),S), ScD1 > ScE1,
% Phrase 10
    membre(ami(_,_,sc(ScD2,ScE2),nantes),S), ScD2 > ScE2,
% Phrase 11
    membre(ami(_,_,sc(2,1),bordeaux),S),
% Phrase 12
    unMoisApres(M1,M4),
% Phrase 13
    membre(ami(_,M6,_,bordeaux),S), membre(ami(_,M7,sc(1,0),_),S), deuxMoisApres(M6,M7),
% Phrase 14
    membre(ami(_,M8,sc(2,3),_),S), membre(ami(_,M9,_,nantes),S), unMoisApres(M9,M8).

```

7. Question 7:

```

solution(S):-
% Phrase 1
    S=[ami(arnaud,M1,S1,V1), ami(christian,M2,S2,V2), ami(hubert,M3,S3,V3),
    ami(julien,M4,S4,V4), ami(stephane,M5,S5,V5) ],
% Phrase 3
    element_membre([V1,V2,V3,V4,V5],[bordeaux,lens,nantes,saint_Etienne,lyon]),
% Phrase 5
    V4 = lyon,
% Phrase 6
    V1 \= lens, V1 \=nantes,
% Phrase 7
    V1 \= bordeaux, V3 \= bordeaux,
% Phrase 4
    element_membre([S1,S2,S3,S4,S5],[sc(1,0),sc(1,1),sc(2,1),sc(2,3),sc(3,2)]),
% Phrase 8
    S5 = sc(ScD,ScE), ScD < ScE,
% Phrase 9
    membre(ami(_,_,sc(ScD1,ScE1),lyon),S), ScD1 > ScE1,
% Phrase 10
    membre(ami(_,_,sc(ScD2,ScE2),nantes),S), ScD2 > ScE2,
% Phrase 11
    membre(ami(_,_,sc(2,1),bordeaux),S),
% Phrase 2
    element_membre([M1,M2,M3,M4,M5],[septembre,octobre,decembre,janvier,mars]),
% Phrase 12
    unMoisApres(M1,M4),
% Phrase 13
    membre(ami(_,M6,_,bordeaux),S), membre(ami(_,M7,sc(1,0),_),S), deuxMoisApres(M6,M7),
% Phrase 14
    membre(ami(_,M8,sc(2,3),_),S), membre(ami(_,M9,_,nantes),S), unMoisApres(M9,M8).

```

8. Question 8:

```

:- use_module(library(clpfd)).

domains(L,T):-
    L ins T.

solution([[Arnaud, Christian, Hubert, Julien, Stephane], [Bordeaux, Lens, Nantes,
    Saint_Etienne, Lyon],[S1, S2, S3, S4, S5]]):-
% Phrase 1, on va prendre comme domaine de chaque variable

```

```

% les valeurs des mois (cf. phrase 2)
domains([Arnaud,Christian,Hubert,Julien,Stephane], (9..10) \ / (12..13) \ / {15}),
all_different([Arnaud,Christian,Hubert,Julien,Stephane]),
% Phrase 2
Septembre is 9, Octobre is 10, Decembre is 12, Janvier is 13, Mars is 15,
% Phrase 3 Idem pour le domaine
domains([Bordeaux,Lens,Nantes,Saint_Etienne,Lyon], (9..10) \ / (12..13) \ / {15}),
all_different([Bordeaux,Lens,Nantes,Saint_Etienne,Lyon]),
% Phrase 4, idem pour le domaine de plus on définit les scores
% S1 = (1,0), S2 = (1,1), S3=(2,1), S4=(2,3), S5=(3,2)
domains([S1,S2,S3,S4,S5], (9..10) \ / (12..13) \ / {15}),
all_different([S1,S2,S3,S4,S5]),
% Phrase 5
Julien #= Lyon,
% Phrase 6
Arnaud #\= Lens, Arnaud #\= Nantes,
% Phrase 7
Arnaud #\= Bordeaux, Hubert #\= Bordeaux,
% Phrase 8
Stephane #= S4,
% Phrase 9
(Lyon #= S1 #\ / Lyon #=S3 #\ / Lyon #=S5),
% Phrase 10
(Nantes #= S1 #\ / Nantes #=S3 #\ / Nantes #=S5),
% Phrase 11
Bordeaux #= S3,
% Phrase 12
Arnaud #= Julien + 1,
% Phrase 13
Bordeaux #= S1 + 2,
% Phrase 14
Nantes #= S4 + 1,
% Enumeration des solutions
labeling([], [Arnaud,Christian,Hubert,Julien,Stephane,Bordeaux,Lens,
Nantes,Saint_Etienne,Lyon,S1,S2,S3,S4,S5]).

```

9. Question 9:

```

S = [ami(arnaud,octobre,sc(1,1),saint_Etienne),
ami(christian,mars,sc(2,1),bordeaux),
ami(hubert,janvier,sc(1,0),nantes),
ami(julien,septembre,sc(3,2),lyon),
ami(stephane,decembre,sc(2,3),lens)]

```

11.5 Exercices chapitre 5

Exercice 1

[Retour à l'énoncé.](#)

1. Question 1 : On pose comme proposition :

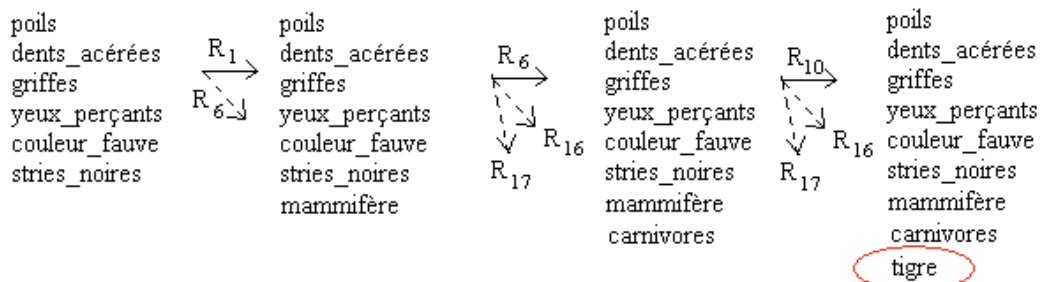
- poils : "animal à poils"
- lait : "animal donne du lait"
- plumes : "animal à des plumes"
- vole : "animal vole"
- pond : "pond des œufs"
- viande : "animal mange de la viande"
- dents_acérées : "animal a des dents acérées"
- griffes : "animal a des griffes"
- yeux_perçants : "animal a des yeux perçants"
- mammifère : "animal est un mammifère"
- oiseau : "animal est un oiseau"
- carnivore : "animal est un carnivore"
- ongulé : "animal est un ongulé"
- rumine : "animal rumine"
- couleur_fauve : "animal est de couleur fauve"
- tâches_sombres : "animal a des tâches sombres"
- guépard : "animal est un guépard"
- stries_noirs : "animal a des stries noirs"
- tigre : "animal est un tigre"
- ongulé : "animal est un ongulé"
- long_cou : "animal a un long cou"
- longues_jambes : "animal a de longues jambes"
- girafe : "animal est une girafe"
- zèbre : "animal est un zèbre"
- blanc_noir : "animal est blanc et noir"
- autruche : "animal est une autruche"
- nage : "animal nage"
- albatros : "animal est un albatros"

Soient les règles traduites en logiques propositionnelles :

- R_1 : poils $\rightarrow$ mammifère
- R_2 : lait $\rightarrow$ mammifère
- R_3 : plumes $\rightarrow$ oiseau
- R_4 : vole $\wedge$ pond $\rightarrow$ oiseau
- R_5 : viande $\rightarrow$ carnivore
- R_6 : dents_acérées $\wedge$ griffes $\wedge$ yeux_perçants $\rightarrow$ carnivore
- R_7 : mammifère $\wedge$ sabots $\rightarrow$ ongulé

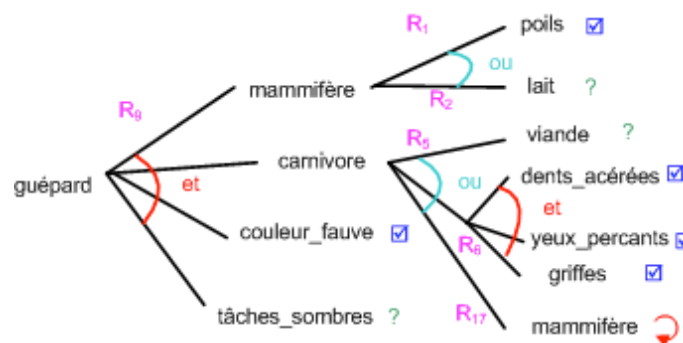
- R_8 : mammifère $\wedge$ rumine $\longrightarrow$ ongulé
- R_9 : mammifère $\wedge$ carnivore $\wedge$ couleur_fauve $\wedge$ taches_sombres $\longrightarrow$ guépard
- R_{10} : mammifère $\wedge$ carnivore $\wedge$ couleur_fauve $\wedge$ stries_noires $\longrightarrow$ tigre
- R_{11} : ongulé $\wedge$ long_cou $\wedge$ longues_jambes $\wedge$ taches_sombres $\longrightarrow$ girafe
- R_{12} : ongulé $\wedge$ stries_noires $\longrightarrow$ zèbre
- R_{13} : oiseau $\wedge$ $\neg$ vole $\wedge$ long_cou $\wedge$ longues_jambes $\wedge$ blanc_noir $\longrightarrow$ autruche
- R_{14} : oiseau $\wedge$ $\neg$ vole $\wedge$ nage $\wedge$ blanc_noir $\longrightarrow$ manchot
- R_{15} : oiseau $\wedge$ vole $\longrightarrow$ albatros
- R_{16} : $\neg$ mammifère $\vee$ $\neg$ oiseau
- R_{17} : mammifère $\longrightarrow$ carnivore ou ongulé

2. Question 2 :



3. Question 3 :

Sur la figure ci-dessous, on utilise un ? pour indiquer qu'il y a une question posée, un check-box pour les faits connus (question 2) et la boucle (pour mammifère qui a déjà été traité) pour indiquer la détection d'un cycle dans le système (on repasse sur un fait déjà présent).



Exercice 2

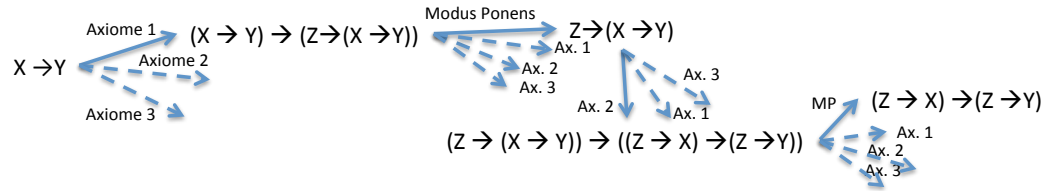
[Retour à l'énoncé.](#)

— Question 1 :

1. Pour le fait $(X \rightarrow Y)$ et le but $(Z \rightarrow X) \rightarrow (Z \rightarrow Y)$
 - (a) $(X \rightarrow Y)$, fait
 - (b) $(X \rightarrow Y) \rightarrow (Z \rightarrow (X \rightarrow Y))$, Axiome 1

- (c) $(Z \rightarrow (X \rightarrow Y))$, Modus Ponens (a) et (b)
 (d) $(Z \rightarrow (X \rightarrow Y)) \rightarrow ((Z \rightarrow X) \rightarrow (Z \rightarrow Y))$, Axiome 2
 (e) $(Z \rightarrow X) \rightarrow (Z \rightarrow Y)$, Modus Ponens (c) et (d)

On aurait pu aussi utiliser une approche visuelle :



2. Pour les faits $\{X \rightarrow Y, Z \rightarrow X, Z\}$ et le but $(Z \rightarrow Y)$
- $(X \rightarrow Y)$, fait
 - $(X \rightarrow Y) \rightarrow (Z \rightarrow (X \rightarrow Y))$, Axiome 1
 - $(Z \rightarrow (X \rightarrow Y))$, Modus Ponens (a) et (b)
 - $(Z \rightarrow (X \rightarrow Y)) \rightarrow ((Z \rightarrow X) \rightarrow (Z \rightarrow Y))$, Axiome 2
 - $(Z \rightarrow X) \rightarrow (Z \rightarrow Y)$, Modus Ponens (c) et (d)
3. Pour les faits $\{X \rightarrow Y, Y \rightarrow Z\}$ et le but $(X \rightarrow Z)$
- $(Y \rightarrow Z)$, fait
 - $(Y \rightarrow Z) \rightarrow (X \rightarrow (Y \rightarrow Z))$, Axiome 1
 - $(X \rightarrow (Y \rightarrow Z))$, Modus Ponens (a) et (b)
 - $(X \rightarrow (Y \rightarrow Z)) \rightarrow ((X \rightarrow Y) \rightarrow (X \rightarrow Z))$, Axiome 2
 - $(X \rightarrow Y) \rightarrow (X \rightarrow Z)$, Modus Ponens (c) et (d)
 - $(X \rightarrow Y)$, fait
 - $(X \rightarrow Z)$, Modus Ponens (e) et (f)

11.6 Exercices chapitre 6

Exercice 1

[Retour à l'énoncé.](#)

1. Question 1 :

On utilise max et min pour dire si l'on veut maximiser ou minimiser le coût, suivant si l'on calcule le meilleur coup pour soi ou pour l'adversaire.

```
%Calcul tous les coups possibles à partir d'une position donnée pour le joueur A (max),
%il crée la liste des fils en position une valeur 1 à la place d'un 0 pour chaque fils
successeurs(max(Position),Children) :-
    setof([A,B,C,D,E,F,G,H,I],succ(max(Position),[A,B,C,D,E,F,G,H,I]),Children).
```

```
%Calcul tous les coups possibles à partir d'une position donnée pour le joueur B (min),
%il crée la liste des fils en position une valeur 2 à la place d'un 0 pour chaque fils
successeurs(min(Position),Children) :-
    setof([A,B,C,D,E,F,G,H,I],succ(min(Position),[A,B,C,D,E,F,G,H,I]),Children).
```

```
%Transforme une case vide (0) en case jouée par le joueur A (1). Ce prédicat
%effectue tous les cas possibles (il y aura autant de points de choix que
%de 0 restant dans la liste).
succ(max([0|X]),[1|X]).
succ(max([A|X]),[A|Y]) :-
    succ(max(X),Y).
```

```
%Transforme une case vide (0) en case jouée par le joueur B(2). Ce prédicat
%effectue tous les cas possibles (il y aura autant de points de choix que
%de 0 restant dans la liste).
succ(min([0|X]),[2|X]).
succ(min([A|X]),[A|Y]) :-
    succ(min(X),Y).
```

2. Question 2 :

```
% Ce prédicat permet de savoir si la configuration correspond à un coup qui peut être gagnant.
gain_possible_max(A,B,C) :-
    A\=2,B\=2,C\=2. % A peut gagner avec cette ligne si B n'a pas joué de coup dessus
gain_possible_min(A,B,C) :-
    A\=1,B\=1,C\=1. % B peut gagner avec cette ligne si A n'a pas joué de coup dessus
```

```
%Si les deux joueurs peuvent finir avec la ligne (aucun des deux n'y a joué)
gain_possible(A,B,C,0) :-
    gain_possible_max(A,B,C),
    gain_possible_min(A,B,C),!.
```

```
% Si A peut finir la ligne, on compte 1
gain_possible(A,B,C,1) :-
    gain_possible_max(A,B,C),!.
```

```
% Si B peut finir la ligne, on compte -1
gain_possible(A,B,C,-1) :-
    gain_possible_min(A,B,C),!.
```

```
% Si aucun des cas précédents n'est possible, la ligne a été jouée par les deux
gain_possible(_,_,_,0).
```

3. Question 3 :

```

% le prédicat p donne les combinaisons des lignes gagnantes
p([X,X,X,_,_,_,_,_],X).
p([_,_,_,X,X,X,_,_,_],X).
p([_,_,_,_,_,_,X,X,X],X).
p([X,_,_,X,_,_,X,_,_],X).
p([_,X,_,_,X,_,_,X,_,_],X).
p([_,_,X,_,_,X,_,_,X],X).
p([X,_,_,_,X,_,_,_,X],X).
p([_,_,X,_,_,X,_,_,_],X).

%Calcul de l'heuristique
h([A1,B1,C1,A2,B2,C2,A3,B3,C3],999) :-
    p([A1,B1,C1,A2,B2,C2,A3,B3,C3],1). % On a une ligne complète pour A
h([A1,B1,C1,A2,B2,C2,A3,B3,C3],-999) :-
    p([A1,B1,C1,A2,B2,C2,A3,B3,C3],2). % On a une ligne complète pour B

% Cas général de l'heuristique, on va évaluer le coût de chaque ligne et
%sommet le résultat pour avoir la valeur renvoyé par l'heuristique
h([A1,B1,C1,A2,B2,C2,A3,B3,C3],Valeur) :-
    gain_possible(A1,B1,C1,A),
    gain_possible(A2,B2,C2,B),
    gain_possible(A3,B3,C3,C),
    gain_possible(A1,A2,A3,D),
    gain_possible(B1,B2,B3,E),
    gain_possible(C1,C2,C3,F),
    gain_possible(A1,B2,C3,G),
    gain_possible(A3,B2,C1,H),
    sum([A,B,C,D,E,F,G,H],Valeur).

% Prédicat permettant d'effectuer la somme des éléments d'une liste
sum([],0).
sum([X|Y],R):-
    sum(Y,R1),
    R is X + R1.

```

4. Question 4:

```

%Le joueur A a gagné si on a une position 999 (cf. heuristique)
gagnant(Position,999):-
    h(Position,999),!.

```

```

%Le joueur B a gagné si on a une position -999 (cf. heuristique)
gagnant(Position,-999):-
    h(Position,-999).

```

5. Question 5:

```

%Permet de choisir le fils ayant le meilleur résultat (heuristique) et
% renvoie le coût Alpha et le chemin correspondant.
% max_children(-Liste,-Alpha,-Beta,+Val,+Path,-Profondeur)
% Liste : La liste des fils d'un noeud
% Alpha : le coefficient Alpha courant
% Beta : le coefficient Beta courant
% Val : la meilleure valeur obtenue
% Path : Le meilleur chemin développé pour la profondeur considérée
% Profondeur : La profondeur de l'arbre de recherche.
max_children([], Alpha, _, Alpha, [], _):-!.
max_children([Child|Children], Alpha, Beta, Val, Path, Profondeur2):-
    alphaBeta(min(Child), Alpha, Beta, Val1, Path0, Profondeur2),
    Alpha1 is max(Alpha, Val1),
    (

```

```

        (Alpha1<Beta) ->
            max_children(Children, Alpha1, Beta, Val2, Path2, Profondeur2),
            compare(max, Alpha1, Val2, Path0, Path2, Val, Path)
    ;
    Val is Beta,
    Path = Path0
).

min_children([], _, Beta, Beta, [], _):-!.
min_children([Child|Children], Alpha, Beta, Val, Path, Profondeur2):-
    alphaBeta(max(Child), Alpha, Beta, Val1, Path0, Profondeur2),
    Beta1 is min(Beta, Val1),
    (
        (Alpha<Beta1) ->
            min_children(Children, Alpha, Beta1, Val2, Path2, Profondeur2),
            compare(min, Beta1, Val2, Path0, Path2, Val, Path)
        ;
        Val is Alpha,
        Path = Path0
    ).

compare(max,V1,V2,P1,_,V1,P1):-
    V1 >= V2,!.
compare(min,V1,V2,P1,_,V1,P1):-
    V1 <= V2,!.
compare(_,_,V2,_,P2,V2,P2).

```

6. Question 6 :

```

% alphaBeta(-f(Position),-Alpha, -Beta, +Val, +Path, -Profondeur)
% max_children(-Liste,-Alpha,-Beta,+Val,+Path,-Profondeur)
% f(Position) : f indiquant le joueur (max ou min) Position est le
% plateau de jeu courant
% Alpha : le coefficient Alpha courant
% Beta : le coefficient Beta courant
% Val : la meilleur valeur obtenue
% Path : Le meilleur chemin développé pour la profondeur considérée
% Profondeur : La profondeur de l'arbre de recherche.
% Pour le dernier coup, on calcule la valeur avec l'heuristique
alphaBeta(max(Position), _, _, Val, [Position], 0):-
    h(Position, Val),!.
alphaBeta(min(Position), _, _, Val, [Position], 0):-
    h(Position, Val),!.

%On regarde si on a un coup gagnant (une ligne a été réalisé)
alphaBeta(max(Position), _, _, Val, [Position], _):-
    gagnant(Position, Val),!.
alphaBeta(min(Position), _, _, Val, [Position], _):-
    gagnant(Position, Val),!.

% Cas d'arret de l'algorithme AlphaBeta où Alpha >= Beta
alphaBeta(max(Position),Alpha, Beta, Val, [Position], _):-
    Alpha >= Beta,
    Val is Beta,!.
alphaBeta(min(Position), Alpha, Beta, Val, [Position], _):-
    Alpha >= Beta,
    Val is Alpha,!.

% Cas général, on calcule pour une profondeur tous les fils puis on
% cherche le meilleur avec l'algorithme AlphaBeta.

```

```
alphaBeta(max(Position), Alpha, Beta, Val, [Position|Path], Profondeur):-
    successeurs(max(Position), Children),
    Profondeur2 is Profondeur-1,
    max_children(Children, Alpha, Beta, Val, Path, Profondeur2).

alphaBeta(min(Position), Alpha, Beta, Val, [Position|Path], Profondeur):-
    successeurs(min(Position), Children),
    Profondeur2 is Profondeur-1,
    min_children(Children, Alpha, Beta, Val, Path, Profondeur2).
```

7. Divers :

%On peut pour simplifier l'appel créer un prédicat prenant les valeurs
%par défaut de Alpha et Beta et limitant la profondeur au nombre de
%coup possible restant :

```
ab(max(Position), S, Profondeur):-
    compte0(Position,L),
    P is min(L,Profondeur),
    alphaBeta(max(Position), -999, 999, _, S, P).

ab(min(Position), S, Profondeur):-
    compte0(Position,L),
    P is min(L,Profondeur),
    alphaBeta(min(Position), -999, 999, _, S, P).

compte0([],0).
compte0([X|L],N):-
    compte0(L,N1),
    ((X=0) -> ValX is 1; ValX is 0),
    N is N1 + ValX.
```

11.7 Exercices chapitre 7

Exercice 1

[Retour à l'énoncé.](#)

1. Question 1 - premier arrivée, premier servi :

Ordre	Date actuelle	Temps traitement	Date promise	Retard
T1	0	11	61	0
T2	11	29	45	0
T3	40	31	31	40
T4	71	1	33	39
T5	72	2	32	42
	74			

Nombre de retard : 3

Retard moyen = $121/5 = 24,2$

2. Question 2 - Temps de traitement le plus court :

Ordre	Date actuelle	Temps traitement	Date promise	Retard
T4	0	1	33	0
T5	1	2	32	0
T1	3	11	61	0
T2	14	29	45	0
T3	43	31	31	43
	74			

Nombre de retard : 1

Retard moyen = $43/5 = 8,6$

3. Question 3 - Date promise la plus tôt :

Ordre	Date actuelle	Temps traitement	Date promise	Retard
T3	0	31	31	0
T5	31	2	32	1
T4	33	1	33	1
T2	34	29	45	18
T1	63	11	61	13
	74			

Nombre de retard : 4

Retard moyen = $33/5 = 6,6$

4. Question 4 - Ratio critique :

Date Actuelle \ Tâches	T1	T2	T3	T4	T5	
0	0,18	0,64	1	0,03	0,06	On choisit T_3
31	0,36	2,07	–	0,5	2	On choisit T_2
60	11	–	–	-0,03	-0,07	On choisit T_1
71	–	–	–	-0,026	-0,051	On choisit T_4
73	–	–	–	–	-0,05	On choisit T_5

Ordre	Date actuelle	Temps traitement	Date promise	Retard
T3	0	31	31	0
T2	31	29	45	15
T1	60	11	61	10
T4	71	1	33	38
T5	72	2	32	42
	74			

Nombre de retard : 4

Retard moyen = $105/5 = 21$

Exercice 2

[Retour à l'énoncé.](#)

- Question 1 - Règle de Johnson : pour chaque machine on donne le temps et entre parenthèses le temps théorique de fin.

Ordre	Durée (A : B)	Machine A	Machine B
T2	1 : 6	0 (1)	1 (7)
T4	3 : 8	1 (4)	7 (15)
T3	9 : 7	4 (13)	15 (22)
T5	10 : 4	13 (23)	23 (27)
T1	5 : 2	23 (28)	28 (30)

- Question 2 - Programme prolog (Solution PLC)

```
:-use_module(library(clpfd)).

jobShopJohnson(Tasks,Temps) :-
% Definition des temps des taches sur
% chaque machine
    Task_A = [5,1,9,3,10],
    Task_B = [2,6,7,8,4],
    johnson(Task_A,Task_B,Tasks,Temps).

johnson(Task_A,Task_B,Tasks,Temps) :-
```

```

    length(Task_A,Nb),
% Definition des variables associees aux
% couples de taches (Ai,Bi)
    declarer_Task(Nb,Task_A,Task_B,[],Temps_Tasks),
% Pose des contraintes
    contraintes_task(Nb,Temps_Tasks,Temps),
% Extraction des variables pour le labeling
    recuperer(Temps_Tasks,LA),
    labeling([],LA),
% Recherche du numero des taches
    temps_to_tasks(Temps_Tasks,Tasks).

% Transformation de la liste en domaine
creerDom([X],Dom):- Dom = {X},!.
creerDom([X|L],Dom):-
    creerDom(L,Dom1),
    Dom = Dom1 \/ {X}.

declarer_Task(0,_,_,X,X):- !.
declarer_Task(N,Task_A,Task_B,L,P):-
    N1 is N-1,
% Definition des variables
% Transformation de la liste en ensemble de CLP-FD
    creerDom(Task_A,SA),
% Pose du domaine
    A in SA,
    creerDom(Task_B,SB),
    B in SB,
% Les deux variables A et B sont liees a la meme tache
    nth1(X,Task_A,A),
    nth1(X,Task_B,B),
    declarer_Task(N1,Task_A,Task_B,[X,A,B]|L,P).

% On pose la contrainte liee a la regle de jonhson
contraintes_task(1,_,0):-!.
contraintes_task(T,[_,A,B],[X,A1,B1]|Task,Temps1):-
    min(A,B1) #< min(A1,B),
    T1 is T-1,
    Temps1 #= Temps + max(A1,B),
    contraintes_task(T1,[X,A1,B1]|Task,Temps).

recuperer([],[]):-!.
recuperer([A,_,_|L],[A|LA]):-recuperer(L,LA).

temps_to_tasks([],[]):-!.
temps_to_tasks([X,_,_|T],[X|L]):-
    temps_to_tasks(T,L).

```

— Question 2 - Programme prolog classique (sans PLC)

```

:- use_module(library(lists)).

% Liste des tâches sous la forme :
% - Nom de la tache
% - Durée sur la machine 1
% - Durée sur la machine 2

listeTaches([
[t1,5,2],
[t2,1,6],

```



```

[t3,9,7],
[t4,3,8],
[t5,10,4]]).

%% Extrait les informations de la liste des tâches pour construire 3 listes :
%% - une liste des noms des tâches
%% - la durée des tâches sur le machine M1
%% - la durée des tâches sur le machine M2
extraitInformation([],[],[],[]).
extraitInformation([[NomTache,DureeM1,DureeM2]|LInfo],
    [NomTache|LTache],[DureeM1|LM1],[DureeM2|LM2]):-
    extraitInformation(LInfo,LTache,LM1,LM2).

% Obtient dans Return les taches Tasks réordonnées selon la
% règle de Johnson à partir de DuréesSurM1 et DuréesSurM2, les
% durées sur chaque machines
regleDeJohnson(Taches, DureeSurM1, DureeSurM2, Solution) :-
    regleDeJohnson(Taches, DureeSurM1, DureeSurM2, [], [], Solution).

% Quand toutes les taches sont traitées on met les listes les unes à
% la suite des autres
regleDeJohnson([], [], [], L1, L2, Solution):-
    append(L1, L2, Solution).
% Cas général avec 2 possibilités suivant la durée de la
% tâche sur M1 ou M2, 1ere possibilité :
regleDeJohnson(Taches, DureeSurM1, DureeSurM2, L1, L2, Solution):-
    min_list(DureeSurM1, MinDureeM1), min_list(DureeSurM2, MinDureeM2),
    % On prend la tache la plus courte peu importe la machine
    MinDureeM1 < MinDureeM2,!,
    nth0(Index, DureeSurM1, MinDureeM1),
    nth0(Index, Taches, TacheChoisie, NTaches),
    % La tache est sur M1 alors on l'ajoute à la fin de L1 et L2 est inchangée
    append(L1, [TacheChoisie], NewL1),
    NewL2 = L2,
    % Nouvelles variables et appel récursif
    nth0(Index, DureeSurM1, _, NDureeSurM1),
    nth0(Index, DureeSurM2, _, NDureeSurM2),
    regleDeJohnson(NTaches, NDureeSurM1, NDureeSurM2, NewL1, NewL2, Solution).
% 2eme Possibilité :
regleDeJohnson(Taches, DureeSurM1, DureeSurM2, L1, L2, Solution):-
    min_list(DureeSurM1, MinDureeM1), min_list(DureeSurM2, MinDureeM2),
    % On prend la tache la plus courte peu importe la machine
    MinDureeM1 >= MinDureeM2,!,
    nth0(Index, DureeSurM2, MinDureeM2),
    nth0(Index, Taches, TacheChoisie, NTaches),
    % La tache est sur M2 alors on l'ajoute au début de L2 et L1 reste inchangé
    NewL2 = [TacheChoisie|L2],
    NewL1 = L1,
    % Nouvelles variables et appel récursif
    nth0(Index, DureeSurM1, _, NDureeSurM1),
    nth0(Index, DureeSurM2, _, NDureeSurM2),
    regleDeJohnson(NTaches, NDureeSurM1, NDureeSurM2, NewL1, NewL2, Solution).

% Obtient le temps total des taches Tasks pour l'ordre spécifié
% par TachesOrdonnées.
% Dans D1 on additionne les temps de calcul sur M1 dans l'ordre des taches
% Dans D2 on prend le max entre la date de fin de la tache précédente
% sur D2 et la date de fin de la tache courante sur D1
% et on y ajoute la durée de la tache sur M2

```

```
calculerDuree(TachesOrdonnees, Taches, DureeSurM1, DureeSurM2, Duree) :-  
    calculerDuree(TachesOrdonnees, Taches, DureeSurM1, DureeSurM2, 0, 0, Duree).  
% On s'arrête quand toutes les taches ordonnées ont été traitées.  
calculerDuree([], _, _, _, _, D2, D2).  
% Cas général pour lequel on extrait une tache pour calculer sa durée.  
calculerDuree([Tache|TachesOrdonnees], Taches, DureeSurM1, DureeSurM2, D1, D2, Duree) :-  
    nth0(Index, Taches, Tache),  
    nth0(Index, DureeSurM1, DureeTacheM1),  
    nth0(Index, DureeSurM2, DureeTacheM2),  
    NewD1 is D1 + DureeTacheM1,  
    NewD2 is max(NewD1, D2) + DureeTacheM2,  
    calculerDuree(TachesOrdonnees, Taches, DureeSurM1, DureeSurM2, NewD1, NewD2, Duree).  
  
% Prédicat pour la résolution  
jobShopJohnson(Duree, TachesOrdonnees):-  
    listeTaches(ListeTaches),  
    extraitInformation(ListeTaches, Taches, DureeSurM1, DureeSurM2),  
    regleDeJohnson(Taches, DureeSurM1, DureeSurM2, TachesOrdonnees),  
    calculerDuree(TachesOrdonnees, Taches, DureeSurM1, DureeSurM2, Duree).
```

11.8 Exercices chapitre 8

Exercice 1

[Retour à l'énoncé.](#)

1. Isoler les comportements indiquant des malfunctions d'un site internet?
C'est une détection d'anomalie.
2. Prédire le salaire d'une personne en fonction de ses années d'expériences et ses performances?
C'est une régression
3. Une entreprise de carte bancaire reçoit des milliers de demandes de nouvelles cartes. Les demandes contiennent des informations sur le client comme son salaire annuel, de grosses dettes le cas échéant, l'âge etc. Qui a un bon profil, un mauvais profil, ou un profil nécessitant plus d'analyse?
C'est une classification.
4. Trouver des groupes de clients ayant des comportements similaires dans les magasins à partir d'une large base de données incluant leurs précédents achats et leur fréquence de venue.
C'est un partitionnement ("*clustering*")

Exercice 2

[Retour à l'énoncé.](#)

1. Première étape est de calculer la moyenne et l'écart type pour la latitude et la longitude :
 - Latitude : moyenne = 47,268 et ecart-type = 0,455
 - Longitude : moyenne = 5,102 et ecart-type = 1,157

Ainsi, on peut normaliser les valeurs (ici nous choisisons pour cela, la standardisation est (valeur - moyenne) / ecart-type) et on obtient les nouvelles données suivantes :

Nom	Latitude normalisée	Longitude normalisée
Autun	-0,697	-0,695
Auxerre	1,158	-1,324
Avallon	0,481	-1,033
Beaune	-0,539	-0,228
Belfort	0,809	1,52
Besançon	-0,067	0,796
Chalon sur Saône	-1,07	-0,215
Dijon	0,115	-0,053
Dole	-0,387	0,336
Lons le Saunier	-1,305	0,389
Lure	0,916	1,202
Mâcon	-2,114	-0,234
Montbard	0,78	-0,661
Montbéliard	0,53	1,464
Nevers	-0,618	-1,681
Pontarlier	-0,8	1,081
Sens	2,039	-1,573
Vesoul	0,769	0,908

Nous obtenons comme écart-type sur les données normalisés 1 pour les 2 variables.

2. Nous pouvons ensuite passer à la classification avec l'algorithme de classification ascendante hiérarchique. Voici la liste des 18 partitions obtenues (on ne détaille pas la première qui contient les 18 villes indépendamment) :

- (1) Contient les 18 villes séparées
- (2) Regroupe Belfort et Montbéliard avec un coût de 0,04 et centroïde de [0,669; 1,492]
- (3) Regroupe Lure et Vesoul avec un coût de 0,053 et centroïde de [0,843; 1,055]
- (4) Regroupe Avallon et Montbard avec un coût 0,110 de et centroïde de [0,631; -0,847]
- (5) Regroupe Autun et Beaune avec un coût de 0,121 et centroïde de [-0,618; -0,461]
- (6) Regroupe Besançon et Dole avec un coût de 0,157 et centroïde de [-0,227; 0,66]
- (7) Regroupe (5) et Chalon sur Saône avec un coût de 0,176 et centroïde de [-0,769; -0,379]
- (8) Regroupe (2) et (3) avec un coût de 0,220 et centroïde de [0,756; 1,274]
- (9) Regroupe (6) et Dijon avec un coût de 0,334 et centroïde de [-0,113; 0,36]
- (10) Regroupe (4) et Auxerre avec un coût de 0,336 et centroïde de [0,807; -1,006]
- (11) Regroupe Lons le Saunier et Pontarlier avec un coût de 0,367 et centroïde de [-1,053; 0,735]
- (12) Regroupe (9) et (11) avec un coût de 1,229 et centroïde de [-0,489; 0,51]
- (13) Regroupe (7) et Nevers avec un coût de 1,287 et centroïde de [-0,731; -0,705]
- (14) Regroupe (10) et Sens avec un coût de 1,379 et centroïde de [1,115; -1,148]
- (15) Regroupe (13) et Mâcon avec un coût de 1,706 et centroïde de [-1,007; -0,611]
- (16) Regroupe (12) et (15) avec un coût de 3,815 et centroïde de [-0,748; -0,05]
- (17) Regroupe (8) et (16) avec un coût de 11,485 et centroïde de [-0,318; 0,328]
- (18) Regroupe (14) et (17) avec un coût de 13,172 et centroïde de [0; 0]

Le bon partitionnement serait celui pour lequel le coût est proche de la somme des écart-types 1 (latitude) et 1 (longitude) soit 2. Ainsi, on prend le 15^e regroupement et on obtient ainsi les 4 partitions suivantes :

1. le regroupement (8) : Lure, Vesoul, Belfort et Montbéliar
 2. le regroupement (12) : Lons le Saunier, Pontarlier, Besançon, Dole et Dijon
 3. le regroupement (14) : Avallon, Montbard, Auxerre et Sens
 4. le regroupement (15) : Autun, Beaune, Chalon sur Saône, Nevers et Mâcon
3. On cherche à trouver les 4 partitions avec l'algorithme k-means. On peut prendre les 4 premiers points qui vont servir de centroïde. On obtient alors le résultat :

	5	6	7	8	9	10	11	12	13	14	15	16	17	18
1	2.604	1.574	0.591	1.007	1.047	1.209	2.421	1.448	1.437	2.414	0.961	1.730	2.793	2.113
2	2.786	2.380	2.420	1.598	2.205	2.917	2.467	3.352	0.741	2.778	1.761	3.015	0.889	2.203
3	2.502	1.856	1.705	1.017	1.577	2.220	2.214	2.639	0.463	2.428	1.241	2.404	1.602	1.908
4	2.146	1.097	0.517	0.658	0.569	0.957	1.984	1.530	1.350	1.946	1.414	1.298	2.826	1.685

Cela donne

- Partition 1 : Autun, Macon, Nevers, nouveau centroïde (-1,111 ; -0,845)
- Partition 2 : Auxerre, Sens, nouveau centroïde (1,553 ; -1,408)
- Partition 3 : Avallon, Montbard, nouveau centroïde (0,613 ; -0,823)
- Partition 4 : Beaune, Belfort, Besançon, Chalon, Dijon, Dôle, Lons, Lure, Montbéliard, Pontarlier, Vesoul nouveau centroïde (-0,091 ; 0,636)

Sur la base des nouveaux centroïdes, la répartition devient :

- Partition 1 : Autun, Beaune Chalon, Macon, Nevers, nouveau centroïde (-0,979 ; -0,593)
- Partition 2 : Auxerre, Sens, nouveau centroïde (1,553 ; -1,408)
- Partition 3 : Avallon, Montbard, nouveau centroïde (0,613 ; -0,823)
- Partition 4 : Belfort, Besançon, Dijon, Dole, Lons, Lure, Montbéliard, Pontarlier, Vesoul nouveau centroïde (0,062 ; 0,825)

Sur la base des nouveaux centroïdes, la répartition devient :

- Partition 1 : Autun, Beaune Chalon, Lons, Macon, Nevers, nouveau centroïde (-1,028 ; -0,432)
- Partition 2 : Auxerre, Sens, nouveau centroïde (1,553 ; -1,408)
- Partition 3 : Avallon, Montbard, nouveau centroïde (0,613 ; -0,823)
- Partition 4 : Belfort, Besançon, Dijon, Dole, Lure, Montbéliard, Pontarlier, Vesoul nouveau centroïde (0,229 ; 0,882)

Sur la base des nouveaux centroïdes, la répartition devient :

- Partition 1 : Autun, Beaune Chalon, Lons, Macon, Nevers, nouveau centroïde (-1,028 ; -0,432)
- Partition 2 : Auxerre, Sens, nouveau centroïde (1,553 ; -1,408)
- Partition 3 : Avallon, Dijon, Montbard, nouveau centroïde (0,446 ; -0,566)
- Partition 4 : Belfort, Besançon, Dole, Lure, Montbéliard, Pontarlier, Vesoul nouveau centroïde (0,246 ; 0,882)

Le système n'évolue plus et nous avons une variance totale de 11.35.

4. Pour réaliser l'analyse meanShift des données, nous regardons le domaine de définition de chacune des variables. Pour la latitude, nous avons $[-2,114; 2,039]$ et pour la longitude $[-1,681; 1,521]$, on peut faire trois partitions de chaque domaine pour avoir une répartition homogène ce qui donne respectivement comme valeurs $(-1,426; -0,04; 1,347)$ et $(-1,15; -0,081; 0,987)$ nous avons ainsi 9 centroïdes avec un rayon respectivement de 0,693 et 0,534. Lors de la première itération nous avons :

- Centroïde initial 1 $[-1.426, -1.15]$ éléments : [], disparaît
- Centroïde initial 2 $[-1.426, -0.081]$ éléments : [Chalon, Lons, Macon], nouveau centroïde 2 $[-1.497, -0.02]$
- Centroïde initial 3 $[-1.426, 0.987]$ éléments : [Pontarlier], nouveau centroïde 3 $[-0.801, 1.082]$
- Centroïde initial 4 $[-0.04, -1.15]$ éléments : [Autun, Avallon, Nevers], nouveau centroïde 4 $[-0.278, -1.137]$
- Centroïde initial 5 $[-0.04, -0.081]$ éléments : [Beaune, Dijon, Dole], nouveau centroïde 5 $[-0.270, 0.018]$
- Centroïde initial 6 $[-0.04, 0.987]$ éléments : [Besancon, Montbeliard], nouveau centroïde 6 $[0.232, 1.130]$
- Centroïde initial 7 $[1.347, -1.15]$ éléments : [Auxerre, Montbard, Sens], nouveau centroïde 7 $[1.326, -1.186]$
- Centroïde initial 8 $[1.347, -0.081]$ éléments : [], disparaît
- Centroïde initial 9 $[1.347, 0.987]$ éléments : [Belfort, Lure, Vesoul], nouveau centroïde 9 $[0.832, 1.211]$

Lors de la 2e itération :

- Centroïde initial 2 $[-1.497, -0.02]$ éléments : [Chalon, Lons, Macon], centroïde inchangé
- Centroïde initial 3 $[-0.801, 1.082]$ éléments : [Pontarlier], centroïde inchangé
- Centroïde initial 4 $[-0.278, -1.137]$ éléments : [Autun], nouveau centroïde 4 $[-0.697, -0.695]$
- Centroïde initial 5 $[-0.270, 0.018]$ éléments : [Beaune, Dijon, Dole], centroïde inchangé
- Centroïde initial 6 $[0.232, 1.130]$ éléments : [Belfort, Besancon, Lure, Montbeliard, Vesoul], nouveau centroïde 6 $[0.592, 1.179]$
- Centroïde initial 7 $[1.326, -1.186]$ éléments : [Auxerre, Montbard], nouveau centroïde 7 $[0.97, -0.993]$
- Centroïde initial 9 $[0.832, 1.211]$ éléments : [Belfort, Lure, Montbeliard, Vesoul], nouveau centroïde 9 $[0.756, 1.274]$

Lors de la 3e itération :

- Centroïde initial 2 $[-1.497, -0.02]$ éléments : [Chalon, Lons, Macon], centroïde inchangé
- Centroïde initial 3 $[-0.801, 1.082]$ éléments : [Pontarlier], centroïde inchangé
- Centroïde initial 4 $[-0.697, -0.695]$ éléments : [Autun, Beaune, Chalon], nouveau centroïde 4 $[-0.769, -0.380]$
- Centroïde initial 5 $[-0.270, 0.018]$ éléments : [Beaune, Dijon, Dole], centroïde inchangé
- Centroïde initial 6 $[0.592, 1.179]$ éléments : [Belfort, Besancon, Lure, Montbeliard, Vesoul], centroïde inchangé
- Centroïde initial 7 $[0.97, -0.993]$ éléments : [Auxerre, Avallon, Montbard], nouveau centroïde 7 $[0.807, -1.006]$
- Centroïde initial 9 $[0.756, 1.274]$ éléments : [Belfort, Lure, Montbeliard, Vesoul], centroïde inchangé

Lors de la 4e itération :

- Centroïde initial 2 $[-1.497, -0.02]$ éléments : [Chalon, Lons, Macon], centroïde inchangé
- Centroïde initial 3 $[-0.801, 1.082]$ éléments : [Pontarlier], centroïde inchangé
- Centroïde initial 4 $[-0.769, -0.380]$ éléments : [Autun, Beaune, Chalon], centroïde inchangé
- Centroïde initial 5 $[-0.270, 0.018]$ éléments : [Beaune, Dijon, Dole], centroïde inchangé
- Centroïde initial 6 $[0.592, 1.179]$ éléments : [Belfort, Besancon, Lure, Montbeliard, Vesoul], centroïde inchangé
- Centroïde initial 7 $[0.807, -1.006]$ éléments : [Auxerre, Avallon, Montbard], centroïde inchangé
- Centroïde initial 9 $[0.756, 1.274]$ éléments : [Belfort, Lure, Montbeliard, Vesoul], centroïde inchangé

A la sortie de la boucle de l'algorithme meanShift, nous fusionnons les centroïdes C4 / C5 et C6 / C9 :

- Centroïde 2 $[-1.497, -0.02]$
- Centroïde 3 $[-0.801, 1.082]$
- Centroïde 4 $[-0.52, -0.181]$
- Centroïde 6 $[0.674, 1.226]$

— Centroïde 7 [0.807, -1.006]

Si on fait le tableau des distances entre chaque point et les 5 centroïdes :

	Centroïde 2	Centroïde 3	Centroïde 4	Centroïde 6	Centroïde 7
Autun	2,296	2,324	1,322	1,921	0,33
Auxerre	2,958	3,103	2,031	2,596	0,474
Avallon	2,223	2,473	1,315	2,267	0,326
Beaune	0,98	1,336	0,051	1,894	1,555
Belfort	2,774	1,669	2,159	0,324	2,527
Besancon	1,647	0,787	1,077	0,857	2,004
Chalon	0,469	1,325	0,552	2,263	2,038
Dijon	1,612	1,458	0,648	1,396	1,178
Dole	1,165	0,852	0,534	1,385	1,798
Lons	0,452	0,857	0,971	2,149	2,532
Lure	2,706	1,722	1,995	0,244	2,212
Macon	0,653	1,859	1,595	3,147	3,021
Montbard	2,366	2,354	1,387	1,891	0,346
Montbeliard	2,513	1,385	1,952	0,278	2,486
Nevers	1,879	2,769	1,504	3,182	1,577
Pontarlier	1,303	0	1,293	1,482	2,635
Sens	3,862	3,888	2,913	3,114	1,356
Vesoul	2,45	1,58	1,688	0,331	1,916

Nous avons au final 5 partitions (ce qui est différent des résultats précédents qui en proposaient 4) :

- Centroïde 2 [-1.497, -0.02], éléments de la partition : Chalon, Lons, Macon
- Centroïde 3 [-0.801 ; 1.082], éléments de la partition : Besançon, Pontarlier
- Centroïde 4 [-0.52, -0.181], éléments de la partition : Beaune, Dijon, Dole, Nevers
- Centroïde 6 [0.674 ; 1.226], éléments de la partition : Belfort, Lure, Montbeliard, Vesoul
- Centroïde 7 [0.807 ; -1.006], éléments de la partition : Autun, Auxerre, Avallon, Montbard, Sens

Exercice 3

[Retour à l'énoncé.](#)

1. Traitement de l'exemple positif : <Ardoises, Pierres, Une, Petites>

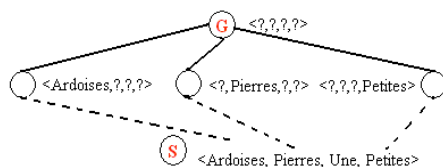
On obtient le graphe suivant :

G <?, ?, ?, ?>

S <Ardoises, Pierres, Une, Petites>

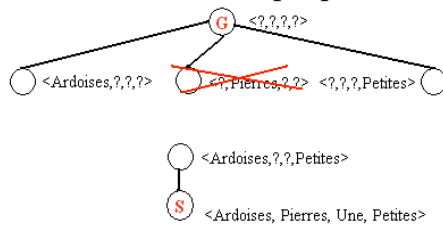
S est le nœud le plus spécifique. G est le nœud le plus général, ? signifiant que l'on peut prendre n'importe quelle valeur

2. Traitement de l'exemple négatif : <Tuilles, Briques, Une, Grandes>



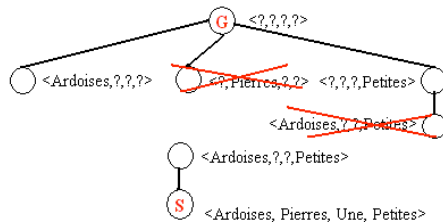
- L'exemple spécialise G de toutes les manières possibles.
- Chaque nouveau modèle est la généralisation d'un modèle de S.
- <?, ?, Une, ?> n'apparaît pas, car il accepte l'exemple.

3. Traitement de l'exemple positif : <Ardoises, Parpaings, Deux, Petites>



On doit éliminer <?, Pierres, ?, ?> car il rejette l'exemple.

4. Traitement de l'exemple négatif : <Chaumes, Pierres, Une, Petites>

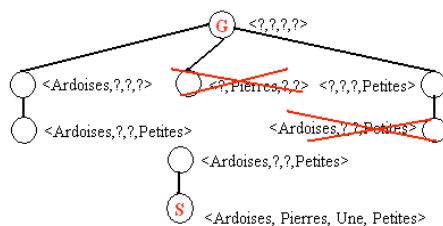


— <Ardoises, ?, ?, ?> n'accepte pas l'exemple donc il n'est pas spécialisé et reste dans l'arbre de généralisation.

— <?, ?, ?, Petites> accepte l'exemple, il est donc spécialisé.

— Aucun nouveau modèle ne doit être la spécialisation d'un autre modèle de G. Donc <Ardoises, ?, ?, Petites> est éliminé.

5. Traitement de l'exemple négatif : <Ardoises, Pierres, Une, Moyennes>



— Le seul moyen pour spécialiser <Ardoises, ?, ?, ?> dans la direction de <Ardoises, ?, ?, Petites> et de créer <Ardoises, ?, ?, Petites>.

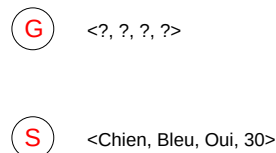
— Les deux arbres convergent. Le modèle <Ardoises, ?, ?, Petites> décrit le concept de "Maison bretonne".

Exercice 4

Retour à l'énoncé.

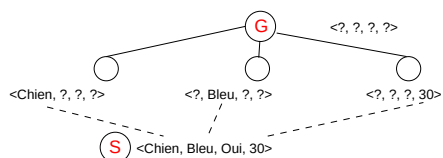
1. Traitement de l'exemple positif : <Chien, Bleu, Oui, 30>

On obtient le graphe suivant :



S est le nœud le plus spécifique. G est le nœud le plus général, ? signifiant que l'on peut prendre n'importe quelle valeur

2. Traitement de l'exemple négatif : <Chat, Rouge, Oui, 40>

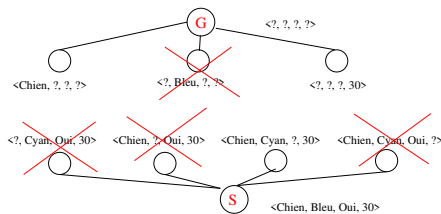


— L'exemple spécialise G de toutes les manières possibles.

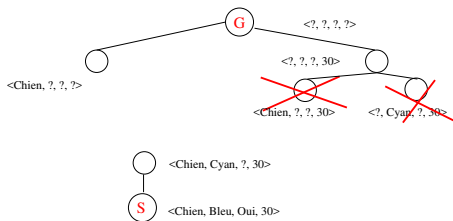
— Chaque nouveau élément de G est la généralisation d'un modèle de S.

— <?, ?, Oui, ?> n'apparaît pas, car il accepte l'exemple.

3. Traitement de l'exemple positif : <Chien, Vert, Non, 30>

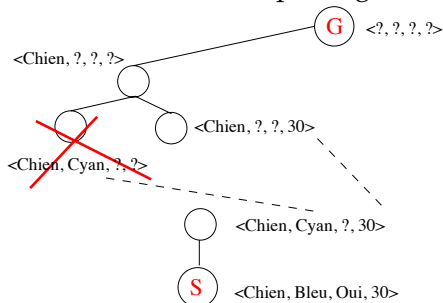


4. Traitement de l'exemple négatif : <Poisson, Bleu, Oui, 30>



- On ne peut pas garder $\langle ?, \text{Bleu}, ?, ? \rangle$ car il rejette l'exemple.
- On fait une généralisation minimal pour le nœud de S, premièrement avec la couleur pour généraliser (vert et bleu) en cyan, puis le fait qu'il est une marque. On ne garde que le 3e qui accepte l'exemple
- $\langle \text{Chien}, ?, ?, ? \rangle$ n'accepte pas l'exemple donc il n'est pas spécialisé et reste dans l'arbre de généralisation.
- $\langle ?, ?, ?, 30 \rangle$ accepte l'exemple, il est donc spécialisé.
- $\langle ?, \text{Cyan}, ?, 30 \rangle$ accepte l'exemple, il est donc éliminé.
- Aucun nouveau modèle ne doit être la spécialisation d'un autre modèle de G. Donc $\langle \text{Chien}, ?, ?, 30 \rangle$ est éliminé.

5. Traitement de l'exemple négatif : <Chien, Bleu, Oui, 50>



- $\langle \text{Chien}, ?, ?, ? \rangle$ accepte l'exemple, il est donc spécialisé.
- $\langle \text{Chien}, \text{Cyan}, ?, ? \rangle$ accepte l'exemple, il est supprimé

Nous avons une solution : Un chien de 30 ans

Exercice 5

[Retour à l'énoncé.](#)

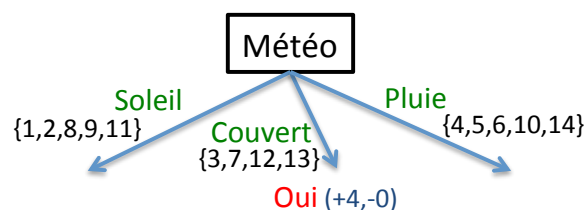
1.

Attribut	Tennis	Non	Oui	Majoritaire	Entropie
Météo	Soleil	3	2	3	0,971
	Couvert	0	4	4	0
	Pluie	2	3	3	0,971
	Bilan	5	9	10	0,693
Température	Chaude	2	2	2	1
	Idéale	2	4	4	0,918
	Froide	1	3	3	0,811
	Bilan	5	9	9	0,911
Humidité	Elevée	4	3	4	0,985
	Normale	1	6	6	0,591
	Bilan	5	9	10	0,788
Vent	Faible	2	6	6	0,811
	Fort	3	3	3	1
	Bilan	5	9	9	0,892

Les deux critères nous donnent le même résultat on va discriminer sur l'attribut Météo. Pourquoi, dans le cas du critère majoritaire, on ne choisit pas Humidité qui donne aussi 10, car on prend le premier critère rencontré (choix arbitraire). On partitionne la population suivant les valeurs de l'attribut Météo. On obtient le contingentement suivant :

- (a) Météo = Soleil avec les jours {J1, J2, J8, J9, J11}
- (b) Météo = Couvert avec les jours {J3, J7, J12, J13}
- (c) Météo = Pluie avec les jours {J4, J5, J6, J10, J14}

Le groupe 2 est bien classé et contient les jours de partie de tennis. Les groupes 1 et 3 restent à classer. Voici le graphe qui résumant l'étape courante :



2.

Attribut	Tennis	Non	Oui	Majoritaire	Entropie	Non	Oui	Majoritaire	Entropie
Température	Chaude	2	0	2	0	0	0	0	0
	Idéale	1	1	1	1	1	2	2	0,918
	Froide	0	1	1	0	1	1	1	1
	Bilan	3	2	4	0,4	2	3	3	0,0,951
Humidité	Elevée	3	0	3	0	1	1	1	1
	Normale	0	2	2	0	1	2	2	0,918
	Bilan	3	2	5	0	2	3	3	0,0,951
Vent	Faible	2	1	2	0,918	0	3	3	0
	Fort	1	1	1	1	2	0	2	0
	Bilan	3	2	3	0,951	2	3	5	0

On partitionne la population suivant les valeurs de l'attribut Humidité pour le soleil et le Vent pour la pluie. On obtient le contingentement suivant :

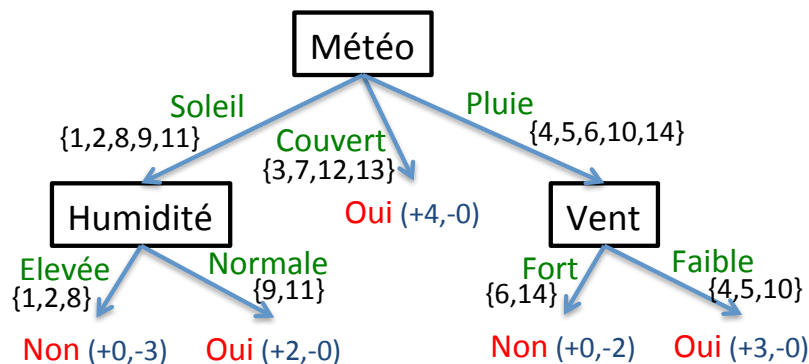
- (a) Météo = Soleil et Humidité = Elevée avec les jours {J1, J2, J8}

(b) Météo = Soleil et Humidité = Normale avec les jours {J9, J11}

(c) Météo = Pluie et Vent = Fort avec les jours {J6, J14}

(d) Météo = Pluie et Vent = Faible avec les jours {J4, J5, J10}

Les groupes 1 et 3 sont bien classés et contiennent les jours sans partie de tennis. Les groupes 2 et 4 sont bien classés et contiennent les jours de partie de tennis. On obtient ainsi l'arbre final :



Exercice 6

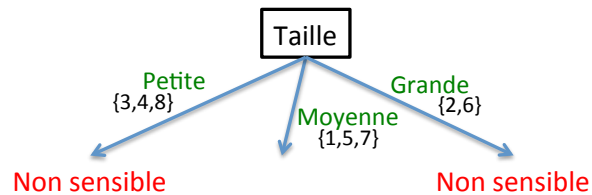
[Retour à l'énoncé.](#)

Attribut	Résultat	Sensible	Non sensible	Entropie
Cheveux	Blond	2	3	0,97
	Brun	0	3	0
	Bilan	2	6	0,60
Taille	Petit	0	3	0
	Moyen	2	1	0,91
	Grand	0	2	0
	Bilan	2	6	0,34
Poids	Léger	1	1	1
	Moyen	0	3	0
	Lourd	1	2	0,91
	Bilan	2	6	0,59
Crème	Oui	0	2	0
	Non	2	4	0,91
	Bilan	2	6	0,68

L'attribut ayant le meilleur résultat est Taille. On obtient ainsi trois fils correspondant aux valeurs de l'attribut :

- petit : les éléments 3,4,8 sont donc non sensible
- moyen : les éléments 1,5,7 restent à discriminer
- grand : les éléments 2,6 sont donc non sensible.

La figure suivante résume cela :



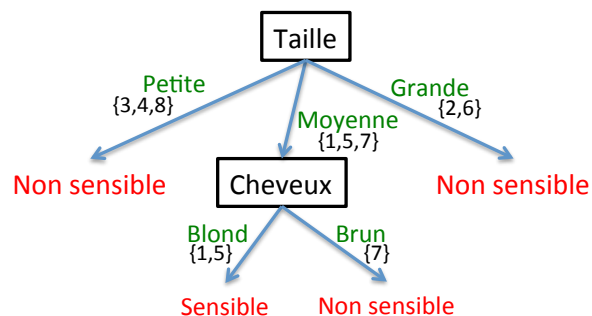
Le calcul sur la valeur "moyen" donne :

Attribut	Résultat	Sensible	Non sensible	Entropie
Cheveux	Blond	2	0	0
	Brun	0	1	0
	Bilan	2	1	0
Poids	Léger	1	0	0
	Moyen	0	0	0
	Lourd	1	1	1
	Bilan	2	1	0,66
Crème	Oui	0	0	0
	Non	2	1	0,92
	Bilan	2	1	0,92

L'attribut Cheveux permet de finir la discrimination, avec :

- Blond : les éléments 1,5 sont sensibles.
- Brun : l'élément 7 est non sensible

Nous obtenons la figure suivante :



Exercice 7

[Retour à l'énoncé.](#)

Les réponses aux questions :

- il y a 54 éléments Iris-setosa dans le data-set.
- Il faut éliminer les données vides (on enlève les données après la ligne 157).
- On trouve un $k = 6$ qui donne le meilleur clustering, mais il n'est pas parfait (entropie $\neq 0$)
- En utilisant le système de filtre outliers on passe de 157 à 143 éléments, la qualité est diminuée et on trouve le meilleur résultat avec $k = 10$.
- On enlève ainsi les données comprises entre la ligne 151 et la fin. On constate une amélioration, le meilleur résultat est donc avec $k = 10$ pour lequel nous avons une entropie de 0.

- Concernant la reprise de l'exercice 2, la normalisation a un vrai impact, car les dendrogrammes ne donnent pas le même résultat (ordre des regroupements des villes). Pour déterminer le bon k , nous ne pouvons pas utiliser l'entropie puisque nous n'avons pas le cluster attendu. Cette fois nous allons utiliser le score de silhouette. Nous prendrons le k pour lequel est le plus bas avant de remonter. Dans ce cas, nous obtenons comme dans le corrigé de l'exercice $k = 5$ (avec un score de 0,556 pour l'ensemble des clusters). On obtient un score de 0 avec $k=17$, mais on peut se demander l'intérêt de partitionner en autant d'éléments.
- Concernant la reprise de l'exercice 5, nous obtenons les mêmes éléments pour les règles.

11.9 Exercices chapitre 9

Exercice 1

[Retour à l'énoncé.](#)

— Question 1 :

```

placer(N,S):-
    creer(N,[],L),
    generer(N,L,[],S).
% Cree une liste de 1 a N
creer(0,R,R):- !.
creer(N,L,R):-
    N1 is N-1,
    creer(N1,[N|L],R).

% Cree une instance aleatoire pour les reines
generer(1,[X],R,[1,X|R]):- !.
generer(N,L1,L,R):-
    random(1,N,Nb),
    nth0(Nb,L1,Y,L2),
    N1 is N-1,
    generer(N1,L2,[N,Y|L],R).

```

— Question 2 :

```

test_en_prisel([],_).
test_en_prisel([C2|R],C1):-
    \+ en_prisel(C1,C2),
    test_en_prisel(R,C1).

en_prisel([A,_],[A,_]).
en_prisel([_,B],[_,B]).
en_prisel([A,B],[C,D]):-
    E is A-C,
    E is B-D.
en_prisel([A,B],[C,D]):-
    E is A-C,
    E is D-B.

```

— Question 3 :

```

% Effectue la permutation des reines en prises
% on arrive au seuil
permutation(0,_,_,_):-fail,!.
% Il n'y a plus d'autre reine a permuter
permutation(_,X,[],[X]):-!.

```

```
% On permute le couple
permutation(_, [XC, YC], [[XY, YY]], [[XC, YY], [XY, YC]]) :-!.
% On effectue toutes les combinaisons de permutation
permutation(_, [XC, YC], [[XY, YY] | L], [[XC, YY], [XY, YC] | L]).
permutation(N, [XC, YC], [C | L], [C | T]) :-
    N1 is N-1,
    permutation(N1, [XC, YC], L, T).
```

— Question 4 :

```
% Partitionne les reines non en prises et les en prises
partition([], Ok, Ok, Ko, Ko) :-!.
partition([C | L], Ok, Ok1, Ko, Ko1) :-
    test_en_prisell(Ok, C), !,
    partition(L, [C | Ok], Ok1, Ko, Ko1).
partition([C | L], Ok, Ok1, Ko, Ko1) :-
    partition(L, Ok, Ok1, [C | Ko], Ko1).
```

— Question 5 :

```
% Effectue une permutation entre les reines en prise
reparation(0, Ok, _, Ok) :-!.
reparation(N, Ok, [A | Ko], S) :-
% Utilisation d'un seuil
    N1 is N // 100 + N mod 50,
    permutation(N1, A, Ko, [X | Kl]),
    test_en_prisell(Ok, X),
    length(Kl, Lko),
    reparation(Lko, [X | Ok], Kl, S).
```

— Question 6 :

```
% test si on a trouve une solution
test(_, R, [], S) :-
    partition(R, [], Ok, [], Ko),
    length(Ko, Lko),
    reparation(Lko, Ok, Ko, S).
% rappel le systeme avec une nouvelle instance
test(N, _, [], S) :-
    placer(N, R1),
    test(N, R1, [], S).
```

— Question 7 :

```
:-use_module(library(random)).
:-use_module(library(lists)).

reines(N, S) :-
    placer(N, R),
    test(N, R, [], S).
```

11.10 Exercices chapitre 10

Exercice 1

[Retour à l'énoncé.](#)

1. Question 1 :

```
phrase1(P):-  
    groupe_nominal1(P, Pn),  
    groupe_verbal1(Pn).  
  
groupe_verbal1([V|C]):-  
    groupe_nominal1(C, []),  
    verbe1(V).  
  
groupe_nominal1([A,M|S], S):-  
    article1(A),  
    nom1(M).  
  
verbe1(mange).  
verbe1(effraye).  
  
article1(le).  
  
nom1(chat).  
nom1(souris).
```

2. Question 2 :

```
phrase2(P):-  
    groupe_nominal2(P, Pn, Nombre),  
    groupe_verbal2(Pn, Nombre).  
  
groupe_verbal2([V|C], Nombre):-  
    groupe_nominal2(C, [], _),  
    verbe2(V, Nombre).  
  
groupe_nominal2([A,M|S], S, Nombre):-  
    article2(A, Genre, Nombre),  
    nom2(M, Genre, Nombre).  
  
verbe2(mange, singulier).  
verbe2(effraye, singulier).  
verbe2(effrayent, pluriel).  
  
article2(les, feminin, pluriel).  
article2(les, masculin, pluriel).  
article2(la, feminin, singulier).
```



```
article2(le, masculin, singulier).
```

```
nom2(chat, masculin, singulier).  
nom2(souris, feminin, singulier).  
nom2(souris, feminin, pluriel).
```

3. Question 3 :

```
phrase(P):-  
    groupe_nominal(P, Pn, Nombre, Concept),  
    groupe_verbal(Pn, Nombre, Concept).
```

```
groupe_verbal([V|C], Nombre, Concept):-  
    groupe_nominal(C, [], _, ConceptCO),  
    verbe(V, Nombre, Concept, ConceptCO).
```

```
groupe_nominal([A,M|S], S, Nombre, Concept):-  
    article(A, Genre, Nombre),  
    nom(M, Genre, Nombre, Concept).
```

```
verbe(mange, singulier, ConcSj, ConcCo):- ConcSj > ConcCo.  
verbe(effraye, singulier, ConcSj, ConcCo):- ConcSj >= ConcCo.  
verbe(effrayent, pluriel, ConcSj, ConcCo):- ConcSj >= ConcCo.
```

```
article(les, feminin, pluriel).  
article(les, masculin, pluriel).  
article(la, feminin, singulier).  
article(le, masculin, singulier).
```

```
nom(chat, masculin, singulier, 1).  
nom(souris, feminin, singulier, 0).  
nom(souris, feminin, pluriel, 2).
```

Liste des annexes

A Principaux prédicats prolog

183

A. Principaux prédicats prolog

Ces commandes sont extraites du manuel SWI-Prolog livré avec le logiciel. Elles correspondent au sous-ensemble des commandes que vous utiliserez dans ce cours.

- `abolish/1`
- `all_different/1`
- `append/3`
- `assert/1`
- `clpfd` : librairie pour la gestion des contraintes à domaine fini.
- `consult/1`
- `debug/0`
- `dif/2`
- `in/2`
- `ins/2`
- `labeling/2`
- `length/2`
- `listing/[0,1]`
- `lists` : librairie pour la gestion des listes.
- `member/2`
- `notrace/0`
- `nth0/[3,4]`
- `nth1/[3,4]`
- `reconsult/1`
- `select/[3,4]`
- `setof/3`
- `statistics/[0,2]`
- `sum/3`
- `trace/0`

Bibliographie

- [Del86] Jean-Paul Delahaye. Outils logique pour l'intelligence artificielle. Eyrolles, 1986.
- [GMJM⁺04] Dreyfus Gérard, Samuelides Manuel, Martinez Jean-Marc, Gordon Mirta B., Badran Fouad, Thiria Sylvie, and Hérault Laurent. Réseaux de neurones. 978-2-212-11464-5. Eyrolles, 2004.
- [GOT04] Groupe GOTHA. Modèles et algorithmes en ordonnancement. 978-2-7298-2171-5. Ellipse, 2004.
- [Qui86] J. R. Quinlan. Induction of decision trees. Machine Learning, 1(1) :81–106, March 1986.
- [Qui93] J. Ross Quinlan. C4.5 : Programs for Machine Learning. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, elsevier science edition, 1993.
- [Qui96] J. R. Quinlan. Improved Use of Continuous Attributes in C4.5. Journal of Artificial Intelligence Research, 4 :77–90, March 1996.
- [Sal94] Steven L. Salzberg. C4.5 : Programs for Machine Learning by J. Ross Quinlan. Morgan Kaufmann Publishers, Inc., 1993. Machine Learning, 16(3) :235–240, September 1994.
- [SYOS97] Thiria Sylvie, Lechevalier Yves, Gascuel Olivier, and Canu Stéphane. Statistiques et méthodes neuronales. 978-2100035441. Dunod, 1997.
- [VSP⁺23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, <https://arxiv.org/abs/1706.03762>, 2023.